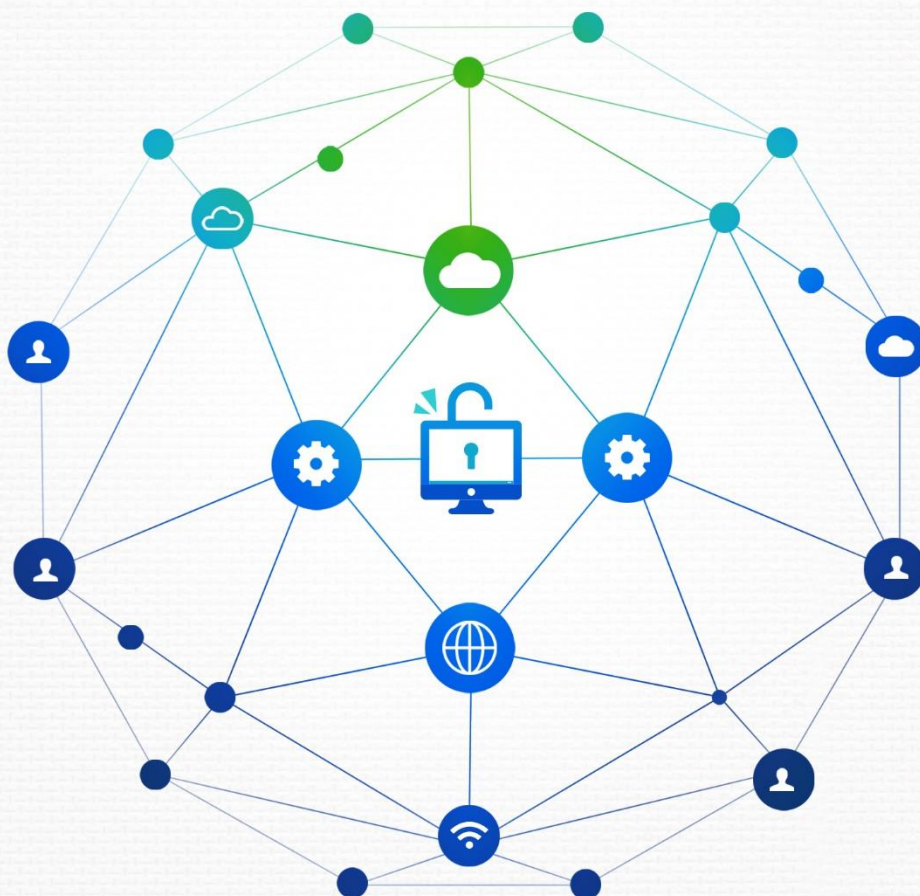


전자정부 SW 개발자·진단원을 위한

공개SW를 활용한 소프트웨어 개발보안 점검가이드

2019. 6



행정안전부



한국인터넷진흥원

전자정부 SW 개발자·진단원을 위한

공개SW를 활용한 소프트웨어 개발보안 점검가이드



제·개정이력(Revision History)

번호	제·개정일	변경 내용	발간팀	비고
1	2016. 2.	[제정] ·공개SW를 활용한 소프트웨어 개발보안 점검가이드	보안평가인증팀	
2	2019. 6.	[개정] ·공개SW 진단도구 변경 및 진단 룰 작성 방법 등 추가	전자정부보호팀	

CONTENTS

PART



제1장 개요

제1절 배경	08
제2절 가이드 목적 및 구성	10

PART



제2장 공개소프트웨어 진단도구 소개 및 설치 환경

제1절 개요	12
제2절 Spotbugs	14
제3절 FindSecurityBugs	16
제4절 PMD	18
제5절 Jenkins	20
제6절 설치환경	22

PART



제3장 Spotbugs/FindSecurityBugs 사용 방법

제1절 Spotbugs 및 FindSecurityBugs 설치	26
제2절 Spotbugs 및 FindSecurityBugs 룰 설정	40
제3절 Spotbugs 및 FindSecurityBugs 검사	43
제4절 Spotbugs에서 플러그인 빌드	46
제5절 FindSecurityBugs에서 플러그인 빌드	49
제6절 플러그인 IDE에 추가	50

PART



제4장 PMD 사용 방법

제1절 PMD 설치	54
제2절 PMD 적용 룰 설정	58
제3절 PMD 검사	63



PART



제5장 Jenkins 사용 방법

제1절 Jenkins 설치	68
제2절 정적분석을 위한 플러그인 설치	76
제3절 Build 관련 공개소프트웨어 연계 설정	78
제4절 Jenkins 실행	88
제5절 Jenkins를 활용한 시큐어코딩 Inspection 활동	96

PART



제6장 사용자 정의 룰(Rule) 작성 및 오탐(False Positive) 관리

제1절 개요	102
제2절 Spotbugs/FindSecurityBugs를 활용한 사용자 정의 룰(Rule) 작성 방법	103
제3절 PMD를 활용한 사용자 정의 룰(Rule) 작성 방법	109
제4절 오탐(False Positive) 관리	124

PART



제7장 행안부 47개 보안약점 관련 공개SW 진단도구 룰(Rule) 분석

제1절 행정안전부 47개 보안약점 관련 공개SW 도구 룰(Rule) 목록	130
제2절 행정안전부 47개 보안약점 관련 공개SW 도구 룰(Rule) 현황	132

PART



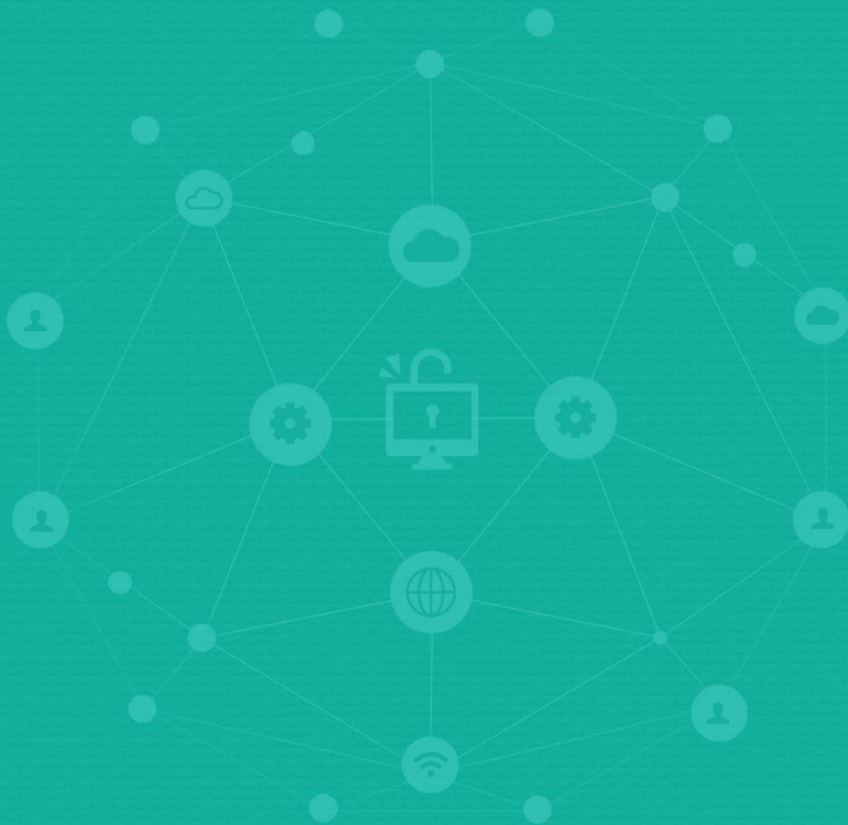
부록

부록

제1절 용어정의	142
제2절 공개소프트웨어 점검도구 목록	143
제3절 시험 예제코드	144
제4절 Spotbugs(Findbugs)/PMD 구조	146
제5절 PMD 상세분석	154



전자정부 SW 개발자·진단원을 위한
공개SW를 활용한 소프트웨어 개발보안 점검가이드



1 PART

제1장 개요

제1절 배경

제2절 가이드 목적 및 구성



제1장 개요



제1절 배경

국가정보화사업으로 개발되는 정보시스템 소프트웨어(이하, SW)는 "행정기관 및 공공 기관 정보시스템 구축·운영 지침"에 따라, SW 개발단계부터 보안취약점의 원인인 보안 약점을 배제하여 개발토록 하고 있다. 지침에 따르면, 사업자(개발자)는 SW 개발보안가 이드를 참고하여 안전한 SW를 개발하고 SW 보안약점을 식별하여 제거해야 하며, 행정기 관등¹⁾의 장은 감리법인을 통해 사업자(개발자)가 SW 보안약점을 제거하였는지 진단해야 한다. 또한 감리법인이 SW 보안약점 진단 을 위해 시큐어코딩 진단도구(이하, 진단도구)를 사용하는 경우, 과학기술정보통신부장관이 고시한 "정보보호시스템 평가·인증 지침"에 따라 정보보호제품 평가·인증된(이하, CC인증) 진단도구를 사용하여야 한다.

CC인증된 진단도구 목록은 IT보안인증사무국 홈페이지(http://www.itscc.or.kr/certifyProd_list.asp)에서 확인할 수 있다. 사업자(개발자)의 경우, CC인증된 진단도구 사용이 의무가 아니기 때문에 사업자(개발자)는 행정기관등의 장과 협의를 통해 진단도구를 선정하여 사용할 수 있다.

본 가이드는 CC인증된 진단도구 의무사용 대상 이외의 사용자가 SW 개발시 참고 및 활용할 수 있는 공개소프트웨어 진단도구 활용 방법에 대한 설명을 담고 있다. 본 가이드 에서 제시한 공개소프트웨어 진단도구는 CC인증된 상용 진단도구는 아니지만, 개발자 스스로 진단도구에 손쉽게 접근할 수 있는 방안을 제시하고 있다.

본 가이드에서 공개 소프트웨어 진단도구 활용 설명을 위해 선정한 진단도구 및 개발언어는 전자정부 정보시스템 SW 개발시 주로 사용하는 자바 언어 기반의 전자정부 표준프레임워크 (<http://www.egovframe.go.kr>)의 개발프레임워크 개발환경 Inspection 도구 (Spotbugs, PMD) 를 참고하였다.

1) 행정기관등 : 전자정부법 제2조에서 정의하는 행정기관, 공공기관

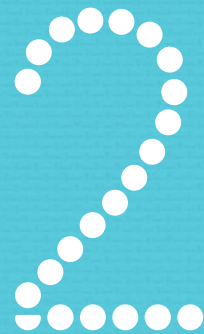
해당 가이드를 통해 무료로 사용할 수 있는 공개 소프트웨어 진단도구의 활용방법을 제시하고, SW 개발보안 교육 및 CC인증된/(또는 상용) 진단도구 사용이 어려운 정보화 사업 추진 시 본 가이드를 활용함으로써 소프트웨어 개발 안전성 향상에 기여 할 수 있을 것으로 기대한다.

제2절 가이드 목적 및 구성

목적	• ‘행정기관 및 공공기관 정보시스템 구축·운영 지침(행안부고시, 제2018-21호)’에 따라 정보화사업 수행시 상용 진단도구를 사용하기 어려운 환경에서 시큐어코딩 점검에 활용
대상	• 전자정부 SW 개발자 • 전자정부 SW 보안약점 진단원
범위	• 신규로 개발되거나 유지보수로 변경되는 소스코드에 대한 시큐어코딩 적용의 적절성 여부를 점검시 참조 ※ 행정기관등이 추진하는 정보화사업 단계 중 구현단계에 초점
구성	• (2장) 공개소프트웨어 진단도구 소개 및 설치환경 • (3장) Spotbugs/FindSecuritybugs 사용 방법 • (4장) PMD 사용 방법 • (5장) Jenkins 사용방법 • (6장) 사용자 정의 룰(Rule) 작성 및 오탐(False Positive) 관리 • (7장) 행안부 47개 보안약점 관련 공개SW 진단도구 룰(Rule) 분석 • (부록) 용어정리 및 참고자료



PART



제2장 공개소프트웨어 진단도구 소개 및 설치 환경

- 제1절 개요
- 제2절 Spotbugs
- 제3절 FindSecurityBugs
- 제4절 PMD
- 제5절 Jenkins
- 제6절 설치환경

PART



제2장 공개소프트웨어 진단도구 소개 및 설치 환경

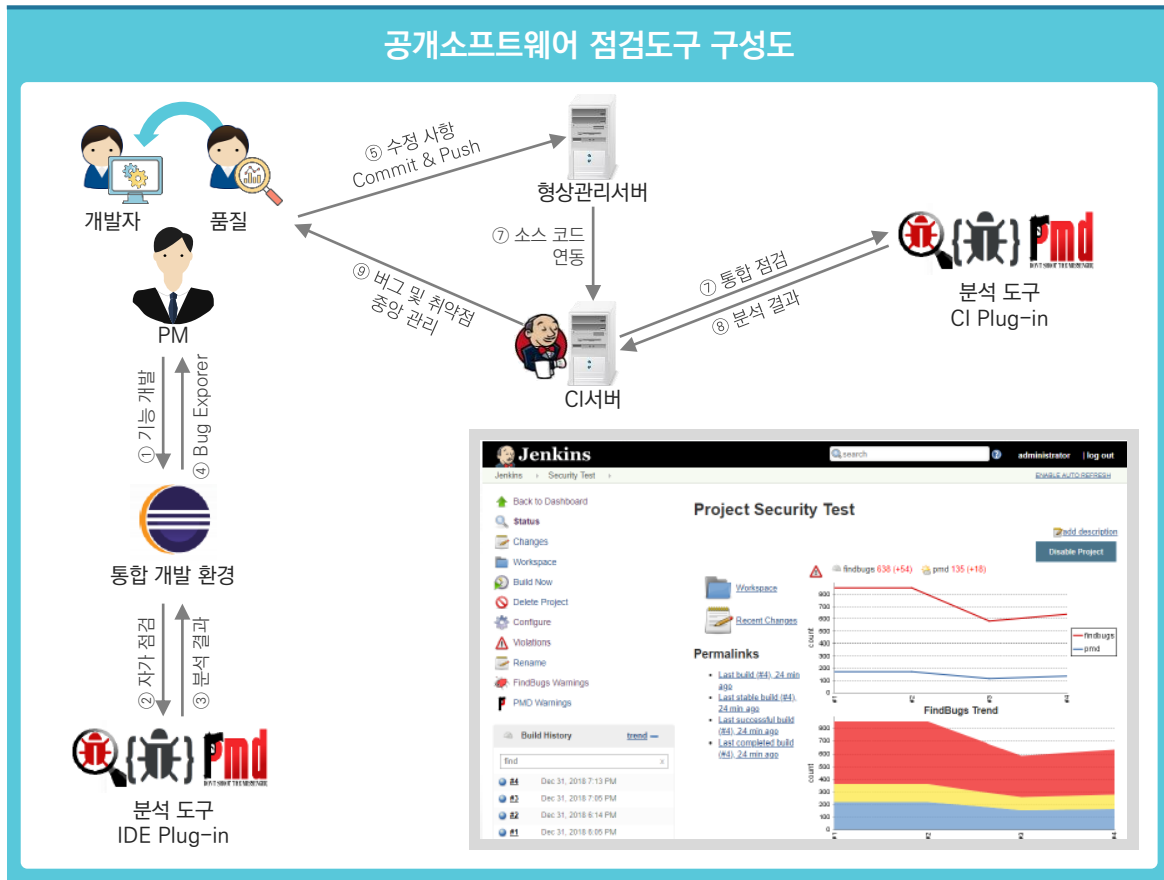


제1절 개요

공개소프트웨어(OSS, Open Source Software)란 소스코드가 공개되어 있는 소프트웨어로서 누구나 자유롭게 사용할 수 있고, 배포할 수 있는 소프트웨어를 말한다. 본 가이드에서 언급하는 공개 소프트웨어의 범위는 자유배포 권리를 가진 소프트웨어와 소스코드가 공개되어 있는 소프트웨어를 포함한다.

본 가이드에서 소개하는 공개 소프트웨어 도구는 총 4가지이다. Spotbugs, FindSecurityBugs, PMD는 보안약점을 진단하는 목적으로 사용한다. Jenkins는 지속적인 통합(Continuous Integration, CI) 도구로서 보안약점을 진단한 결과를 리포팅하는 용도로 사용한다.

다음은 Spotbugs, FindSecurityBugs, PMD와 Jenkins를 연계한 개념도이고 개발자, 품질관리자, PM 등 정보시스템 구축에 관련된 이해관계자 들은 아래의 단계로 SW 개발보안을 SW개발에 적용할 수 있다. 이 과정은 매 Build 마다 지속적인 통합과정에서 이루어져 소스코드의 보안성을 SW 보안약점 정적 분석을 통해 향상시킬 수 있다.



①~④ 개발자는 Eclipse에 플러그인 된 Spotbugs, FindSecurityBugs, PMD를 활용하여 SW보안 약점을 자가 점검한다. 개발이 완료된 코드는 원격의 형상관리 서버에 반영한다.

⑤~⑧ Jenkins 서버는 주기적으로 형상 관리 서버로부터 개발코드를 연동하여 플러그인 된 Spotbugs, FindSecurityBugs, PMD를 통해 SW 보안약점 관련 정적분석을 실시하고, 분석된 결과를 측정하고 리포팅 한다.

⑨ 측정된 SW 보안약점 결과는 Jenkins를 통해 PM/품질담당자/개발자에게 전달된다.

제2절 Spotbugs

1. Spotbugs의 개요

Spotbugs는 자바 바이트 코드(byte code)를 분석하여 버그 패턴을 발견하는 정적분석 공개소프트웨어이다(GNU LGPL 라이선스를 적용). 기존에 존재하던 정적분석 공개소프트웨어인 FindBugs를 계승한 프로젝트다. 미국의 Maryland 대학에서 2006 년에 개발하였으며 Java 프로그램에서 발생 가능한 100여개의 잠재적인 에러에 대해 scariest, scary, troubling, concern의 4등급으로 구분하여 탐지하고, 그 결과를 XML 로 저장할 수 있도록 지원한다.

Spotbugs는 Linux, Windows, MacOSX 운영체제를 지원하며, GUI기반의 단독 실행(Stand alone) 응용프로그램 방식과 Eclipse, NetBeans, IntelliJ IDEA, Gradle, Hudson and Jenkins 에 대한 플러그인(Plug-in) 방식을 지원하고 있다.

2018년 12월을 기준으로 최신 버전 3.1.9까지 출시되었으며 Spotbugs 실행을 위해서는 512MB 이상의 메모리와 Java 1.8 버전 이상이 요구된다.

공식 웹사이트는 <https://spotbugs.github.io/>로 Spotbugs에 대한 소개와 매뉴얼, 소프트웨어 다운로드 서비스를 제공하고 있다.

Spotbugs 로고



2. Spotbugs에서 탐지 가능한 오류 유형

Spotbugs에서는 Bad practice, Correctness, Dodgy code, Experimental,

Internationalization, Malicious code vulnerability, Multithreaded correctness, Performance, Security 등 9개 이상의 카테고리과 400개가 넘는 규칙이 등록되어 있어 잠재적인 버그를 알려 주며, 룰셋(ruleset)의 커스터마이징이 가능해 기업 마다 원하는 형 태로 만들 수 있다.

최신 버전인 Spotbugs 3.1.9에서 제공하는 탐지유형은 다음과 같다.

번호	탐지 유형	사례 및 설명
1	Bad practice	클래스 명명규칙, null 처리 실수 등 개발자의 나쁜 습관을 탐지
2	Correctness	잘못된 상수, 무의미한 메소드 호출 등 문제의 소지가 있는 코드를 탐지
3	Dodgy code	int의 곱셈결과를 long으로 변환하는 등 부정확하거나 오류를 발생시킬 수 있는 코드를 탐지
4	Experimental	메소드에서 생성된 stream이나 리소스가 해제하지 못한 코드를 탐지
5	Internationalization	Default 인코딩을 지정하지 않은 경우 등 지역특성을 고려하지 않은 코드 탐지
6	Malicious code vulnerability	보안 코드에 취약한 가변적인 배열이나 컬렉션, Hashtable 탐지
7	Multithreaded correctness	멀티쓰레드에 안전하지 않은 객체 사용 등을 탐지
8	Performance	미사용 필드, 비효율적 객체생성 등 성능에 영향을 주는 코드를 탐지
9	Security	CSS, DB 패스워드 누락 등 보안에 취약한 코드를 탐지

제3절 FindSecurityBugs

1. FindSecurityBugs의 개요

FindSecurityBugs는 자바 웹 어플리케이션에 대한 보안 감사를 지원하는 Spotbugs의 플러그인으로, Philippe Arteau에 의해 만들어졌다.

2018년 6월에 최신 버전인 1.8.0 버전이 배포되었고, Spring-MVC, Struts, Tapestry 등 다양한 SW 개발 프레임워크와 Eclipse, IntelliJ, Android Studio, NetBeans 등 통합개발환경(IDE) 도구, Jenkins와 SonarQube와 같은 CI(Continuous integration) 도구에서 활용가능하다.

또한, FindSecurityBugs는 200개 이상의 시그니처를 활용하여 OWASP TOP 10과 CWE를 커버하는 78개의 버그패턴(Bug Pattern)을 탐지할 수 있다.

공식 홈페이지인 <http://find-sec-bugs.github.io>를 통해 소프트웨어에 대한 소개와 튜토리얼을 제공하고 있으며, LGPL 공개소프트웨어 라이선스에 따라 소프트웨어 다운로드 기능을 제공한다.

Find Security Bugs 로고



2. FindSecurityBugs에서 탐지할 수 있는 주요 점검내용

FindSecurityBugs에서는 <http://find-sec-bugs.github.io/bugs.htm>을 통해 탐지 가능한 버그 패턴 78개에 대한 오용된 코드, 솔루션, 레퍼런스에 대한 목록을 제공하고 있다.

번호	탐지 유형	사례 및 설명
1	부적절한 입력 탐지	Untrusted Servlet parameter, session Cookie, Query String 등
2	SQL Injection 탐지	Hibernate, JDO, JPA, Spring JDBC, LDAP Injection 등
3	XSS	JSP, Servlet, JavaScript(Android) XSS를 탐지
4	취약한 암호화 알고리즘	SHA-1, MD2/MD4/MD5 또는 NullCipher 등 취약한 암호화 함수를 탐지
5	취약한 URL Redirection	유효하지 않은 URL로 Redirection 취약점 탐지
6	기타	보안 플래그가 없는 쿠키, Regex DOS 취약점, 신뢰할 수 없는 Context-type 또는 헤더

제4절 PMD

1. PMD의 개요

큰 규모의 프로젝트에서는 개발자참서, 동료 검토(Peer Review)만으로는 시스템의 소스코드에 산재되어 있는 잠재적인 문제점을 찾기가 어렵다. 따라서 소스코드 전체를 일괄로 스캔하여 문제가 되는 패턴을 자동으로 찾아서 수정함으로써 잠재적인 문제를 최소화 할 수 있는 방안이 필요하다.

PMD는 JAVA 프로그램의 소스코드(source code)를 분석하여 프로그램의 부적절한 부분을 찾아내고 성능을 높이도록 도와주는 공개소프트웨어 점검 도구로, 사용하지 않는 변수, 아무런 처리도 하지 않은 catch block, 불필요한 Object 생성 등을 찾아내며, 시스템 개발공정의 구현 및 테스트단계에서 정적분석에 활용할 수 있다.

PMD 로고



2. PMD의 주요 점검 내용

PMD에서는 https://pmd.github.io/pmd-6.9.0/pmd_rules_java.html을 통해 탐지 가능한 버그 패턴에 대한 오용된 코드, 솔루션, 레퍼런스에 대한 목록을 제공하고 있다.

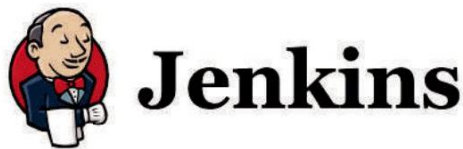
번호	점검 항목	점검 내용	예시
1	Basic (rulesets/basic.xml)	대부분의 개발자들이 동의하는 규칙	catch 블록들은 비어있어서는 안 된다, equals()를 overriding 할 때 마다 hashCode()를 override한다.
2	Naming (rulesets/naming.xml)	표준 자바 naming 규약을 위한 테스트	변수 이름들은 너무 짧아서 안 된다, 메소드 이름은 너무 길어서 안 된다, 클래스 이름은 대문자로 시작해야 한다, 메소드와 필드 이름들은 소문자로 시작해야 한다
3	Unused code (rulesets/unusedcode.xml)	불필요한 코드 제거	결코 읽히지 않은 private field와 local variable, 접근할 수 없는 문장, 결코 호출되지 않는 private method 등
4	Design (rulesets/design.xml)	디자인 원리 체크	switch 문장은 default 블록을 갖고 있어야 한다, 심하게 중첩된 if 블록은 피해야 한다
5	Import statements (rulesets/imports.xml)	import 문장에 대한 문제들 점검	같은 클래스를 두 번 반입하는 것, java.lang에서 클래스를 import하는 것
6	JUnit tests (rulesets/junit.xml)	Test Case와 Test Method 관련 특정 문제 검색	Method 이름의 정확한 스펠링 등
8	Strings (rulesets/string.xml)	스트링 관련 작업을 할 때 발생하는 일반적인 문제들 규명	
9	Braces (rulesets/braces.xml)	for, if, while, else 문장이 괄호를 사용하는지 여부 검사	
9	Code size (rulesets/codesize.xml)	과도하게 긴 메소드, 너무 많은 메소드를 가진 클래스 검사	
10	Javabeans (rulesets/javabeans.xml)	직렬화 될 수 없는 bean 클래스같이 JavaBeans 코딩 규약을 위반하는 JavaBeans 컴포넌트 검사	
11	Finalizers	finalize() 메소드 관련한 다양한 문제들을 검사	비어있는 finalizer, 다른 메소드를 호출하는 finalize() 메소드, finalize()로의 호출 등
12	Clone (rulesets/clone.xml)	clone() 메소드에 대한 규칙	clone()을 오버라이드하는 클래스는 Cloneable을 구현해야 한다
13	Coupling (rulesets/coupling.xml)	클래스들간 과도한 커플링 표시 검색	지나치게 많은 import, 너무 적은 필드, 변수, 클래스 내의 리턴 유형 등
14	Strict exceptions (rulesets/ strictexception.xml)	예외 테스트	메소드는 java.lang.Exception을 발생 시키도록 선언되어서는 안 됨, 예외는 플로우 제어에 사용 되어서는 안 됨
15	Controversial (rulesets/controversial.xml)	좀 더 의심스러운 검사	변수에 null 할당하기
16	Logging (rulesets/logging-java.xml)	java.util.logging.Logger를 위험하게 사용하는 경우 검색	끝나지 않고 정적이지 않은 logger와 한 클래스 에 한 개 이상의 logger 등

제5절 Jenkins

1. Jenkins의 개요

Jenkins는 www.jenkins-ci.org에서 다운로드 받을 수 있는 공개소프트웨어 CI(Continuous Integration) 서버이다. 자바로 개발되어 있으며, 2015년 10월 현재 빌드와 테스트를 지원하는 1,097개의 플러그인이 제공 되는 확장성이 뛰어난 CI 도구 이다.

Jenkins의 로고



Jenkins를 처음 만든 개발자는 Kohsuke Kawaguchi로, Jenkins의 전신인 Hudson 을 만든 2004년 여름부터 지금까지 계속 영향을 주고 있다.

지금까지 수많은 공개소프트웨어와 상용 CI툴이 존재해왔는데, 그 중에서 사용자들이 Jenkins를 선호하게 된 이유는 무엇일까? 크게 세 가지로 살펴본다면 다음과 같다.

2. Jenkins의 장점

1) 설치의 용이성

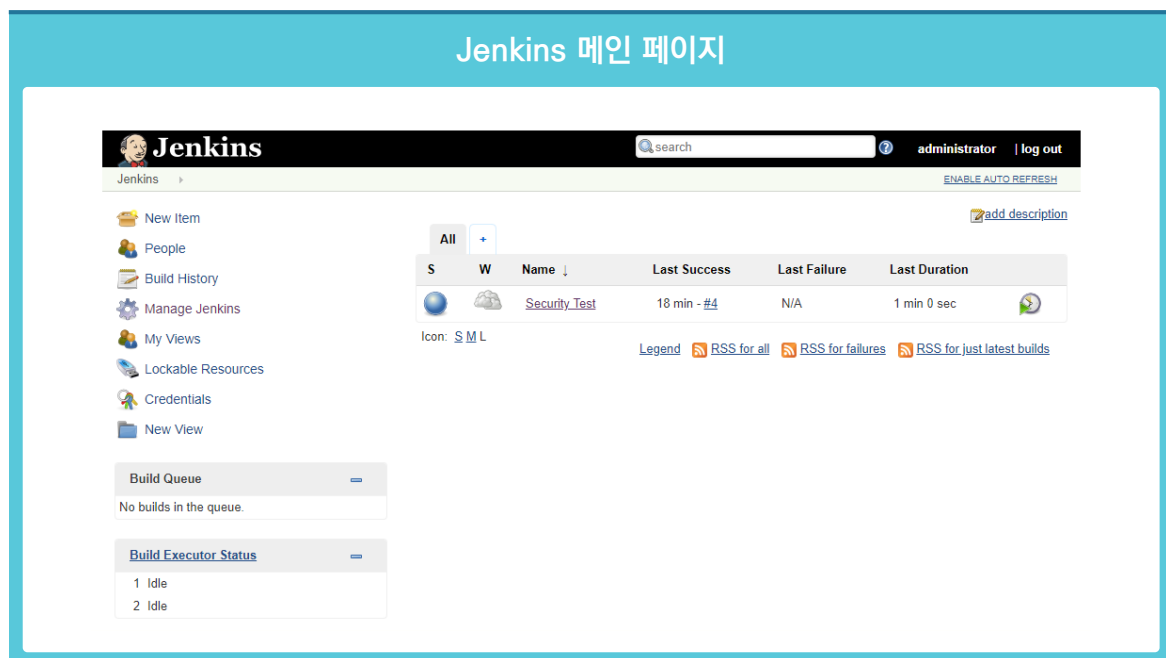
Jenkins는 압축 파일 하나에 웹 서버까지 내장하고 있고, 별도의 DB 설정이 필요 없이 바로 설치, 실행이 가능하다. 환경 구성하는 부분도 실행 이후 하나씩 확장할 수 있으므로 다른 툴에 비해서 첫 화면을 만나는 시간이 매우 짧다. 이는 인터넷 접속이 어려운 환경, 개발 환경이 아직 확정되지 않은 상황에서 특히 빛을 발하게 되며, 짧은 시간 안에 설치하고 환경변화에 따라서도 영향을 적게 받는다.

2) 확장성

Jenkins의 두 번째 장점은 Jenkins가 CI의 플랫폼이 되어준다는 점이다. 사용 환경이나 개발 언어, 빌드 툴에 따라서 플러그인을 맞춰주면 그 상황에 최적화 된 개발 환경이 제공된다. 즉, C++에 관련된 플러그인을 세팅하면 그 프로젝트가 C++에 최적화 된 구성이 되고, Java에 관련된 플러그인, 테스트와 관련된 플러그인을 설치하면 테스트를 강화 하는 java 프로젝트가 구성된다. 원하지 않는 플러그인은 바로 제거가 가능하니, 다양한 시도와 피드백이 가능하다.

3) 가시성

Jenkins는 세팅과 플러그인에 따라서 메인 화면이 유기적으로 변화한다. 또한 기능이 확장 될 때에도 전반적인 표준에 따라 일관성을 유지하면서 확장된다. 그래서 익숙하지 않은 사용자도 원하는 동작과 그에 따른 결과를 예측하기 쉽다. 가시성의 핵심은 빌드의 상태를 알려주는 신호등이다. 어떤 과정과 점검을 거치더라도 그 결과가 정상인지, 비정상인지, 안정적이지 않은지에 따라서 신호등 표시가 된다. 문제의 원인을 파악할 때에도 소스코드의 해당 라인까지 drill-down이 가능하고, 전체적인 내용을 보다가도 과거 빌드 이력을 로그 정보까지 접근해서 확인할 수 있다. 다른 측면에서 바라본다면 개발자와 개발자, 개발자와 관리자 사이에서도 서로 소스코드와 빌드 결과를 한 화면에 보면서 의사소통하기 쉬운 장점이 있다.



Jenkins 메인 페이지에는 성공/실패/불안정한지 그 상태가 신호등으로 표현되어 있고, 각 프로젝트의 최근 상태를 한 눈에 알아볼 수 있는 날씨 표시가 대시보드 형태로 구성되어 있다.

제6절 설치환경

설치에 사용하는 개발환경은 다음과 같다.

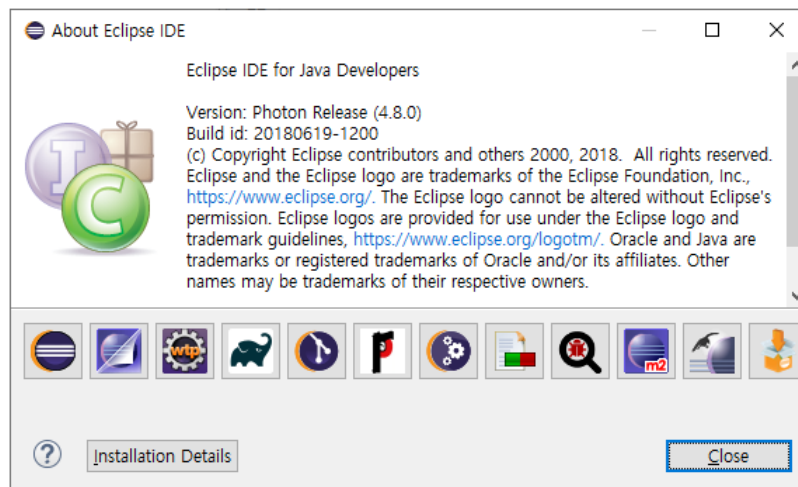
1. JDK(Java Development Kit) 권장버전

Java 1.8 버전

```
PS D:\> java -version
java version "1.8.0_181"
Java(TM) SE Runtime Environment (build 1.8.0_181-b13)
Java HotSpot(TM) 64-Bit Server VM (build 25.181-b13, mixed mode)
PS D:\>
```

2. 통합개발환경(IDE, Integrated development environment) 권장버전

Eclipse Photon(4.8.0)



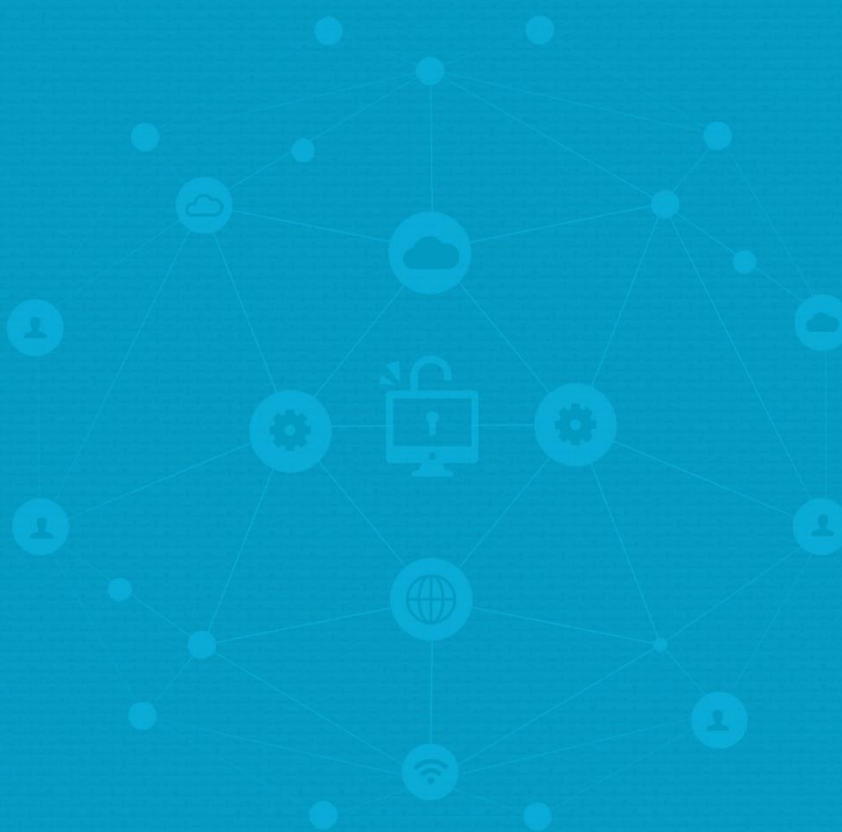


3. 사용되는 공개소프트웨어 SW 버전

공개소프트웨어 명	사용버전	용도	비고(권장 버전)
Eclipse	4.4.2	통합개발도구	JDK 1.5 이상 Eclipse 3.3 이상
Spotbugs (plugin)	3.1.9 (3.1.5)	시큐어코딩 분석	JDK 1.8 이상 Eclipse 3.3 이상 IntelliJ 2018.2.2 이상
FindSecurityBugs	1.8.0		JDK 1.8 이상 Eclipse 3.3 이상 IntelliJ 2018.2 이상
PMD (plugin)	6.7.0 (4.0.18)		JDK 1.8 이상 Eclipse 4.0 이상 IntelliJ 2018.2.2 이상
Jenkins	1.643	정적분석 리포팅, 지속적 통합	JDK 1.6 이상
maven	3.3.9	프로젝트관리도구	JDK 1.7 이상



전자정부 SW 개발자·진단원을 위한
공개SW를 활용한 소프트웨어 개발보안 점검가이드



PART



제3장 Spotbugs/ FindSecurityBugs 사용 방법

- 제1절 Spotbugs 및 FindSecurityBugs 설치
- 제2절 Spotbugs 및 FindSecurityBugs 룰 설정
- 제3절 Spotbugs 및 FindSecurityBugs 검사
- 제4절 Spotbugs에서 플러그인 빌드
- 제5절 FindSecurityBugs에서 플러그인 빌드
- 제6절 플러그인 IDE에 추가



제3장 Spotbugs/FindSecurityBugs 사용 방법

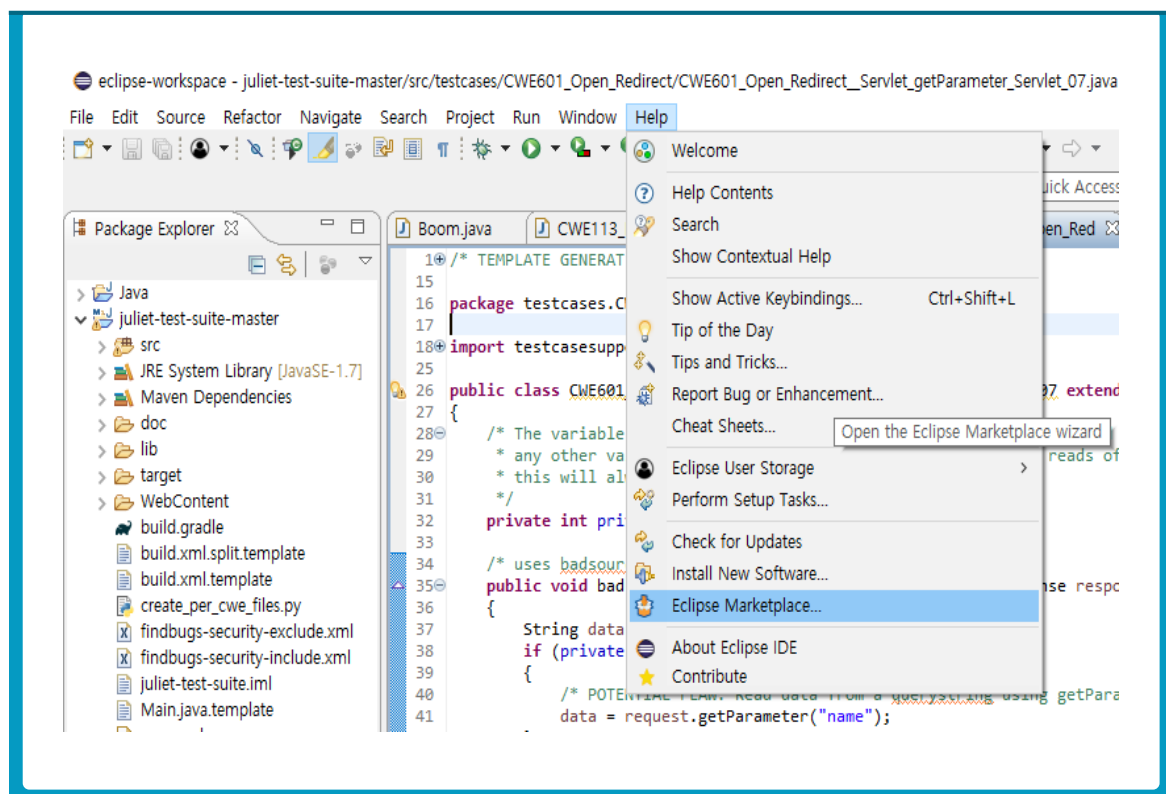


제1절 Spotbugs 및 FindSecurityBugs 설치

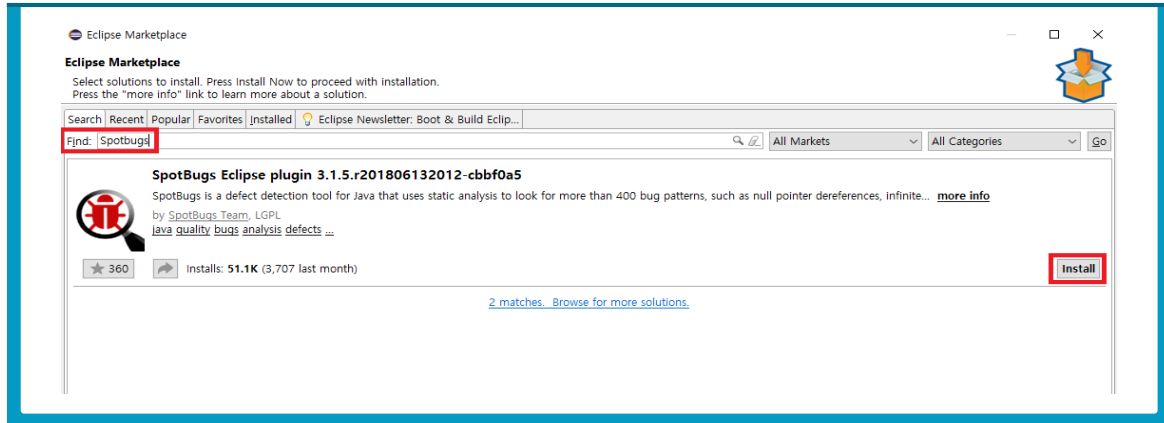
Spotbugs, FindSecurityBugs 구동을 위한 핵심 플러그인을 다음과 같이 설치한다. IDE 플러그인에서는 Spotbugs의 전신인 Findbugs라는 이름을 유지하는 경우도 있다.

1. Spotbugs 이클립스 플러그인 설치

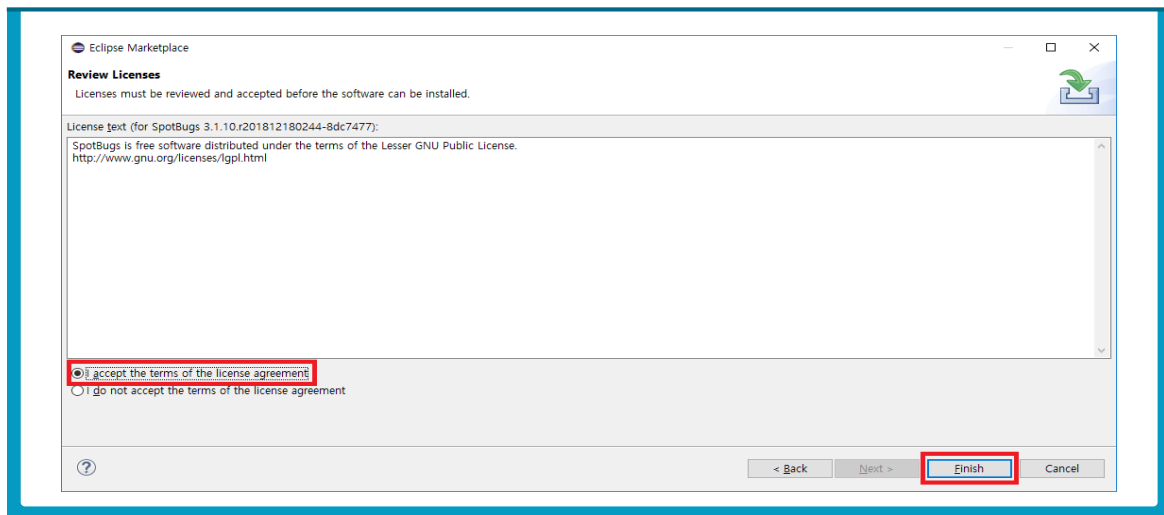
- ❶ 이클립스를 구동한다.
- ❷ Help > Eclipse Marketplace...를 선택한다.



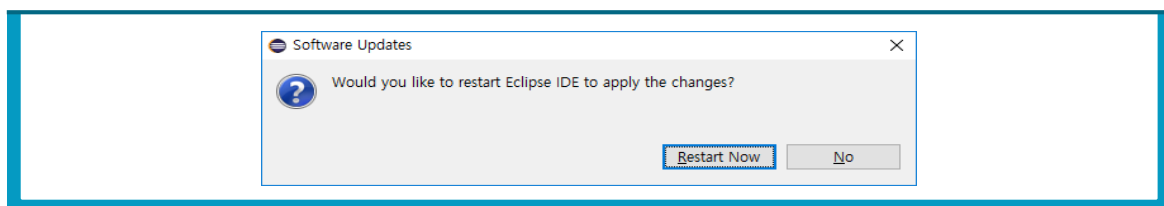
- ③ Eclipse Marketplace에서 Find창에 Spotbug를 검색하고 Install 버튼을 클릭한다.



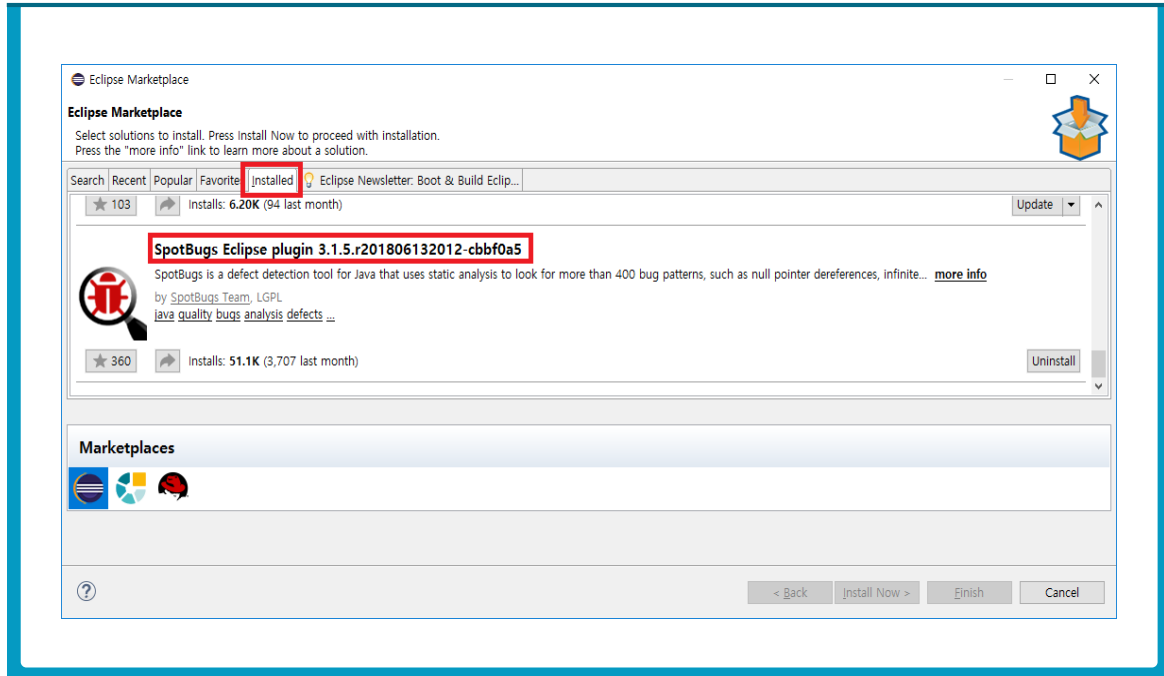
- ④ License를 승인하고 Finish를 선택한다.



- ⑤ 설치가 완료되고 Eclipse IDE를 재실행할지 확인하는 알림이 나오면 Restart Now를 클릭하여 재실행한다.

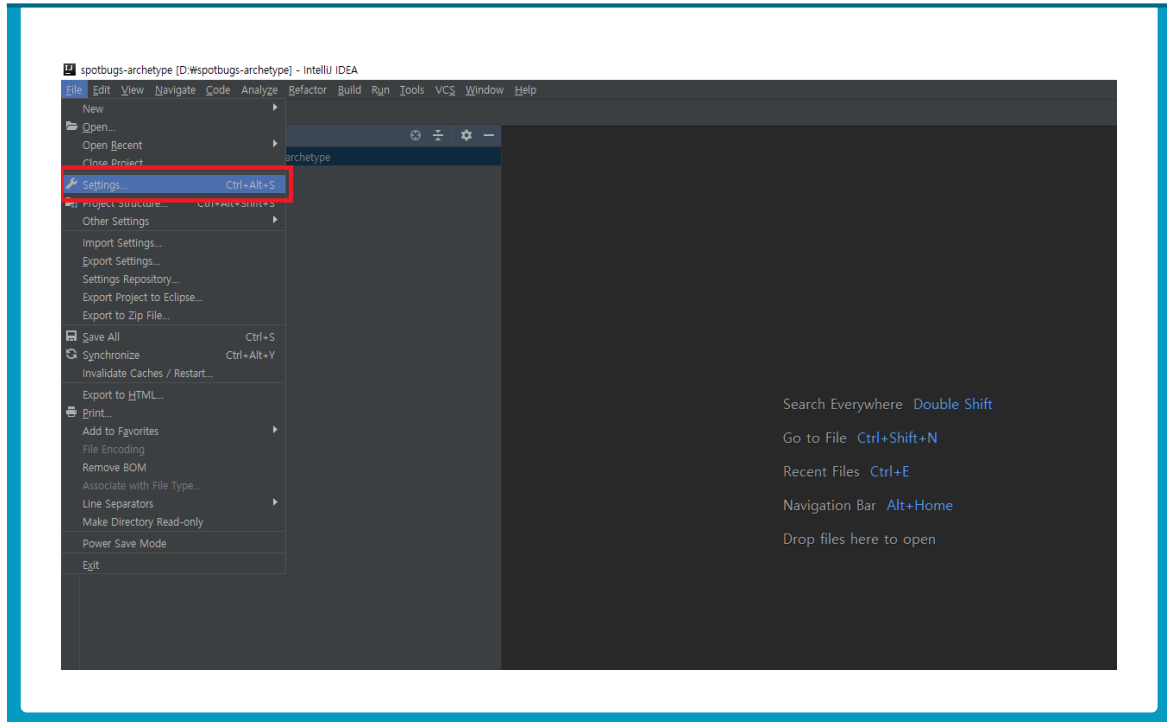


- ⑥ 재실행 후 Help > Eclipse Marketplace... > Installed 항목을 선택하여 Spotbugs 플러그인이 설치되었는지 확인한다.

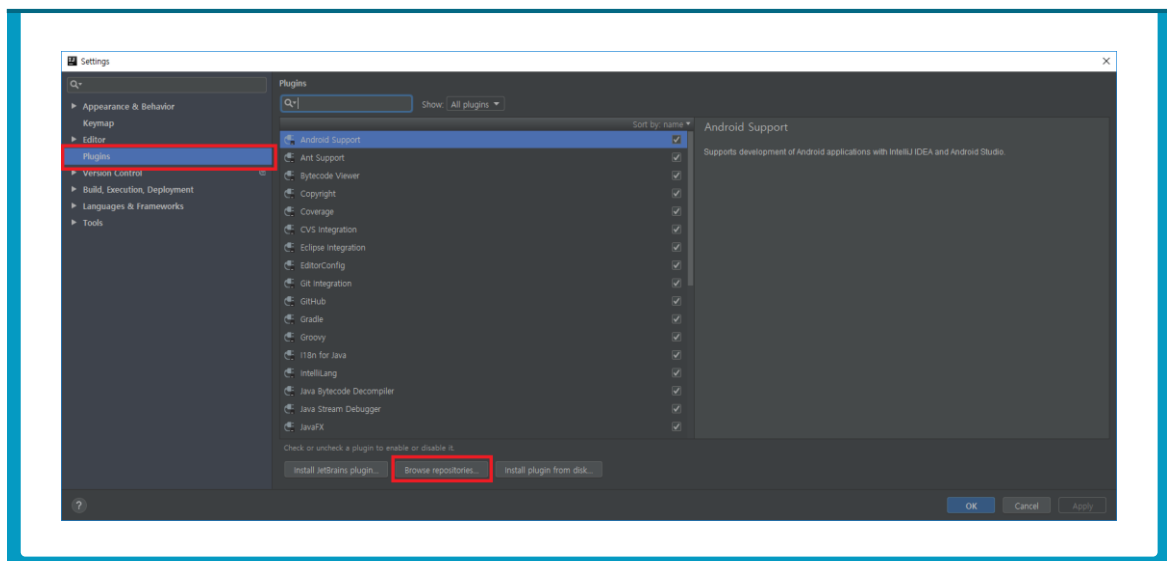


2. Spotbugs IntelliJ 플러그인 설치

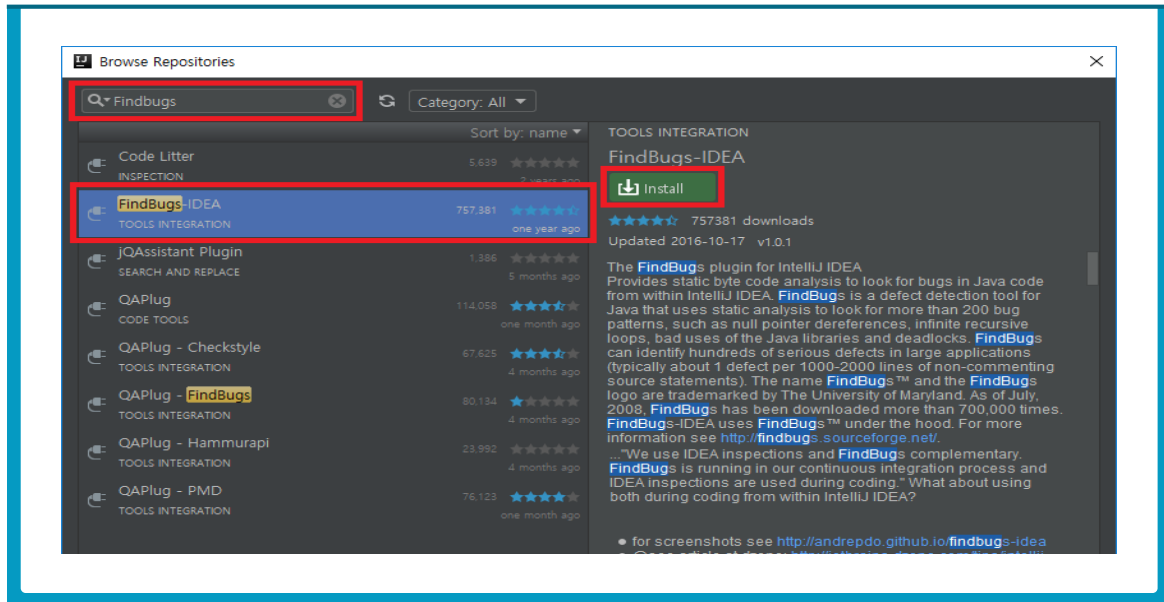
- ❶ IntelliJ를 구동한다.
- ❷ File > Settings를 선택한다.



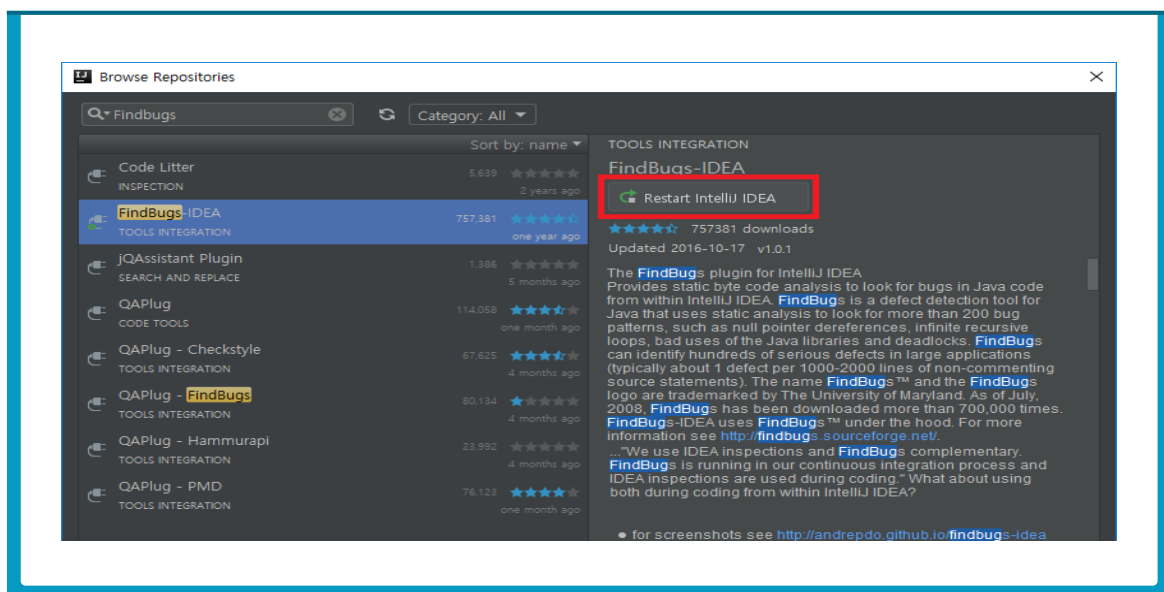
- ❸ Settings 창에서 Plugins을 선택하고 Browse repositories...를 클릭한다.



- ④ Browse Repositories 창에서 Findbugs를 검색하고 FindBugs-IDEA를 클릭한다. 창 우측에 해당 플러그인의 정보가 출력된다. Install 버튼을 눌러 설치를 진행한다.

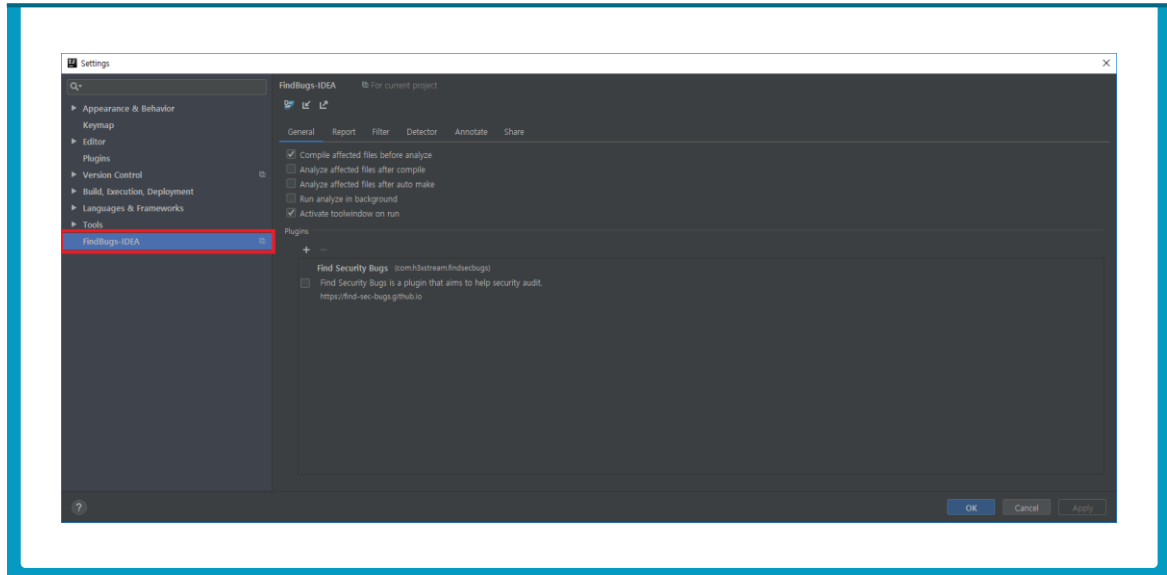


- ⑤ 설치가 완료되면, Restart IntelliJ IDEA 버튼을 클릭하여 IntelliJ를 다시 시작한다.



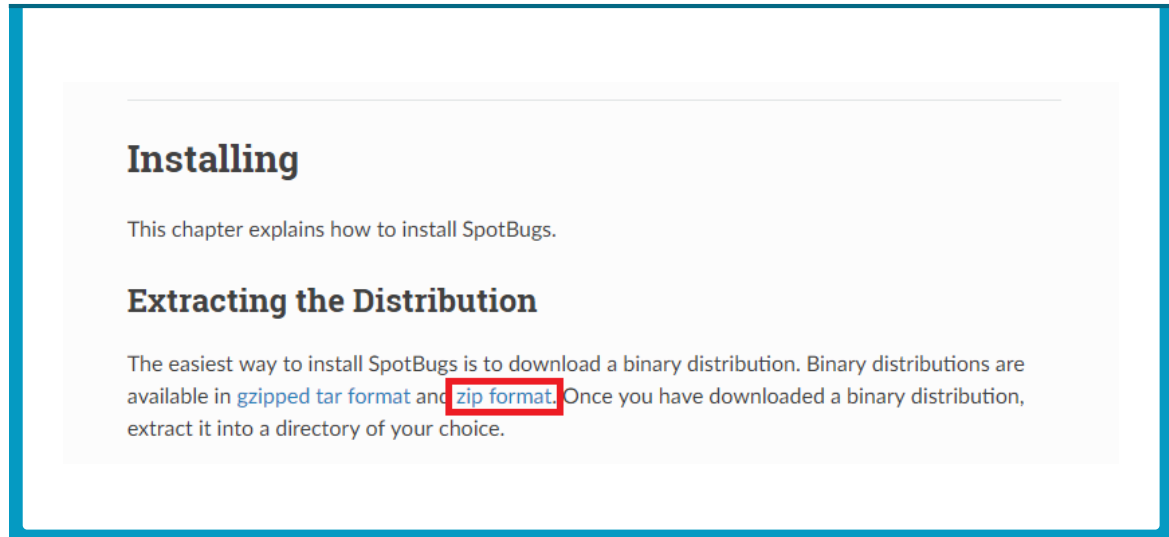
최신 버전은 v1.0.1이다.

- ⑥ File > Setting를 선택하고 Settings 창에서 FindBugs-IDEA 탭이 생성되었는지 확인한다.

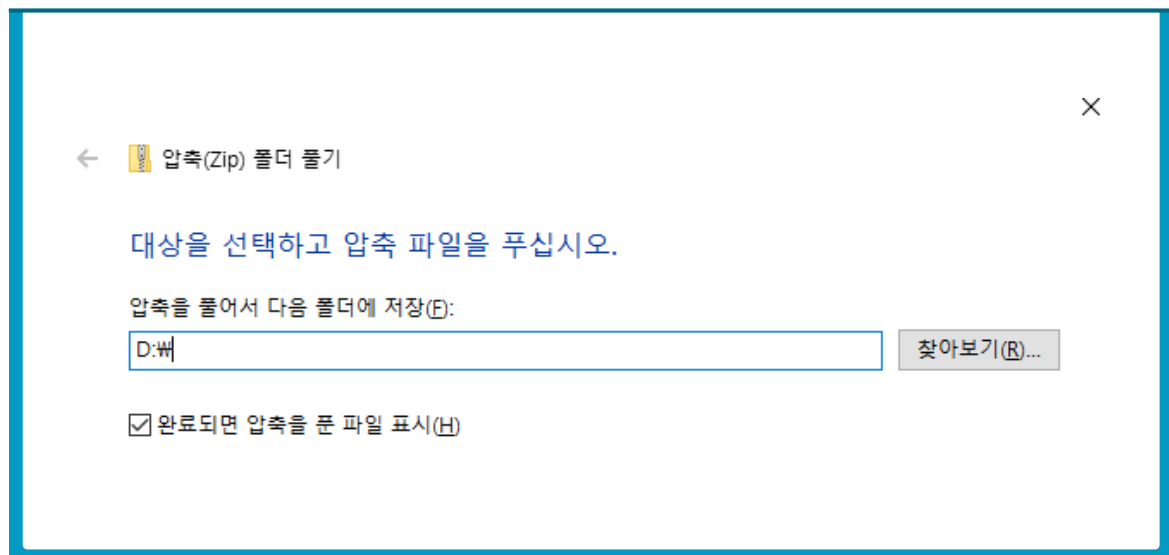


3. Spotbugs stand-alone 설치

- ① 공식 매뉴얼 사이트에서 Docs->Installing->Extracting the Distribution에서 “zip format”을 클릭하여 압축 파일을 다운로드 받는다.

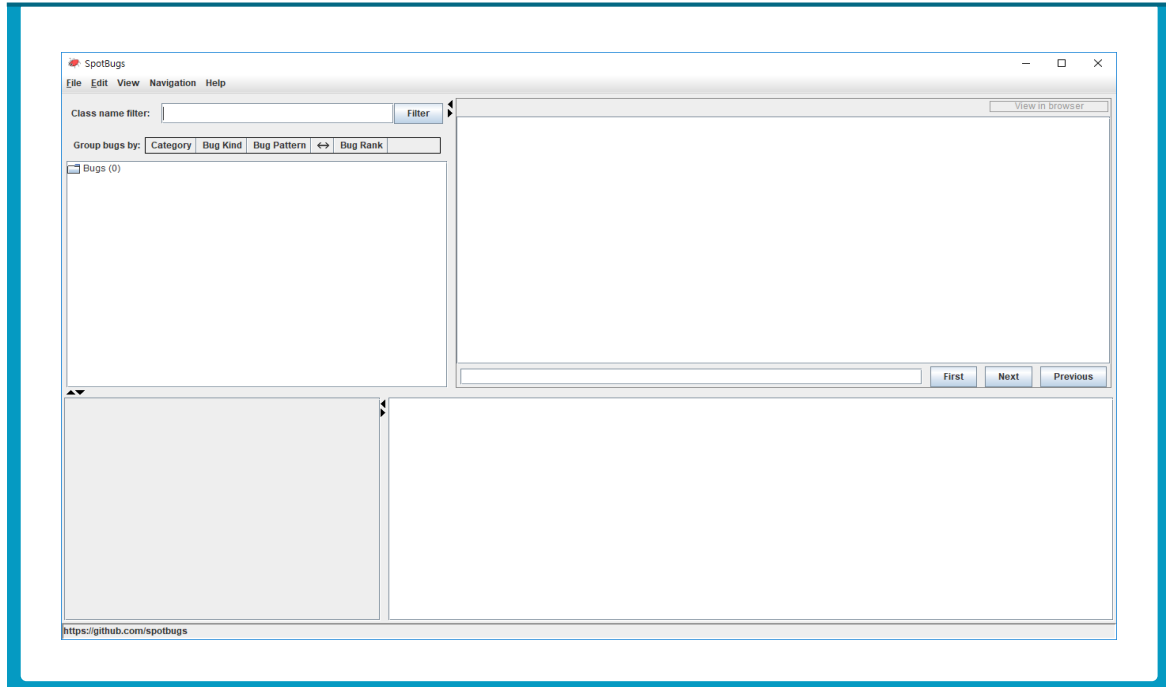


- ② 다운로드 받은 압축 파일을 해제한다. 압축 파일에 마우스 오른쪽을 클릭하여 “압축 풀기(T)”를 선택하여 진행한다.



- ③ 압축이 해제된 D:\Spotbugs-3.1.6 폴더를 '%SPOTBUGS_HOME%'으로 하자

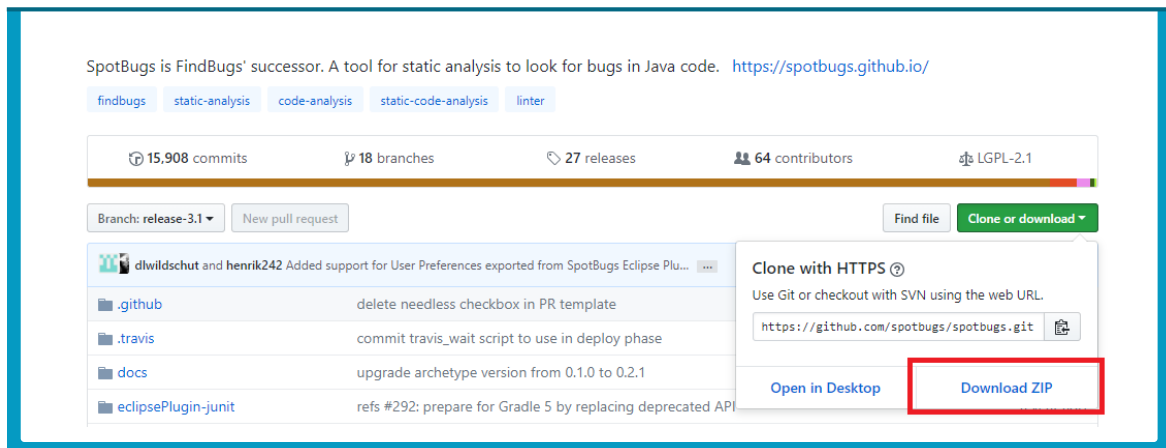
- ④ '%SPOTBUGS_HOME%\lib\spotbugs.jar'를 더블 클릭해서 Spotbugs GUI를 실행한다.



4. Spotbugs 소스 코드 빌드

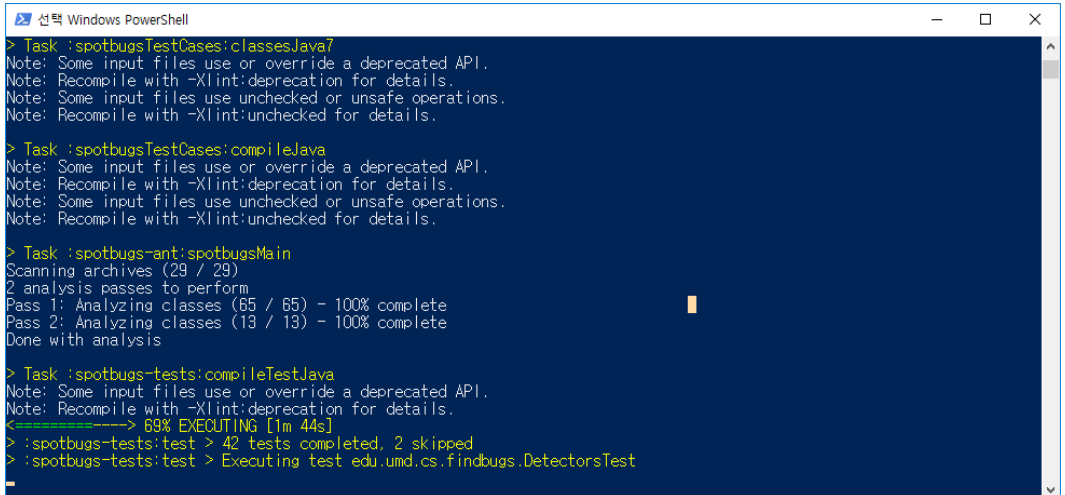
Spotbugs는 공개 소프트웨어이기 때문에 직접 github에서 소스 코드를 받아 빌드하여 사용할 수 있다. Spotbugs 프로젝트는 빌드를 위해 gradle을 사용한다.

- ① Github에서 spotbugs를 다운로드 받는다: <https://github.com/spotbugs/spotbugs>



- ❷ 다운로드 받은 압축 파일을 해제한다. 압축이 해제된 D:\spotbugs-release-3.1 폴더를 %SPOTBUGS_HOME%으로 한다.
- ❸ %SPOTBUGS_HOME%에서 왼쪽 Shift 키를 누른 채로 마우스 오른쪽을 클릭 하여 “여기에 PowerShell 창 열기”을 선택한다.
- ❹ Spotbugs 프로젝트는 빌드를 위해 gradle을 사용한다. 루트 프로젝트에 gradle wrapper(gradlew.bat)이 포함되어 있기 때문에 별도의 gradle 설치가 필요없고 gradle 버전에 관계없이 gradle 빌드가 가능하다. 다음 명령어를 통해 gradle을 빌드한다.

\$gradlew.bat build



```
> Task :spotbugsTestCases:classesJava7
Note: Some input files use or override a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
Note: Some input files use unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.

> Task :spotbugsTestCases:compileJava
Note: Some input files use or override a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
Note: Some input files use unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.

> Task :spotbugs-ant:spotbugsMain
Scanning archives (29 / 29)
2 analysis passes to perform
Pass 1: Analyzing classes (65 / 65) - 100% complete
Pass 2: Analyzing classes (13 / 13) - 100% complete
Done with analysis

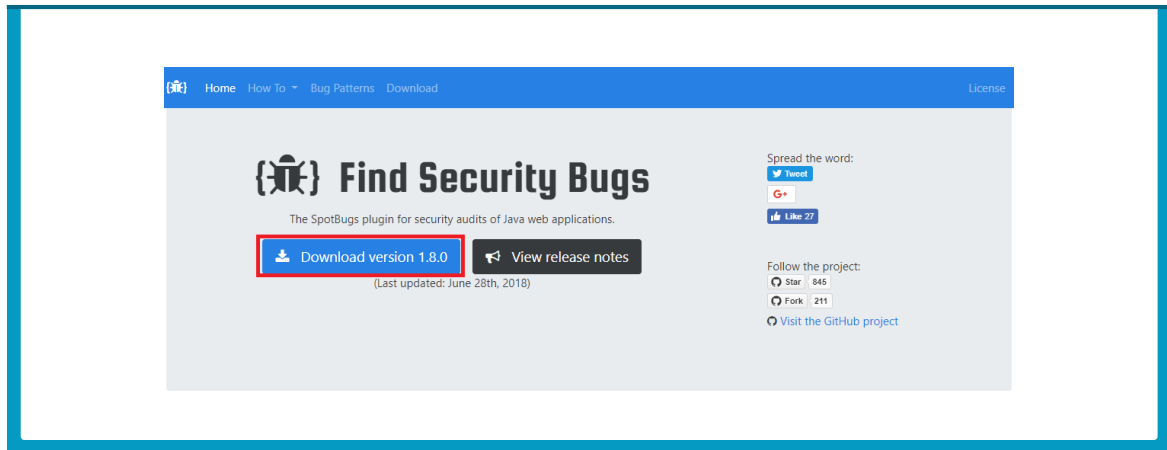
> Task :spotbugs-tests:compileTestJava
Note: Some input files use or override a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
<=====--> 69% EXECUTING [1m 44s]
> :spotbugs-tests:test > 42 tests completed, 2 skipped
> :spotbugs-tests:test > Executing test edu.umd.cs.findbugs.DetectorsTest
```

- ❺ build가 오류 없이 종료되면 %SPOTBUGS_HOME%\spotbugs build\distributions 경로로 이동한다. 제2장 1절에서 다운로드 받은 것과 같은 tgz 파일과 zip 파일이 생성되었다. 위에서 기술한 방식대로 실행하면 된다.

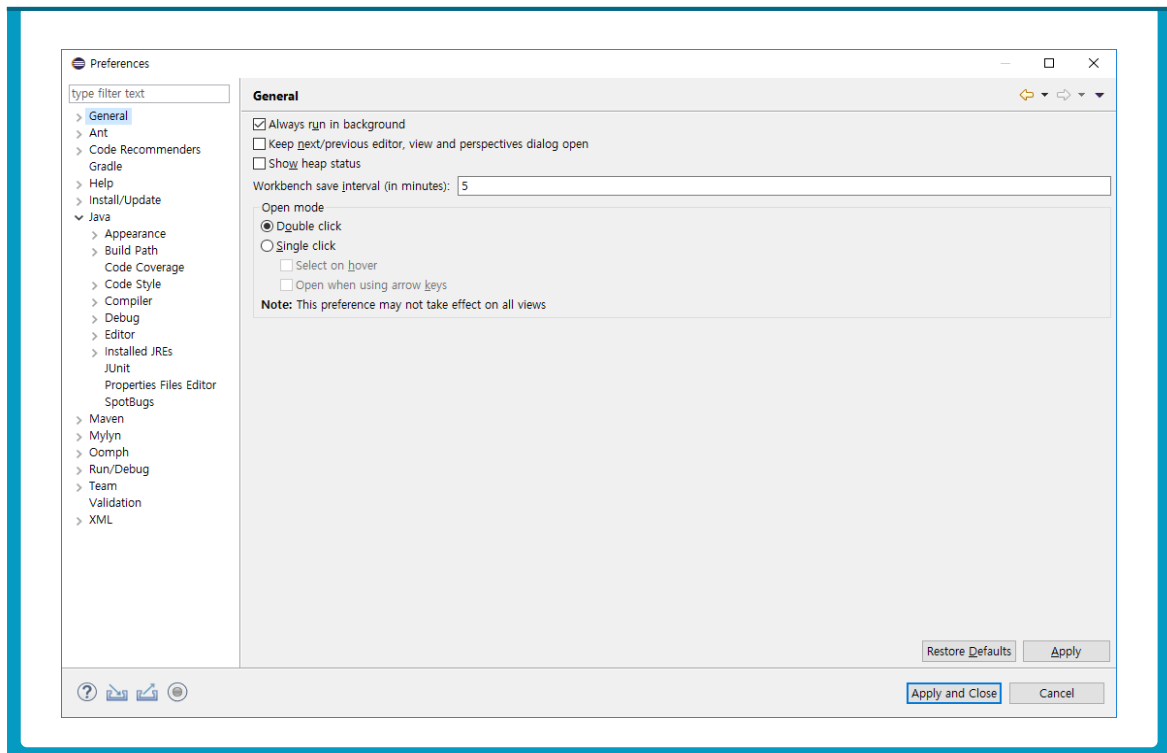
5. FindSecurityBugs 이클립스 설치

FindSecurityBugs는 Spotbugs의 플러그인 방식으로 동작한다.

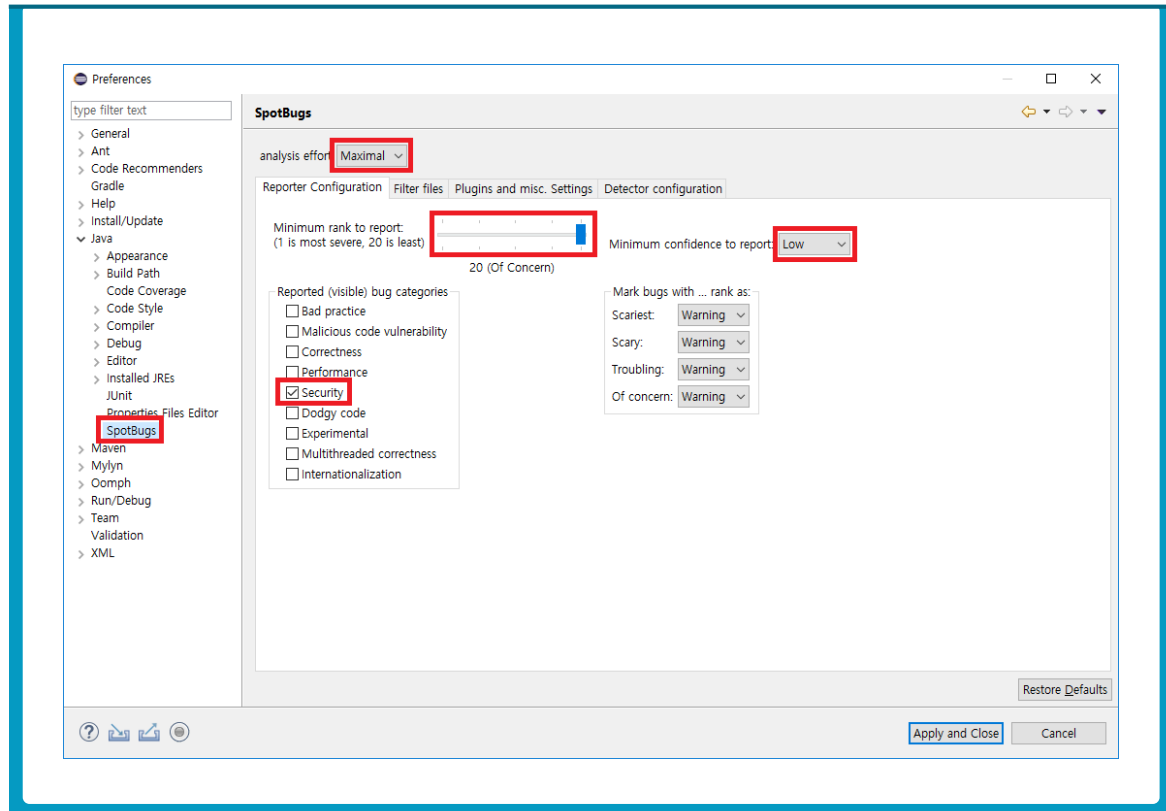
- 1 <https://find-sec-bugs.github.io/> 에서 플러그인 1.8.0을 다운로드 받는다.



- 2 Eclipse에서 Windows > Preferences를 클릭해서 Preferences화면을 띄운다.

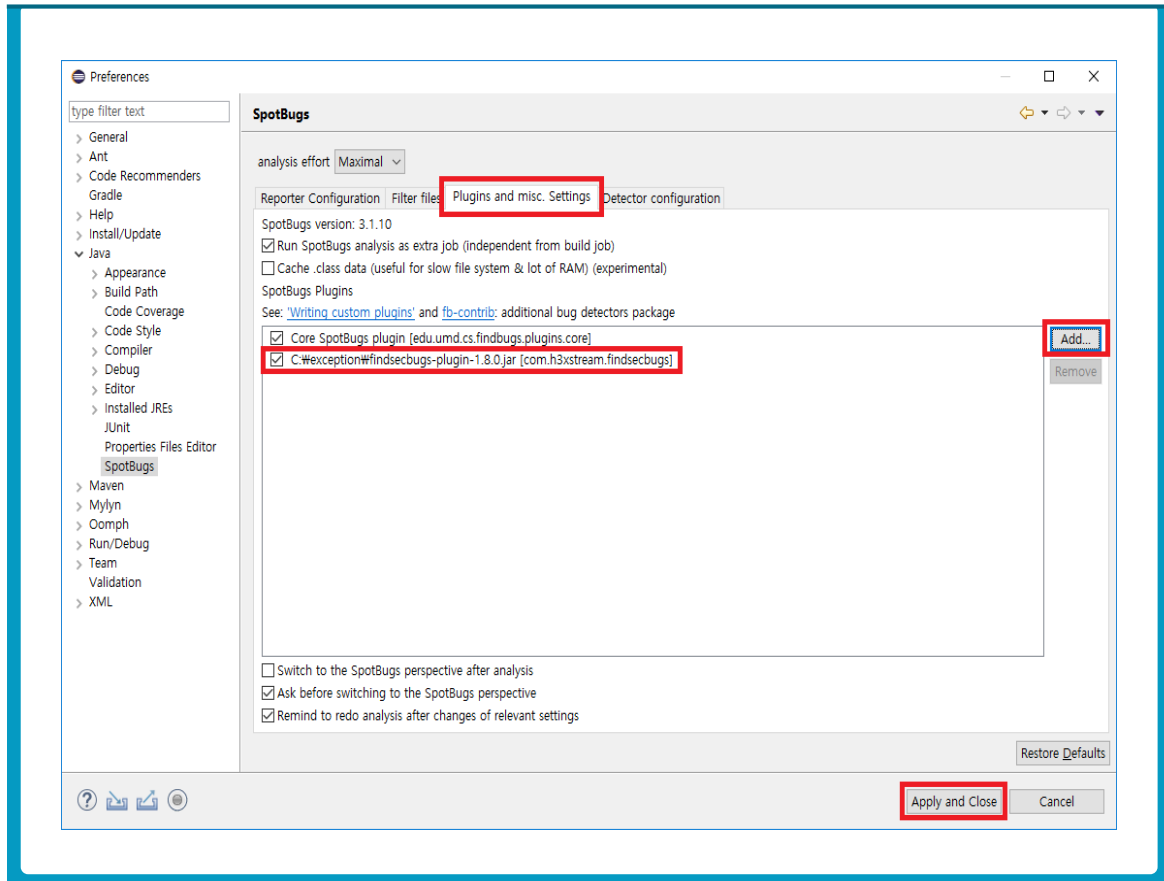


- ③ Preferences화면에서 Java > Spotbugs를 선택한 뒤 다음과 같이 세팅 정보를 변경한다.

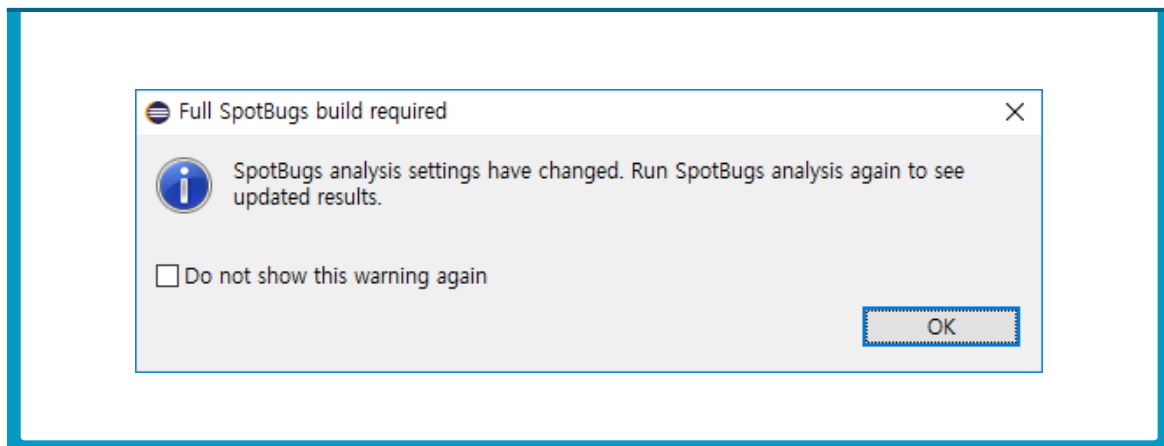


설정항목	변경한 정보
analysis effort	Maximal
Minimum rank to report	20(Of Concern)
Minimum confidence to report	Low
Reported (visible) bug categories	Security

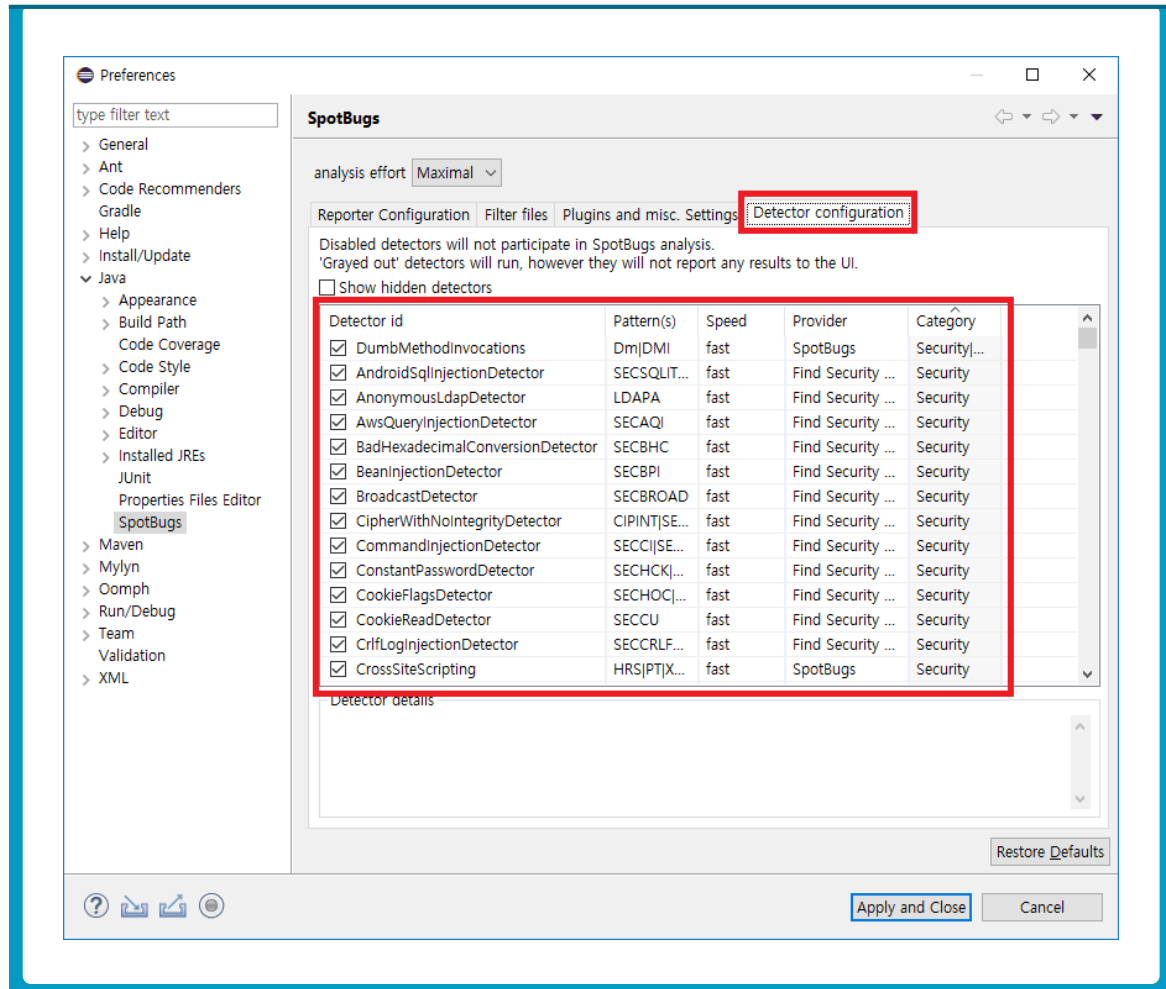
- ④ Plugins and misc. Settings 탭을 클릭하고, Add 버튼을 클릭하여 다운로드 받은 findsecbugs-plugin-1.8.0.jar 파일을 선택한다. 플러그인이 추가되었는지 확인하고 Apply and Close 버튼을 클릭하여 플러그인 설치를 완료한다.



- ⑤ Spotbugs 설정이 변경되었다는 알림을 확인하고 OK 버튼을 클릭한다.



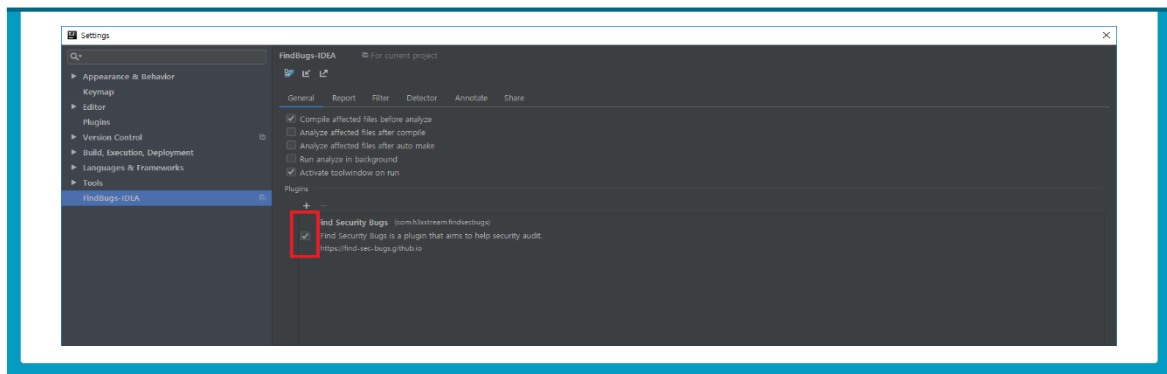
- ⑥ 다시 Spotbugs Preference 창을 띄워서 Detector configuration 탭을 선택한다. Rule 목록에서 Category 탭을 두 번 클릭하여 Category 내림차순으로 정렬하여 Security detector들이 상단에 위치하도록 한다. FindSecurityBugs의 룰셋들이 추가된 것을 확인할 수 있다.



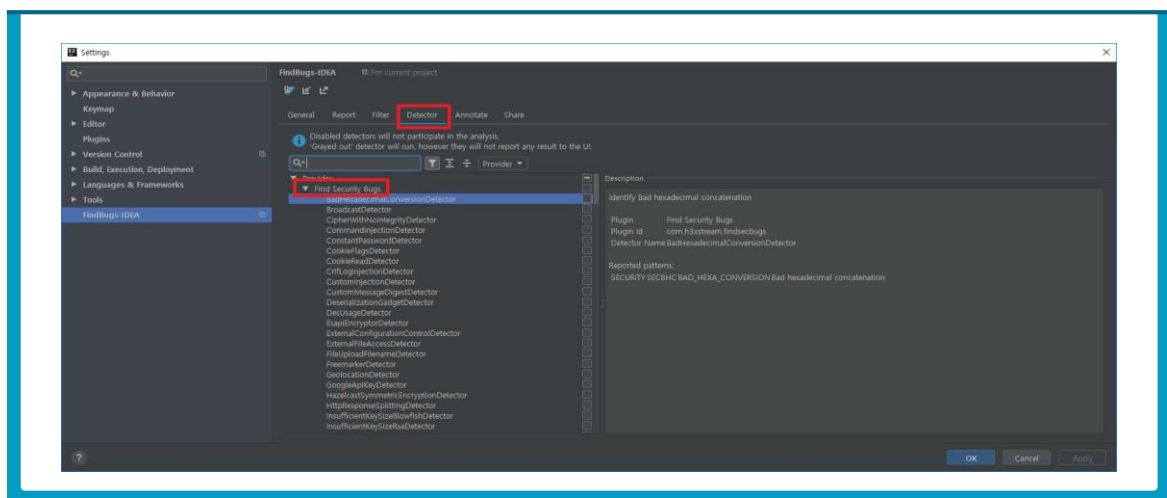
6. FindSecurityBugs IntelliJ 설치

FindSecurityBugs는 spotbugs의 플러그인으로 보안 관련 detector들의 집합체이다. 단순 사용자 정의 detector의 집합이 아니라, 취약점을 탐지하는데 유용한 기능(e.g., taint analysis)들이 추가적으로 구현되어 있다. Spotbugs의 플러그인 형태로, IDE 플러그인을 설치하면 생성되는 설정 화면에서 FindSecurityBugs를 추가할 수 있다.

- ❶ File > Setting를 선택하고 Settings 창에서 FindBugs-IDEA 탭을 클릭한다.
- ❷ 최신 버전의 FindBugs-IDEA를 설치한 경우, 기본으로 FindSecurityBugs가 추가되어 있다. 체크박스를 클릭하여 플러그인을 활성화한다.



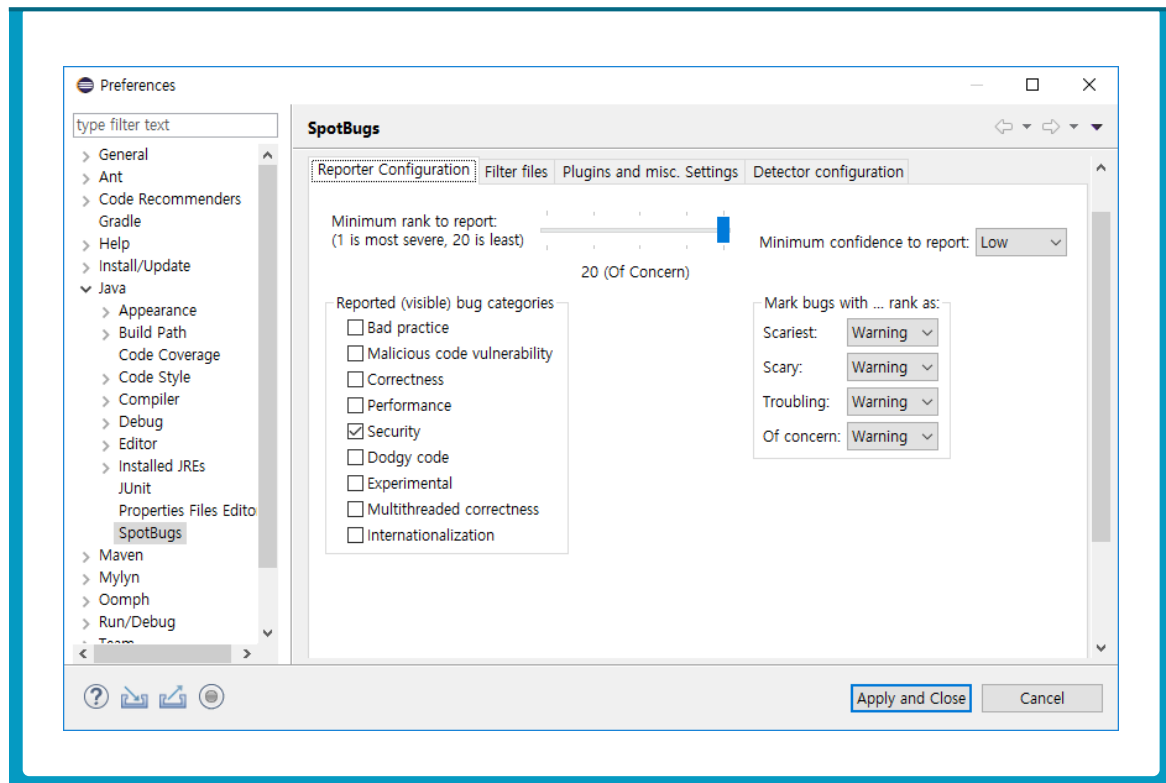
- ❸ OK 버튼을 눌러 설정을 저장하고 다시 FindBugs-IDEA 설정 창을 연다.
- ❹ Detector 탭을 클릭하여 FindSecurityBugs detector들이 추가된 것을 확인한다.



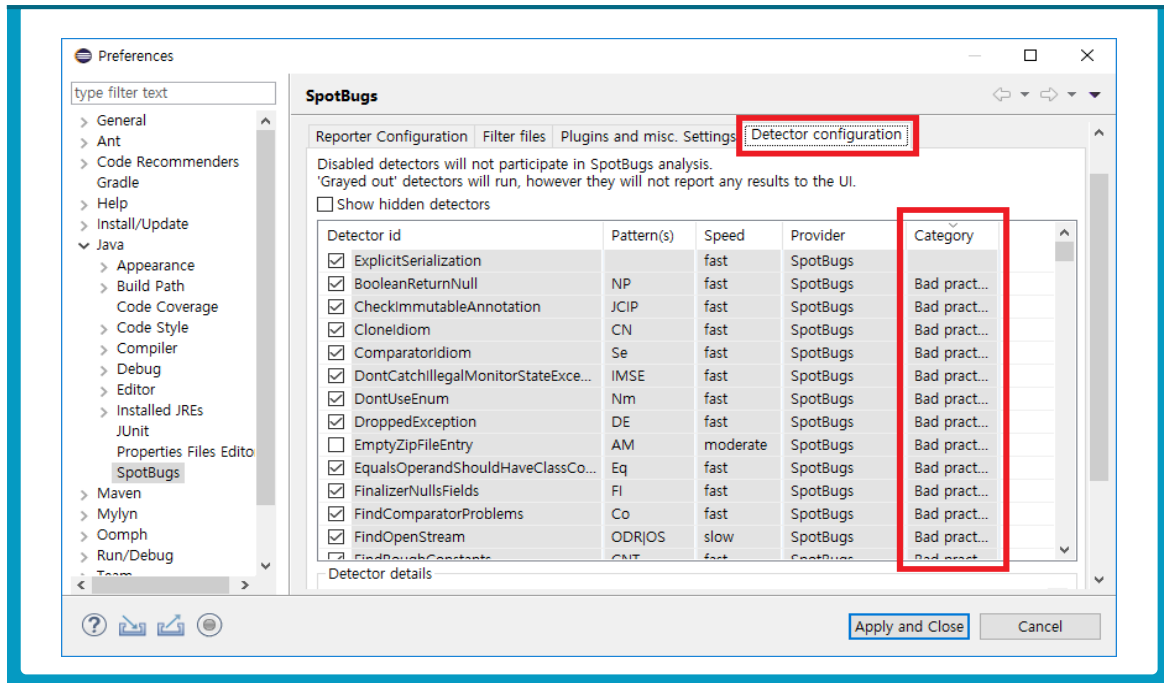
제2절 Spotbugs 및 FindSecurityBugs 룰 설정

본 가이드에서 사용되는 룰셋은 제7장 2절 Spotbugs 룰셋 목록을 참조한다. Spotbugs 및 FindSecurityBugs에서는 사용하길 원하지 않는 detector들을 제외하거나 검출하기 원하는 detector만 검사할 수 있다. 이를 적용하기 위해서 룰을 설정하는 방법을 설명한다.

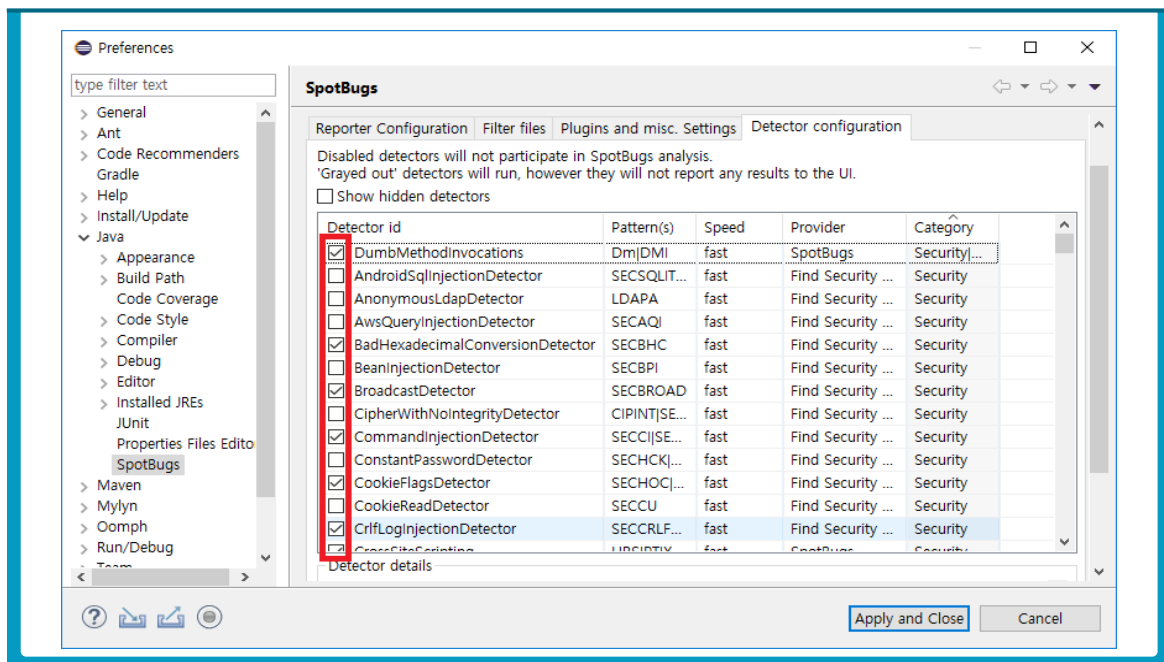
- ❶ Eclipse 메뉴에서 Windows > Preference를 클릭하고 Java > Spotbugs를 클릭하여 Spotbugs 설정 창을 띄운다.



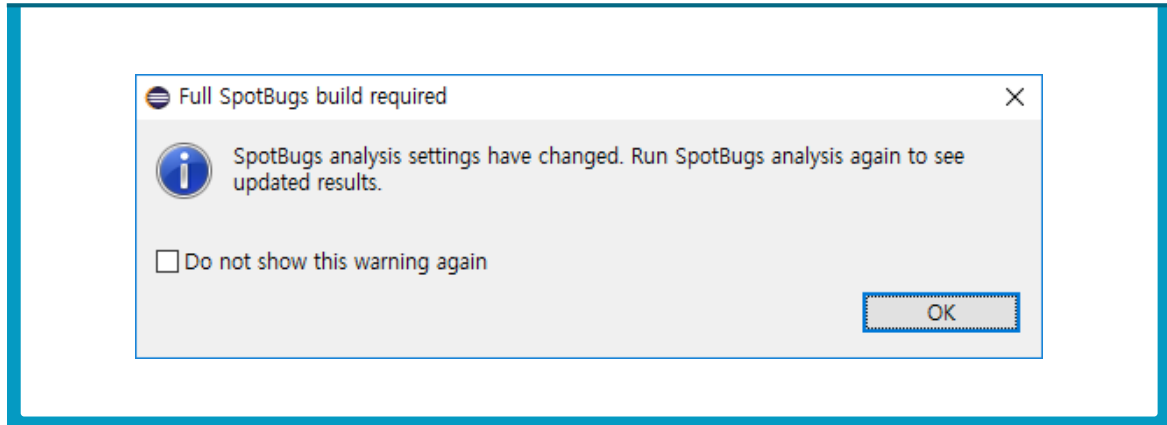
- ② Detector configuration 탭을 선택하고 detector 목록에 Category 탭을 클릭하여 Category 순서로 정렬한다.



- ③ 포함하고 싶은 Detector는 체크 박스를 클릭하고 제외하고 싶은 detector는 체크 박스를 해제한다.



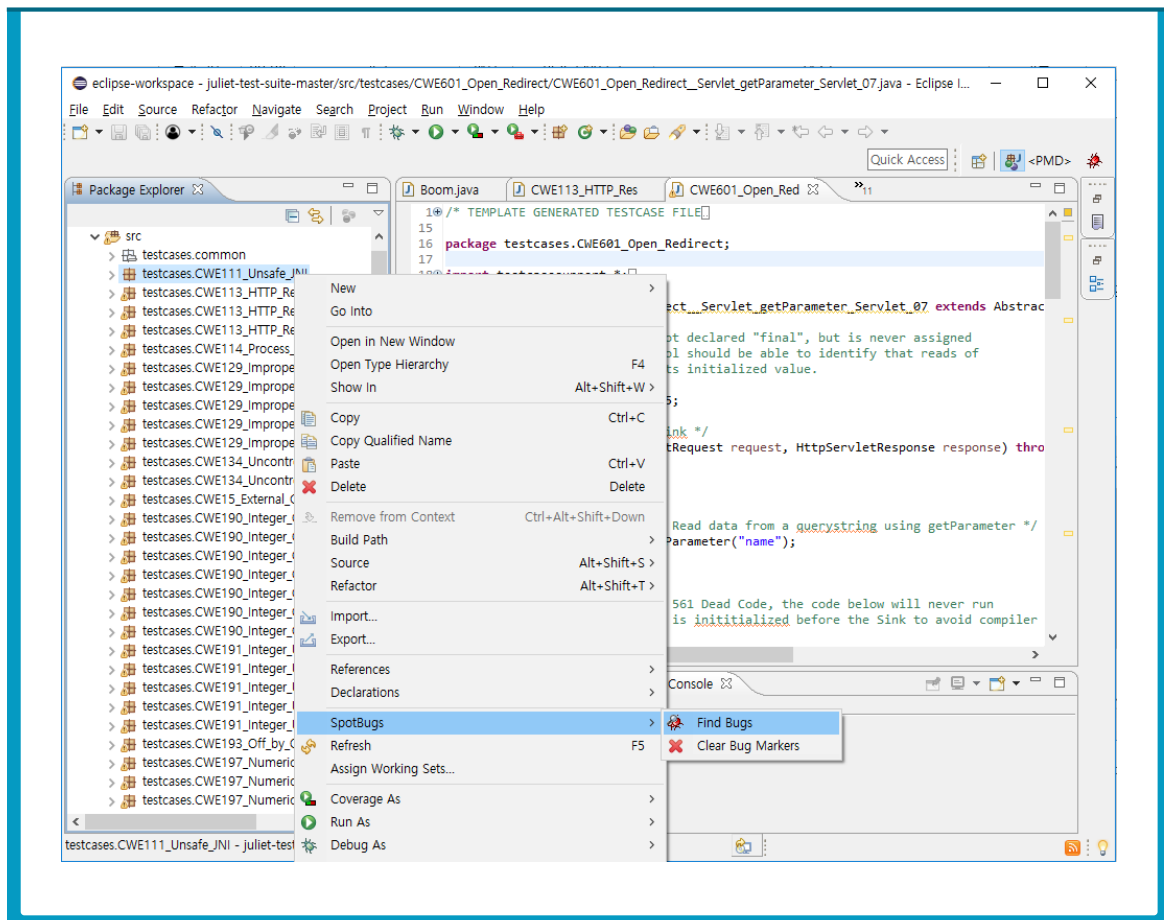
- ④ Apply and Close 버튼을 누르고 Spotbugs 설정이 변경되었다는 알림을 확인한 뒤, OK 버튼을 클릭한다.



제3절 Spotbugs 및 FindSecurityBugs 검사

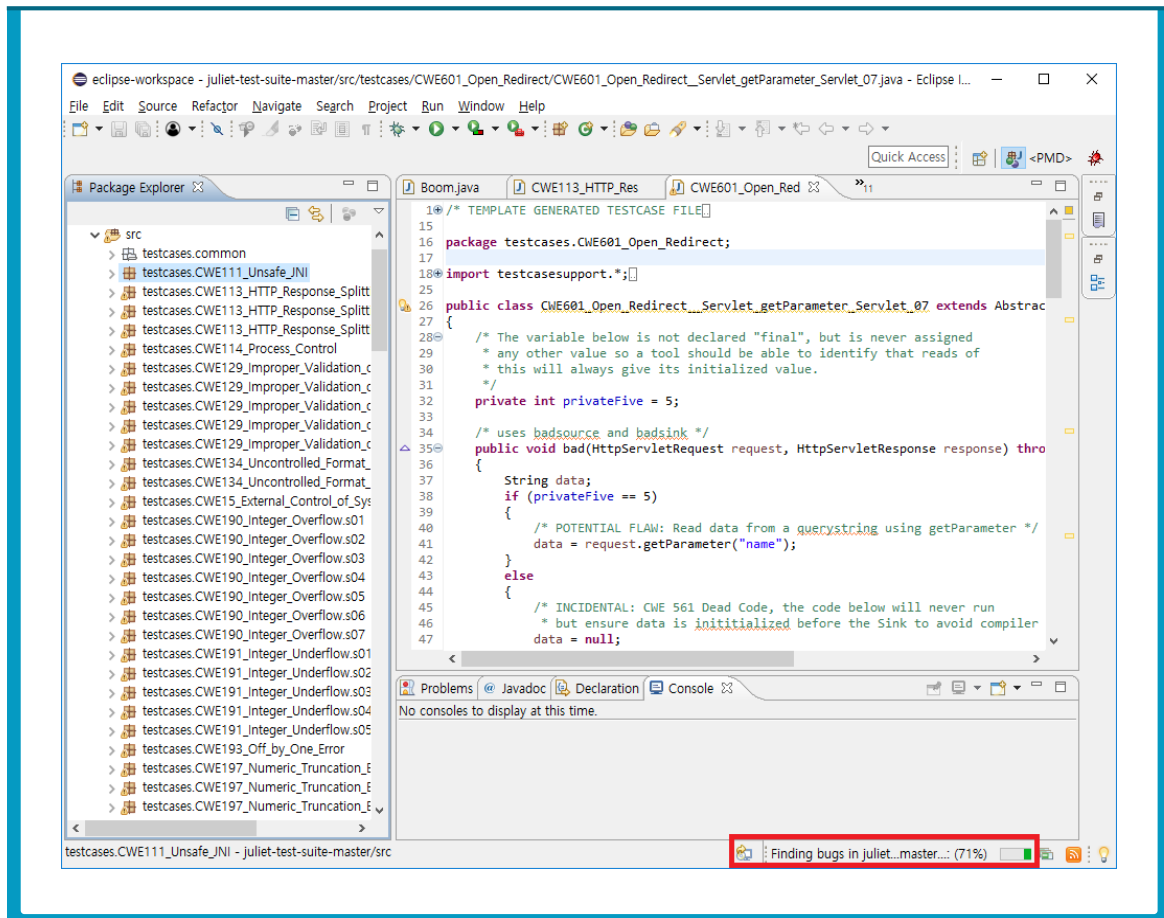
다음은 Spotbugs와 FindSecurityBugs 플러그인을 통해 코드 검사를 수행하는 방법을 설명한다.

- 1 분석하려는 프로젝트 또는 패키지를 선택하고 마우스 오른쪽 버튼을 클릭한 뒤, 메뉴에서 Spotbugs > Findbugs를 선택한다.

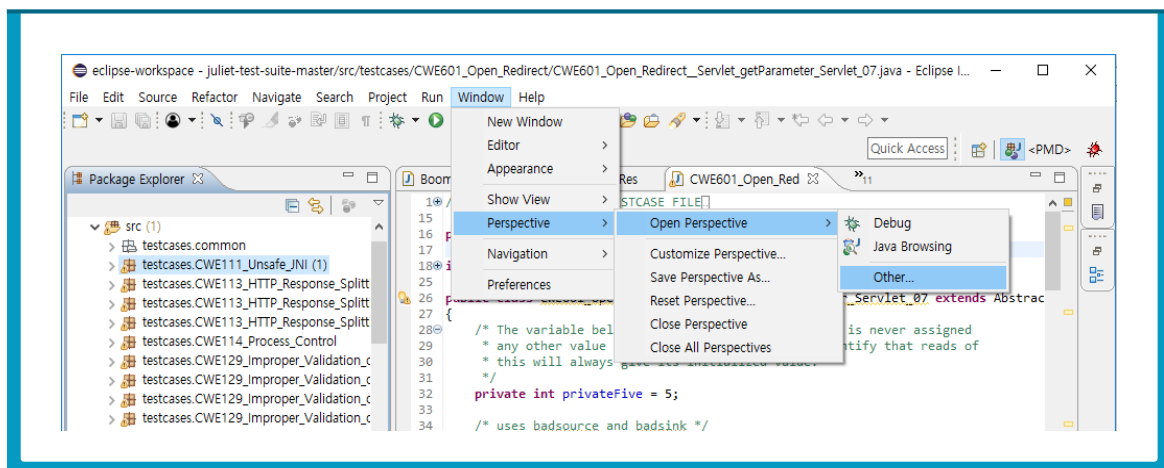


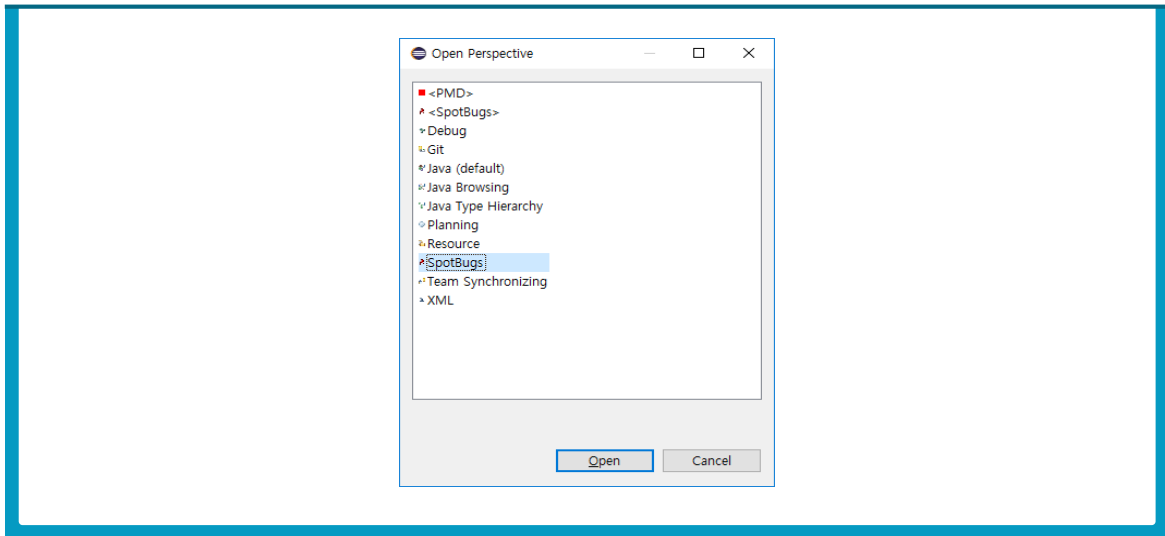
- ※ 참고 : 특정 파일을 선택해서 분석하려면, 이클립스의 Package Explorer에서 특정 파일을 선택하고 마우스 오른쪽 버튼을 누르고 분석한다.
- ※ 참고2: 기존에 분석한 내용을 깨끗하게 지우고 다시 분석하려면 마우스 오른쪽 버튼을 클릭하고 Spotbugs > Clear Bug Marks를 선택한다.

❷ Eclipse IDE 오른쪽 아래에 분석 진행 현황이 표시된다.

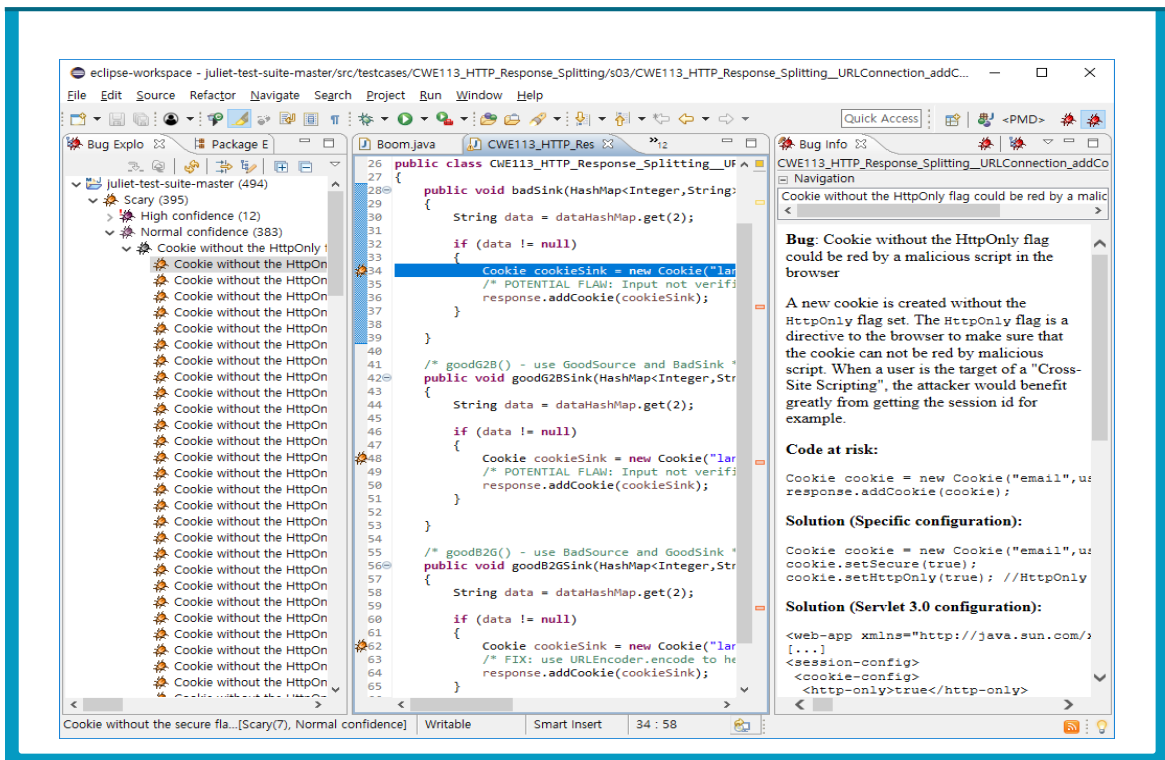


❸ 메뉴에서 Windows > Perspective > Open Perspective > Other...를 선택하고 Spotbugs를 선택한다.





- ④ Bug Explorer에 분석 단위 별로 버그 심각도와 항목별로 목록이 나타난다. 검출된 버그를 더블 클릭하면 버그가 발견된 소스 라인으로 이동한다. Bug Info 탭을 활용하여 검출된 버그에 대한 정보를 확인할 수 있다.



- ⑤ 버그 수정 후 다시 분석을 진행하여 수정된 버그가 검출되지 않는 것을 확인한다.

제4절 Spotbugs에서 플러그인 빌드

사용자 정의 detector들은 하나의 jar 파일(플러그인)로 패키징되어 spotbugs에 추가된다. 빌드는 maven을 사용하며 maven 설치 과정은 생략한다. Spotbugs는 플러그인을 쉽게 개발할 수 있도록 템플릿을 제공한다. 템플릿은 maven-archetype 명령어를 이용하거나 github에서 다운로드할 수 있다.

- ❶ Spotbugs 플러그인 프로젝트를 생성할 곳(e.g., D:\)으로 이동한다.
- ❷ 왼쪽 Shift키를 누른 채로 마우스 오른쪽을 클릭하여 “여기에 PowerShell 창 열기”을 선택한다. 다음 명령어를 통해 archetype을 생성한다.

\$mvn archetype:generate

```

Windows PowerShell
2184: remote -> se.hiq.oss:oss-project-archetype (Archetype for HiQ OSS Projects)
2185: remote -> se.vgregion.javg.maven.archetypes:javg-minimal-archetype (-)
2186: remote -> se.walkercrou:ghp-maven-archetype (Quickstart for developers wanting to integrate the GHP Maven Plugin)
2187: remote -> sk.seges.sesam:sesam-annotation-archetype (-)
2188: remote -> sk.upjs.jpaz2.archetypes:jpaz2-archetype-launcher (An archetype which contains a sample Java project with an empty launcher and JPAZ2 as a dependency.)
2189: remote -> sk.upjs.jpaz2.archetypes:jpaz2-archetype-novice (An archetype which contains a sample Java project for JPAZ2 novices. The launcher instantiates a pane which animates moves and turnings.)
2190: remote -> sk.upjs.jpaz2.archetypes:jpaz2-archetype-quickstart (An archetype which contains a sample Java project with a launcher. The launcher instantiates WinPane, SmartTurtle (extending Turtle), and ObjectInspector.)
2191: remote -> sk.upjs.jpaz2.archetypes:jpaz2-archetype-theater (An archetype which contains a sample Java project based on theater subpackage included in JPAZ2.)
2192: remote -> systems.manifold:archetype (Archetype to demonstrate the structure of a basic project using Manifold.)
2193: remote -> tech.iooo.maven.archetypes:iooo-spring-mvc-quickstart-archetype (iooo spring mvc quickstart archetype)
2194: remote -> tk.skuro:clojure-maven-archetype (A simple Maven archetype for Clojure)
2195: remote -> top.lshaci:framework-archetype (top lshaci framework maven archetype)
2196: remote -> top.marchand.archetype:sie-xf-prio-dep-import-generic ([ELS] Mod?le de projet d'import Flash bas? sur l'importeur g?n?rique)
2197: remote -> tr.com.lucidcode:kite-archetype (A Maven Archetype that allows users to create a Fresh Kite project)
2198: remote -> tr.com.obss.sdlic.archetype:obss-archetype-java (This archetype provides a common skelton for the Java packages.)
2199: remote -> tr.com.obss.sdlic.archetype:obss-archetype-webapp (This archetype provides a skelton for the Java Web Application packages.)
2200: remote -> ua.co.gravy.archetype:single-project-with-junit-and-slf4j (Create a single project with junit, Mockito and slf4j dependencies.)
2201: remote -> uk.ac.ebi.gxa:atlas-archetype (Archetype for generating a custom Atlas webapp)
2202: remote -> uk.ac.gate:gate-plugin-archetype (Maven archetype to create a new GATE plugin project.)
2203: remote -> uk.ac.gate:gate-pr-archetype (Maven archetype to create a new GATE plugin project including a sample PR class (an empty LanguageAnalyser).)
2204: remote -> uk.ac.nactem.argo:argo-analysis-engine-archetype (An archetype which contains a sample Argo (UIMA) Analysis Engine)
2205: remote -> uk.ac.nactem.argo:argo-reader-archetype (An archetype which contains a sample Argo (UIMA) Reader)
2206: remote -> uk.ac.rdg.resc.edal-ncwms-based-webapp (-)
2207: remote -> uk.co.nemstix:basic-javase7-archetype (A basic Java EE7 Maven archetype)
2208: remote -> uk.co.solong:angular-spring-archetype (So Long archetype for RESTful spring services with an AngularJS frontend. Includes debian deployment)
2209: remote -> us.fatehi:schemacrawler-archetype-maven-project (-)
2210: remote -> us.fatehi:schemacrawler-archetype-plugin-command (-)
2211: remote -> us.fatehi:schemacrawler-archetype-plugin-dbconnector (-)
2212: remote -> us.fatehi:schemacrawler-archetype-plugin-lint (-)
2213: remote -> ws.osiris:osiris-archetype (Maven Archetype for Osiris)
2214: remote -> xyz.luan.generator:xyz-gae-generator (-)
2215: remote -> xyz.luan.generator:xyz-generator (-)
2216: local -> com.github.spotbugs:spotbugs-archetype-archetype (spotbugs-archetype)
Choose a number or apply filter (format: [groupId:artifactId, case sensitive contains]): 1225:
  
```

- ❸ archetype으로 등록된 템플릿 리스트들이 출력된다. spotbugs를 입력한다.

- ④ 다운로드 받을 수 있는 spotbugs-archetype이 출력된다. 원격에 있는 Spotbugs 플러그인 프로젝트를 선택한다. 아래 그림에서는 1을 입력한다.

```

Windows PowerShell
2193: remote -> tech.iooo.maven.archetypes:iooo-spring-mvc-quickstart-archetype (iooo spring mvc quickstart archetype)
2194: remote -> tk.skuro:clojure-maven-archetype (A simple Maven archetype for Clojure)
2195: remote -> top.lshaci:framework-archetype (top lshaci framework maven archetype)
2196: remote -> top.marchand.archetype:sie-xf-prio-dep-import-generic ([ELS] Mod?le de projet d'import Flash bas? sur l'
Importeur g?n?rique)
2197: remote -> tr.com.lucidcode:kite-archetype (A Maven Archetype that allows users to create a Fresh Kite project)
2198: remote -> tr.com.obss.sdlc.archetype:obss-archetype-java (This archetype provides a common skelton for the Java pa
ckages.)
2199: remote -> tr.com.obss.sdlc.archetype:obss-archetype-webapp (This archetype provides a skelton for the Java Web App
lication packages.)
2200: remote -> ua.co.gravy.archetype:single-project-with-junit-and-slf4j (Create a single project with junit, Mockito a
nd slf4j dependencies.)
2201: remote -> uk.ac.ebi.gxa:atlas-archetype (Archetype for generating a custom Atlas webapp)
2202: remote -> uk.ac.gate:gate-plugin-archetype (Maven archetype to create a new GATE plugin project.)
2203: remote -> uk.ac.gate:gate-pr-archetype (Maven archetype to create a new GATE plugin project including a sample PR
class (an empty LanguageAnalyser).)
2204: remote -> uk.ac.nactem.argo:argo-analysis-engine-archetype (An archetype which contains a sample Argo (UIMA) Analy
sis Engine)
2205: remote -> uk.ac.nactem.argo:argo-reader-archetype (An archetype which contains a sample Argo (UIMA) Reader)
2206: remote -> uk.ac.rdg.resc.edal-ncwms-based-webapp (-)
2207: remote -> uk.co.nemstix:basic-javase7-archetype (A basic Java EE7 Maven archetype)
2208: remote -> uk.co.solong:angular-spring-archetype (So Long archetype for RESTful spring services with an AngularJS f
rontend. Includes debian deployment)
2209: remote -> us.fatehi:schemacrawler-archetype-maven-project (-)
2210: remote -> us.fatehi:schemacrawler-archetype-plugin-command (-)
2211: remote -> us.fatehi:schemacrawler-archetype-plugin-dbconnector (-)
2212: remote -> us.fatehi:schemacrawler-archetype-plugin-lint (-)
2213: remote -> ws.osiris:osiris-archetype (Maven Archetype for Osiris)
2214: remote -> xyz.luan.generator:xyz-gae-generator (-)
2215: remote -> xyz.luan.generator:xyz-generator (-)
2216: local -> com.github.spotbugs:spotbugs-archetype-archetype (spotbugs-archetype)
Choose a number or apply filter (format: [groupId]artifactId, case sensitive contains): 1225: spotbugs
Choose archetype:
1: remote -> com.github.spotbugs:spotbugs-archetype (A Maven archetype for SpotBugs plugin project)
2: local -> com.github.spotbugs:spotbugs-archetype-archetype (spotbugs-archetype)
Choose a number or apply filter (format: [groupId]artifactId, case sensitive contains): 1
  
```

- ⑤ 생성될 플러그인 프로젝트에 필요한 정보를 다음과 같이 입력한다.

spotbugs-archetype version: 리스트 중 최신 버전(5)

artifactId : spotbugs-archetype

groupId : com.github.spotbugs

version : default 값 입력(enter)

package : default 값 입력(enter)

```

Windows PowerShell
2216: local -> com.github.spotbugs:spotbugs-archetype-archetype (spotbugs-archetype)
Choose a number or apply filter (format: [groupId]artifactId, case sensitive contains): 1225: spotbugs
Choose archetype:
1: remote -> com.github.spotbugs:spotbugs-archetype (A Maven archetype for SpotBugs plugin project)
2: local -> com.github.spotbugs:spotbugs-archetype-archetype (spotbugs-archetype)
Choose a number or apply filter (format: [groupId]artifactId, case sensitive contains): 1
Choose com.github.spotbugs:spotbugs-archetype version:
1: 0.1.0
2: 0.2.0
3: 0.2.1
4: 0.2.2-SNAPSHOT
5: 0.3.0-SNAPSHOT
Choose a number: 5: 5
Define value for property 'groupId': com.github.spotbugs
Define value for property 'artifactId': spotbugs-archetype
Define value for property 'version' 1.0-SNAPSHOT:
Define value for property 'package' com.github.spotbugs:
  
```

⑥ 입력한 정보가 출력된다. Y를 입력하여 프로젝트를 생성한다.

```
Windows PowerShell
Confirm properties configuration:
groupid: com.github.spotbugs
artifactId: spotbugs-archetype
version: 1.0-SNAPSHOT
package: com.github.spotbugs
Y: Y
[INFO] Using following parameters for creating project from Archetype: spotbugs-archetype:0.3.0-SNAPSHOT
[INFO]
[INFO] Parameter: groupId, Value: com.github.spotbugs
[INFO] Parameter: artifactId, Value: spotbugs-archetype
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] Parameter: package, Value: com.github.spotbugs
[INFO] Parameter: packageInPathFormat, Value: com/github/spotbugs
[INFO] Parameter: package, Value: com.github.spotbugs
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] Parameter: groupId, Value: com.github.spotbugs
[INFO] Parameter: artifactId, Value: spotbugs-archetype
[WARNING] The directory D:\spotbugs-archetype already exists.
[INFO] Project created from Archetype in dir: D:\spotbugs-archetype
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 12:09 min
[INFO] Finished at: 2018-08-30T15:10:32+09:00
[INFO]
PS D:\spotbugs-archetype>
```

⑦ 입력한 artifactId로 spotbugs 플러그인 프로젝트 폴더가 생성된다.

```
Windows PowerShell
PS D:\spotbugs-archetype> tree .\spotbugs-archetype
폴더 PATH의 목록입니다.
볼륨 일련 번호가 0000023F 0E9C:B4EE입니다.
D:\spotbugs-archetype
.
├── src
│   ├── main
│   │   ├── java
│   │   │   └── com
│   │   │       └── github
│   │           └── spotbugs
│   └── resources
└── test
    ├── java
    │   ├── com
    │   └── github
    └── spotbugs
PS D:\spotbugs-archetype>
```

⑧ 프로젝트 폴더로 이동한다. 다음 명령어를 통해 플러그인을 빌드한다. 하위 폴더 target이 생성되고 jar 파일(플러그인)이 target 폴더에 안에 생성된다.

\$mvn install

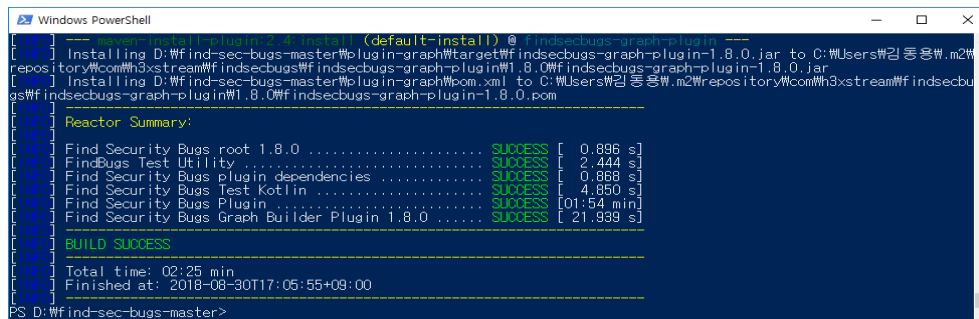
```
Windows PowerShell
PS D:\spotbugs-archetype> cd .\spotbugs-archetype\
PS D:\spotbugs-archetype> mvn install
[INFO] Scanning for projects...
[INFO]
[INFO] < com.github.spotbugs:spotbugs-archetype >-----
[INFO] Building spotbugs-archetype 1.0-SNAPSHOT
[INFO] [ jar ]-----
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ spotbugs-archetype ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 2 resources
[INFO]
[INFO] --- maven-compiler-plugin:3.1:compile (default-compile) @ spotbugs-archetype ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ spotbugs-archetype ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory D:\spotbugs-archetype\src\test\resources
[INFO]
```

제5절 FindSecurityBugs에서 플러그인 빌드

FindSecurityBugs는 그 자체로 spotbugs의 플러그인이기 때문에 따로 플러그인 제작 템플릿이 필요하지 않다. 다만, 기존에 구현되어 있는 FindSecurityBugs detector들과 같이 빌드되기 때문에 구현한 사용자 정의 detector만 따로 빌드할 수 있는 spotbugs-archetype 방식과 장단점이 있다.

- 1 Github에서 FindSecurityBugs 프로젝트를 다운로드 받는다.
<https://github.com/find-sec-bugs/find-sec-bugs>
- 2 다운로드 받은 압축을 해제하고 해당 폴더로 이동한다. find-sec-bugs-master는 루트 프로젝트이며, plugin 폴더에서 서브 프로젝트가 플러그인을 생성한다.
- 3 왼쪽 Shift 키를 누른 채로 마우스 오른쪽을 클릭하여 “여기에 PowerShell 창 열기”를 선택한다. 다음 명령어를 통해 spotbugs 플러그인을 빌드한다.

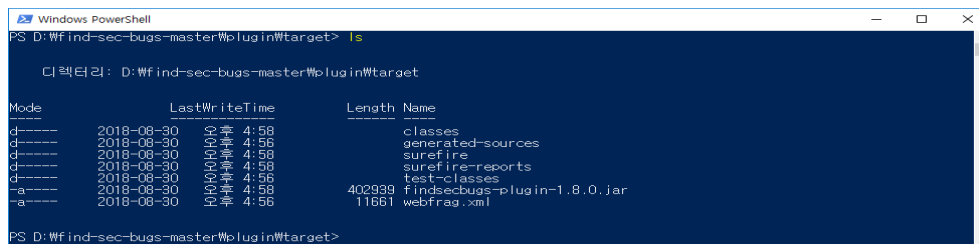
\$mvn clean install



```

[INFO] --- (default-install) @ findsecbugs-graph-plugin ---
[INFO] Installing D:\find-sec-bugs-master\plugin-graph\target\findsecbugs-graph-plugin-1.8.0.jar to C:\Users\김동용\m2\repository\com\h3xstream\findsecbugs-graph-plugin\1.8.0\findsecbugs-graph-plugin-1.8.0.jar
[INFO] Installing D:\find-sec-bugs-master\plugin-graph\pom.xml to C:\Users\김동용\m2\repository\com\h3xstream\findsecbugs-graph-plugin\1.8.0\findsecbugs-graph-plugin-1.8.0.pom
[INFO] Reactor Summary:
[INFO] Find Security Bugs root 1.8.0 ..... SUCCESS [ 0.896 s]
[INFO] FindBugs Test Utility ..... SUCCESS [ 2.444 s]
[INFO] Find Security Bugs plugin dependencies ..... SUCCESS [ 0.868 s]
[INFO] Find Security Bugs Test Kotlin ..... SUCCESS [ 4.850 s]
[INFO] Find Security Bugs Plugin ..... SUCCESS [01:54 min]
[INFO] Find Security Bugs Graph Builder Plugin 1.8.0 ..... SUCCESS [ 21.939 s]
[INFO] BUILD SUCCESS
[INFO] Total time: 02:25 min
[INFO] Finished at: 2018-08-30T17:05:55+09:00
[INFO]
PS D:\find-sec-bugs-master>
  
```

- 4 하위 폴더 plugin/target으로 이동한다. 플러그인이 생성된 것을 확인한다.



```

PS D:\find-sec-bugs-master\plugin\target> ls

디렉터리: D:\find-sec-bugs-master\plugin\target

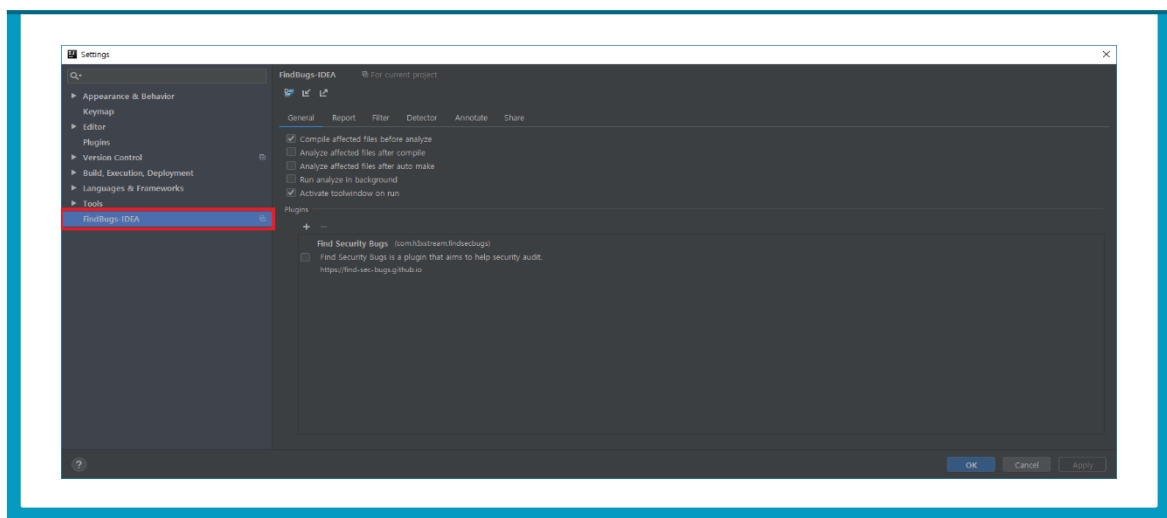
Mode                LastWriteTime         Length Name
----                -
d-----         2018-08-30 오후 4:58             classes
d-----         2018-08-30 오후 4:58      generated-sources
d-----         2018-08-30 오후 4:58             surefire
d-----         2018-08-30 오후 4:58      surefire-reports
d-----         2018-08-30 오후 4:58          test-classes
-a-----         2018-08-30 오후 4:58      402939 findsecbugs-plugin-1.8.0.jar
-a-----         2018-08-30 오후 4:58       11661 webfrag.xml

PS D:\find-sec-bugs-master\plugin\target>
  
```

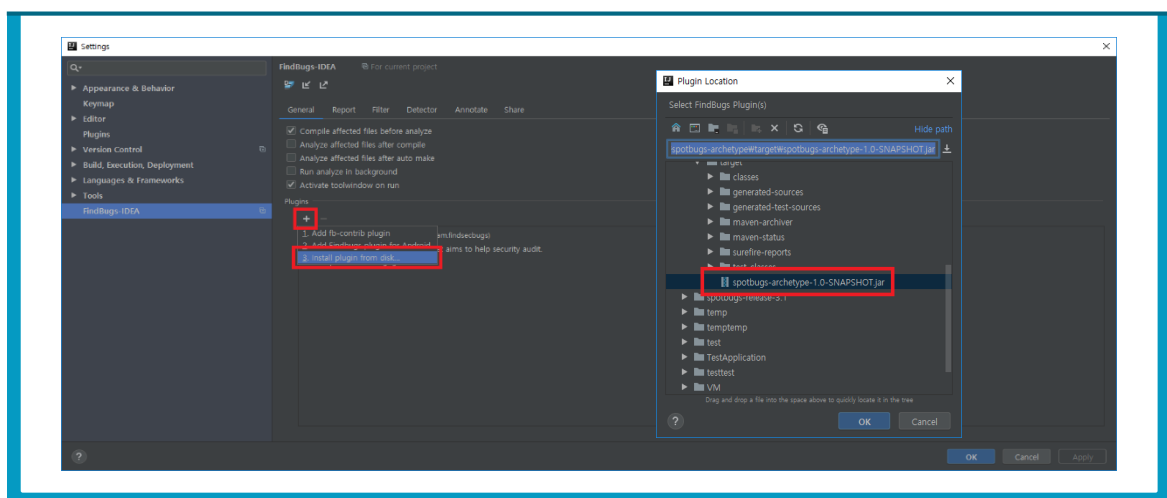

제6절 플러그인 IDE에 추가

위의 두 가지 방법으로 spotbugs 플러그인을 빌드했다. 생성된 플러그인을 IntelliJ에서 추가하는 방법을 소개한다. 추가하는 방법은 두 가지 빌드 방법 모두 동일하다. 플러그인이 추가되면 플러그인에 포함된 detector들을 IDE 설정 창에서 확인할 수 있다.

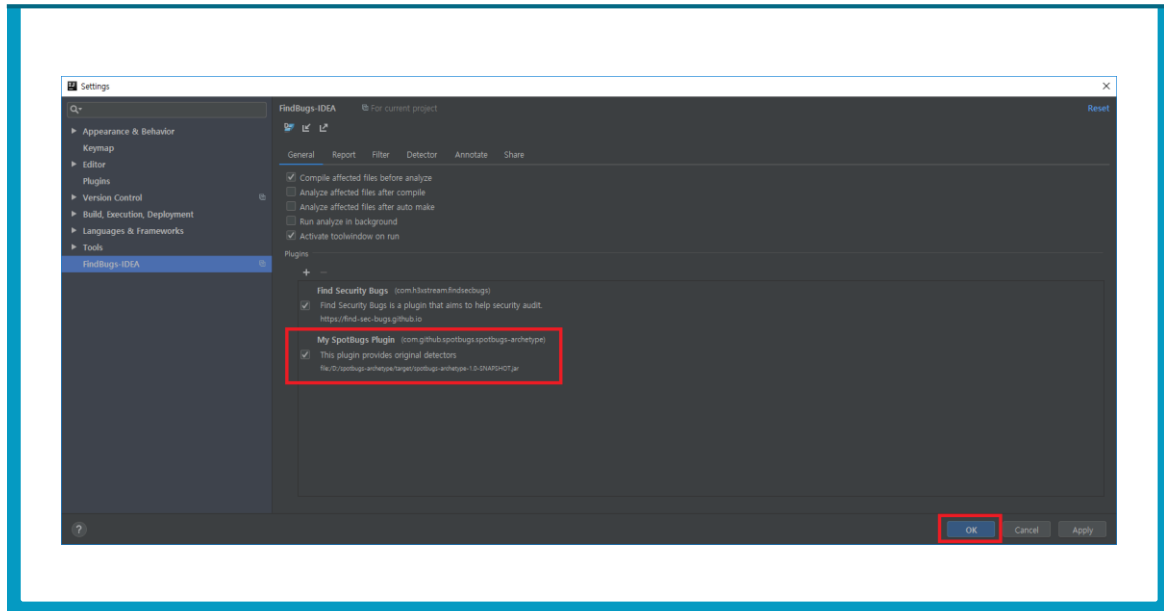
- ❶ IntelliJ에서 File > Setting를 선택하고 Settings 창에서 FindBugs-IDEA 탭을 클릭한다.



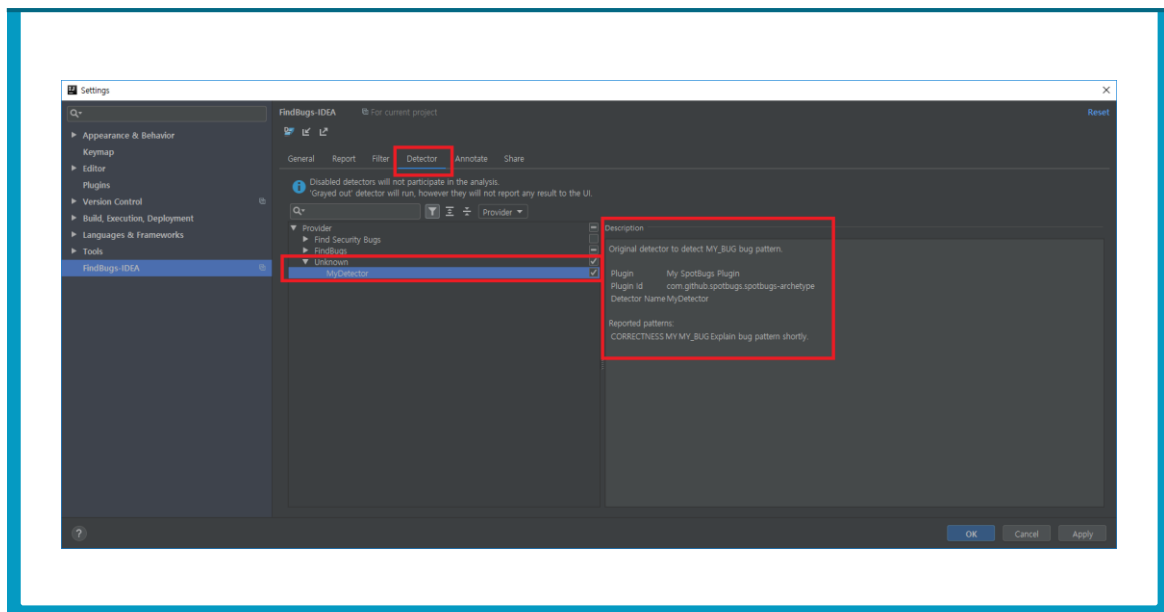
- ❷ FindBugs-IDEA 탭 중 Plugins에서 “+”버튼을 클릭해 Install plugin from disk...를 선택한다. 빌드한 jar 파일을 선택하여 추가한다.

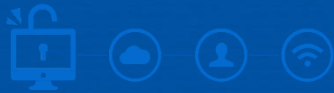


- ③ 빌드된 jar 파일이 정상적으로 추가되었는지 확인한다. OK 버튼을 누르고 FindBugs-IDEA 설정 창을 다시 실행한다

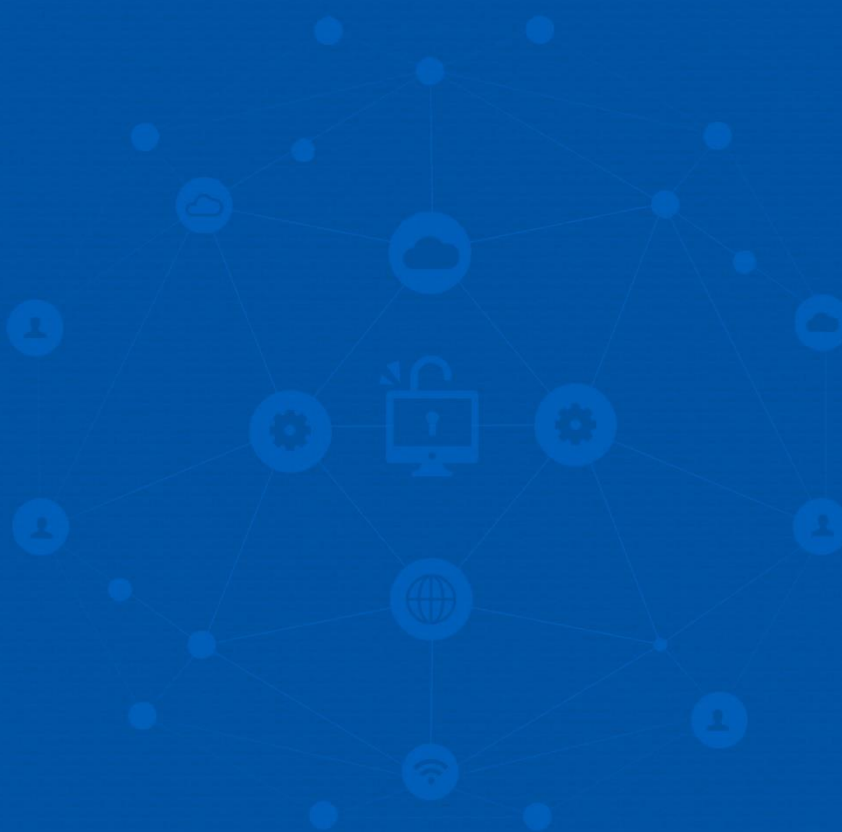


- ④ Detector 탭에서 사용자 정의 detector인 MyDetector가 추가되었는지 확인한다. 우측에 detector를 생성할 때 입력한 메시지가 출력된다.





전자정부 SW 개발자·진단원을 위한
공개SW를 활용한 소프트웨어 개발보안 점검가이드



PART 4

제4장 PMD 사용 방법

- 제1절 PMD 설치
- 제2절 PMD 적용 룰 설정
- 제3절 PMD 검사

PART



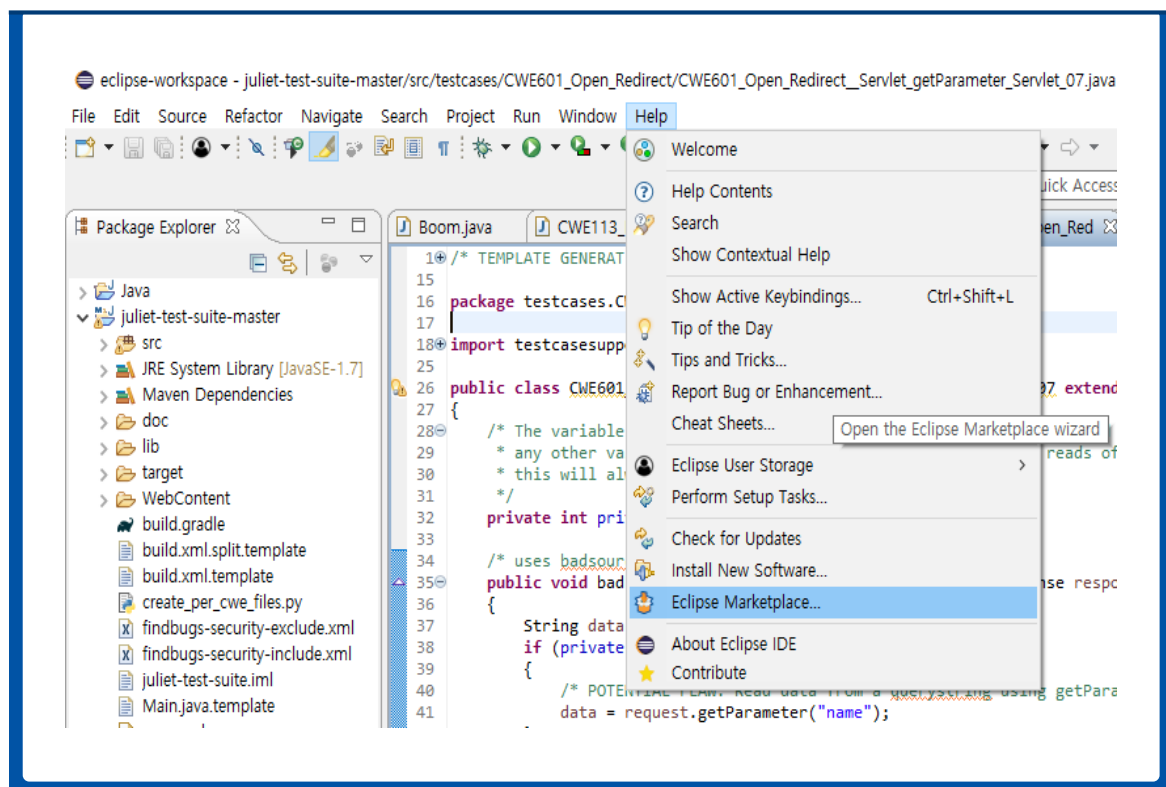
제4장 PMD 사용 방법



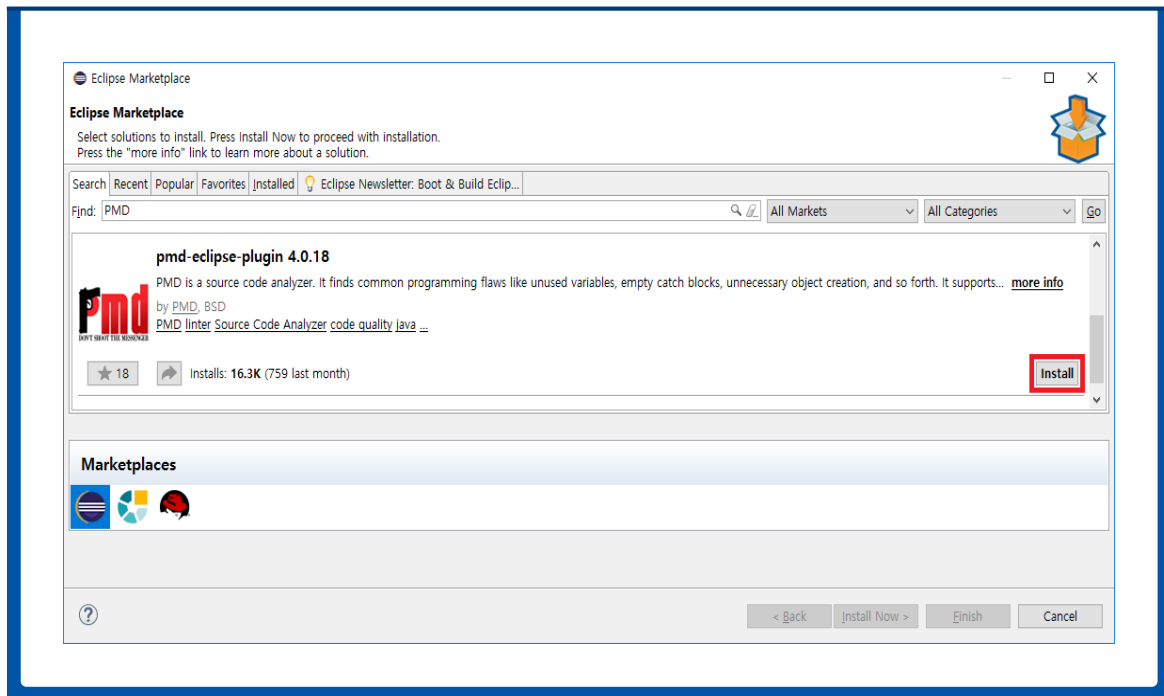
제1절 PMD 설치

1. 이클립스에서 PMD 플러그인 설치

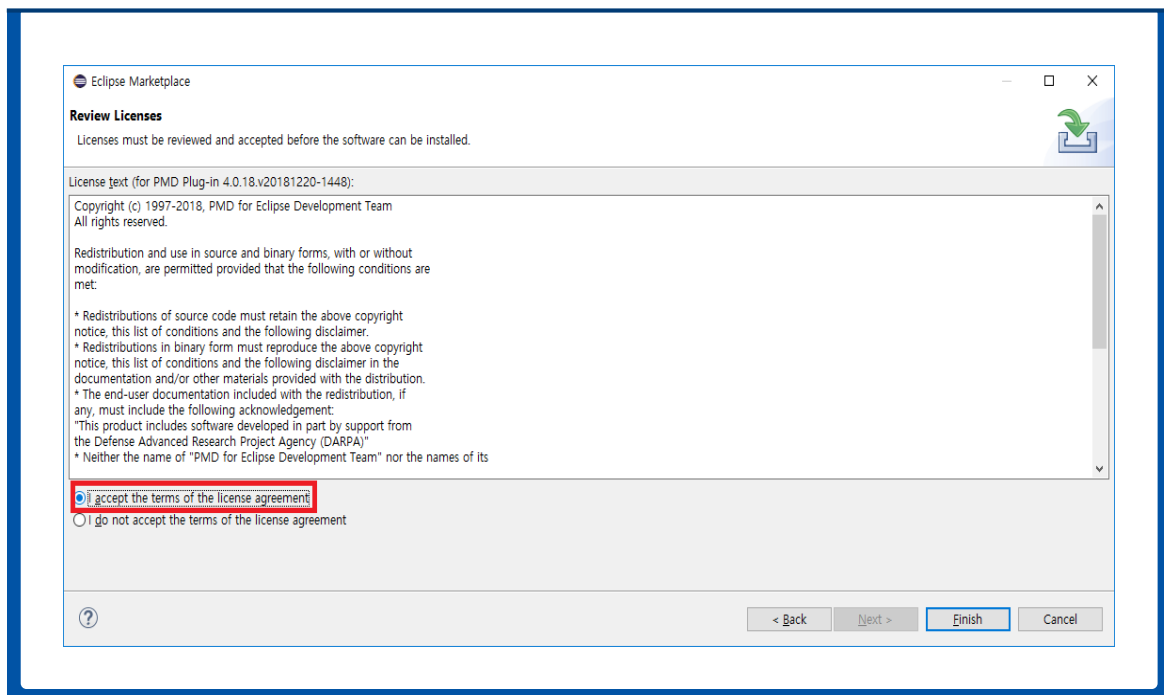
- ❶ 이클립스를 구동한다.
- ❷ Help > Eclipse Marketplace...를 선택한다.



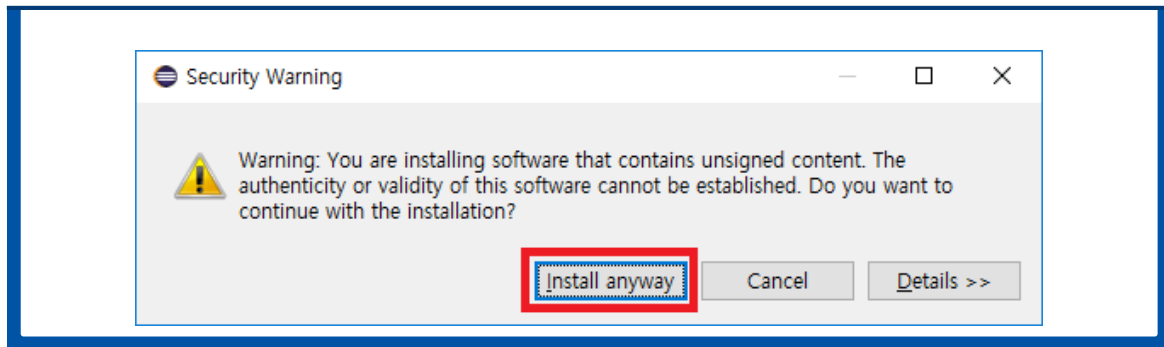
③ Eclipse Marketplace에서 Find창에 PMD를 검색하고 Install 버튼을 클릭한다.



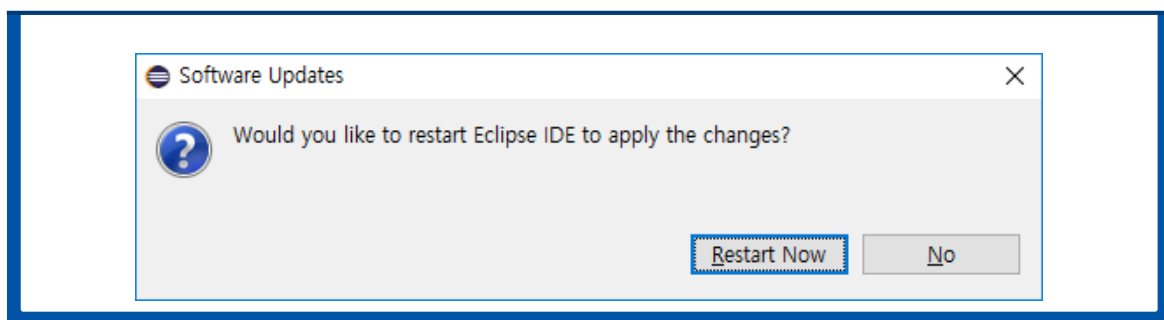
④ 설치 중 라이선스 확인 알림이 나오면 I accept the terms of the license agreement 를 클릭해 라이선스에 동의하고 Finish를 클릭한다.



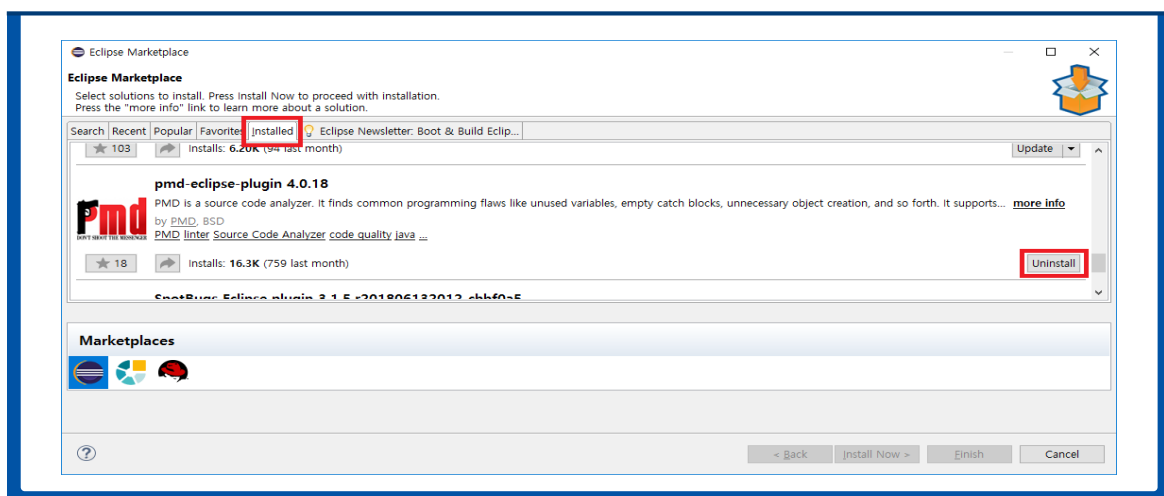
- ⑤ 플러그인을 설치하는 도중 Security Warning 알림이 나오면 OK를 선택한다.



- ⑥ 설치가 완료되고 Eclipse IDE를 재실행할지 확인하는 알림이 나오면 Restart Now 를 클릭하여 재실행한다.



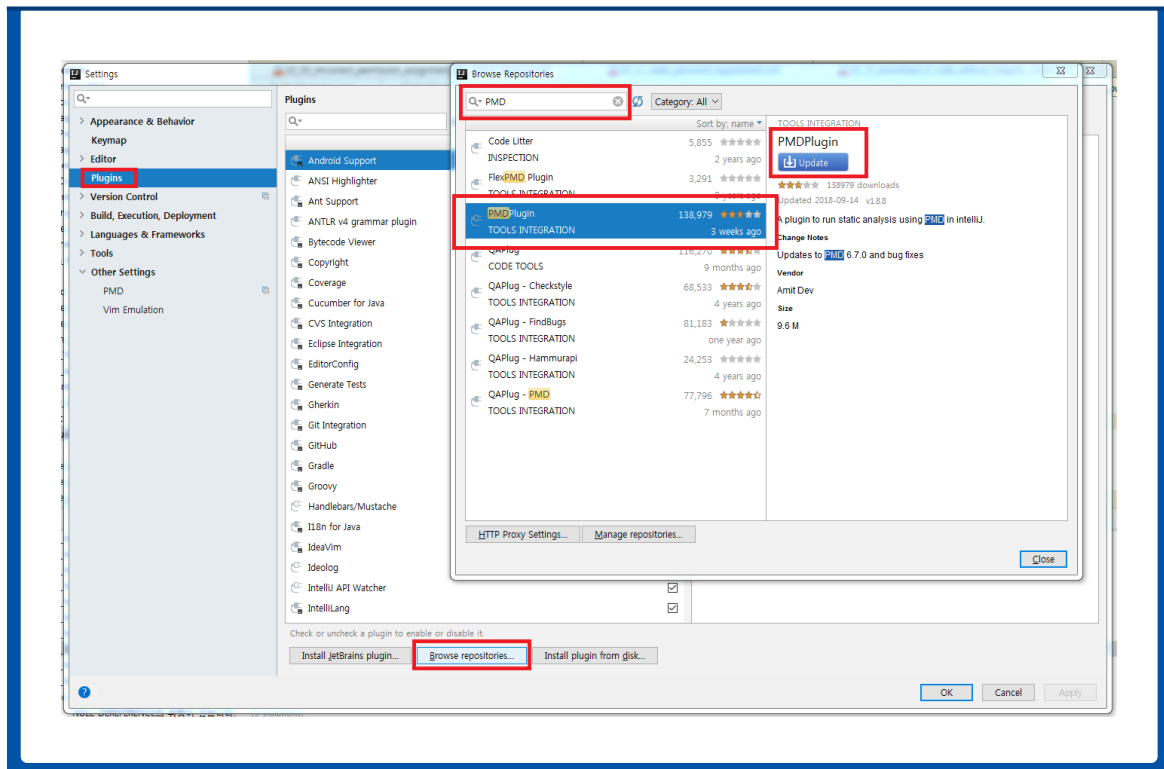
- ⑦ 재실행 후 Help > Eclipse Marketplace... > Installed 항목을 선택하여 PMD 플러그인이 설치되었는지 확인한다.



2. IntelliJ에서 PMD 플러그인 설치

PMD는 IDE 플러그인을 설치하는 설정 화면에서 PMD를 추가할 수 있다.

- ❶ Settings – Plugin에서 Browse repositories를 선택한다.
- ❷ PMD 플러그인을 검색하여 설치한다.

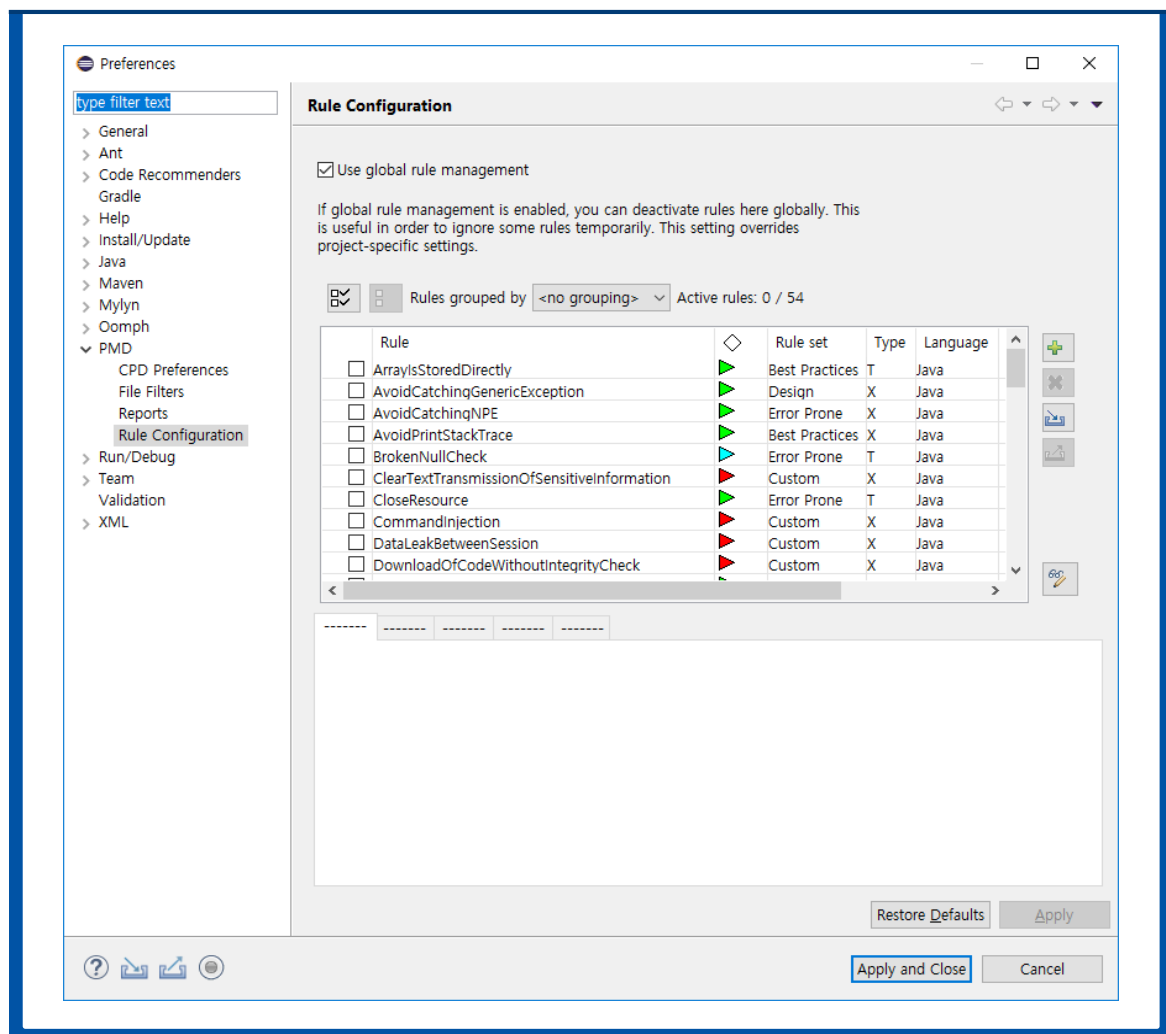


제2절 PMD 적용 룰 설정

본 가이드에 사용되는 룰셋은 제7장 2절 PMD 룰셋 목록을 참조한다. 이를 적용하기 위해서 룰을 설정하는 방법을 설명한다.

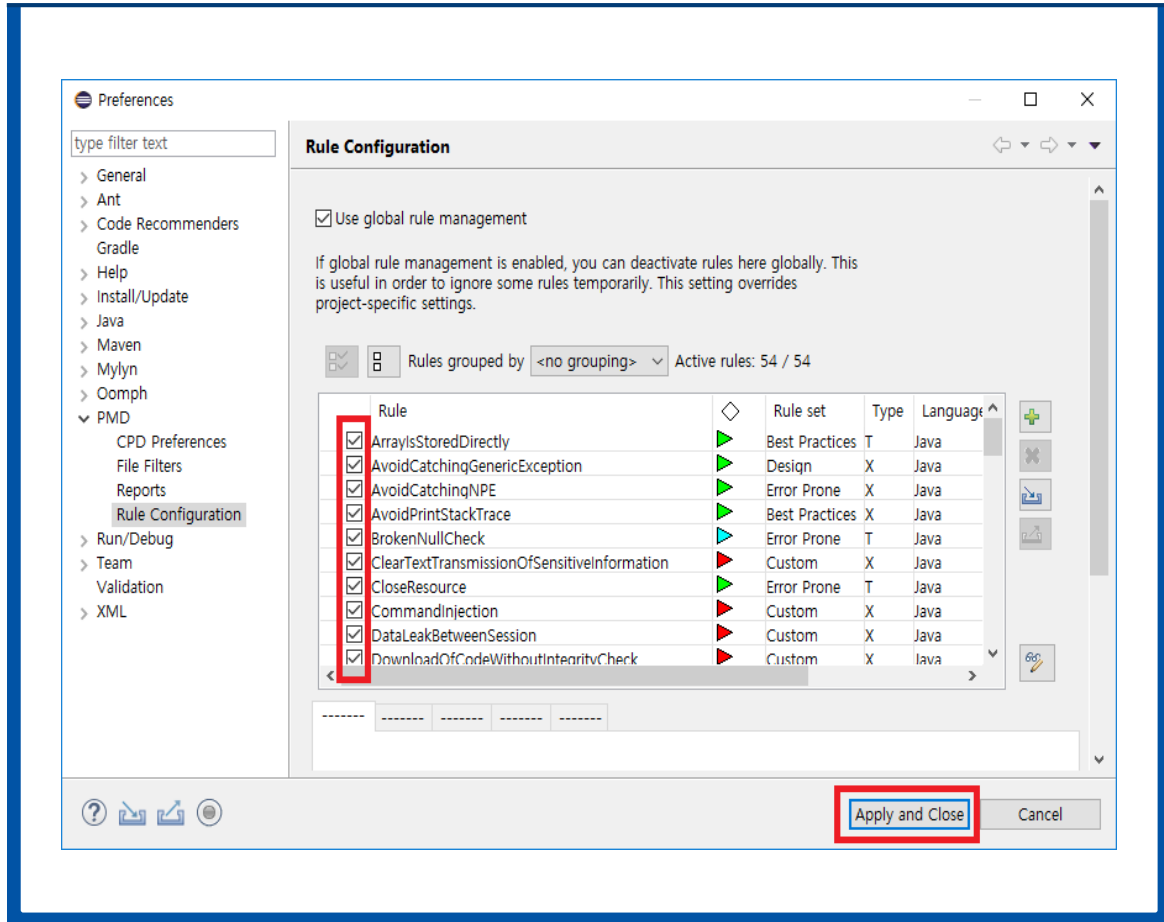
1. 이클립스에서 PMD 룰셋 설정

- ❶ Eclipse 메뉴에서 Windows > Preference를 클릭하고 PMD > Rule Configuration을 클릭하여 PMD 룰셋 설정 창을 띄운다.

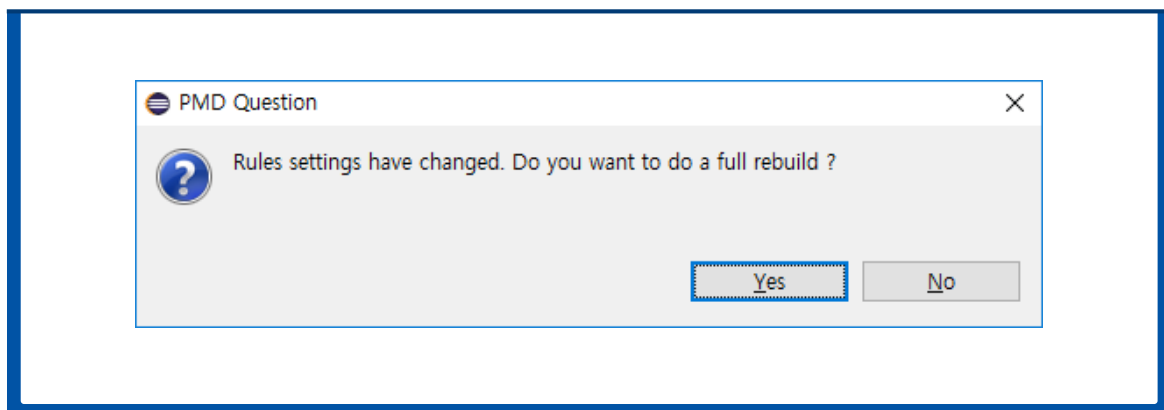


- ❷ Use global rule management를 선택하여 룰셋 수정이 가능하도록 한다.

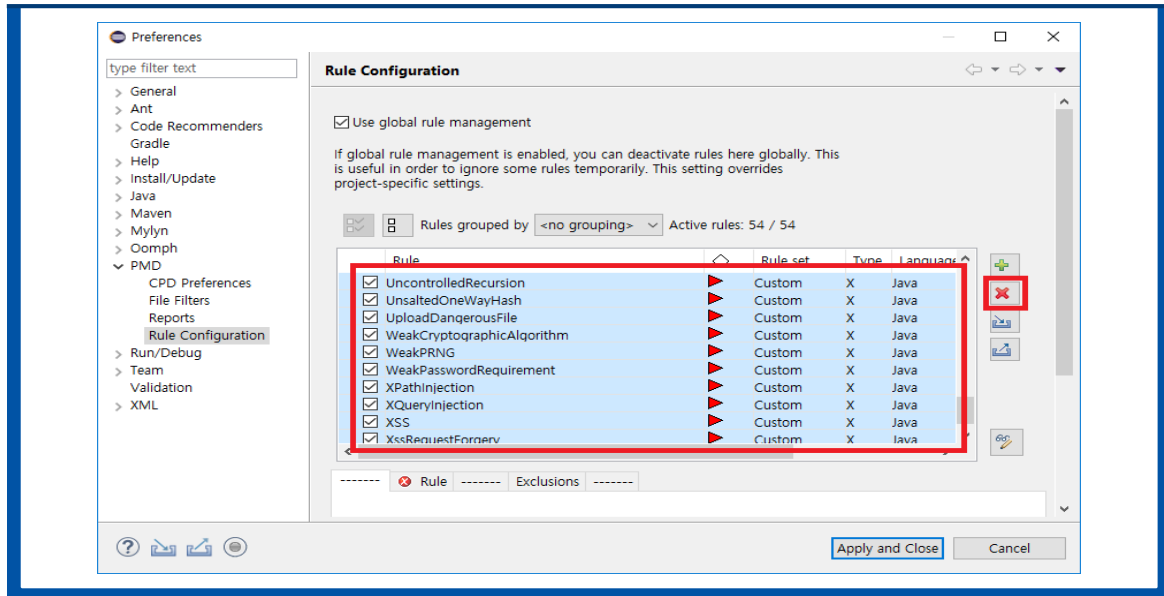
- ③ 분석에 포함시키고 싶은 룰들을 선택한다. 체크 박스를 해제하면 분석에서 제외된다. Apply and Close 버튼을 클릭해 설정을 적용한다.



- ④ 설정이 변경되었다는 알림을 확인하고 rebuild 실행을 묻는 알림에 Yes 버튼을 클릭한다.

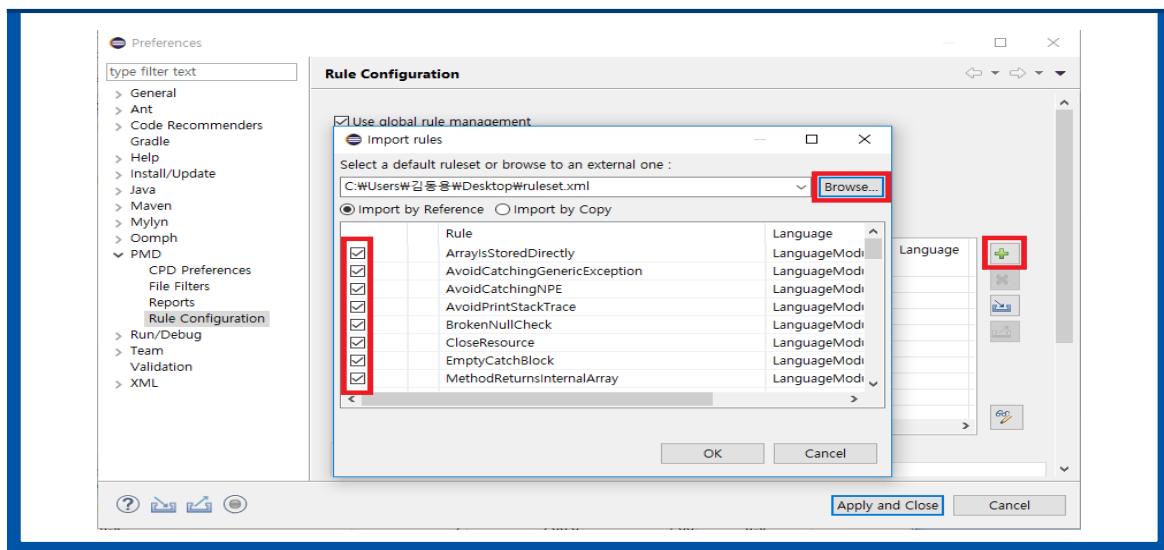


- ⑤ xml 파일을 통해 룰셋을 관리할 수 있다. 기존 룰셋과의 혼동을 피하기 위해 기존에 존재하는 모든 룰들을 삭제한다.



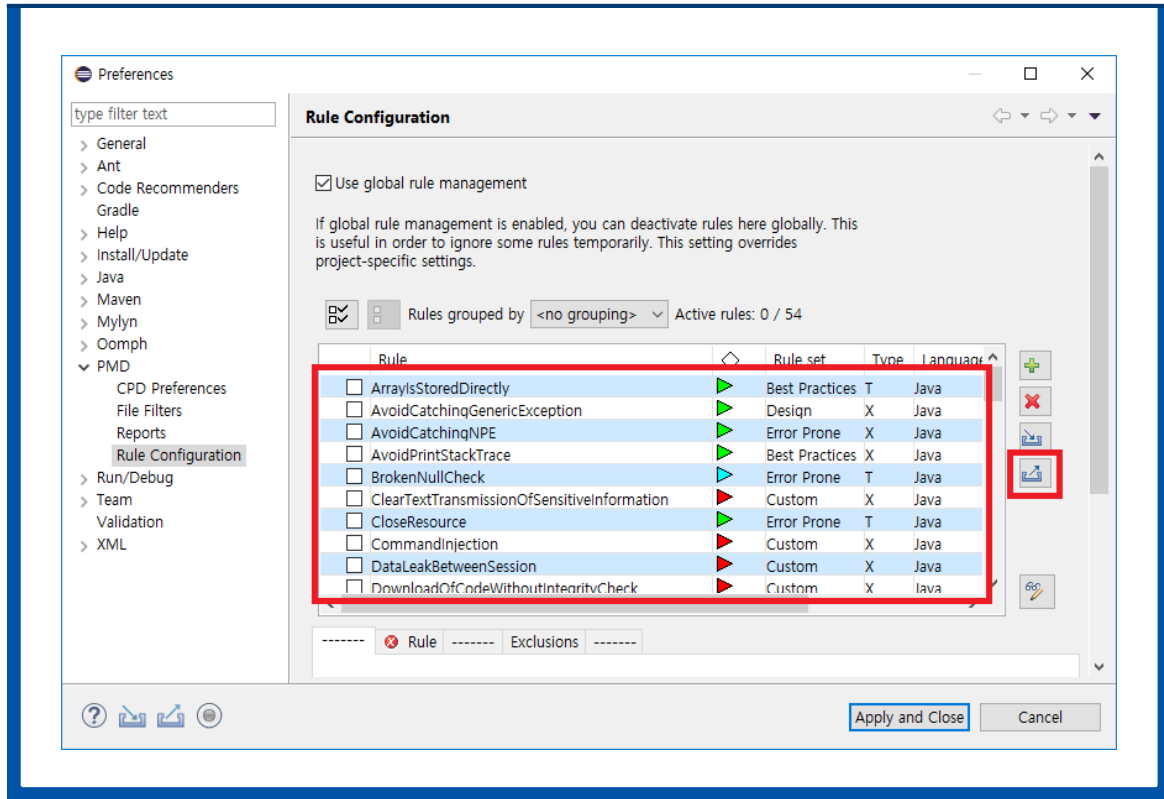
※ 참고 : 룰을 삭제할 때, 체크 박스를 해제하는 것이 아니라 룰을 마우스로 클릭해서 선택해야 한다. 선택된 룰들은 그림과 같이 파란색으로 표기된다.

- ⑥ 설정 창에서 add 버튼을 클릭해 외부에 있는 룰셋 xml 파일을 불러온다. 파일을 선택하면 Import rules 창에 룰셋에 포함된 룰들이 표기된다. 체크박스를 선택하여 원하는 룰들만 포함시킬 수 있다.



※ 참고 : 룰셋을 반영하여도 설정 창에 바로 보이지 않는다. 설정 창에 Apply and Close 버튼을 눌러 적용하고 다시 설정 창을 키면 반영된 것을 확인할 수 있다.

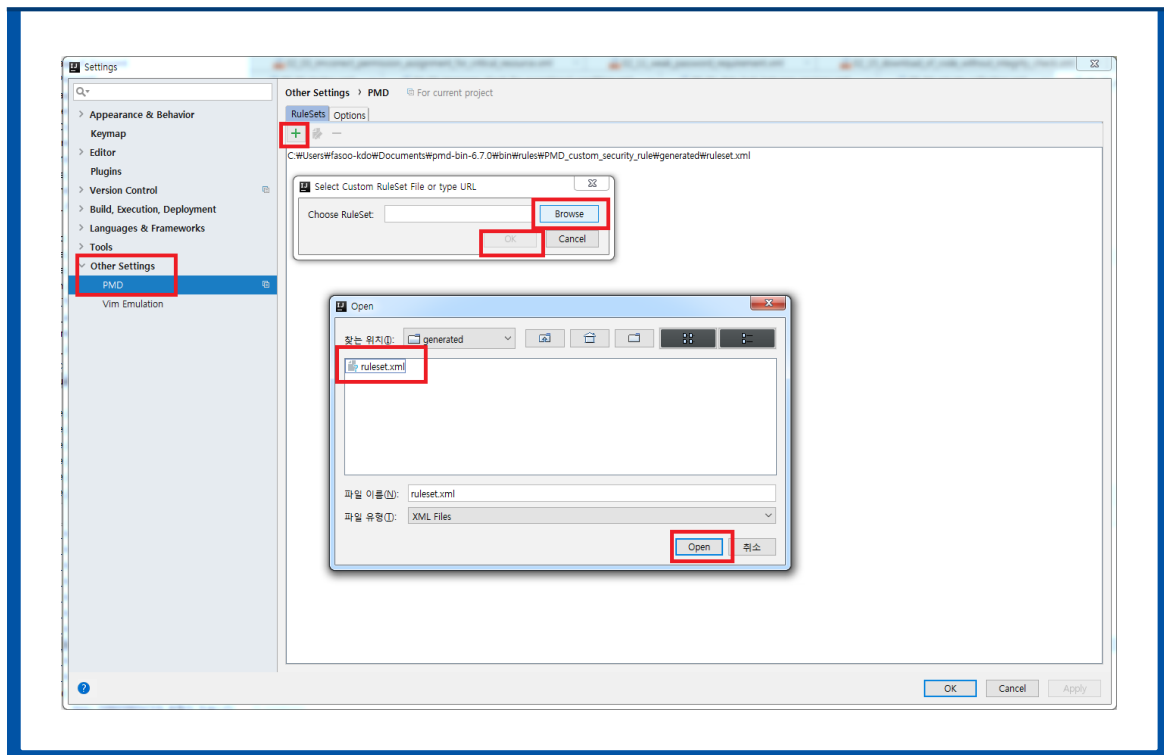
- ⑦ 사용자가 선별한 룰셋을 xml 파일로 내보내는 기능을 제공한다. 따로 룰셋으로 구성하고 싶은 룰들을 선별한다. Export selected rules... 버튼을 클릭해 선별된 룰셋을 파일로 내보낸다.



※ 참고 : 삭제와 마찬가지로 룰을 선별할 때 체크 박스가 아닌 마우스로 클릭하여야 한다.

2. IntelliJ에서 PMD 룰셋 설정

- ❶ Settings – Other Settings – PMD에서 +를 눌러준다.
- ❷ 대화상자에서 Browse를 누르고 ruleset xml파일을 찾아서 열어준다.
- ❸ OK를 클릭하면 ruleset 목록에 올라오게 된다.

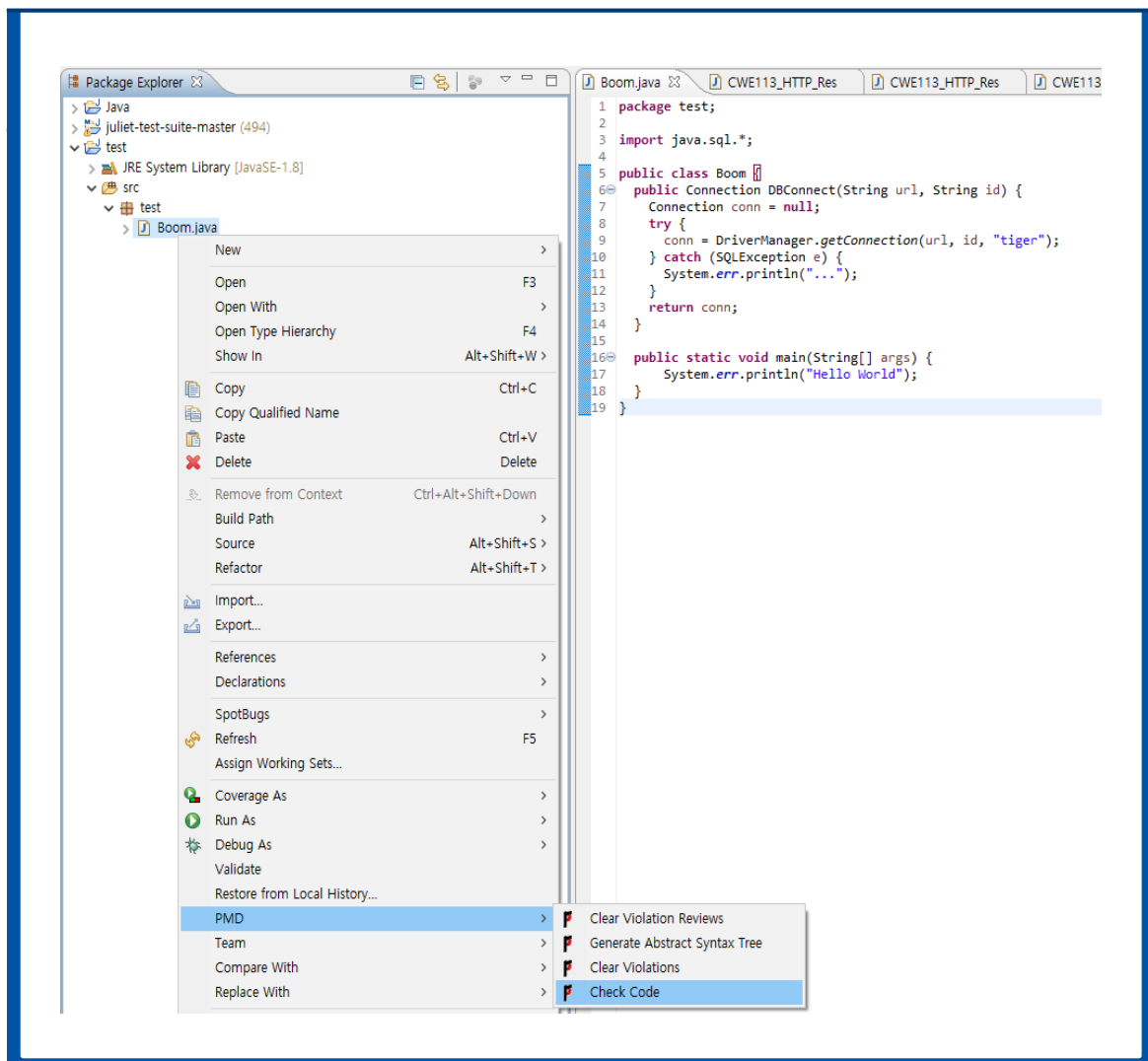


제3절 PMD 검사

다음은 PMD 플러그인을 통해 코드 검사를 수행하는 법을 설명한다.

1. 이클립스에서 PMD 검사 수행

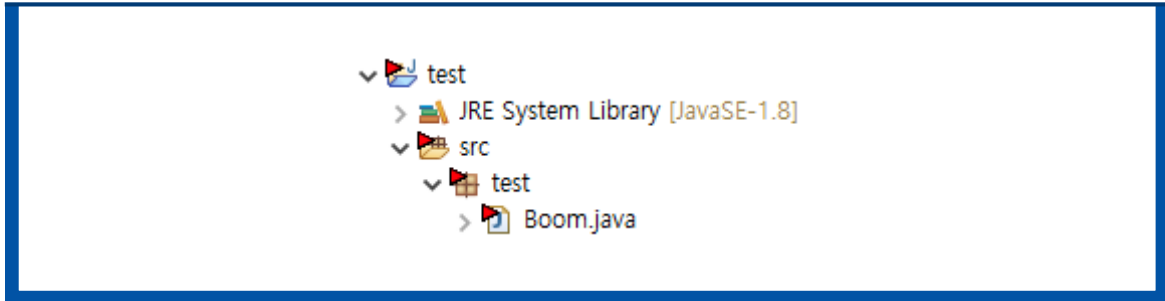
- ❶ 프로젝트를 선택하고 마우스 오른쪽 버튼을 클릭한 뒤 메뉴에서 PMD > Check Code를 선택한다.



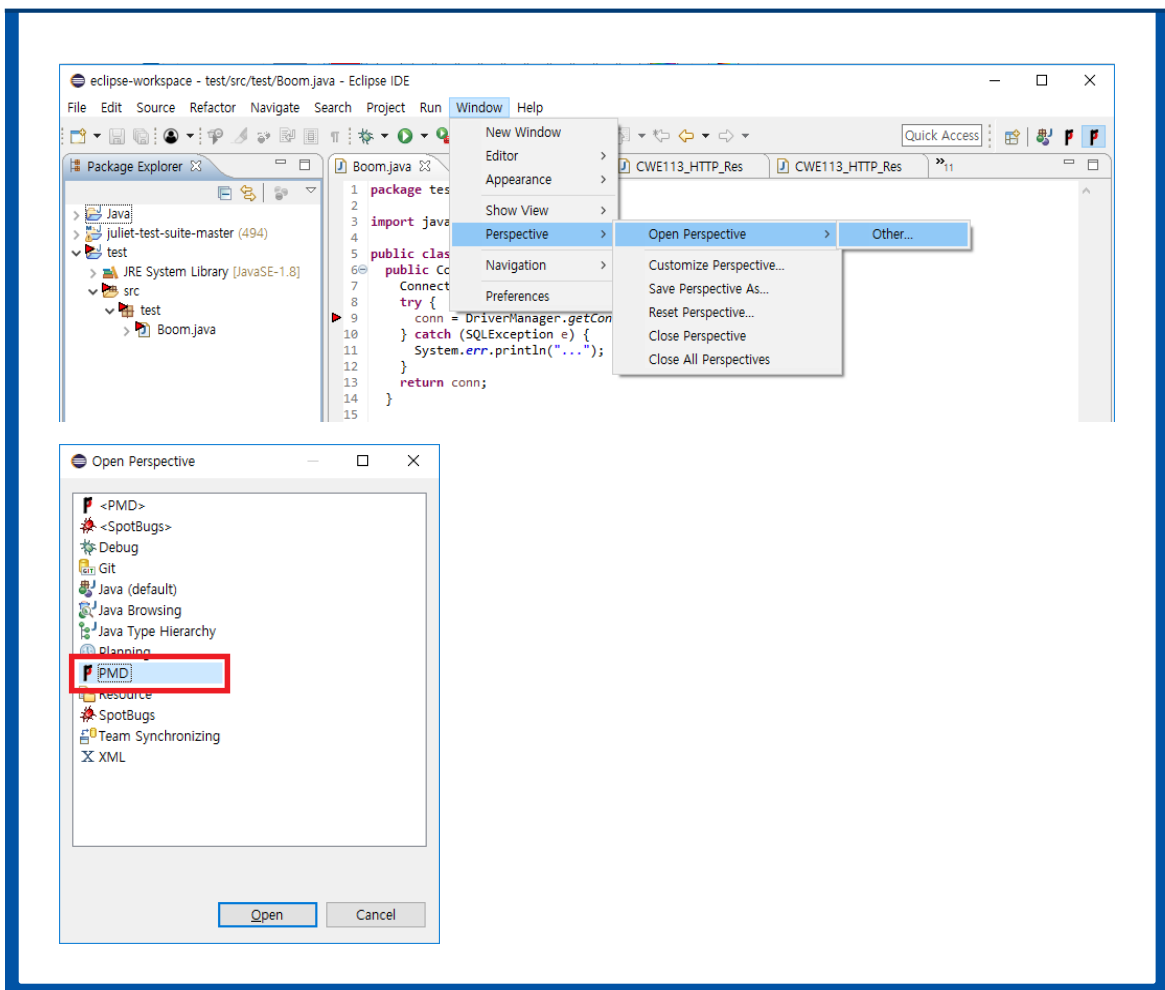
※ 참고1 : 특정 파일을 선택해서 분석하려면, 이클립스의 Package Explorer에서 특정 파일을 선택하고 마우스 오른쪽 버튼을 누르고 분석한다.

※ 참고2 : 기존에 분석한 내용을 깨끗하게 지우고 다시 분석하려면 마우스 오른쪽 버튼을 클릭하고 PMD > Clear Violations를 선택한다.

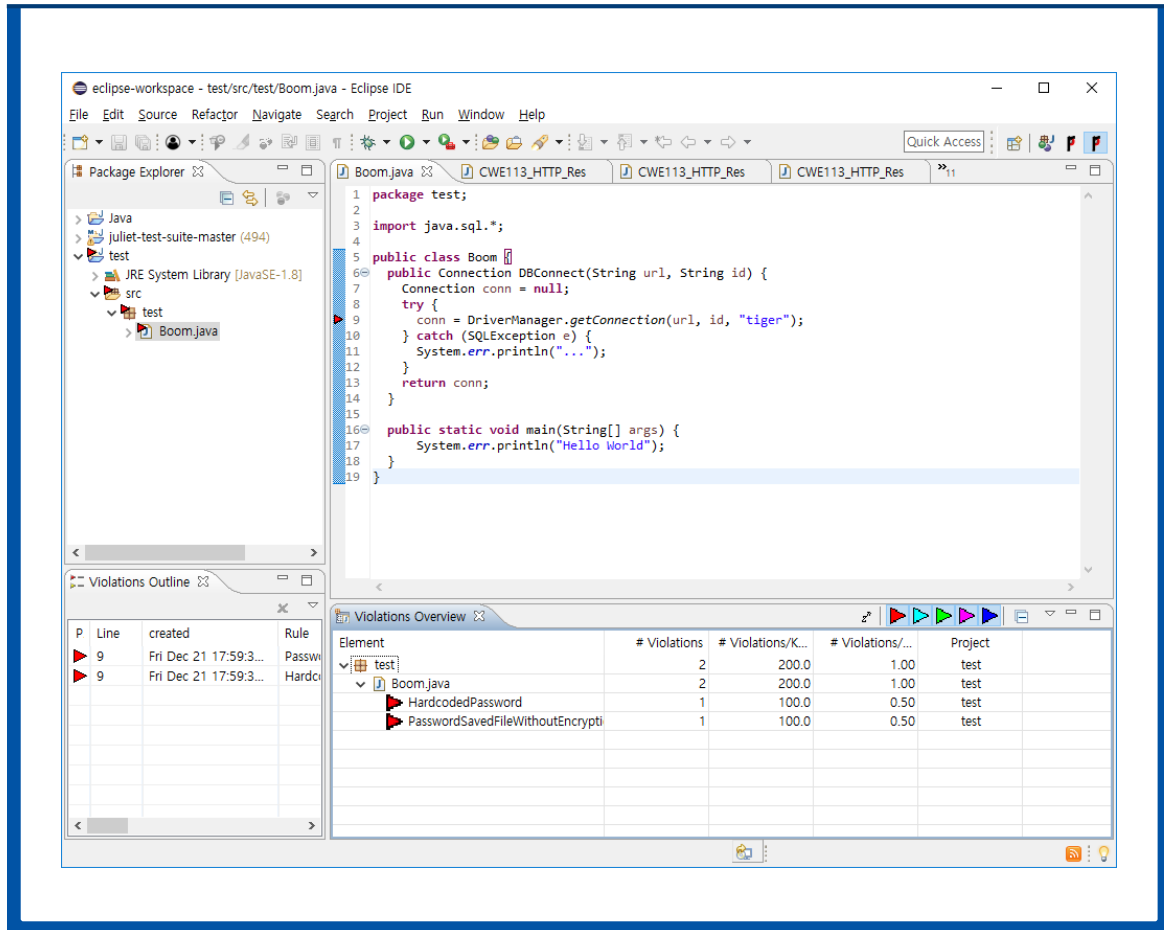
- ❷ 잠시 분석화면이 나타나고 Package Explorer의 프로젝트와 자바 파일 이름 왼쪽에 삼각형이 나타난다. 이는 PMD에서 발견한 위반 사항이 존재한다는 것을 의미한다.



- ❸ 검출 결과를 확인하기 위해서 Windows > Open Perspective > Other...에서 PMD를 선택한다.



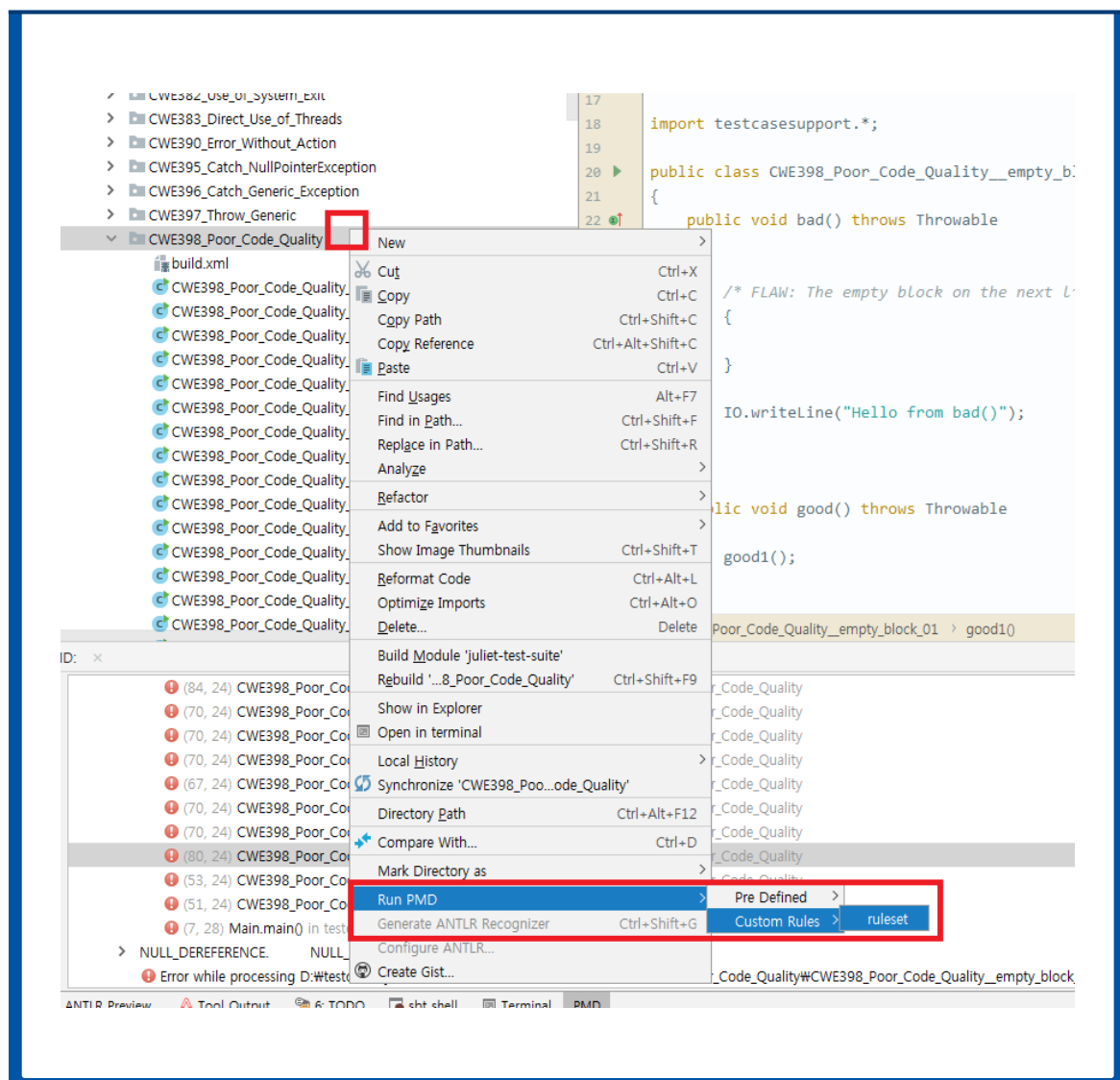
- ④ Package Explorer 하단에 Violation Outline과 Violation Overview 목록이 나타난다. 더블클릭 하면 해당 소스로 바로 이동한다.

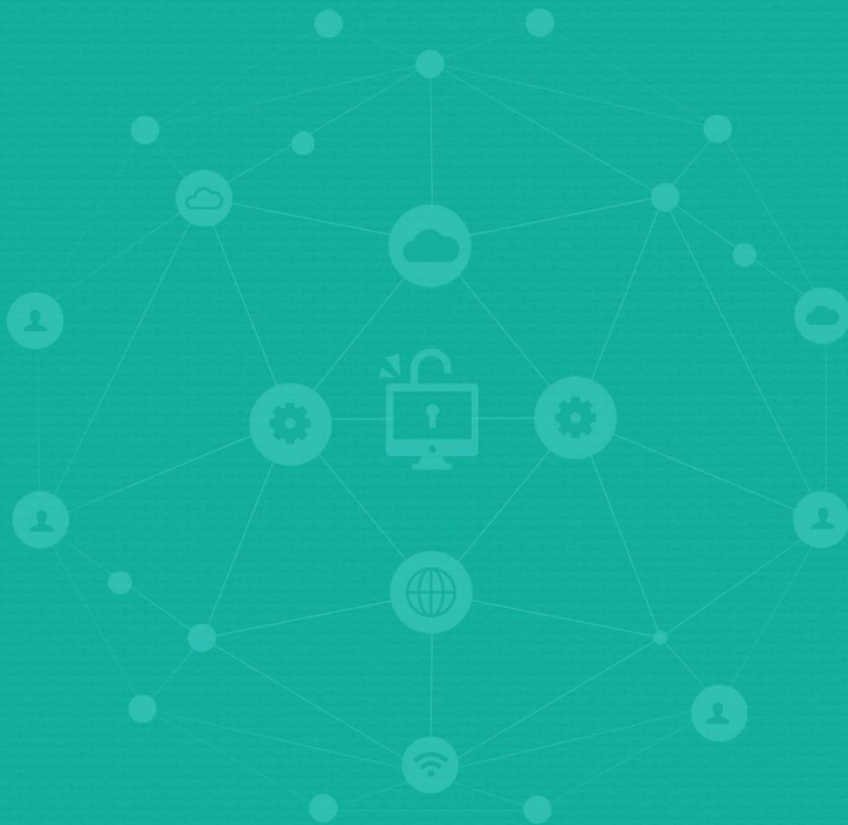


- ⑤ 수정 후 프로젝트를 다시 선택하고 마우스 오른쪽 버튼을 클릭하고 PMD > Check Code를 선택하면 새로 분석해서 남아있는 버그가 존재하는지 확인한다.

2. IntelliJ에서 PMD 검사 수행

- ❶ 분석을 수행하기 원하는 파일 또는 디렉토리에 오른쪽 클릭을 통해 팝업창을 띄운다.
- ❷ Run PMD – Custom Rules – {원하는 ruleset}을 선택하면 분석이 시작된다.
- ❸ 분석결과는 아래 PMD 보조 탭에 나오게 되고 목록을 클릭하면 해당 소스코드 지점으로 이동할 수 있다.





PART

5

제5장 Jenkins 사용 방법

- 제1절 Jenkins 설치
- 제2절 정적분석을 위한 플러그인 설치
- 제3절 Build 관련 공개소프트웨어 연계 설정
- 제4절 Jenkins 실행
- 제5절 Jenkins를 활용한 시큐어코딩 Inspection 활동

PART



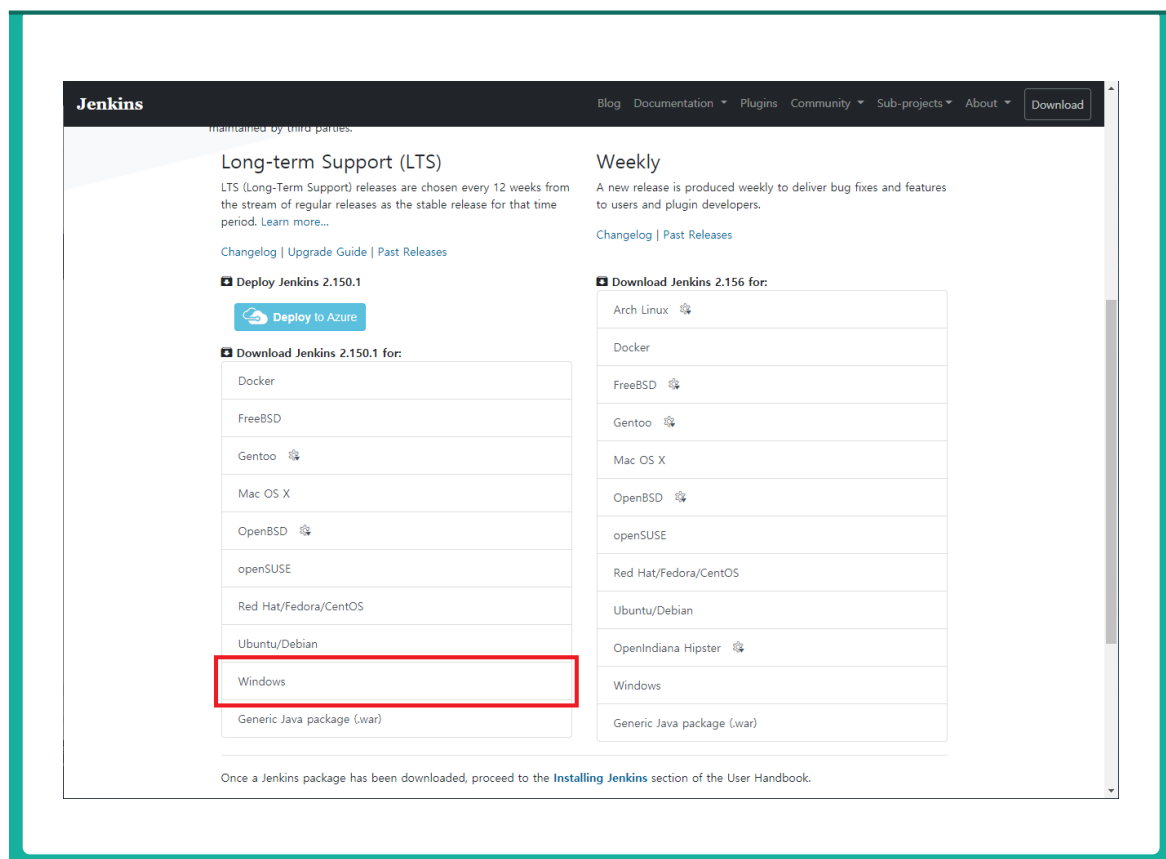
제5장 Jenkins 사용 방법



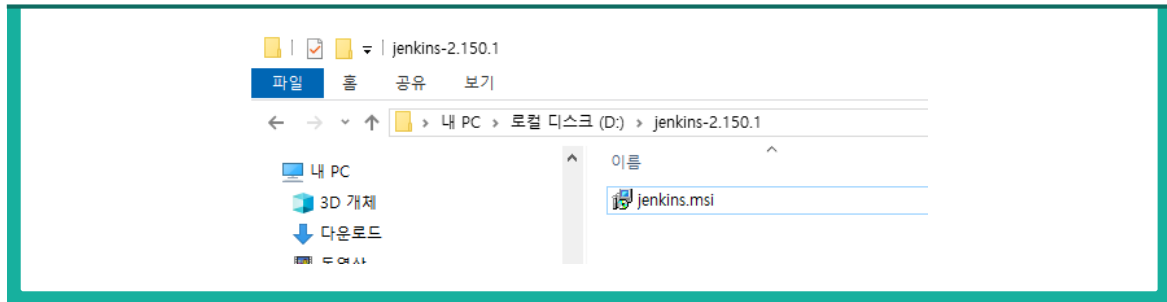
제1절 Jenkins 설치

1. Jenkins 다운로드 및 설치

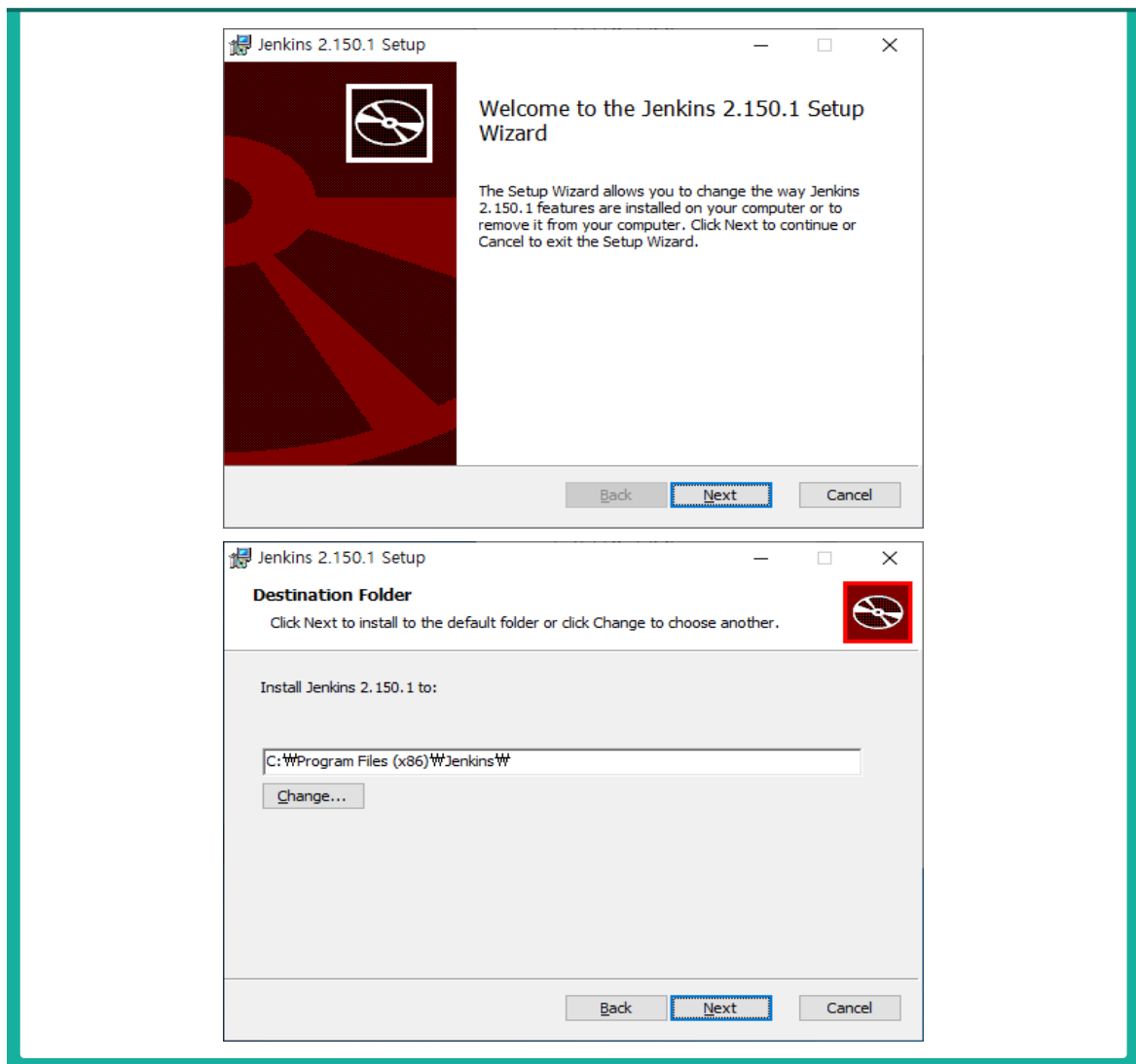
- 1 Jenkins는 <http://jenkins-ci.org/>에서 다운로드 받는다. 최신 버전인 jenkins-2.150.1.zip이 다운로드 된다.

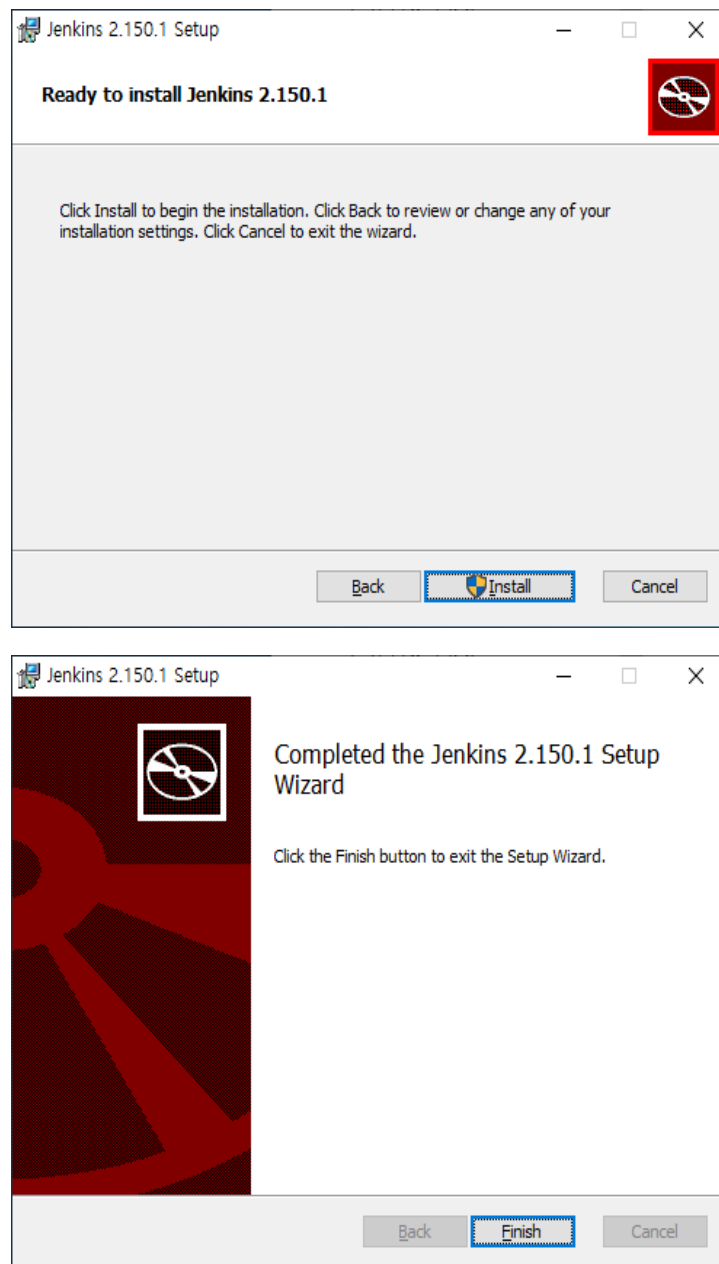


- ❷ 다운받은 Jenkins-2.150.1.zip 파일을 적당한 위치(예 D:\)에 압축을 해제한다.



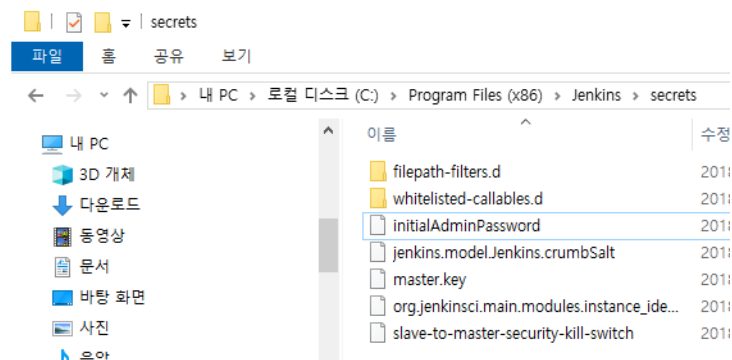
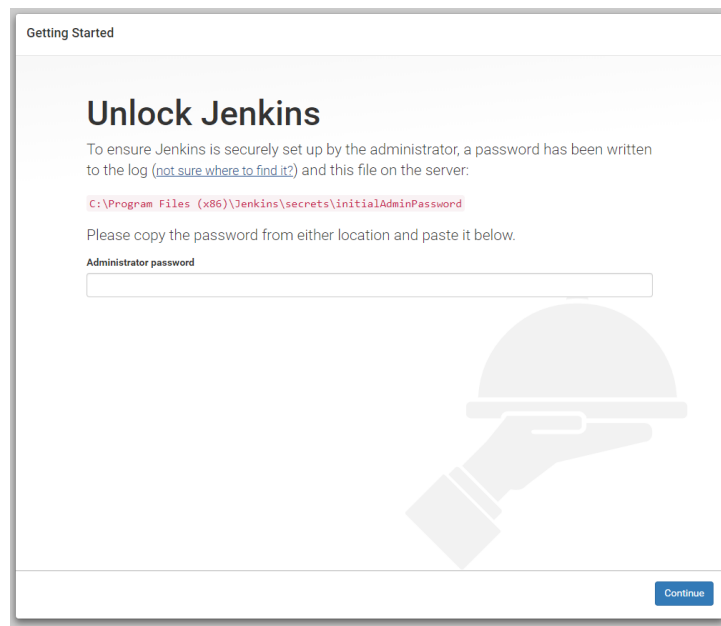
- ❸ Windows 설치 파일(msi)을 실행하여 Jenkins 설치를 진행한다. 설치 시 관리자 권한을 요구한다.



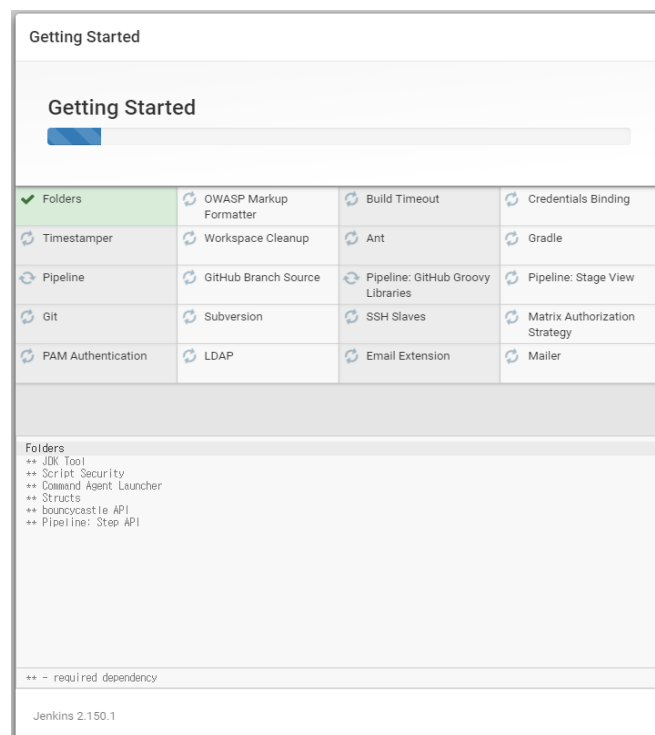
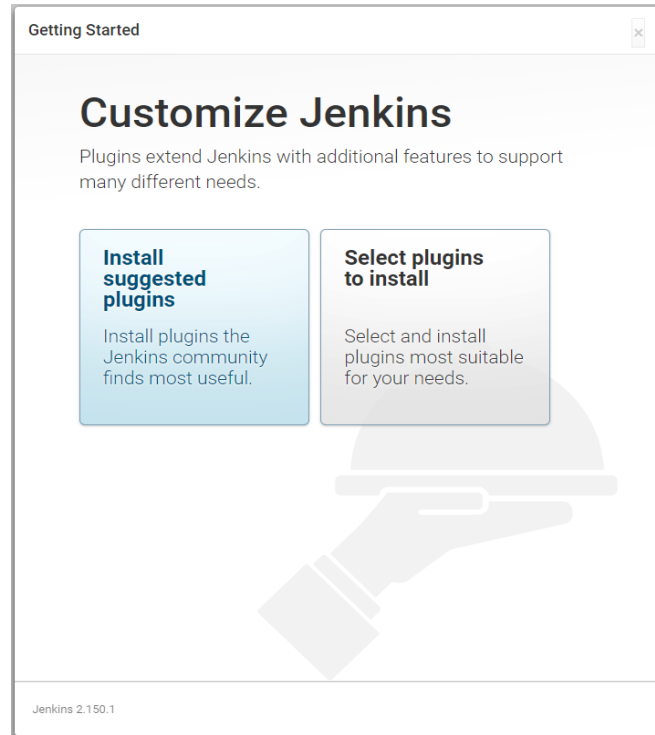


- ④ 설치가 완료되면 자동으로 웹 브라우저가 실행되면서 Jenkins 서버로 연결이 진행된다. Jenkins 서버의 기본 설정 값은 <http://localhost:8080>이다.

- ⑤ Jenkins 서버(웹 브라우저)에서 나머지 설정이 진행된다. 설정을 진행하기 전 Jenkins 서버에 접속한 사용자가 관리자인지 확인한다. 초기 비밀번호는 Jenkins 설치 폴더에 secrets\initialPassword 파일에서 확인할 수 있다. 해당 파일은 Jenkins 서버가 설치된 PC에서만 확인할 수 있으므로 서버 관리자만 비밀번호를 확인할 수 있다.



- ⑥ 올바른 비밀번호를 입력하면 설정화면으로 넘어간다. ‘Install suggested plugins’ 을 선택한다. 주로 사용되는 플러그인들과 함께 설치가 진행된다.



- ⑦ 설치가 완료되면 관리자 계정 설정 페이지가 나온다. 이후에 사용될 계정 정보를 입력하고 Save and Continue 버튼을 클릭한다.

- ⑧ Jenkins 서버 URL을 설정한다. 기본 설정은 <http://localhost:8080/> 이다. 사용할 URL로 변경하고 'Save and Finish' 버튼을 클릭한다.

- ⑨ 설치가 완료되었음을 확인한다. ‘Start using Jenkins’ 버튼을 클릭하여 설정을 마친다. Jenkins 서버에 다시 접속하면 로그인 화면을 확인할 수 있다.

Getting Started

Jenkins is ready!

Your Jenkins setup is complete.

[Start using Jenkins](#)



Welcome to Jenkins!

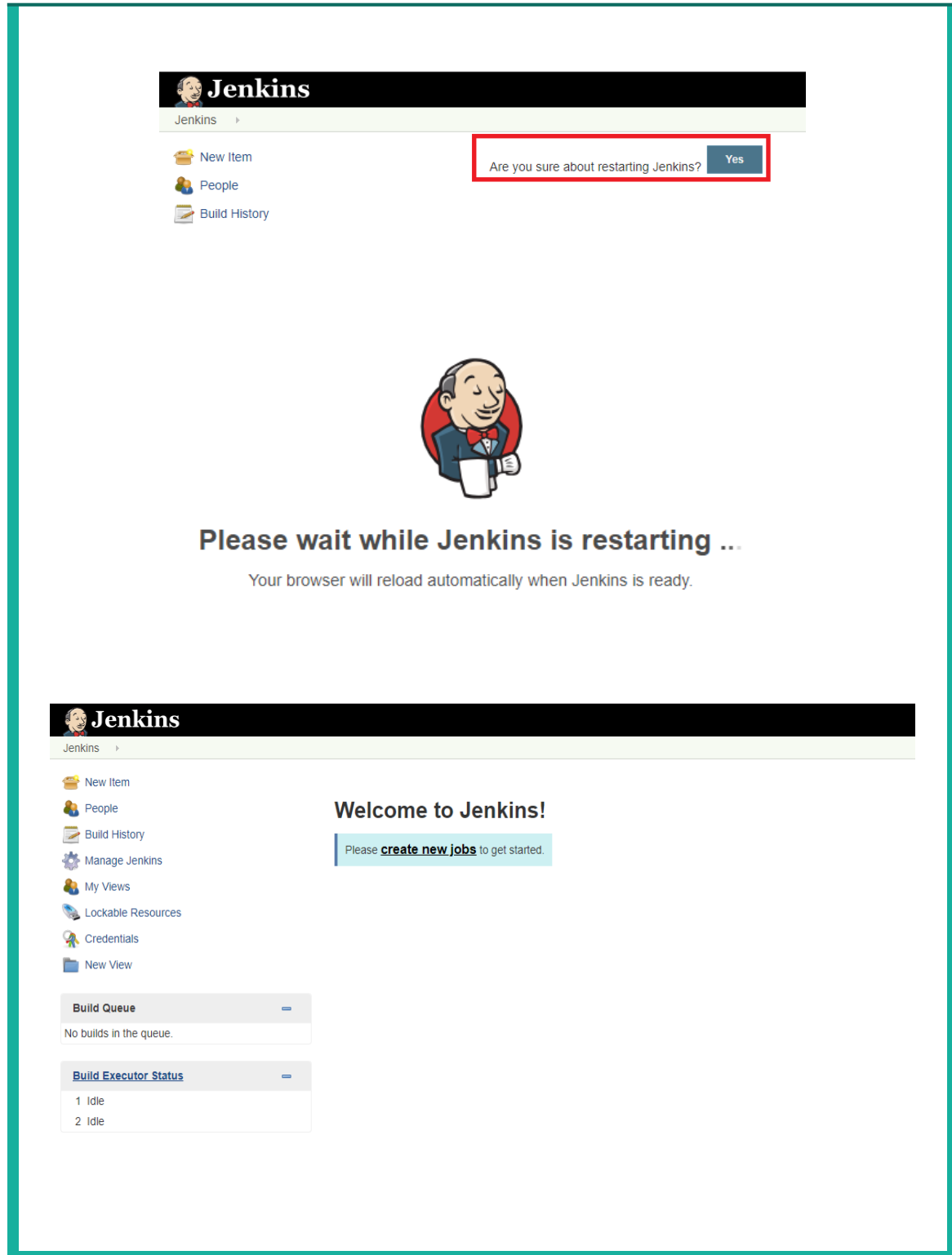
[Sign in](#)

☐ Keep me signed in

※ 참고: Jenkins에서 로그인을 진행하면 빈 페이지가 출력된다. 초기 설정 페이지랑 Jenkins 메인 URL이 동일하여 발생하는 버그로 추정되며 Jenkins를 재실행하면 해결된다.

※ 참고2: 빈 화면이 출력되는 것은 Jenkins 서버가 살아있음을 의미한다.

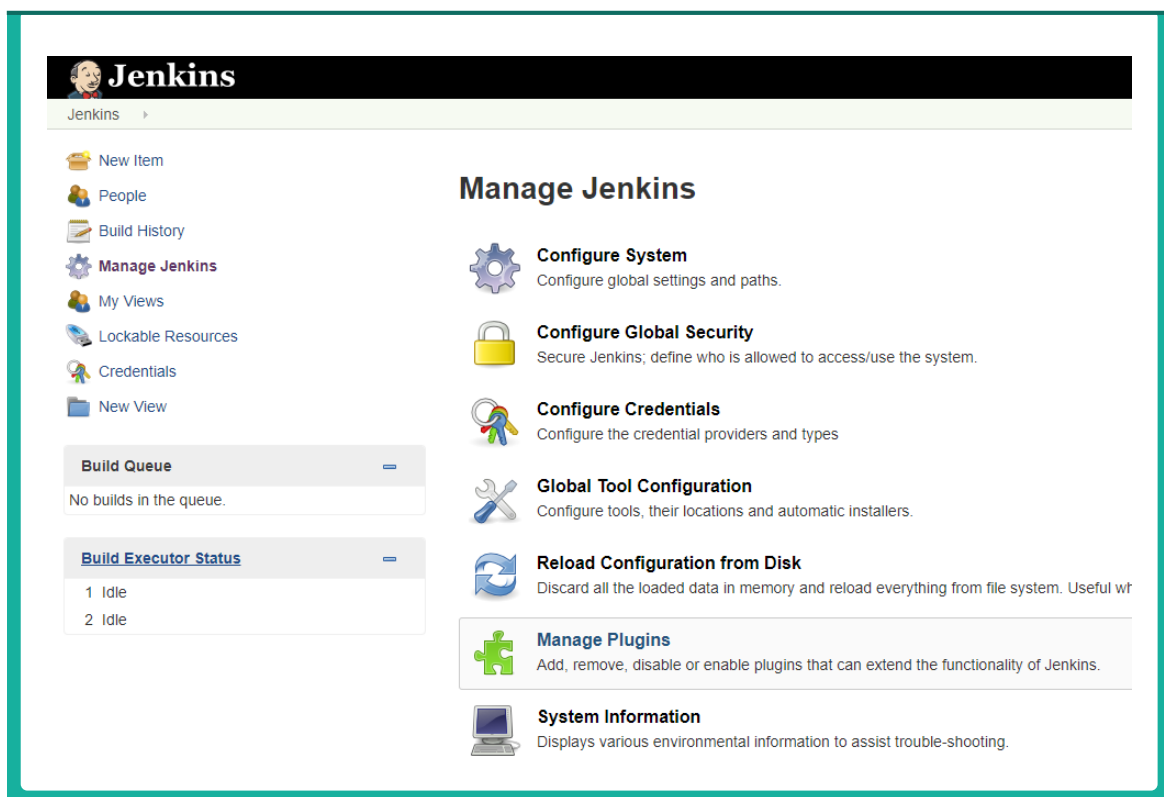
- ⑩ 주소창에 '<http://localhost:8080/restart>'을 입력하고 재실행을 묻는 질문에 'Yes' 버튼을 클릭하여 진행한다. 이후 로그인을 진행하면 정상적으로 Jenkins 메인 화면이 출력되는 것을 확인할 수 있다.



제2절 정적분석을 위한 플러그인 설치

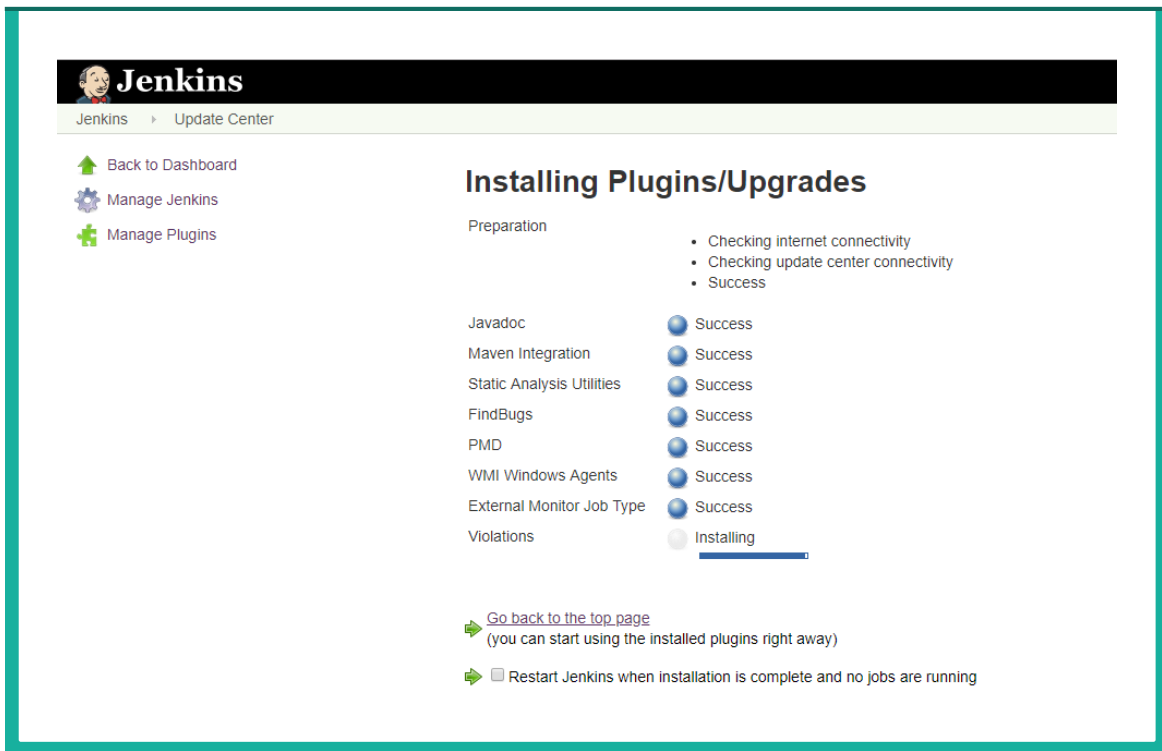
Jenkins의 가장 큰 장점은 플러그인을 통한 확장성이다. 여기에서는 Secure Coding 을 지원하는 플러그인의 목록과 이를 설치하는 과정을 설명하도록 한다.

- ① 왼쪽의 메뉴에서 Manage Jenkins를 클릭한다. 나타나는 메뉴에서 Manage Plugins 를 선택한다.



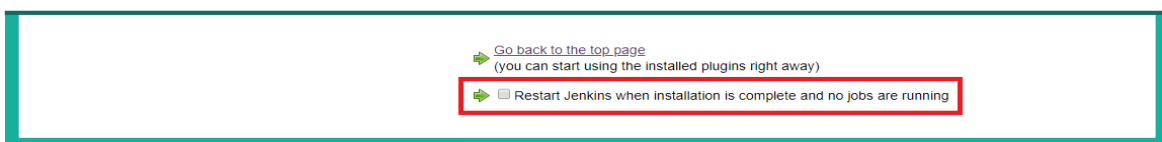
- ② Plugin Manager 화면에서 Available 탭을 선택한다. 오른쪽 상단에 Filter에 다음 목록을 참조하여 설치하려는 Plugin의 이름을 입력한다.

플러그인 이름	목 적
PMD	PMD 분석결과를 확인할 수 있는 플러그인
FindBugs Plug-in	FindBugs, FindSecurityBugs 분석결과를 확인할 수 있는 플러그인
Violations	PMD, FindBugs등 다양한 정적분석 도구의 결과를 통합해서 볼 수 있는 플러그인



※ 참고: 의존성이 있는 플러그인은 자동으로 연관된 플러그인들이 설치된다. 위의 그림은 PMD, FindBugs Plug-in, Violations만 선택하고 설치를 진행하였다. Static Analysis Utilities 등 의존성 플러그인들이 자동으로 설치되는 것을 확인할 수 있다.

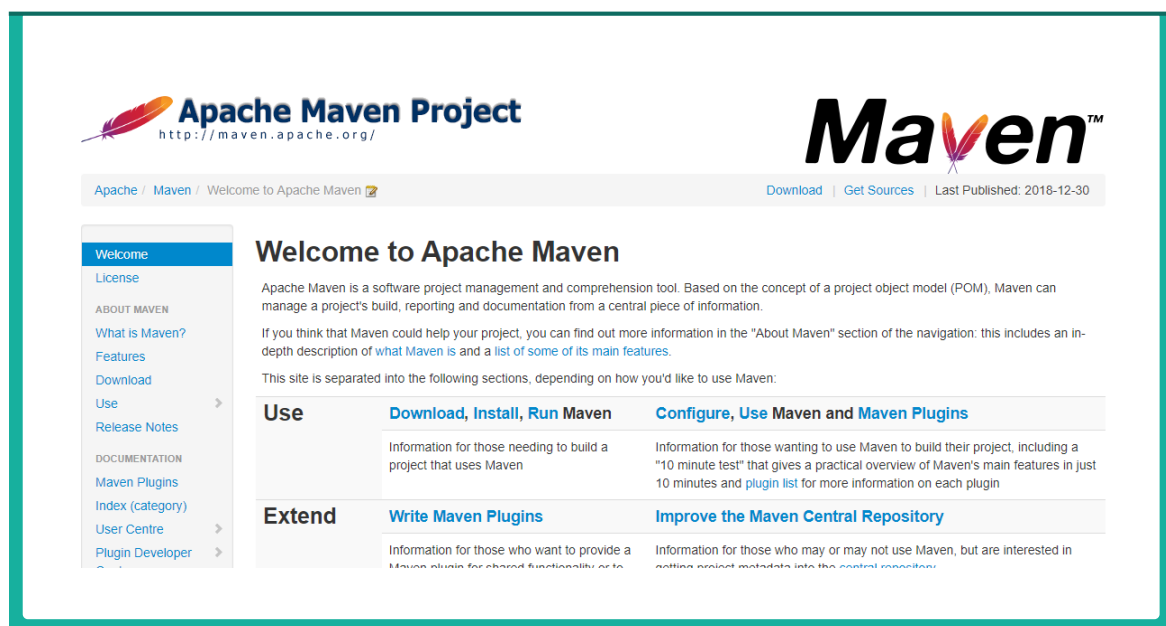
- ③ 플러그인 설치가 완료 된 후, ‘Restart Jenkins when installation is complete and no jobs are running’의 체크박스를 클릭하여 Jenkins를 재실행한다



- ④ Jenkins가 재실행되며 브라우저에 로그인 페이지가 출력된다. 로그인하여 플러그인 관리 페이지로 이동한 뒤, Installed 탭에서 플러그인이 올바르게 설치되었는지 확인한다.

제3절 Build 관련 공개소프트웨어 연계 설정

소프트웨어 개발에서 Build는 원본 소스를 가져와서 컴파일, 테스트, 패키징 작업을 진행하는 것을 의미한다. Jenkins에서는 매 Build 마다 정적분석을 실시하고 그 결과를 분석할 수 있다. 이를 위해 Build에 관련된 공개소프트웨어(Maven, Ant 등)와 연계를 해야한다. 본 가이드에서는 Maven을 사용하여 연계하는 방법을 설명한다. Maven에 대한 자세한 내용은 <https://maven.apache.org/>를 참조한다. 또한, 전자정부 표준 프레임워크 기반으로 SW개발을 하는 경우에는, 표준 프레임워크센터(<http://www.egovframe.go.kr>)에서 제공하는 개발가이드를 참조할 수 있다.



본 가이드에서는 maven 설정파일인 pom.xml에 Secure Coding 점검 툴을 추가하는 것에 대해서 설명하도록 한다. pom.xml은 Project Object Model의 약자로 maven 프로젝트에 관련된 정보를 갖고 있다. 아래는 pom.xml 파일의 예이다.

```

1 <project xmlns="http://maven.apache.org/POM/4.0.0"
2   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
4
5   <modelVersion>4.0.0</modelVersion>
6   <groupId>gov.nist.samate</groupId>
7   <artifactId>juliet-test-suite</artifactId>
8
9   <packaging>jar</packaging>
10  <version>1.2.0</version>
11
12  <name>Juliet Test Suite</name>
13
14  <description>A collection of test cases in the Java language. It contains examples for 112 different CWEs.</description>
15
16  <properties>
17    <!-- Encoding configuration -->
18    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
19    <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
20  </properties>
21
22  <build>
23    <sourceDirectory>src</sourceDirectory>
24    <plugins>
25      <!-- Force the use the compatibility to Java 7 -->
26      <plugin>
27        <artifactId>maven-compiler-plugin</artifactId>
28        <version>2.3.2</version>
29        <configuration>
30          <source>1.7</source>
31          <target>1.7</target>
32        </configuration>
33      </plugin>
34    </plugins>
35  </build>
36
37  <dependencies>
38    <!-- The list of dependencies is derived from the "lib" directory -->
39
40    <dependency>
41      <groupId>commons-codec</groupId>
42      <artifactId>commons-codec</artifactId>
43      <version>1.10</version>
44    </dependency>
45    <dependency>
46      <groupId>commons-lang</groupId>
47      <artifactId>commons-lang</artifactId>
48      <version>2.6</version>
49    </dependency>
50    <dependency>
51      <groupId>javax.mail</groupId>
52      <artifactId>mail</artifactId>
53      <version>1.4.7</version>
54    </dependency>
55    <dependency>
56      <groupId>javax.servlet</groupId>
57      <artifactId>javax.servlet-api</artifactId>
58      <version>4.0.0</version>
59      <scope>provided</scope>
60    </dependency>
61  </dependencies>
62
63 </project>

```


1. Spotbugs, FindSecurityBugs 추가

Spotbugs와 FindSecurityBugs를 추가하려면 plugins 태그 안에 다음의 내용을 추가하면 된다.

```
<plugins>
  <!-- Force the use the compatibility to Java 7 -->
  <plugin>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>2.3.2</version>
    <configuration>
      <source>1.7</source>
      <target>1.7</target>
    </configuration>
  </plugin>
  <!-- SpotBugs Static Analysis -->
  <plugin>
    <groupId>com.github.spotbugs</groupId>
    <artifactId>spotbugs-maven-plugin</artifactId>
    <version>3.1.1</version>
    <configuration>
      <effort>Max</effort>
      <threshold>Low</threshold>
      <failOnError>true</failOnError>
      <includeFilterFile>${session.executionRootDirectory}/spotbugs-security-include.xml</includeFilterFile>
      <excludeFilterFile>${session.executionRootDirectory}/spotbugs-security-exclude.xml</excludeFilterFile>
      <plugins>
        <!-- Install findsecbugs plugin -->
        <plugin>
          <groupId>com.h3xstream.findsecbugs</groupId>
          <artifactId>findsecbugs-plugin</artifactId>
          <version>LATEST</version> <!-- Auto-update to the latest stable -->
        </plugin>
      </plugins>
    </configuration>
  </plugin>
</plugins>
```

2. PMD 추가

PMD도 Spotbugs 설정과 유사하게 plugins 아래에 추가한다.

```
<!-- PMD Static Analysis -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-pmd-plugin</artifactId>
  <version>3.11.0</version>
  <configuration>UTF-8</configuration>
</plugin>
```

3. Maven 실행

pom.xml이 위치하고 있는 디렉토리에서 maven을 실행한다. 이 때 maven의 추가 명령어를 통해서 Spotbugs, PMD를 실행하고 그 분석결과를 얻을 수 있다.

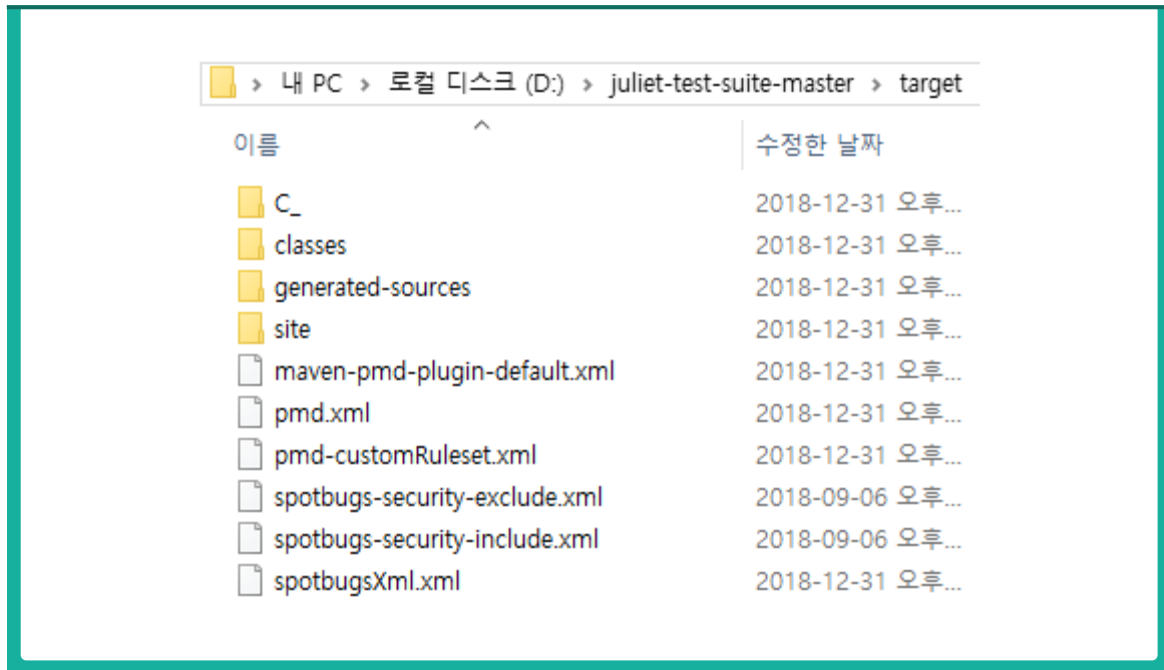
Spotbugs, FindSecurityBugs를 실행하고 분석결과를 xml로 받으려면 spotbugs: spotbugs 명령어를 입력하면 된다. PMD를 실행하고 분석결과를 xml로 받으려면 pmd:pmd의 명령어를 입력하면 된다. 두 개를 모두 실행하려면 mvn spotbugs:spotbugs pmd:pmd 명령어를 입력하고, 그 결과는 다음과 같다.

명령어	설 명
mvn spotbugs:spotbugs	Spotbugs와 FindSecurityBugs를 실행하고 분석결과를 xml 파일(spotbugsXml.xml)로 저장
mvn pmd:pmd	PMD를 실행하고 분석결과를 xml 파일(pmd.xml)로 저장
mvn findbugs:findbugs pmd:pmd	Spotbugs와 FindSecurityBugs와 PMD를 실행하고 분석결과를 xml 파일(spotbugsXml.xml, pmd.xml)로 저장

```

PS D:\juliet-test-suite-master> mvn pmd:pmd
[INFO] Scanning for projects...
[INFO] -----< gov.nist.samate:juliet-test-suite >-----
[INFO] Building Juliet Test Suite 1.2.0
[INFO] -----[ jar ]-----
[INFO] --- maven-pmd-plugin:3.11.0:run (default-cli) @ juliet-test-suite ---
[WARNING] Unable to locate Source XRef to link to - DISABLED
[INFO] BUILD SUCCESS
[INFO] Total time: 4.786 s
[INFO] Finished at: 2018-12-31T15:22:31+09:00
[INFO]
PS D:\juliet-test-suite-master>
  
```

그리고, maven 실행 후 결과 xml파일(spotbugsXml.xml, pmd.xml)이 target 폴더 아래 다음과 같이 생성된다.



※ 파일 설명

- spotbugsXml.xml : findbugs 분석 결과가 저장되는 파일
- spotbugs-security-include.xml: spotbugs의 룰을 설정하는 파일
- spotbugs-security-exclude.xml: spotbugs의 제외할 룰을 설정하는 파일
- pmd.xml : pmd 분석 결과가 저장되는 파일
- pmd-customRuleset.xml : pmd의 룰을 설정하는 파일

4. Spotbugs, FindSecurityBugs, PMD 룰(Rule) 파일 상세 설정 및 반영

위에서 가이드한 내용으로 수행하면 모든 Spotbugs, FindSecurityBugs, PMD 룰로 분석한 결과가 나온다. 여기에서는 시큐어코딩 관련 룰을 선별적으로 구성하는 방법을 설명한다.

❶ Spotbugs /FindSecurityBugs 룰(Rule) 설정

Spotbugs /FindSecurityBugs에서는 Filter 개념을 가지고 있다. Include Filter는 포함해야 할 규칙을, exclude Filter는 제외해야 할 룰을 담게 된다. 다음은 Secure Coding 과 관련된 규칙을 포함하는 include Filter인 spotbugs-security-include.xml 파일과 제외하는 내용을 담을 수 있는 spotbugs-security-exclude.xml 파일이다. 자세한 사항은 <https://spotbugs.readthedocs.io/en/latest/filter.html>를 참조한다.

spotbugs-security-include.xml

```

1  <FindBugsFilter>
2      <Match>
3          <Bug category="SECURITY"/>
4          <Bug pattern="
5              SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE,
6              SQL_PREPARED_STATEMENT_GENERATED_FROM_NONCONSTANT_STRING,
7              SQL_INJECTION, SQL_INJECTION_TURBINE, SQL_INJECTION_HIBERNATE,
8              SQL_INJECTION_JDO, SQL_INJECTION_JPA,
9              SQL_INJECTION_SPRING_JDBC,
10             SQL_INJECTION_JDBC,
11             SCALA_SQL_INJECTION_SLICK,
12             SCALA_SQL_INJECTION_ANORM,
13             SQL_INJECTION_ANDROID,
14             SERVLET_PARAMETER,
15             PATH_TRAVERSAL_IN,
16             PATH_TRAVERSAL_OUT,
17             SCALA_PATH_TRAVERSAL_IN,
18             FILE_UPLOAD_FILENAME,
19             PT_RELATIVE_PATH_TRAVERSAL,
20             PT_ABSOLUTE_PATH_TRAVERSAL,
21             XSS_REQUEST_PARAMETER_TO_SERVLET_WRITER,
22             XSS_REQUEST_PARAMETER_TO_JSP_WRITER,
23             XSS_REQUEST_PARAMETER_TO_SEND_ERROR,
24             XSS_REQUEST_WRAPPER,
25             JSP_JSTL_OUT,
26             XSS_JSP_PRINT,
```

```

27      XSS_SERVLET,
28      SCALA_XSS_TWIRL,
29      SCALA_XSS_MVC_API,
30      COMMAND_INJECTION,
31      UNVALIDATED_REDIRECT,
32      PLAY_UNVALIDATED_REDIRECT,
33      SPRING_UNVALIDATED_REDIRECT,
34      SERVLET_CONTENT_TYPE,
35      XPATH_INJECTION,
36      LDAP_ANONYMOUS,
37      LDAP_INJECTION,
38      SPRING_CSRF_PROTECTION_DISABLED,
39      SPRING_CSRF_UNRESTRICTED_REQUEST_MAPPING,
40      HRS_REQUEST_PARAMETER_TO_COOKIE,
41      HTTP_RESPONSE_SPLITTING,
42      HRS_REQUEST_PARAMETER_TO_HTTP_HEADER,
43      INTEGER_OVERFLOW,
44  }
45      SERVLET_SERVER_NAME,
46      FORMAT_STRING_MANIPULATION,
47      JAXWS_ENDPOINT,
48      JAXRS_ENDPOINT,
49      EXTERNAL_CONFIG_CONTROL,
50      INAPPROPRIATE_AUTHORIZATION,
51      DP_INSIDE_DO_PRIVILEGED,
52      DP_CREATE_CLASSLOADER_INSIDE_DO_PRIVILEGED,
53      INSECURE_SMTP_SSL,
54      ANDROID_EXTERNAL_FILE_ACCESS,
55      HTTPONLY_COOKIE,
56      INSECURE_COOKIE,
57      COOKIE_PERSISTENT,
58      CUSTOM_MESSAGE_DIGEST,
59      DES_USAGE,
60      TDES_USAGE,
61      RSA_NO_PADDING,
62      ECB_MODE,
63      PADDING_ORACLE,
64      HAZELCAST_SYMMETRIC_ENCRYPTION,
65      WEAK_MESSAGE_DIGEST_MD5,
66      WEAK_MESSAGE_DIGEST_SHA1,
67      UNENCRYPTED_SOCKET,
68      UNENCRYPTED_SERVER_SOCKET,
69      SENSITIVE_CLEARTEXT,
70      COOKIE_USAGE,
71      DMI_CONSTANT_DB_PASSWORD,
72      DMI_EMPTY_DB_PASSWORD,
73      HARD_CODE_PASSWORD,
74      BLOWFISH_KEY_SIZE,
75      RSA_KEY_SIZE,
76      DMI_RANDOM_USED_ONLY_ONCE,
77      PREDICTABLE_RANDOM,
78      PREDICTABLE_RANDOM_SCALA,
79      HARD_CODE_KEY,
80      INSUFFICIENT_SESSION_EXPIRATION,
81      INFO_EXPOSURE_BY_COMMENTS_SERVLET,
82      UNSALTED_ONE_WAY_HASH,
83      STRUTS_FILE_DISCLOSURE,
84      SPRING_FILE_DISCLOSURE,
85      URLCONNECTION_SSRF_FD,
      WL_USING_GETCLASS_RATHER_THAN_CLASS_LITERAL,

```

```

86      RACE_CONDITION_FILE_IO,
87      IL_INFINITE_RECURSIVE_LOOP,
88      INFINITE_LOOP_PATTERN,
89      INFORMATION_EXPOSURE_THROUGH_AN_ERROR_MESSAGE,
90      ERROR_CONDITION_WITHOUT_ACTION,
91      USE_OF_NULL_POINTER_EXCEPTION_CATCH,
92      USE_OF_GENERIC_EXCEPTION_CATCH,
93      NP_ALWAYS_NULL,
94      NP_ALWAYS_NULL_EXCEPTION,
95      NP_GUARANTEED_DEREF,
96      NP_GUARANTEED_DEREF_ON_EXCEPTION_PATH,
97      NP_NULL_ON_SOME_PATH,
98      NP_NULL_ON_SOME_PATH_EXCEPTION,
99      NP_NULL_PARAM_DEREF_ALL_TARGETS_DANGEROUS,
100     NP_NULL_PARAM_DEREF_NONVIRTUAL,
101     NULL_CHECK_AFTER_DEREF,
102     UL_UNRELEASED_LOCK,
103     UL_UNRELEASED_LOCK_EXCEPTION_PATH,
104     SWL_SLEEP_WITH_LOCK_HELD,
105     INCOMPLETE_CLEANUP,
106     UR_UNINIT_READ,
107     UR_UNINIT_READ_CALLED_FROM_SUPER_CONSTRUCTOR,
108     UWF_FIELD_NOT_INITIALIZED_IN_CONSTRUCTOR,
109     SERVLET_SESSION_ID,
110     LEFTOVER_DEBUG_CODE,
111     REQUESTDISPATCHER_FILE_DISCLOSURE,
112     MS_EXPOSE_REP,
113     EI_EXPOSE_REP,
114     EI_EXPOSE_REP2,
115     DM_RUN_FINALIZERS_ON_EXIT,
116     DEPRECATED_API_RUNTIME_EXIT,
117     DEPRECATED_API_SYSTEM_EXIT
118     "
119   />
120 </Match>
121 </FindBugsFilter>

```

spotbugs-security-exclude.xml

```

1 <FindBugsFilter>
2 </FindBugsFilter>

```

② PMD 룰(Rule) 설정

PMD에 적용하는 룰셋은 별도의 xml 파일로 구성해서 ruleset으로 설정할 수 있다.

다음은 PMD 룰셋을 담은 pmd-customRuleset.xml 파일이다. 자세한 내용은 https://pmd.github.io/pmd/pmd_userdocs_making_rulesets.html을 참조한다.

※ 제6장 사용자 정의 룰(Rule) 작성 및 오답 관리에서 작성한 pmd-customRuleset.xml 파일을 사용한다.

```

1  <?xml version="1.0"?>
2
3  <ruleset name="Custom Rules"
4      xmlns="http://pmd.sourceforge.net/ruleset/2.0.0"
5      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
6      xsi:schemaLocation="http://pmd.sourceforge.net/ruleset/2.0.0 http://pmd.sourceforge.net/ruleset_2_0_0.xsd">
7      <description>KISA Security Rule</description>
8      <rule ref="category/java/bestpractices.xml/ArrayIsStoredDirectly"/>
9      <rule ref="category/java/design.xml/AvoidCatchingGenericException"/>
10     <rule ref="category/java/errorprone.xml/AvoidCatchingNPE"/>
11     <rule ref="category/java/bestpractices.xml/AvoidPrintStackTrace"/>
12     <rule ref="category/java/errorprone.xml/BrokenNullCheck"/>
13     <rule ref="category/java/errorprone.xml/CloseResource"/>
14     <rule ref="category/java/errorprone.xml/EmptyCatchBlock"/>
15     <rule ref="category/java/bestpractices.xml/MethodReturnsInternalArray"/>
16     <rule ref="category/java/errorprone.xml/MisplacedNullCheck"/>
17     <rule name="SQLInjection"
18         language="java"
19         message="SQL_INJECTION의 위험이 있습니다."
20         class="net.sourceforge.pmd.lang.rule.XPathRule">
21         <!-- externalInfoUrl="TODO"-->
22         <description>
23             SQL_INJECTION의 위험이 있습니다.
24         </description>
25         <priority>1</priority>
26         <properties>
27             <property name="version" value="1.0 compatibility" />
28             <property name="xpath">
29                 <value>
30                     <![CDATA[
31
32                     //PrimaryExpression[
33                         (: Detect a usage of execute() method :)
34                         (: execute() 메소드 사용 탐지 :)
35                         PrimaryPrefix/Name[contains(@Image, 'execute')]
36                         and
37                         PrimarySuffix/Name
38                         and
39                         (: If a readLine or getParameter exists before the execute in the code block in which the node satisfying t
40                         (: 위의 조건을 만족하는 노드가 포함된 코드 블록에서 execute 함수 이전에 readLine 또는 getParameter가 사용된 경우
41                         (count(ancestor::BlockStatement/preceding-sibling::BlockStatement//Name[(contains(@Image, "readLine"))])>0
42                         or
43                         count(ancestor::BlockStatement/preceding-sibling::BlockStatement//Name[(contains(@Image, "getParameter"))])
44                     )
45                     ]
46                 </value>
47             </property>
48         </properties>
49         <!--<example><![CDATA[]]></example-->
50     </rule>

```

③ 룰(Rule) 반영

pom.xml 파일에 Spotbugs/FindSecurityBugs의 filter를 반영하려면 configuration안에 includeFilterFile과 excludeFilterFile을 추가하고 앞에서 작성한 파일을 연결한다. PMD 룰셋 파일을 반영하려면 마찬가지로 configuration 안에 rulesets -> ruleset 안에 룰셋 파일을 연결한다. 적용한 내용은 다음과 같다.

```
<!-- SpotBugs Static Analysis -->
<plugin>
  <groupId>com.github.spotbugs</groupId>
  <artifactId>spotbugs-maven-plugin</artifactId>
  <version>3.1.1</version>
  <configuration>
    <effort>Max</effort>
    <threshold>Low</threshold>
    <failOnError>true</failOnError>
    <includeFilterFile>${session.executionRootDirectory}/spotbugs-security-include.xml</includeFilterFile>
    <excludeFilterFile>${session.executionRootDirectory}/spotbugs-security-exclude.xml</excludeFilterFile>
  </configuration>
</plugin>

<!-- Install findsecbugs plugin -->
<plugin>
  <groupId>com.h3xstream.findsecbugs</groupId>
  <artifactId>findsecbugs-plugin</artifactId>
  <version>LATEST</version> <!-- Auto-update to the latest stable -->
</plugin>

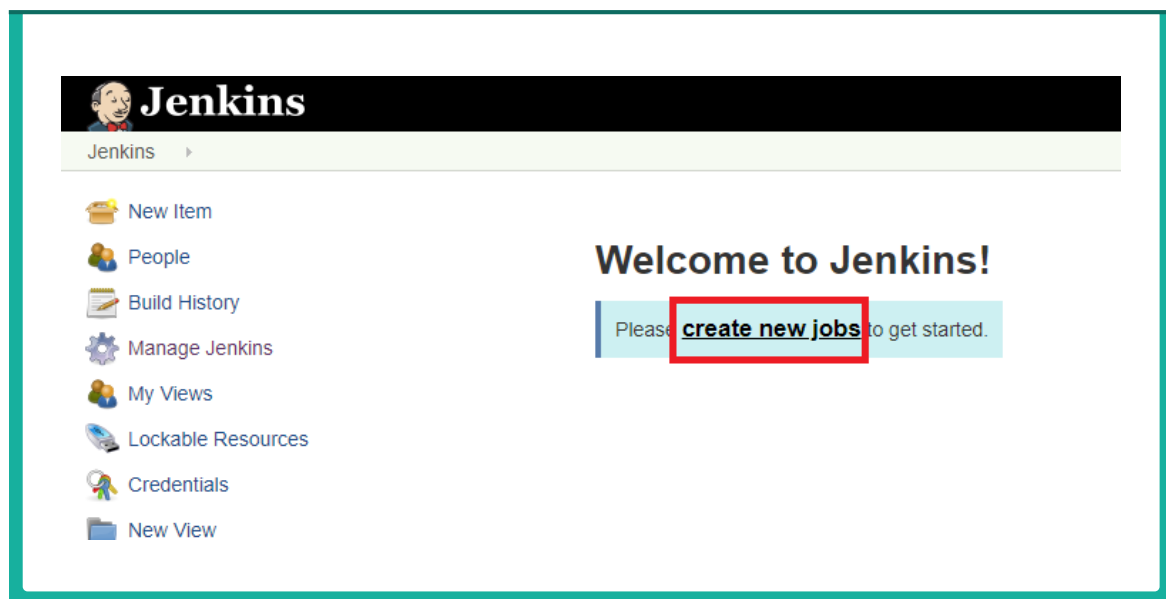
<!-- PMD Static Analysis -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-pmd-plugin</artifactId>
  <version>3.11.0</version>
  <configuration>
    <sourceEncoding>UTF-8</sourceEncoding>
    <rulesets>
      <ruleset>pmd-customRuleset.xml</ruleset>
    </rulesets>
  </configuration>
</plugin>
```


제4절 Jenkins 실행

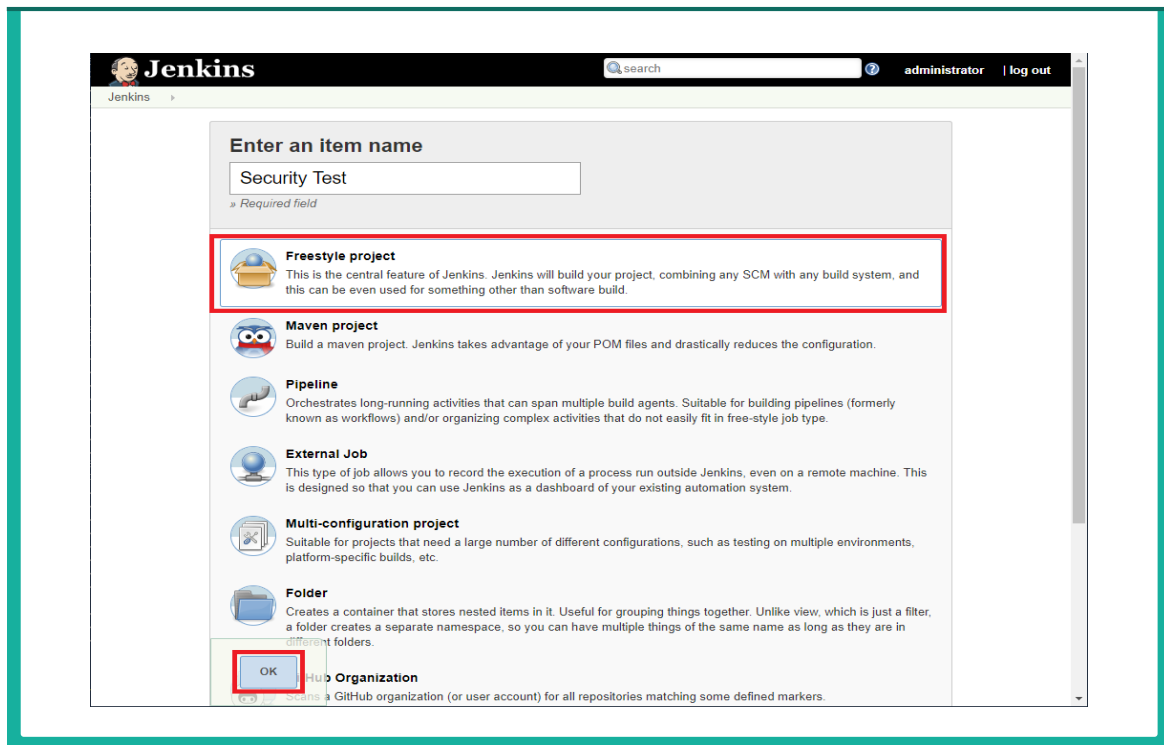
Jenkins에서 동일한 작업을 반복적으로 실행하고 이를 가시화 하려면 job을 생성해서 관리하면 된다.

1. 새 작업(Job) 생성

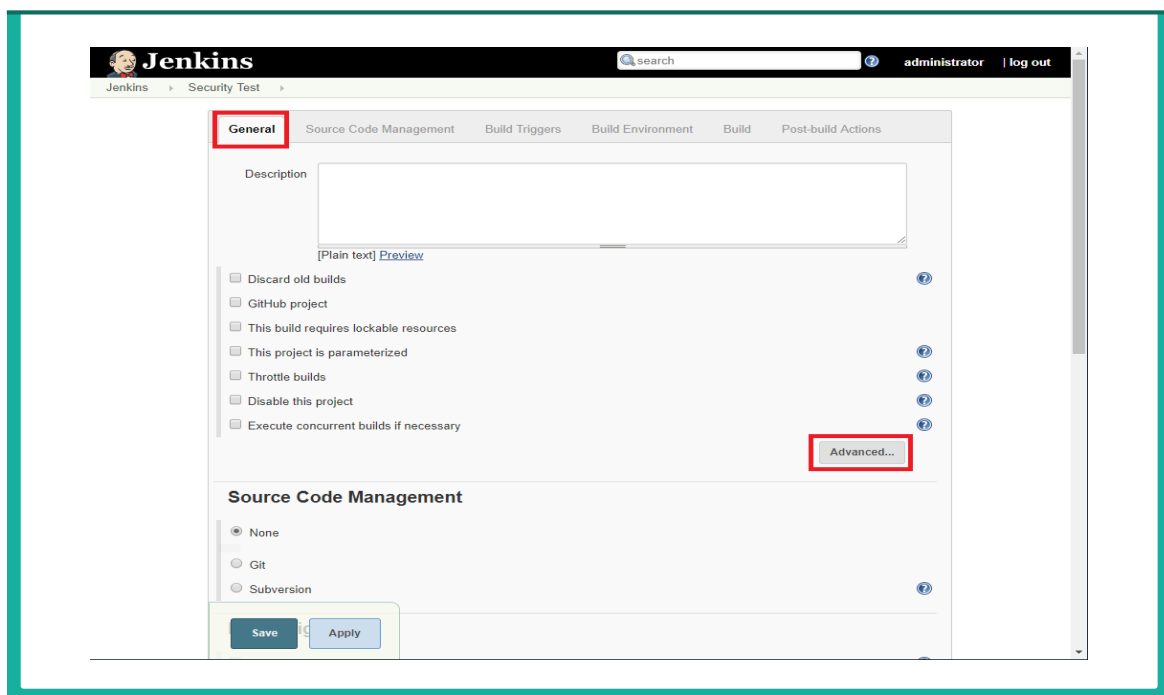
❶ 메인 화면에서 ‘create new jobs’을 클릭한다.



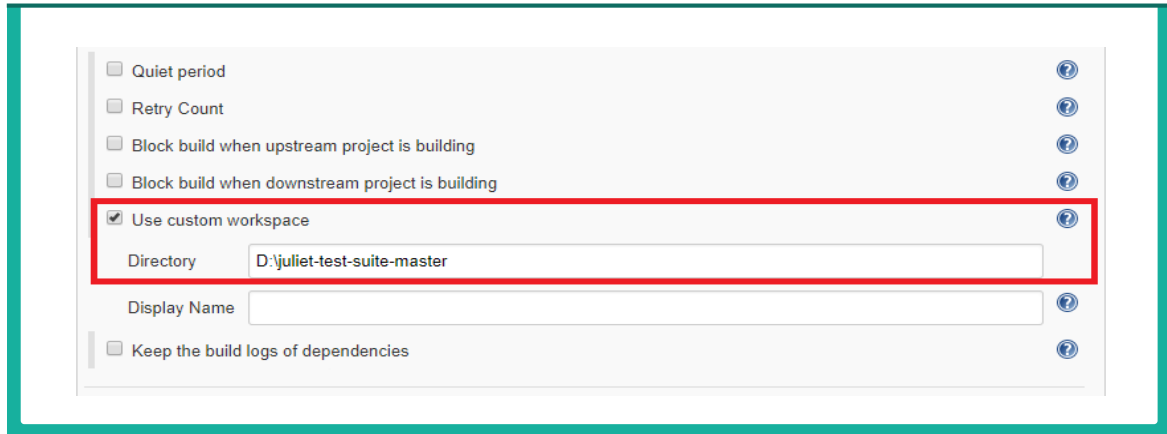
- ② Job의 이름을 입력하고 Freestyle project를 선택한 뒤 OK를 선택한다.



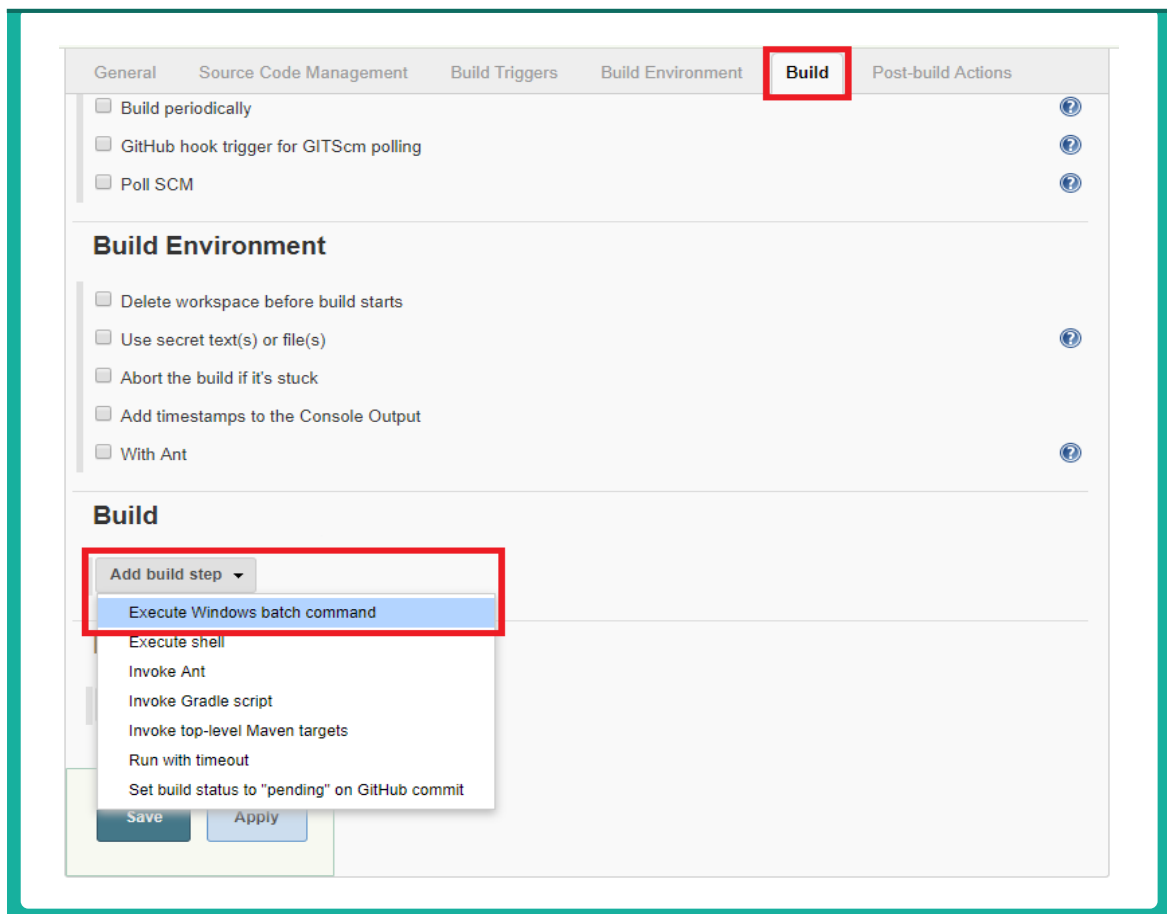
- ③ General 탭에서 'Advanced...'를 선택한다.



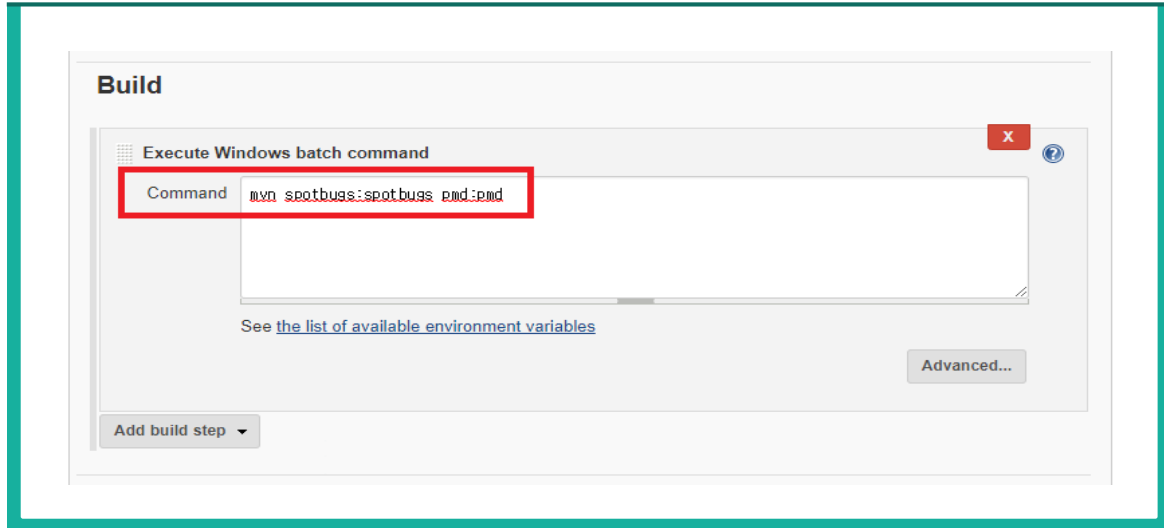
- ④ 추가된 메뉴에서 Use custom workspace의 체크박스를 클릭하고 위에서 설정한 maven 프로젝트의 루트 디렉토리를 입력한다.



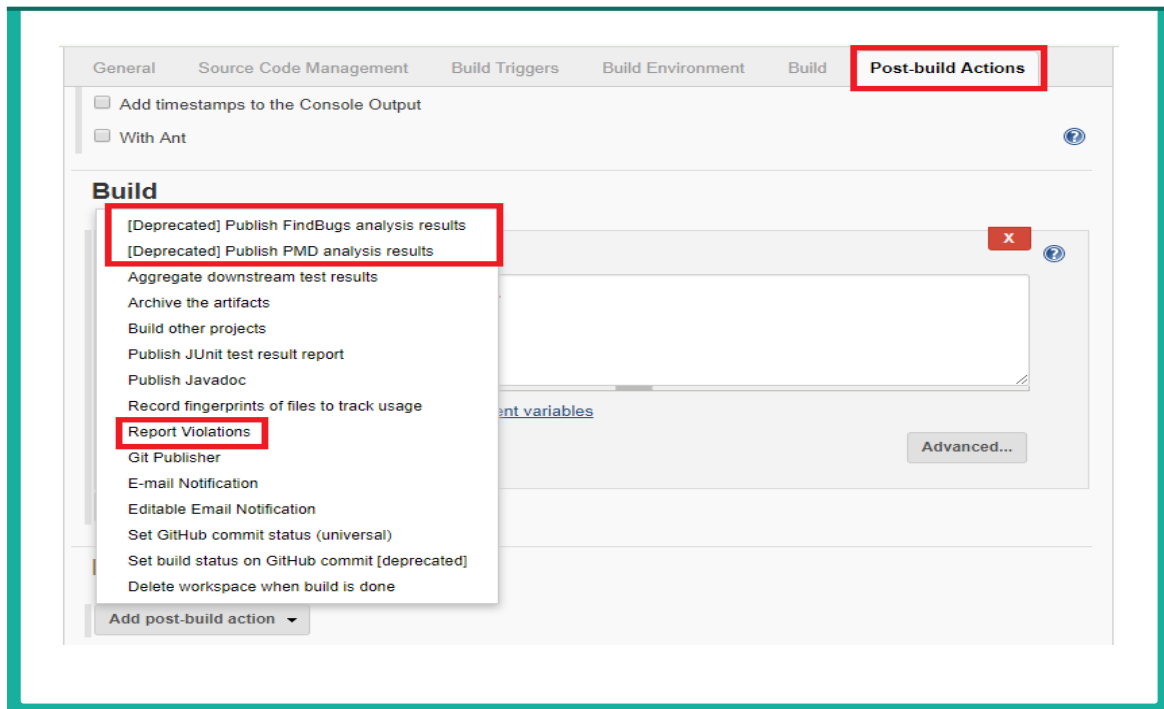
- ⑤ Build 탭에서 Add build step을 선택한 뒤, Execute Windows batch command를 선택한다.



- ⑥ 실행하고 싶은 Windows 명령어를 선택한다. 본 예제에서는 Spotbugs와 PMD 분석을 진행할 것이므로 `mvn spotbugs:spotbugs pmd:pmd`를 입력한다.



- ⑦ Post-build Actions을 선택한다. Spotbugs, PMD, Violation 플러그인을 추가한다.



※ 참고: Jenkins는 Eclipse와 마찬가지로 아직 Findbugs라는 이름을 유지하고 있다. Jenkins Findbugs 플러그인은 분석된 xml을 Jenkins에서 출력해주는 역할을 한다. 분석을 진행하고 xml을 생성하는 주체는 maven spotbugs 플러그인이다. spotbugs와 findbugs는 같은 xml 형식을 사용하기 때문에 Jenkins Findbugs 플러그인을 이용해서 spotbugs를 무리없이 사용할 수 있다.

- ⑧ 위에서 입력한 Windows batch command를 통해 spotbugs와 pmd가 실행되고 분석된 파일이 xml 형식으로 저장되었다. 각각 해당하는 xml 파일을 플러그인에 입력한다.

Post-build Actions

[Deprecated] Publish FindBugs analysis results

FindBugs results:

[Fileset includes](#) setting that specifies the generated raw FindBugs XML report files, such as `**/findbugs.xml` or `**/findbugsXml.xml`. Basedir of the fileset is [the workspace root](#). If no value is set, then the default `**/findbugsXml.xml` or `**/findbugs.xml` are used for maven or ant builds, respectively. Be sure not to include any non-report files into this pattern.

Use rank as priority ☐

Uses the bug rank when evaluating the priority of the warnings (otherwise the FindBugs priority is used).

[Advanced...](#)

[Deprecated] Publish PMD analysis results

PMD results:

[Fileset includes](#) setting that specifies the generated raw PMD XML report files, such as `**/pmd.xml`. Basedir of the fileset is [the workspace root](#). If no value is set, then the default `**/pmd.xml` is used. Be sure not to include any non-report files into this pattern.

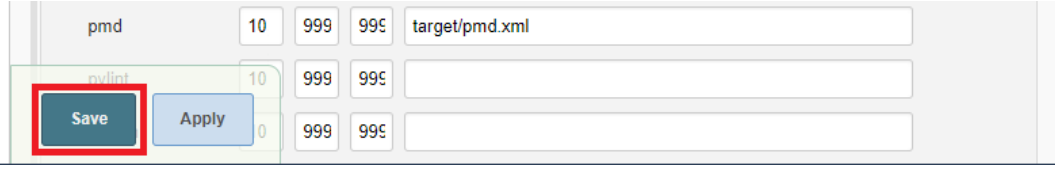
[Advanced...](#)

Report Violations

XML filename pattern

checkstyle	10	999	99%	
codenarc	10	999	99%	
cpd	10	999	99%	
cpplint	10	999	99%	
csslint	10	999	99%	
findbugs	10	999	99%	target/spotbugsXml.xml
fxcop	10	999	99%	
gendarme	10	999	99%	
jcreport	10	999	99%	
jslint	10	999	99%	
pep8	10	999	99%	
perlritic	10	999	99%	
pmd	10	999	99%	target/pmd.xml

- ⑨ 하단에 Save 버튼을 클릭하여 Job을 생성한다.



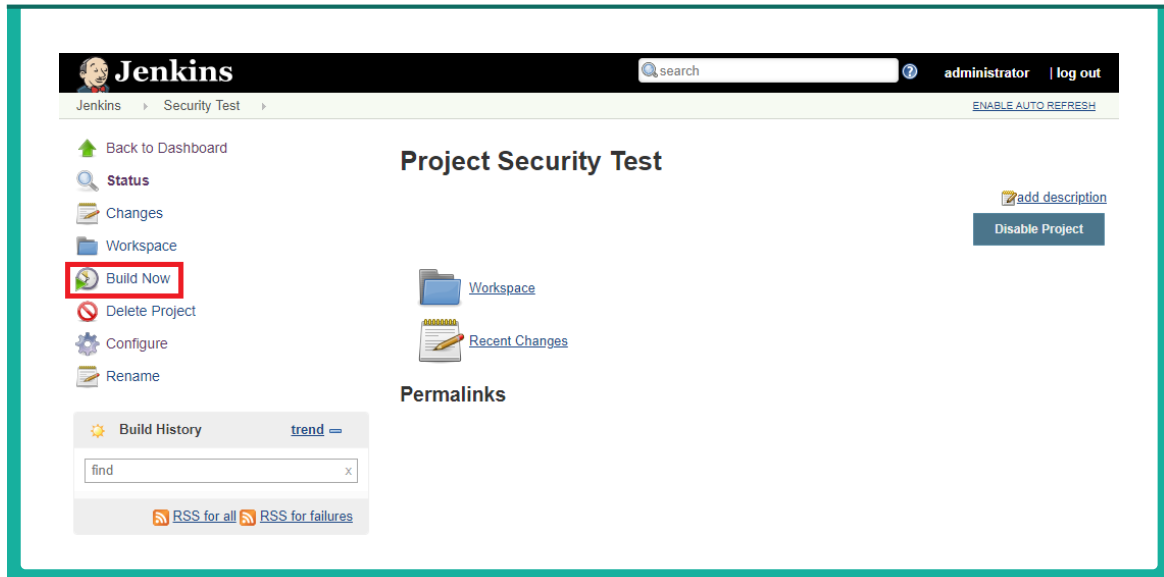
The screenshot shows the Jenkins job configuration page for a job named 'pmd'. The configuration includes a table with columns for name, type, and value. The 'Save' button is highlighted with a red box.

Name	Type	Value
pmd	10	999 99%
target/pmd.xml		
pylint	10	999 99%
	0	999 99%

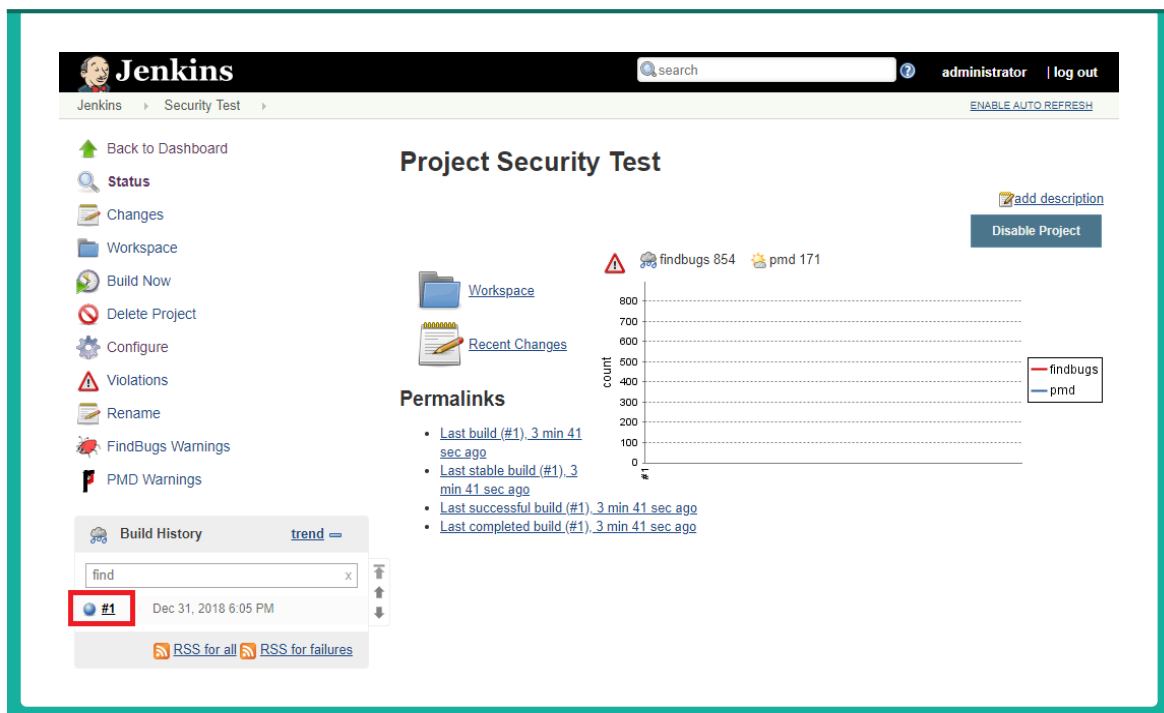
Buttons: Save, Apply

2. 새 작업(Job) 실행

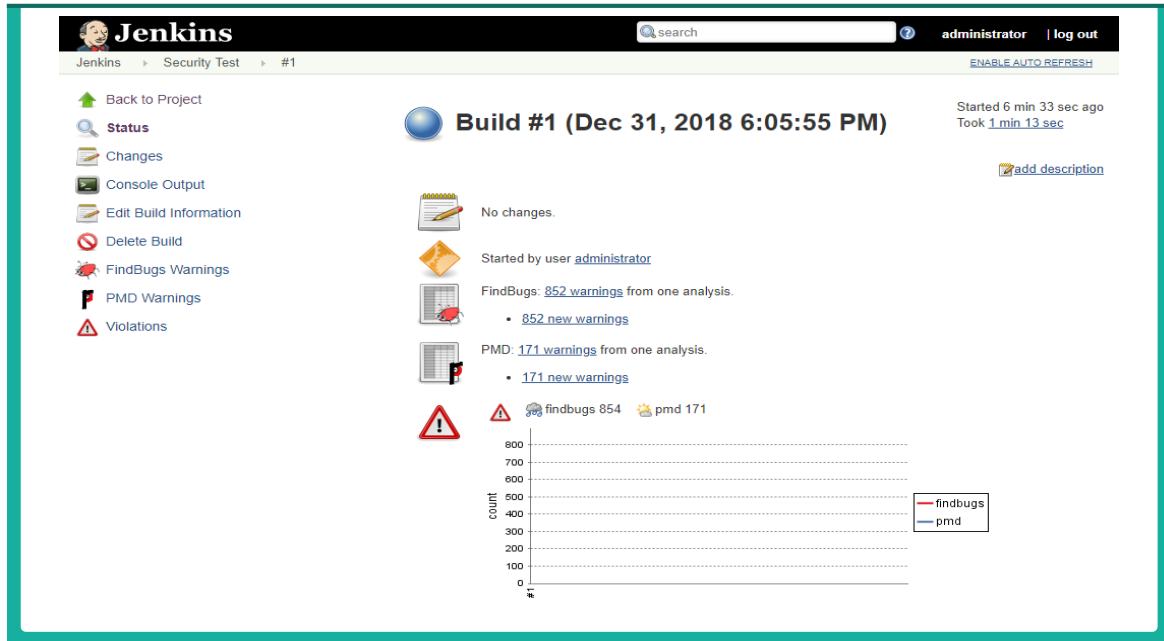
- 1 Job을 실행하기 위해서는 Jenkins 좌측 메뉴에서 Build Now 를 클릭한다.



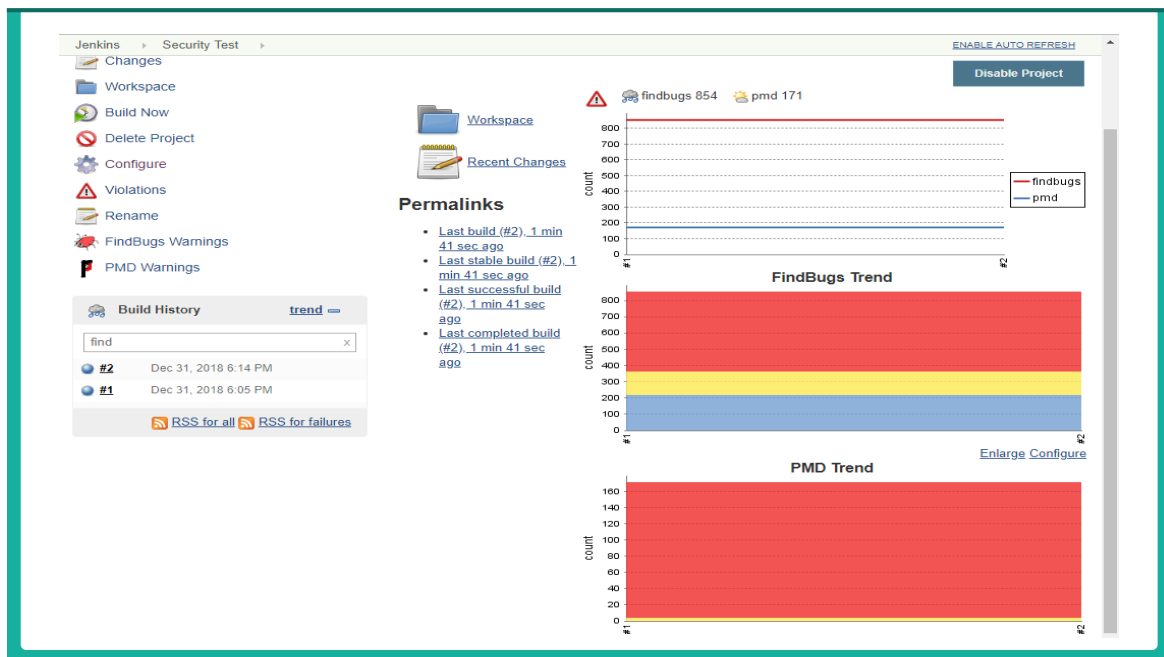
- 2 Build가 성공하면 좌측 Build History에 파란색 불과 Build 숫자가 나타난다. Main 화면에서 Build의 간략한 정보를 파악할 수 있다.



- ③ Build 숫자를 클릭하면 해당 build의 정보를 볼 수 있다. 각 플러그인(Findbugs, PMD, Violation)에서 검출된 결과를 확인할 수 있다.



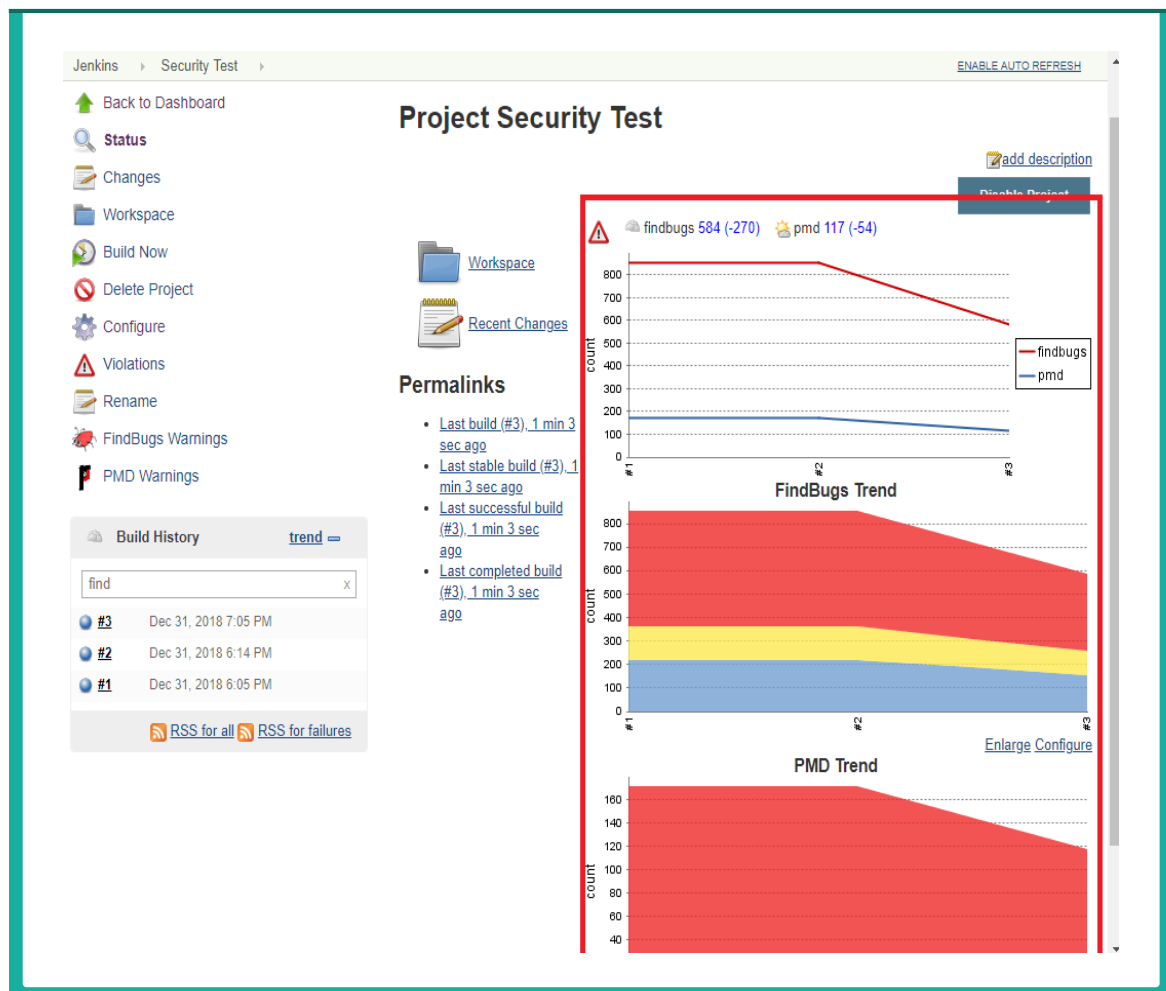
- ④ 추이 변화를 확인하기 위해서 Build를 다시 진행한다. #2로 빌드가 생성되며 메인 화면에 각 플러그인에서 검출된 버그 추이 변화 그래프가 업데이트된다. 아래 그림에서는 같은 프로젝트를 그대로 빌드하여 그래프 상 변동은 없다.



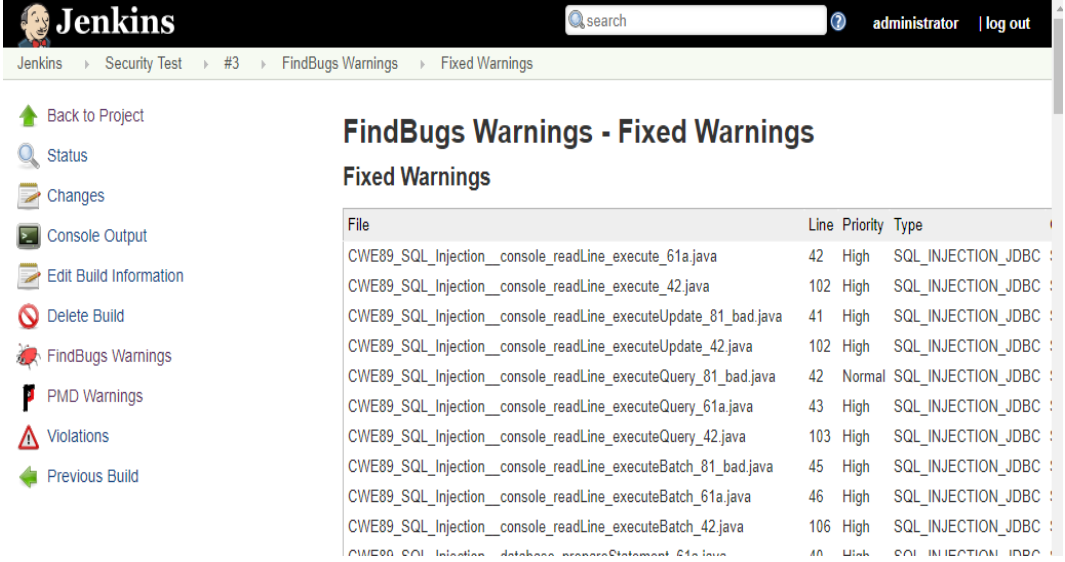
제5절 Jenkins를 활용한 시큐어코딩 Inspection 활동

시큐어코딩(Securecoding)은 개발 도중 발생된 보안약점을 중앙에서 관리하는 활동이 중요하다. 이러한 활동은 프로젝트의 규모에 따라서 PM/PL이 할 수도 있고 품질관리조직 또는 보안 코드 품질관련 조직이 수행 할 수 있다. Jenkins를 통한 Inspection활동은 이러한 활동을 도와줄 수 있다. Build마다 소스가 생성/수정/삭제되고 이에 따라서 시큐어코딩 Inspection 결과가 달라진다. 이전 빌드와 변동된 정보가 있으면 해 당 빌드의 메인 페이지에 신규/fix된 검사결과가 무엇인지 나타난다. 이를 통해서 추가 발견된 문제와 fix된 문제를 확인할 수 있다.

아래 화면을 보면 빌드#2보다 빌드#3에서 FindBugs와 PMD에서 발견한 보안약점이 감소됨을 알 수 있다.



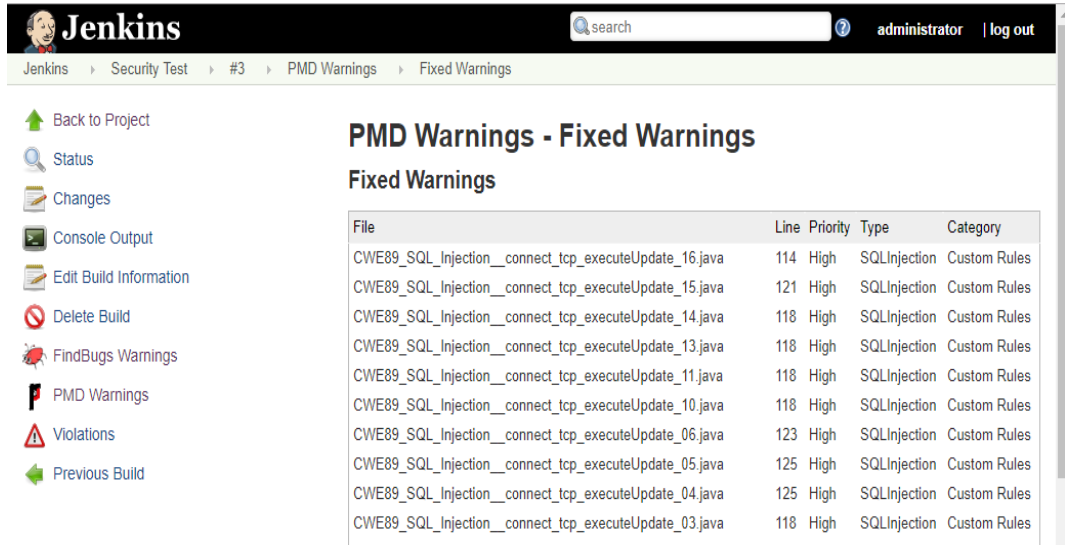
또한 어떠한 파일에서 보안약점이 수정되었는지 알 수 있다. 아래 화면은 FindBugs를 통해서 발견된 보안약점을 수정한 결과이다. 해당 페이지는 Jenkins 좌측 메뉴 FindBugs Warning에서 Warning Trend→Fixed Warnings를 클릭하여 확인할 수 있다.



The screenshot shows the Jenkins interface with the 'FindBugs Warnings - Fixed Warnings' page. The left sidebar contains navigation links: Back to Project, Status, Changes, Console Output, Edit Build Information, Delete Build, FindBugs Warnings, PMD Warnings, Violations, and Previous Build. The main content area displays a table of fixed warnings.

File	Line	Priority	Type
CWE89_SQL_Injection__console_readLine_execute_61a.java	42	High	SQL_INJECTION_JDBC
CWE89_SQL_Injection__console_readLine_execute_42.java	102	High	SQL_INJECTION_JDBC
CWE89_SQL_Injection__console_readLine_executeUpdate_81_bad.java	41	High	SQL_INJECTION_JDBC
CWE89_SQL_Injection__console_readLine_executeUpdate_42.java	102	High	SQL_INJECTION_JDBC
CWE89_SQL_Injection__console_readLine_executeQuery_81_bad.java	42	Normal	SQL_INJECTION_JDBC
CWE89_SQL_Injection__console_readLine_executeQuery_61a.java	43	High	SQL_INJECTION_JDBC
CWE89_SQL_Injection__console_readLine_executeQuery_42.java	103	High	SQL_INJECTION_JDBC
CWE89_SQL_Injection__console_readLine_executeBatch_81_bad.java	45	High	SQL_INJECTION_JDBC
CWE89_SQL_Injection__console_readLine_executeBatch_61a.java	46	High	SQL_INJECTION_JDBC
CWE89_SQL_Injection__console_readLine_executeBatch_42.java	106	High	SQL_INJECTION_JDBC

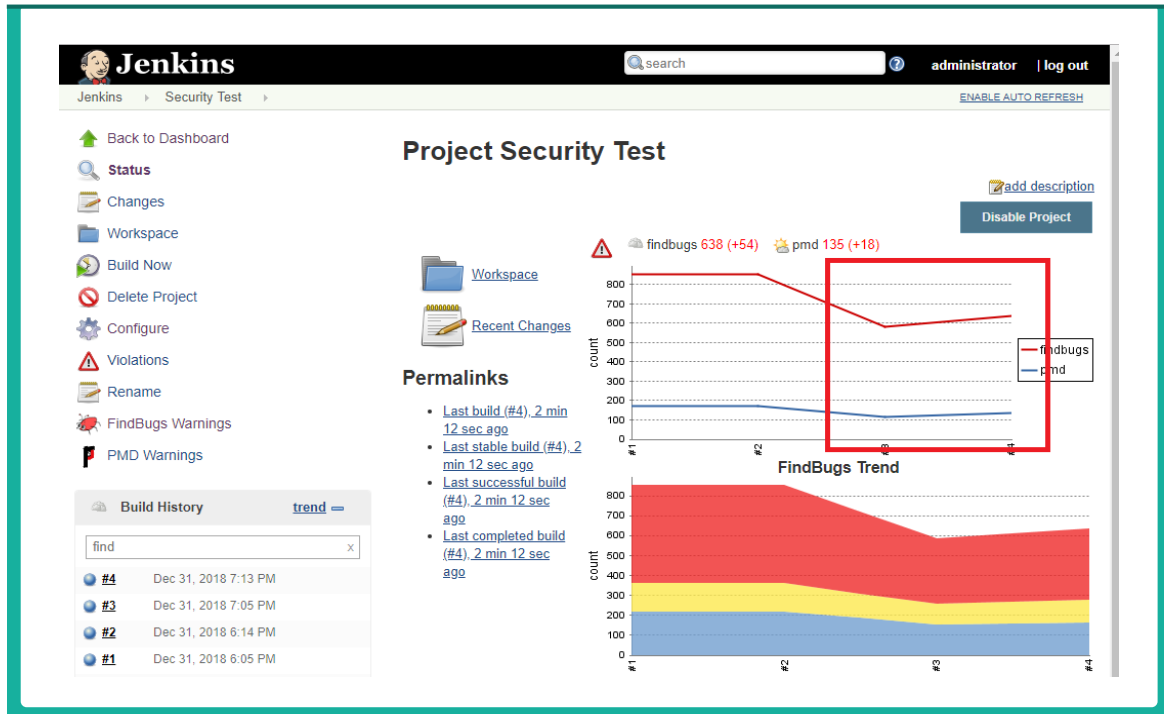
아래 화면은 PMD를 통해서 발견된 보안약점을 수정한 결과이다.



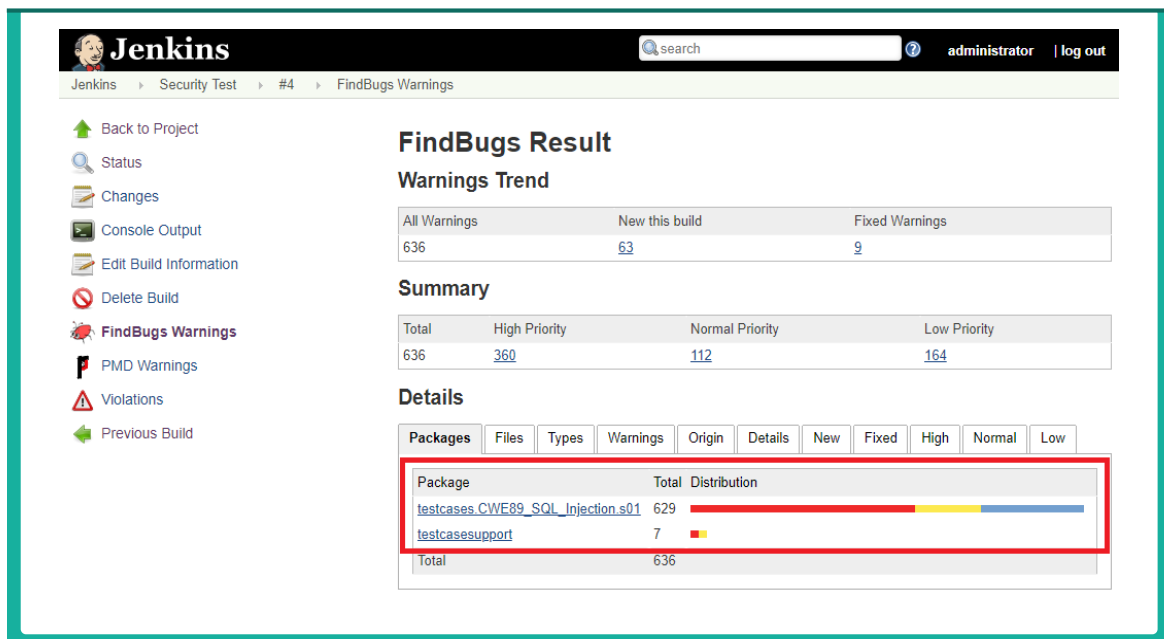
The screenshot shows the Jenkins interface with the 'PMD Warnings - Fixed Warnings' page. The left sidebar contains navigation links: Back to Project, Status, Changes, Console Output, Edit Build Information, Delete Build, FindBugs Warnings, PMD Warnings, Violations, and Previous Build. The main content area displays a table of fixed warnings.

File	Line	Priority	Type	Category
CWE89_SQL_Injection__connect_tcp_executeUpdate_16.java	114	High	SQLInjection	Custom Rules
CWE89_SQL_Injection__connect_tcp_executeUpdate_15.java	121	High	SQLInjection	Custom Rules
CWE89_SQL_Injection__connect_tcp_executeUpdate_14.java	118	High	SQLInjection	Custom Rules
CWE89_SQL_Injection__connect_tcp_executeUpdate_13.java	118	High	SQLInjection	Custom Rules
CWE89_SQL_Injection__connect_tcp_executeUpdate_11.java	118	High	SQLInjection	Custom Rules
CWE89_SQL_Injection__connect_tcp_executeUpdate_10.java	118	High	SQLInjection	Custom Rules
CWE89_SQL_Injection__connect_tcp_executeUpdate_06.java	123	High	SQLInjection	Custom Rules
CWE89_SQL_Injection__connect_tcp_executeUpdate_05.java	125	High	SQLInjection	Custom Rules
CWE89_SQL_Injection__connect_tcp_executeUpdate_04.java	125	High	SQLInjection	Custom Rules
CWE89_SQL_Injection__connect_tcp_executeUpdate_03.java	118	High	SQLInjection	Custom Rules

아래 화면은 다음 빌드에서 취약점 개수가 증가한 경우이다. 빌드#3보다 빌드#4에서 보안약점이 증가됨을 알 수 있다.



아래 화면은 Findbugs 플러그인이 어떠한 패키지에서 보안약점을 많이 검출했는지 보여준다. 탭을 변경하면 파일, 타입, 경고 등 분류하여 결과를 확인할 수 있다.



아래 화면은 파일 별로 검출된 보안약점을 보여준다.

The screenshot shows the Jenkins 'FindBugs Result' page. The left sidebar contains navigation links: Back to Project, Status, Changes, Console Output, Edit Build Information, Delete Build, FindBugs Warnings (selected), PMD Warnings, Violations, and Previous Build. The main content area is titled 'FindBugs Result' and includes a 'Warnings Trend' table, a 'Summary' table, and a 'Details' section with a file list.

All Warnings	New this build	Fixed Warnings
636	63	9

Total	High Priority	Normal Priority	Low Priority
636	360	112	164

File	Total	Distribution
AbstractTestCaseServlet.java	2	
AbstractTestCaseServletBadOnly.java	2	
CWE89_SQL_Injection__Environment_executeBatch_21.java	2	
CWE89_SQL_Injection__Environment_executeBatch_22b.java	2	
CWE89_SQL_Injection__Environment_executeBatch_41.java	2	

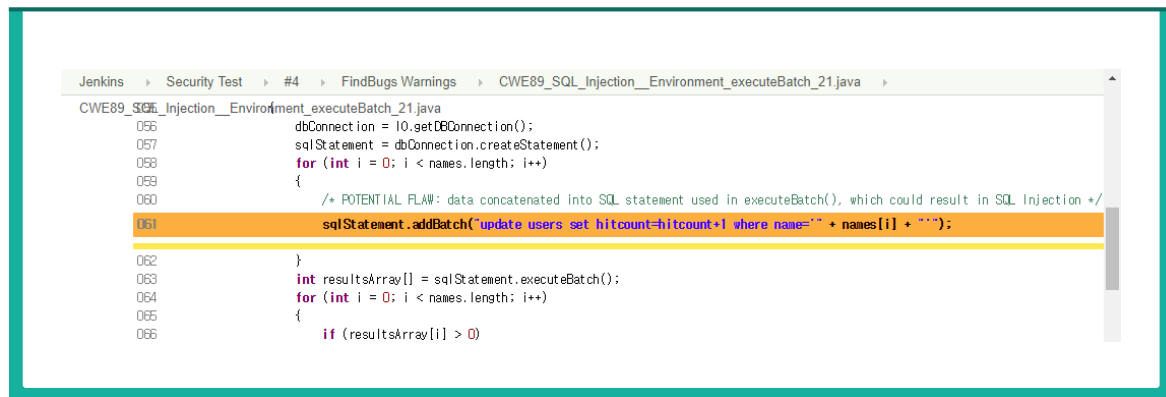
해당 파일을 클릭하면 해당 파일에 어떠한 보안약점이 있는지 보여준다.

The screenshot shows the Jenkins 'FindBugs Warnings - File' page for the file 'CWE89_SQL_Injection__Environment_executeBatch_21.j'. The left sidebar is the same as the previous screenshot. The main content area is titled 'FindBugs Warnings - File' and includes a 'Summary' table and a 'Details' section with a 'Warnings' table.

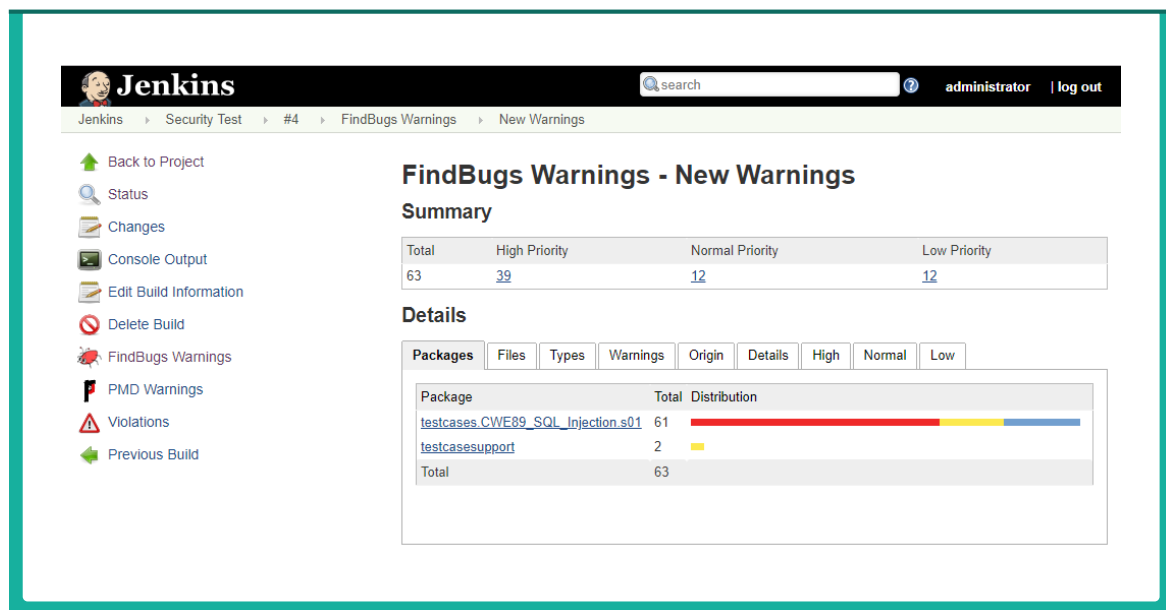
Total	High Priority	Normal Priority	Low Priority
2	0	0	2

File	Priority	Rank	Type	Category
CWE89_SQL_Injection__Environment_executeBatch_21.java:61	Low	20	SQL_INJECTION_JDBC	SECURIT
CWE89_SQL_Injection__Environment_executeBatch_21.java:309	Low	20	SQL_INJECTION_JDBC	SECURIT

검출된 보안약점을 클릭하면 소스코드의 어떤 부분이 보안약점으로 의심되는지 보여준다.



아래 화면은 새로운 Build에서 발견된 신규 보안약점을 보여준다. Findbugs Warning > Warnings Trend > New this build 를 클릭하여 확인할 수 있다.





PART



제6장 사용자 정의 룰(Rule) 작성 및 오탐(False Positive) 관리

제1절 개요

제2절 Spotbugs/FindSecurityBugs를
활용한 사용자 정의 룰(Rule) 작성 방법

제3절 PMD를 활용한 사용자 정의 룰(Rule)
작성 방법

제4절 오탐(False Positive) 관리



제6장 사용자 정의 룰(Rule) 작성 및 오탐(False Positive) 관리



제1절 개요

본 장에서는 Spotbugs, FindSecurityBugs, PMD를 활용한 사용자 정의 룰(Rule) 작성 및 오탐(False Positive) 관리 방법에 대해 설명한다.

FindSecurityBugs는 Spotbugs의 플러그인 형태이므로, Spotbugs 프로젝트를 이용하여 플러그인을 제작하거나 FindSecurityBugs 프로젝트를 이용하여 플러그인을 제작하는 결과물은 모두 Spotbugs 플러그인이자 Spotbugs의 사용자 정의 룰이다. FindSecurityBugs 프로젝트에는 Spotbugs 프로젝트에서는 지원하지 않는 semantic 버그 패턴을 감지할 수 있는 엔진이 지원되므로 FindSecurityBugs 프로젝트를 이용하여 사용자 정의 룰을 제작하는 방법을 설명한다.

PMD에서는 Rule designer를 이용하여 룰을 제작하는 방법에 대해서 설명한다. 독자적으로(stand-alone)으로 제공되는 Rule designer GUI를 이용한 방법과 플러그인에서 제공되는 Rule Designer를 이용한 방법 두 가지를 설명한다.

제2절 Spotbugs/FindSecurityBugs를 활용한 사용자 정의 룰(Rule) 작성 방법

플러그인 안에 들어갈 detector 작성법에 대해 설명한다. 예시는 FindSecurityBugs로 진행한다. 원문은 해당 링크에서 확인할 수 있다. 원문에 기술된 내용만으로는 바로 작성하기에 어려움이 있어 추가적인 설명을 추가했다. FindSecurityBugs 루트 프로젝트(find-sec-bugs-master)를 '%FSB_HOME%'로 하고 진행한다.

1. 취약한 샘플 코드 생성

```

1 package testcode.xmldecoder;
2
3 import java.beans.XMLDecoder;
4 import java.io.InputStream;
5
6 public class XmlDecodeUtil {
7
8     public static Object handleXml(InputStream in) {
9         XMLDecoder d = new XMLDecoder(in);
10        try {
11            Object result = d.readObject(); //Deserialization happen here
12            return result;
13        }
14        finally {
15            d.close();
16        }
17    }
18
19    public static void main(String[] args) {
20
21        InputStream in = XmlDecodeUtil.class.getResourceAsStream("/testcode/xmldecoder/obj1.xml");
22
23        XmlDecodeUtil.handleXml(in);
24    }
25
26 }

```

XMLDecoder usage 버그는 신뢰할 수 없는 사용자 입력값을 XMLDecoder가 파싱하는 경우 발생한다. 사용자의 입력값을 역직렬화(deserialization)하는 과정에서 임의 코드 실행이 발생할 수 있다. 역직렬화는 11번째 줄 readObject()에서 호출된다. 그러나 역직렬화되는 값은 9번째 줄 new XMLDecoder(in)에서 신뢰할 수 없는 사용자 입력값 in으로 XMLDecoder를 생성하는 과정에서 초기화된다.

2. 새로운 Detector 클래스 작성

```

PS D:\find-sec-bugs-master\plugin\target\test-classes\testcode\xmldecoder> javap -c .\XmlDecodeUtil.class
Compiled from "XmlDecodeUtil.java"
public class testcode.xmldecoder.XmlDecodeUtil {
    public testcode.xmldecoder.XmlDecodeUtil();
    Code:
    0: aload_0
    1: invokespecial #1           // Method java/lang/Object.<init>:()V
    4: return

    public static java.lang.Object handleXml(java.io.InputStream);
    Code:
    0: new           #2           // class java/beans/XMLDecoder
    3: dup
    4: aload_0
    5: invokespecial #3           // Method java/beans/XMLDecoder.<init>:(Ljava/io/InputStream;)V
    8: astore_1
    9: aload_1
    10: invokevirtual #4          // Method java/beans/XMLDecoder.readObject:()Ljava/lang/Object;
    13: astore_2
    14: aload_2
    15: astore_3
    16: aload_1
    17: invokevirtual #5          // Method java/beans/XMLDecoder.close:()V
    20: aload_3
    21: areturn
    22: astore       4
    24: aload_1
    25: invokevirtual #5          // Method java/beans/XMLDecoder.close:()V
    28: aload       4
    30: athrow

    Exception table:
        from    to  target type
         9      16    22     any
        22     24    22     any

    public static void main(java.lang.String[]);
    Code:
    0: ldc          #6           // class testcode/xmldecoder/XmlDecodeUtil
    2: ldc          #7           // String /testcode/xmldecoder/obj1.xml
    4: invokevirtual #8          // Method java/lang/Class.getResourceAsStream:(Ljava/lang/String;)Ljava/io/I
InputStream;
    7: astore_1
    8: aload_1
    9: invokestatic #9           // Method handleXml:(Ljava/io/InputStream;)Ljava/lang/Object;
    12: pop
    13: return
}
PS D:\find-sec-bugs-master\plugin\target\test-classes\testcode\xmldecoder>
  
```

FindSecurityBugs와 Spotbugs는 ByteCode Engineering Library(BCEL)과 ASM을 이용하여 자바 바이트코드 수준에서 분석을 진행한다. Detector를 개발할 때 사용되는 상수나 변수들이 바이트코드를 이용하므로 자바 소스 코드 뿐 아니라 바이트코드를 참고하면 도움이 된다. 샘플 코드를 컴파일한 클래스를 javap(Java 역어셈블러)를 이용해서 역어셈블하면 바이트코드 수준에서 샘플 코드를 분석할 수 있다.

샘플 코드를 작성하면 취약점이 발생하는 원인이 드러나기 때문에 샘플 코드를 작성하면서 어떻게 버그를 탐지할지 구상한다. 임의 코드 실행은 11번째 줄에서 발생하지만, 원인은 9번째 줄에서 XMLDecoder usage는 신뢰할 수 없는 사용자 입력값으로 XMLDecoder를 초기화하는 것이다. 따라서 detector에서는 XMLDecoder의 생성자에서 사용자 입력값(InputStream)을 인자로 사용하는 경우를 탐지하면 된다.

해당하는 내용은 바이트 코드 상에서 handXML(java.op.InputStream)의 5번째 opcode인 invokespecial #3이다. invokespecial는 메소드 호출을 의미하는 opcode이며 #3은 세번째 인덱스에 해당하는 메소드를 의미한다. 즉, 세번째 인덱스에 해당되는 메소드를 호출하라는 명령어이다.

인덱스에 해당되는 메소드는 주석(// Methodjava/beans/XMLDecoder."<init>":(Ljava/io/InputStream;)V)으로 알려준다. 주석은 [클래스.“메소드”(인자)반환값] 형식으로 구성되며, 이를 참조하여 해당 경로에 detector를 작성한다. 아래 그림은 XMLDecoder usage 버그를 탐지하는 detector 코드이다.

```

18 package com.h3xstream.findsecbugs.xml;
19
20 import com.h3xstream.findsecbugs.common.matcher.InvokeMatcherBuilder;
21 import edu.umd.cs.findbugs.BugInstance;
22 import edu.umd.cs.findbugs.BugReporter;
23 import edu.umd.cs.findbugs.Priorities;
24 import edu.umd.cs.findbugs.bcel.OpcodeStackDetector;
25 import org.apache.bcel.Const;
26
27 import static com.h3xstream.findsecbugs.common.matcher.InstructionDSL.invokeInstruction;
28
29 public class XmlDecoderDetector extends OpcodeStackDetector {
30
31     private static final String XML_DECODER = "XML_DECODER";
32     private static final InvokeMatcherBuilder XML_DECODER_CONSTRUCTOR = invokeInstruction().atClass("java/beans/XMLDecoder").atMethod("<init>");
33
34     private BugReporter bugReporter;
35
36     public XmlDecoderDetector(BugReporter bugReporter) {
37         this.bugReporter = bugReporter;
38     }
39
40     @Override
41     public void sawOpcode(int seen) {
42
43         if (seen == Const.INVOKESPECIAL && XML_DECODER_CONSTRUCTOR.matches(this)) {
44             bugReporter.reportBug(new BugInstance(this, XML_DECODER, Priorities.HIGH_PRIORITY) //
45                 .addClass(this).addMethod(this).addSourceLine(this));
46         }
47     }
48 }

```

OpcodeStackDetector는 코드를 opcode 단위로 읽는다. opcode는 특정 기능을 수행하는 명령어이며 한 바이트로 표현되는 고유한 값이다. 개발자는 sawOpcode 메소드를 @Override하여 알람을 발생할 조건문을 추가할 수 있다.

인자로 들어오는 seen에는 현재 detector가 멈춰있는 opcode의 값이 들어있다. seen == Const.INVOKESPECIAL을 통해 opcode가 특수한 메소드(e.g., 생성자)를 호출하는 명령어인지 확인하고 XML_DECODER_CONSTRUCTOR를 통해 특정한 클래스의 생성자가 호출한 것을 확인한다. 탐지하고 싶은 역어셈블 주석을 참고하여 검출하고 싶은 클래스와 메소드를 XML_DECODER_CONSTRUCTOR에 작성한다.

3. 테스트 케이스 작성

Detector와 샘플 코드를 연결해서 테스트할 수 있도록 테스트 케이스를 작성한다. Spotbugs는 테스트 주도 개발방법론(Test Driven Development, TDD)을 지원한다. Detector가 개발될 때 마다 샘플 코드에서 의도한대로 버그를 검출하는지 테스트하여 검증한다.

기본적인 골자는 다른 테스트 케이스를 복사해서 사용하는 것이 편하다. 테스트 케이스 클래스 이름은 detector 이름과 동일하게 하고 뒤에 Test (XmlDecoderDetectorTest)를 붙여준다.

files에는 샘플 코드의 경로를 입력해주고, verify() 안에 bugDefinition()에 테스트에서 검출되어야 하는 버그 정보를 입력한다. 클래스XmlDecodeUtil의 메소드 handleXml 9번째 줄에서 XML_DECODER가 검출되어야 함을 의미한다.

```

29 public class XmlDecoderDetectorTest extends BaseDetectorTest {
30
31
32     @Test
33     public void detectXmlDecoder() throws Exception {
34         //Locate test code
35         String[] files = {
36             getClassFilePath("testcode/xmldecoder/XmlDecodeUtil")
37         };
38
39         //Run the analysis
40         EasyBugReporter reporter = spy(new SecurityReporter());
41         analyze(files, reporter);
42
43         verify(reporter).doReportBug(
44             bugDefinition()
45                 .bugType("XML_DECODER")
46                 .inClass("XmlDecodeUtil").inMethod("handleXml").atLine(9)
47                 .build()
48         );
49     }
50 }

```

4. 플러그인에 Detector 추가

플러그인 설정 파일에 개발한 detector를 추가한다. 플러그인은 findbugs.xml 파일을 파싱하여 사용할 detector를 추가한다. findbugs.xml은 다음 경로에 있다.

```
<Detector class="com.h3xstream.findsecbugs.jsp.XslTransformJspDetector" reports="JSP_XSLT" />
<Detector class="com.h3xstream.findsecbugs.xml.StdXmlTransformDetector" reports="MALICIOUS_XSLT" />
<Detector class="com.h3xstream.findsecbugs.xml.XmlDecoderDetector" reports="XML_DECODER"/>
<Detector class="com.h3xstream.findsecbugs.crypto.StaticIVDetector" reports="STATIC_IV"/>
<Detector class="com.h3xstream.findsecbugs.crypto.CipherWithNoIntegrityDetector" reports="ECB_MODE," />

<BugPattern type="MALICIOUS_XSLT" abbrev="SECXSLT" category="SECURITY" cweid="94"/>
<BugPattern type="XSS_SERVLET" abbrev="SECXSS2" category="SECURITY" cweid="79"/>
<BugPattern type="XML_DECODER" abbrev="XMLDEC" category="SECURITY"/>
<BugPattern type="STATIC_IV" abbrev="STAIV" category="SECURITY" cweid="329"/>
<BugPattern type="ECB_MODE" abbrev="SECECB" category="SECURITY" cweid="327"/>
```

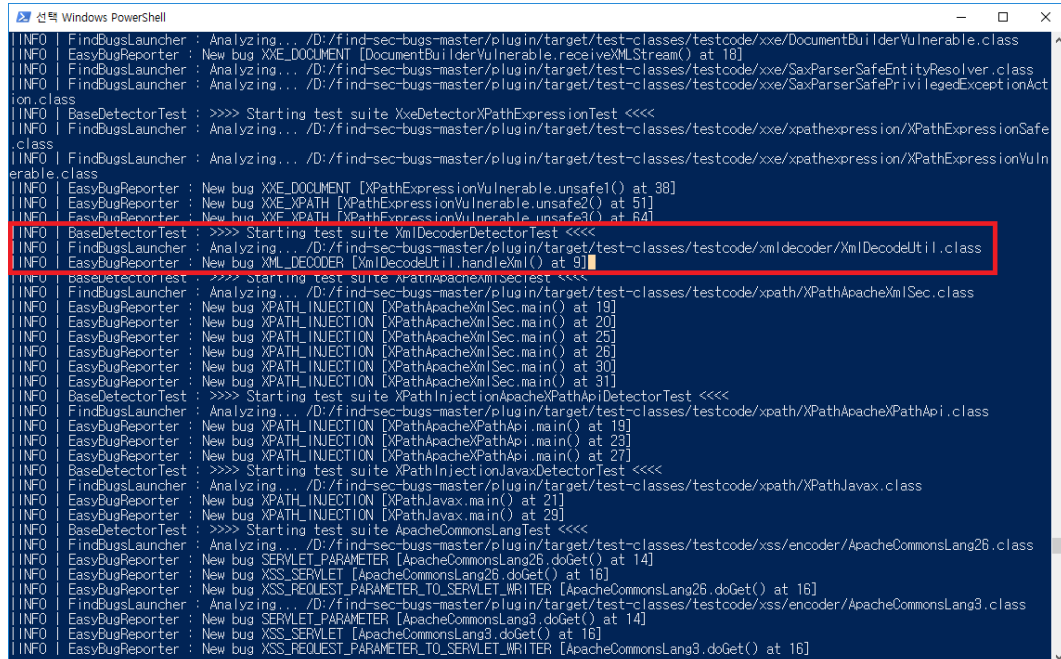
5. Detector 설명 추가

사용자에게 보여줄 detector에 대한 설명을 messages.xml에 추가한다. messages.xml은 다음 경로에서 찾을 수 있다. Details 부분은 비워두고 추후에 작성해도 된다.

```
3917 <!-- XML decoder -->
3918 <Detector class="com.h3xstream.findsecbugs.xml.XmlDecoderDetector">
3919   <Details>Identify use of XMLDecoder (a dangerous XML serializer).</Details>
3920 </Detector>
3921 <BugPattern type="XML_DECODER">
3922   <ShortDescription>XMLDecoder usage</ShortDescription>
3923   <LongDescription>It is not safe to use an XMLDecoder to parse user supplied data</LongDescription>
3924   <Details>
3925     <![CDATA[
3926 <p>
3927   XMLDecoder should not be used to parse untrusted data. Deserializing user input can lead to arbitrary code execution.
3928   This is possible because XMLDecoder supports arbitrary method invocation. This capability is intended to call setter methods,
3929   but in practice, any method can be called.
3930 </p>
3931   ]]>
3932 </Details>
3933 </BugPattern>
3934 <BugCode abbrev="XMLDEC">XMLDecoder usage</BugCode>
```

6. Detector 테스트

위의 과정을 성공적으로 수행하면 플러그인을 빌드할 때 자동으로 테스트가 실행된다.



```

선택 Windows PowerShell
INFO | FindBugsLauncher : Analyzing... /D:/find-sec-bugs-master/plugin/target/test-classes/testcode/xxe/DocumentBuilderVulnerable.class
INFO | EasyBugReporter : New bug XQE_DOCUMENT [DocumentBuilderVulnerable.receiveXMLStream() at 18]
INFO | FindBugsLauncher : Analyzing... /D:/find-sec-bugs-master/plugin/target/test-classes/testcode/xxe/SaxParserSafeEntityResolver.class
INFO | FindBugsLauncher : Analyzing... /D:/find-sec-bugs-master/plugin/target/test-classes/testcode/xxe/SaxParserSafePrivilegedExceptionAct
ion.class
INFO | BaseDetectorTest : >>>> Starting test suite XxeDetectorXPathExpressionTest <<<<
INFO | FindBugsLauncher : Analyzing... /D:/find-sec-bugs-master/plugin/target/test-classes/testcode/xxe/xpathexpression/XPathExpressionSafe
.class
INFO | FindBugsLauncher : Analyzing... /D:/find-sec-bugs-master/plugin/target/test-classes/testcode/xxe/xpathexpression/XPathExpressionVuln
erable.class
INFO | EasyBugReporter : New bug XQE_DOCUMENT [XPathExpressionVulnerable.unsafe1() at 38]
INFO | EasyBugReporter : New bug XQE_XPATH [XPathExpressionVulnerable.unsafe2() at 51]
INFO | EasyBugReporter : New bug XQE_XPATH [XPathExpressionVulnerable.unsafe3() at 64]
INFO | BaseDetectorTest : >>>> Starting test suite XmlDecoderDetectorTest <<<<
INFO | FindBugsLauncher : Analyzing... /D:/find-sec-bugs-master/plugin/target/test-classes/testcode/xmldecoder/XmlDecodeUtil.class
INFO | EasyBugReporter : New bug XML_DECODER [XmlDecodeUtil.handleXml() at 91]
INFO | BaseDetectorTest : >>>> Starting test suite XpathApacheXmlSecTest <<<<
INFO | FindBugsLauncher : Analyzing... /D:/find-sec-bugs-master/plugin/target/test-classes/testcode/xpath/XPathApacheXmlSec.class
INFO | EasyBugReporter : New bug XPATH_INJECTION [XPathApacheXmlSec.main() at 19]
INFO | EasyBugReporter : New bug XPATH_INJECTION [XPathApacheXmlSec.main() at 20]
INFO | EasyBugReporter : New bug XPATH_INJECTION [XPathApacheXmlSec.main() at 25]
INFO | EasyBugReporter : New bug XPATH_INJECTION [XPathApacheXmlSec.main() at 26]
INFO | EasyBugReporter : New bug XPATH_INJECTION [XPathApacheXmlSec.main() at 30]
INFO | EasyBugReporter : New bug XPATH_INJECTION [XPathApacheXmlSec.main() at 31]
INFO | BaseDetectorTest : >>>> Starting test suite XPathInjectionApacheXPathApiDetectorTest <<<<
INFO | FindBugsLauncher : Analyzing... /D:/find-sec-bugs-master/plugin/target/test-classes/testcode/xpath/XPathApacheXPathApi.class
INFO | EasyBugReporter : New bug XPATH_INJECTION [XPathApacheXPathApi.main() at 19]
INFO | EasyBugReporter : New bug XPATH_INJECTION [XPathApacheXPathApi.main() at 23]
INFO | EasyBugReporter : New bug XPATH_INJECTION [XPathApacheXPathApi.main() at 27]
INFO | BaseDetectorTest : >>>> Starting test suite XPathInjectionJavaxDetectorTest <<<<
INFO | FindBugsLauncher : Analyzing... /D:/find-sec-bugs-master/plugin/target/test-classes/testcode/xpath/XPathJavax.class
INFO | EasyBugReporter : New bug XPATH_INJECTION [XPathJavax.main() at 21]
INFO | EasyBugReporter : New bug XPATH_INJECTION [XPathJavax.main() at 29]
INFO | BaseDetectorTest : >>>> Starting test suite ApacheCommonsLangTest <<<<
INFO | FindBugsLauncher : Analyzing... /D:/find-sec-bugs-master/plugin/target/test-classes/testcode/xss/encoder/ApacheCommonsLang26.class
INFO | EasyBugReporter : New bug SERVLET_PARAMETER [ApacheCommonsLang26.doGet() at 14]
INFO | EasyBugReporter : New bug XSS_SERVLET [ApacheCommonsLang26.doGet() at 16]
INFO | EasyBugReporter : New bug XSS_REQUEST_PARAMETER_TO_SERVLET_WRITER [ApacheCommonsLang26.doGet() at 16]
INFO | FindBugsLauncher : Analyzing... /D:/find-sec-bugs-master/plugin/target/test-classes/testcode/xss/encoder/ApacheCommonsLang3.class
INFO | EasyBugReporter : New bug SERVLET_PARAMETER [ApacheCommonsLang3.doGet() at 14]
INFO | EasyBugReporter : New bug XSS_SERVLET [ApacheCommonsLang3.doGet() at 16]
INFO | EasyBugReporter : New bug XSS_REQUEST_PARAMETER_TO_SERVLET_WRITER [ApacheCommonsLang3.doGet() at 16]

```

제3절 PMD를 활용한 사용자 정의 룰(Rule) 작성 방법

PMD에서는 Java 또는 XPath를 활용하여 사용자가 직접 점검 룰(Rule)을 작성하고 이를 활용할 수 있다. 본 가이드에서는 XPath를 활용한 사용자 정의 점검 룰(Rule) 작성 방법을 설명한다. PMD 룰(Rule)을 작성하는데 필요한 기반 기술은 AST와 XPath 이다. 이에 대한 설명은 다음을 참고한다.

1. AST(Abstract Syntax Tree)

AST는 추상구문트리의 약어로, 프로그램 소스코드의 문법/어휘 내용을 추상화, 구조화 한 것을 의미한다. PMD는 Java AST를 이용하여 사용자 정의 룰을 적용할 소스코드를 분석한다.

AST는 자바 클래스의 각 요소를 참조할 수 있도록 소스코드의 요소들을 generic descriptor로 표현한다. 또한 인스턴스를 구분할 수 있는 모든 정보들은 개체의 속성(attribute)으로 저장된다.

AST는 XML 문서와 구조가 유사하므로 XML을 연상하면 이해하기 편하다. 아래 그림은 간단한 소스코드를 AST로 표현한 것이다.



2. XPath

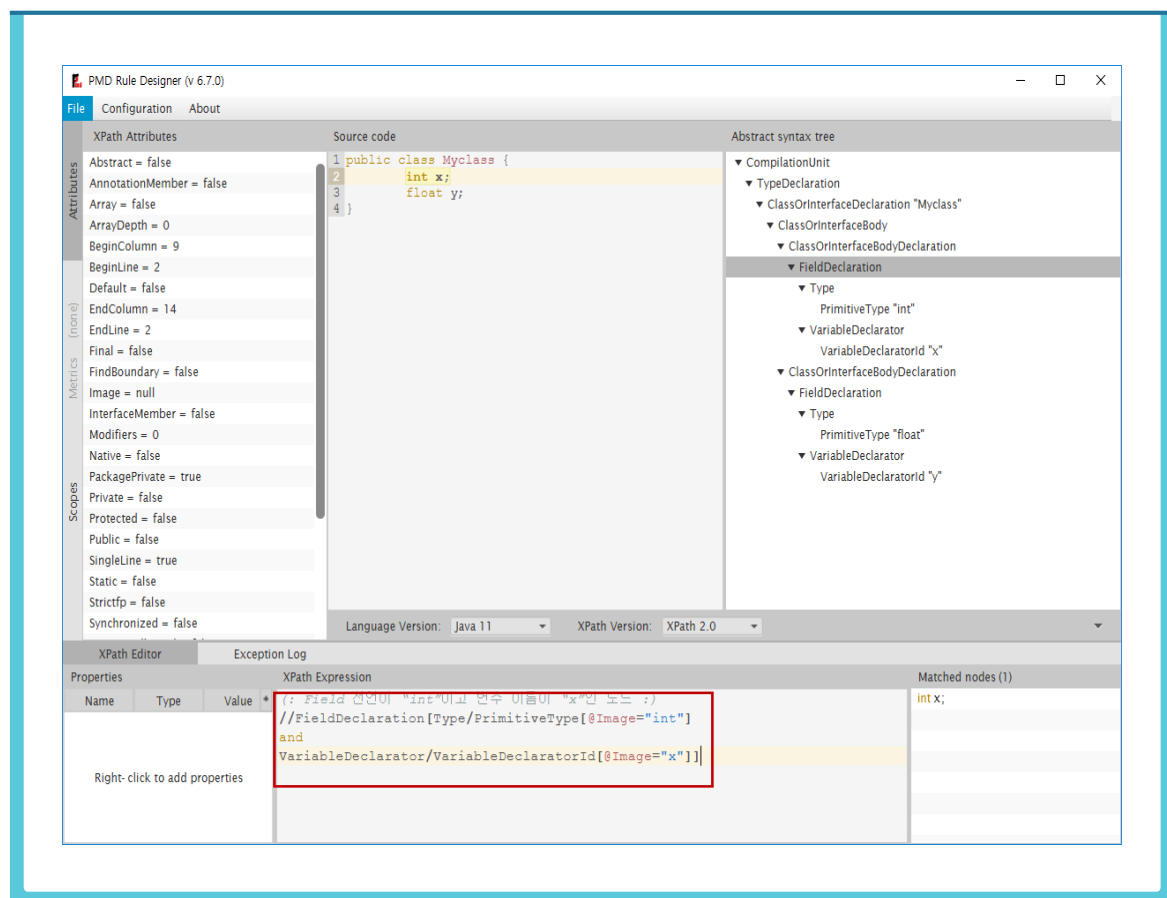
AST로 표현되는 개체들은 XPath 구문을 통해서 접근 가능하다.

예시로 아래의 그림에서 `int x;` 구문에 접근하고 싶으면 아래와 같이 입력하면 된다.

- ‘(:’와 ‘:’) 사이에 있는 문구는 주석이므로 실제 작성 시에는 작성하지 않아도 무방하다. 주석은 XPath 1.0에서는 제공되지 않는 기능이므로, 사용하고 싶다면 XPath 1.1 compatability 또는 XPath 2.0 버전을 사용하도록 한다.

(입력)

```
(: Field 선언이 "int"이고 변수 이름이 "x"인 노드 :)
//FieldDeclaration[Type/PrimitiveType[@Image="int"]
and
VariableDeclarator/VariableDeclaratorId[@Image="x"]]
```

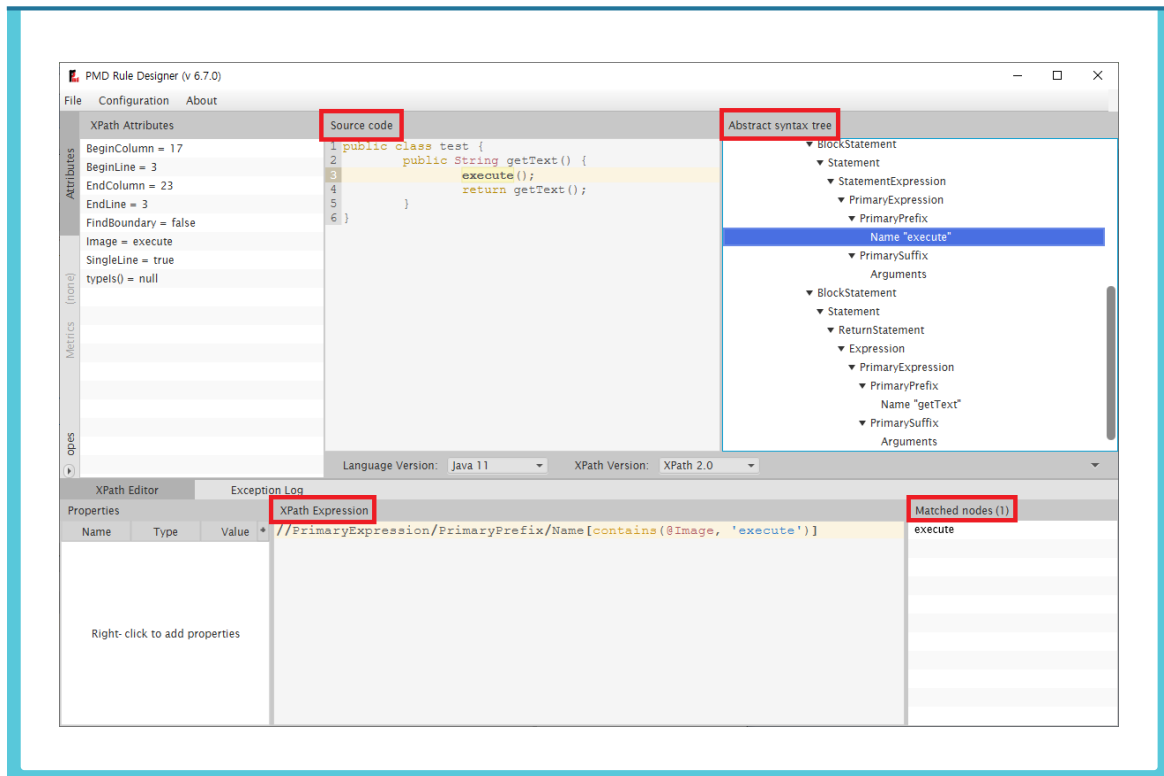


3. PMD Rule Designer 설명

PMD는 사용자 정의 룰을 개발한 코드를 빠르게 테스트할 수 있는 PMD Rule Designer를 제공한다. Source code 칸에 테스트할 샘플 코드를 입력하면 자동으로 분석하여 Abstract syntax tree 칸에 트리 구조를 보여준다.

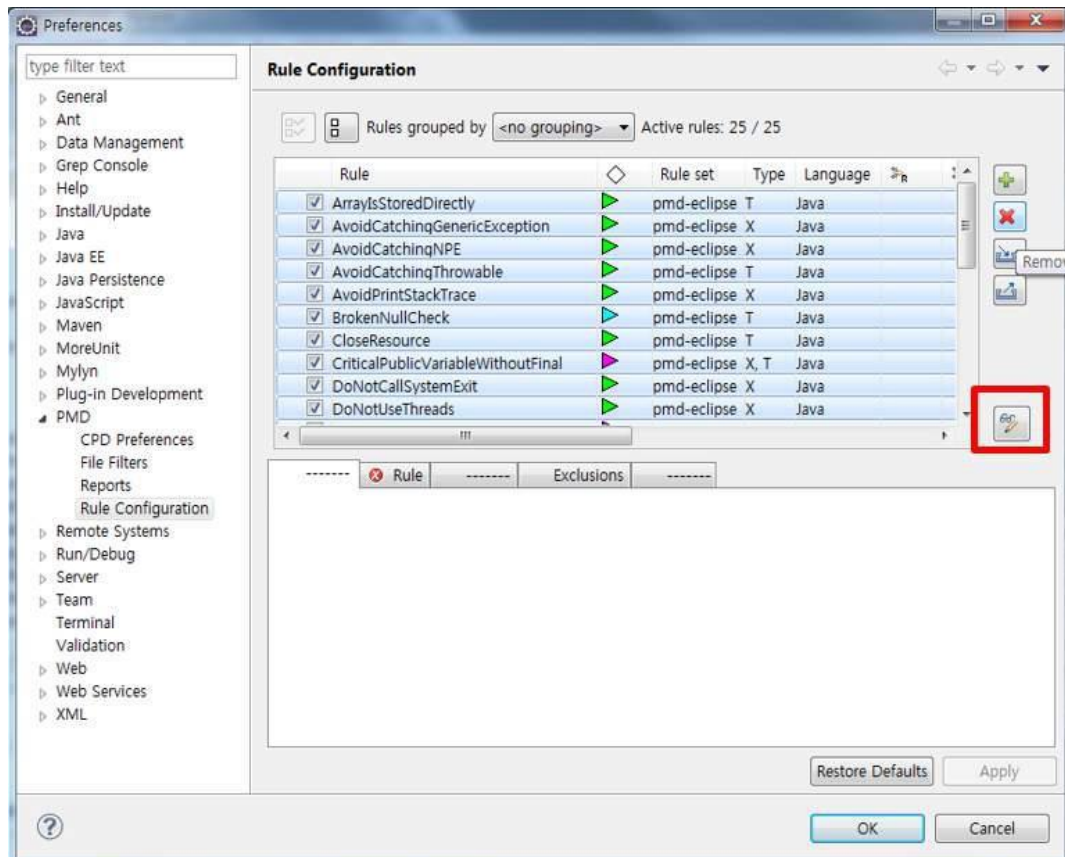
다음은 아래의 Rule Designer 화면에 대한 설명이다.

- Source code: 룰 위반사항과 정상적인 경우를 담은 sample 소스 작성
- XPath Expression: XPath 룰셋을 조절하면서 위반사항 발견 및 정상경우를 통과하는 룰셋 작성
- Abstract Syntax Tree(AST): AST를 확인하면서 XPath 룰셋 작성
- Matched nodes: 원하는 위반사항을 잡아내는지 확인



4. Eclipse플러그인에서PMD Rule Designer 사용

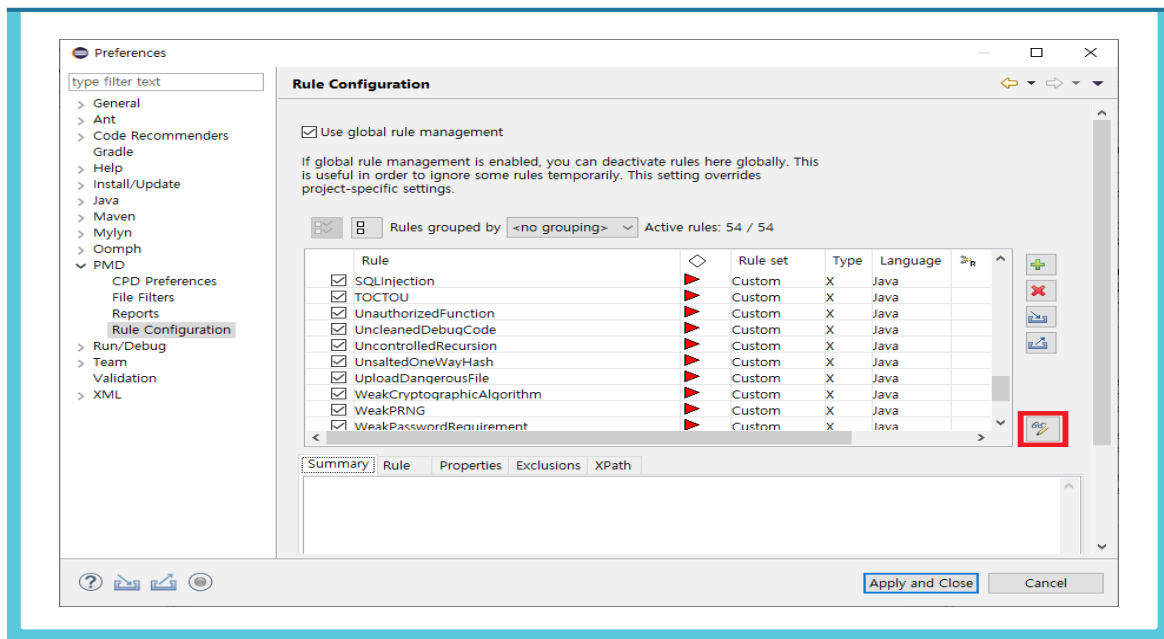
PMD로 룰셋을 만드는 법은 다음과 같다. 룰 작성은 Rule Designer을 통해서 작성한다. 이클립스에서 Windows > Preference > PMD 클릭 후 Rule Configuration 화면에서 Rule Designer 버튼을 클릭한다.



5. PMD Rule Designer를 사용한 작성법 사례

다음은 행정안전부의 47개 보안약점 중 하드코딩된 비밀번호에 대한 사용자 정의 룰 (Rule) 작성 예이다.

- ❶ 이클립스에서 Windows > Preference > PMD 클릭 후 Rule Configuration 화면에서 Rule Designer 버튼 선택한다.



- ❷ Rule Designer에서 Source Code 부분에 점검 대상이 되는 안전하지 않은 코드 (예제1)를 넣는다. 여기서는 소프트웨어 개발보안 가이드의 소스를 예제로 사용한다.

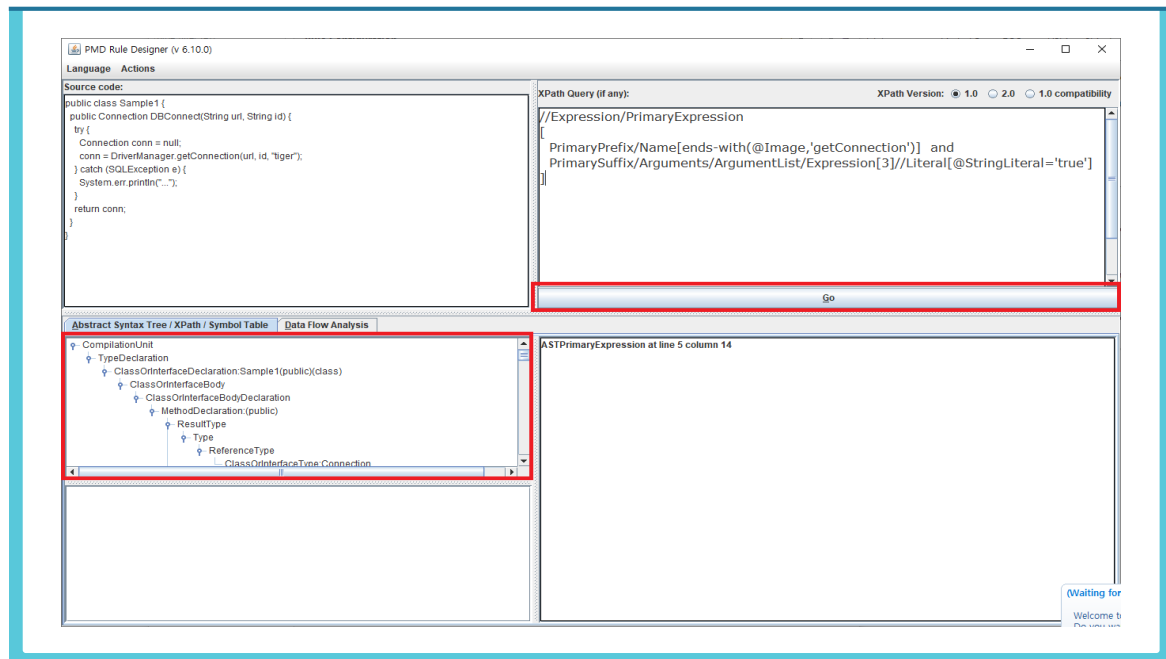
예제 1

```

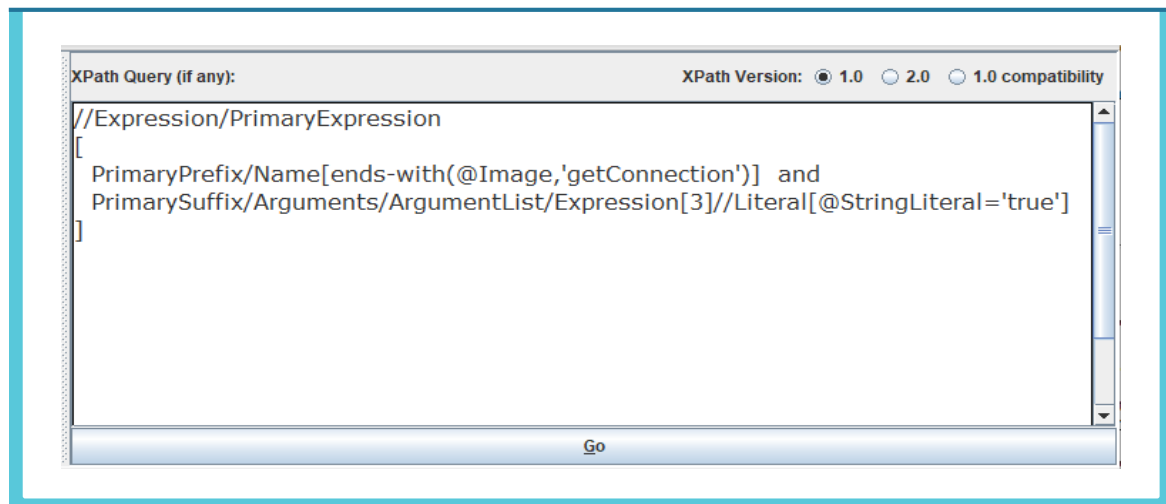
1 public class Sample1 {
2     public Connection DBconnet(String url,String id) {
3         try {
4             conn = DriverManager.getConnection(url,id, "tiger");
5         } catch (SQLException e) {
6             System.out.println("...");
7         }
8         return conn;
9     }
10 }

```

③ 오른쪽의 Go 버튼을 누른 후 해당 소스코드의 AST를 생성한다.



④ AST(Abstract Syntax Tree)를 참고하여 XPath문법에 맞게 룰(Rule)을 작성한다. 예시는 XPath 1.0 버전을 활용하였다.

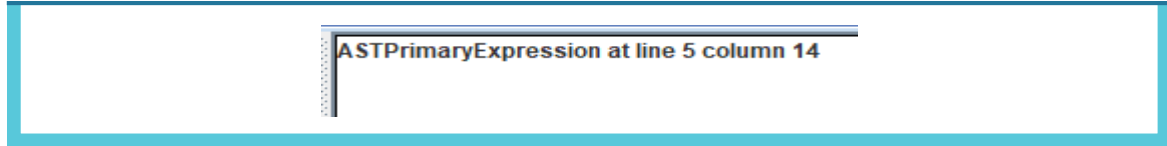


- '룰(Rule) 설명 : DB연결시 사용되는 getConnection 메소드가 사용되었는지 확인하고 getConnection 메소드의 세 번째 매개변수가 String 값을 직접 받는지 확인

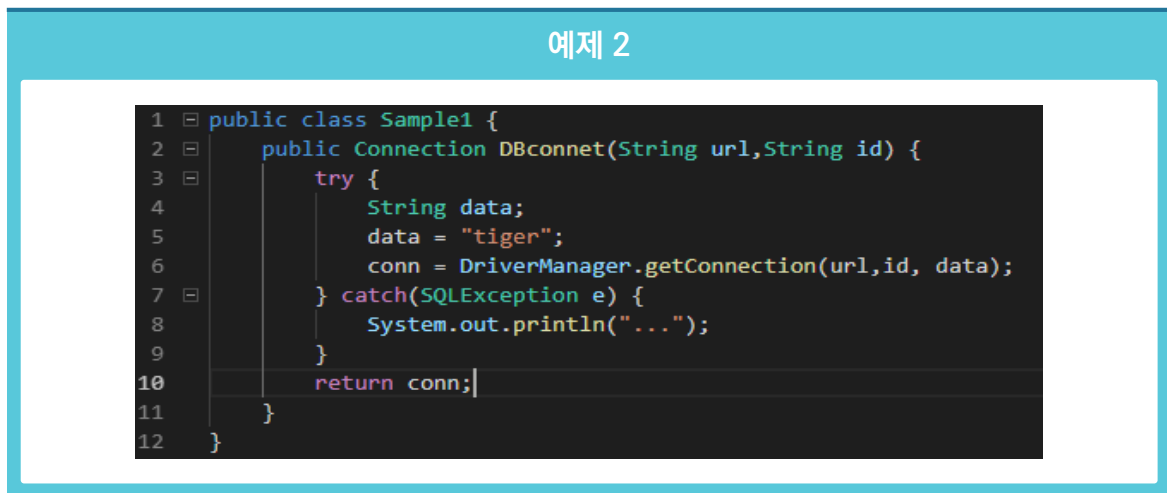
※ 참고: PMD 룰 작성 관련 자료는 아래의 사이트를 참고한다.

(<https://pmd.sourceforge.io/pmd-5.4.1/customizing/howtowritearule.html>)

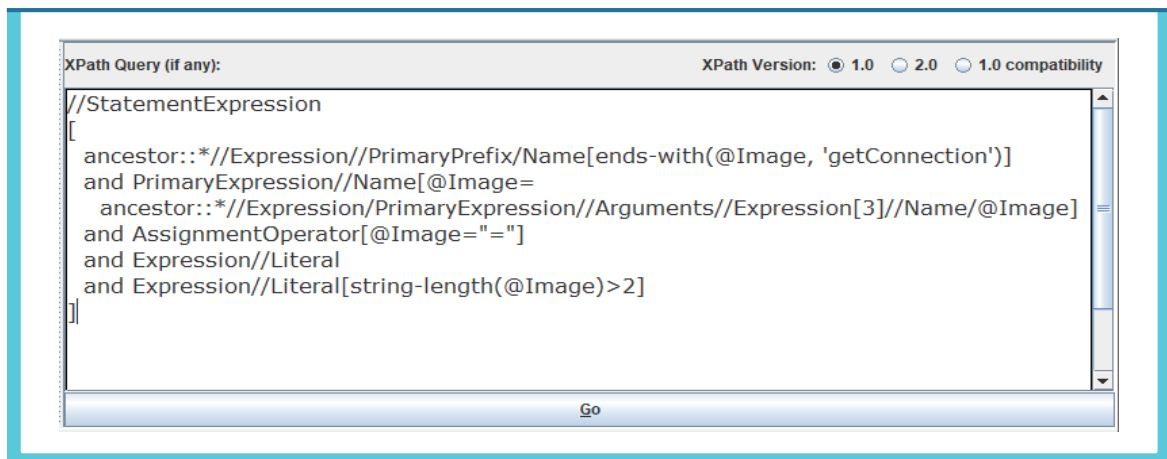
- ⑤ Go 버튼을 눌러서 룰이 맞게 적용되는지 결과 화면을 확인한다. 아래의 결과는 안전하지 않은 코드의 5번째 줄에서 보안약점을 찾았다는 결과를 나타낸다.



- ⑥ 위에서 만들어진 룰은 getConnection 메소드의 세번째 인자가 String 값을 직접 받는 패턴을 찾을 수 있으나 아래의 소스(예제2)와 같이 변수로 받는 패턴으로 변경되면 찾지 못하는 문제점이 있다.

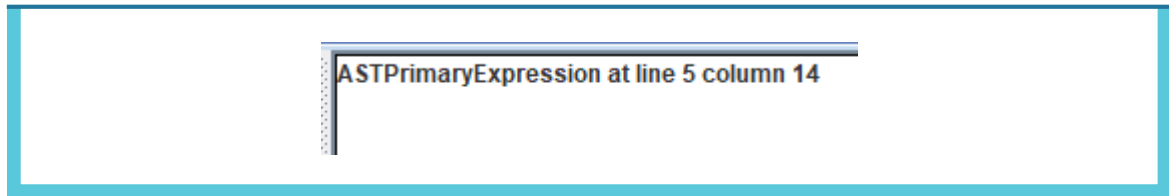


- ⑦ 위의 같은 경우를 대응하기 위해 아래와 같이 룰을 만들 수 있다.



- 룰(Rule) 설명 : DB 연결시 사용되는 getConnection 메소드가 사용되었는지 확인하고 선언된 변수가 getConnection 메소드의 인자로 사용되었는지 확인한 후에 변수에 문자열 값이 할당되었는지 확인

- ⑧ 안전하지 않은 코드 예제2와 대응되는 룰(Rule)을 Rule Designer에서 실행시키면 아래와 같이 보안약점을 찾을 수 있다.



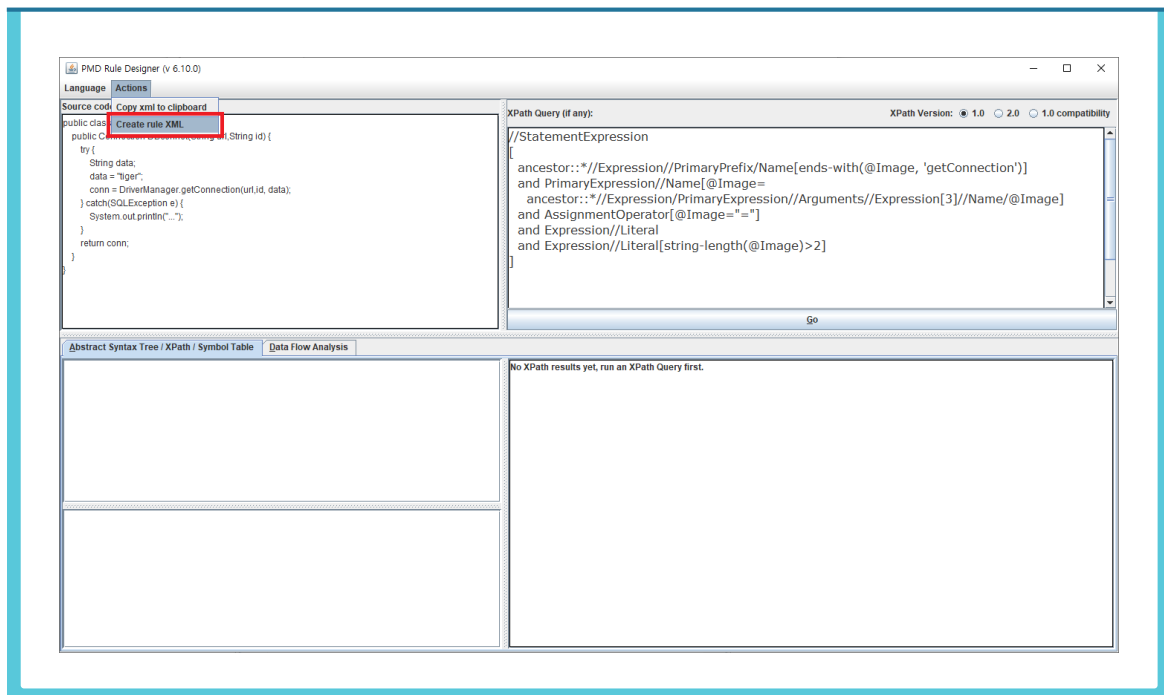
- ⑨ 예제1과 예제2의 룰을 같이 사용하면 하드코드된 비밀번호를 진단하는 확률이 높아질 것이다.

6. Eclipse Rule Designer PMD 룰 저장

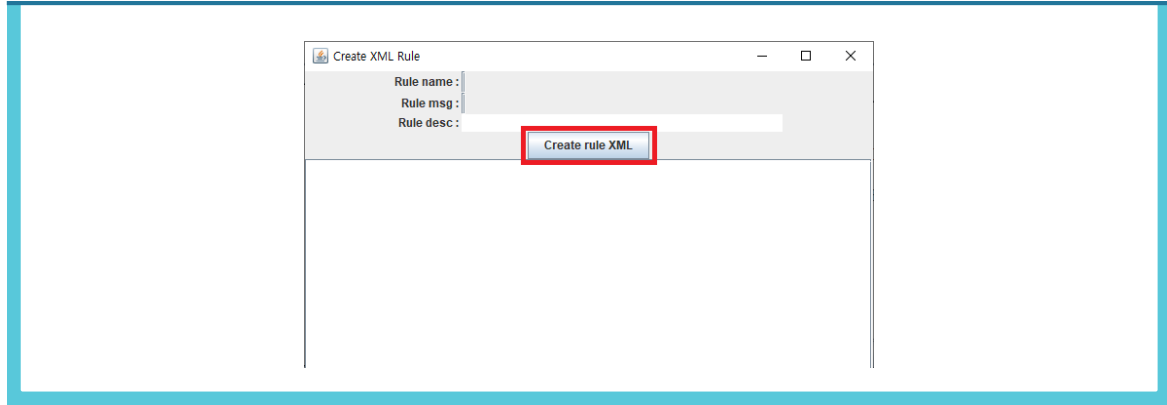
Rule Configuration 화면에서 'Add rule...'을 선택하여 룰을 등록할 수 있으나 최신 버전 Eclipse 와 PMD에서는 버그가 있어 룰 등록이 진행되지 않는다. XML 파일을 따로 저장하여 등록하는 방법을 소개한다.

저장된 XML 룰을 나중에 PMD에서 추가해서 사용하게 되는데 매번 새로운 룰을 만들 때마다 XML 파일이 생성된다면 사용자가 나중에 자신만의 커스텀 룰을 IDE에 추가할 때 매우 번거로운 것이다. 때문에 PMD에서는 ruleset 이라는 XML 파일에 여러 룰들을 포함하여 한 번에 관리할 수 있다. 다음은 ruleset을 만드는 방법이다.

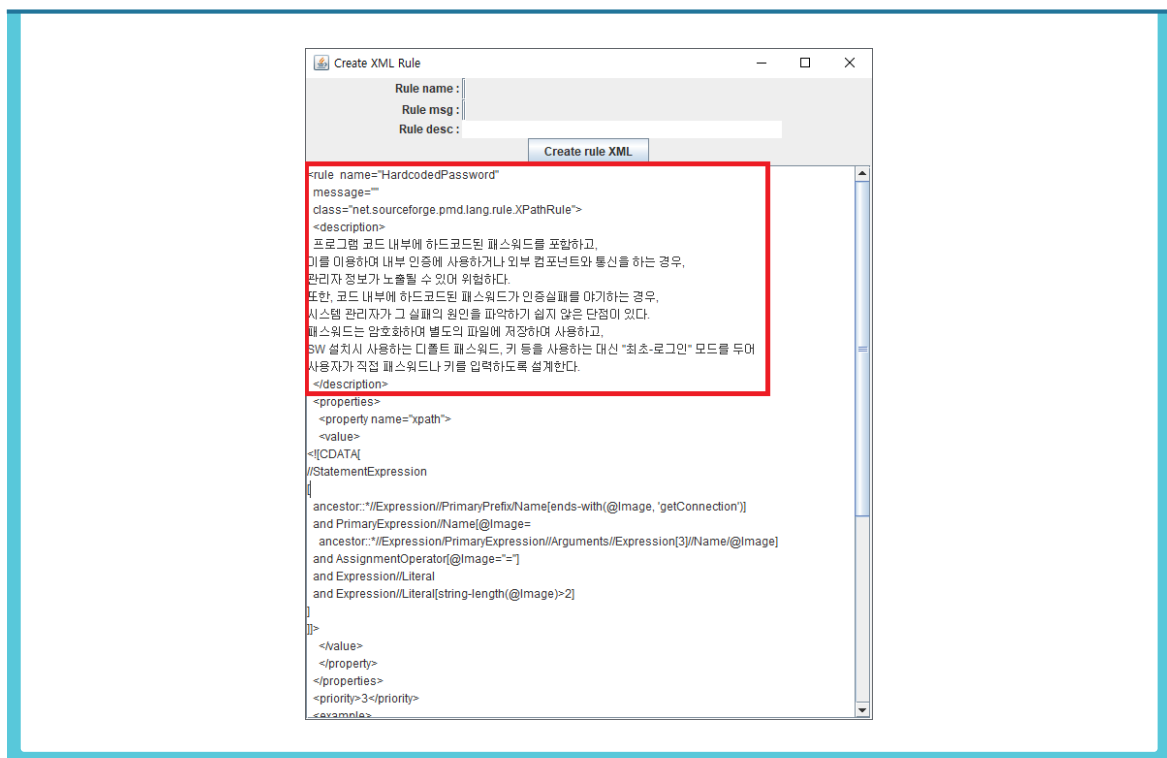
❶ Rule Designer > Actions > Create rule XML 메뉴를 선택한다.



- ② 팝업 창에서 'Create rule XML'을 클릭한다. 비어 있는 아래 칸이 XML 형식의 룰로 채워진다.



- ③ XML에서 rule name, message, description을 채워준다.



- ④ 위의 XML 형식의 룰을 포함할 룰셋을 제작한다. 룰셋 형식은 해당 링크에서 확인할 수 있으며 다음과 같다. <ruleset> 태그에 룰셋 정보를 입력하고 <description>에 룰셋에 대한 설명을 추가한다.

```

1 <?xml version="1.0"?>
2 <ruleset name="Custom Rules"
3   xmlns="http://pmd.sourceforge.net/ruleset/2.0.0"
4   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5   xsi:schemaLocation="http://pmd.sourceforge.net/ruleset/2.0.0 http://pmd.sourceforge.net/ruleset_2_0_0.xsd">
6   <description>KISA Security Rule</description>
7
8   <!-- Rule here ... -->
9
10 </ruleset>

```

- ⑤ 위에서 제작한 XML을 복사하여 <description>과 같은 레벨로 넣어준다. 그림에서 <!-- Rules here ...-->에 해당되는 부분이다. XML을 복사할 때는 Rule Designer의 팝업 창에서 XML을 부분을 클릭하고 전체 선택(Ctrl+A)과 복사하기(Ctrl+C)를 통해 편하게 코드를 옮길 수 있다.

```

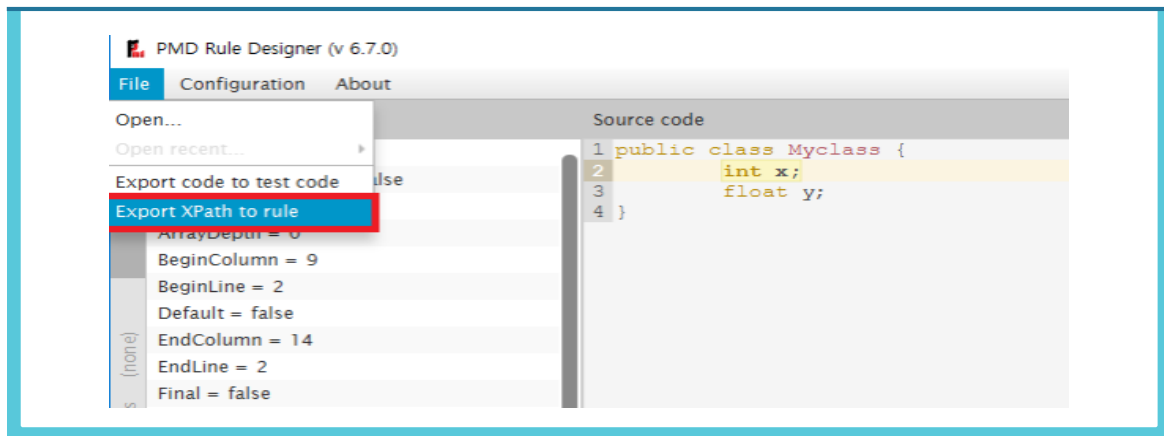
1 <?xml version="1.0"?>
2 <ruleset name="Custom Rules"
3   xmlns="http://pmd.sourceforge.net/ruleset/2.0.0"
4   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5   xsi:schemaLocation="http://pmd.sourceforge.net/ruleset/2.0.0 http://pmd.sourceforge.net/ruleset_2_0_0.xsd">
6   <description>KISA Security Rule</description>
7
8   <rule name="HardcodedPassword"
9     message="
10     class="net.sourceforge.pmd.lang.rule.XPathRule">
11     <description>
12       프로그램 코드 내부에 하드코딩된 패스워드를 포함하고,
13       이를 이용하여 내부 인증에 사용하거나 외부 컴포넌트와 통신을 하는 경우,
14       관리자 정보가 노출될 수 있어 위험하다.
15       또한, 코드 내부에 하드코딩된 패스워드가 인증실패를 야기하는 경우,
16       시스템 관리자가 그 실패의 원인을 파악하기 쉽지 않은 단점이 있다.
17       패스워드는 암호화하여 별도의 파일에 저장하여 사용하고,
18       SW 설치시 사용하는 디폴트 패스워드, 키 등을 사용하는 대신 "최초-로그인" 모드를 두어
19       사용자가 직접 패스워드나 키를 입력하도록 설계한다.
20     </description>
21     <properties>
22       <property name="xpath">
23         <value>
24           <![CDATA[
25             //StatementExpression
26             [
27               ancestor::*//Expression//PrimaryPrefix/Name[ends-with(@Image, 'getConnection')]
28               and PrimaryExpression//Name[@Image=
29                 ancestor::*//Expression/PrimaryExpression//Arguments//Expression[3]//Name[@Image]
30               and AssignmentOperator[@Image="="]
31               and Expression//Literal
32               and Expression//Literal[string-length(@Image)>2]
33             ]
34           ]>
35         </value>
36       </property>
37     </properties>
38     <priority>3</priority>
39     <example>
40     <![CDATA[
41       public class Sample1 {
42         public Connection DBconnet(String url,String id) {
43           try {
44             String data;
45             data = "tiger";
46             conn = DriverManager.getConnection(url,id, data);
47           } catch(SQLException e) {
48             System.out.println("...");
49           }
50         }
51       }
52     ]>
53     </example>
54   </rule>
55 </ruleset>

```

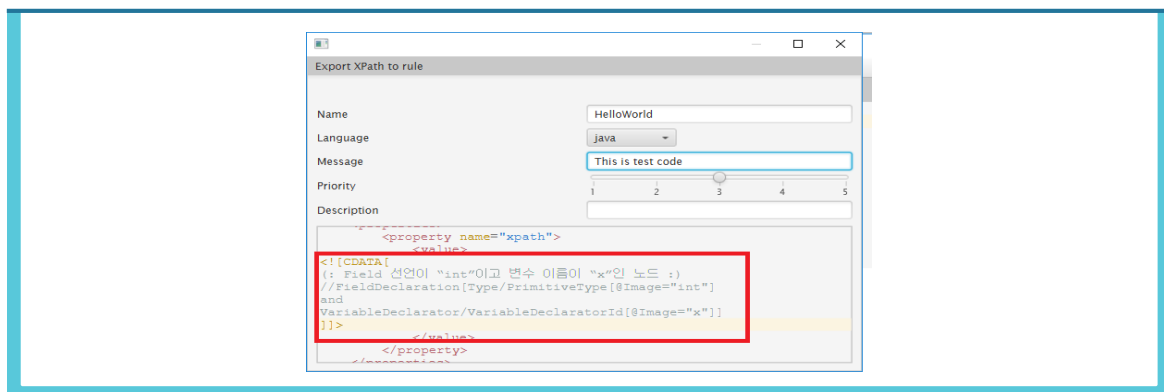

7. Stand-alone Rule Designer PMD 룰 저장

PMD의 바이너리 파일을 받게 되면, bin 폴더 아래 designer.bat 형태로 제공된다. 해당 파일을 더블 클릭하여 실행한다. Rule Designer에서 작성한 XPath 문법을 XML 파일로 저장한다. 제작한 룰을 룰셋으로 만드는 방법은 Eclipse 플러그인과 동일하다.

- ① Rule Designer에서 File > Export XPath to rule를 선택한다.



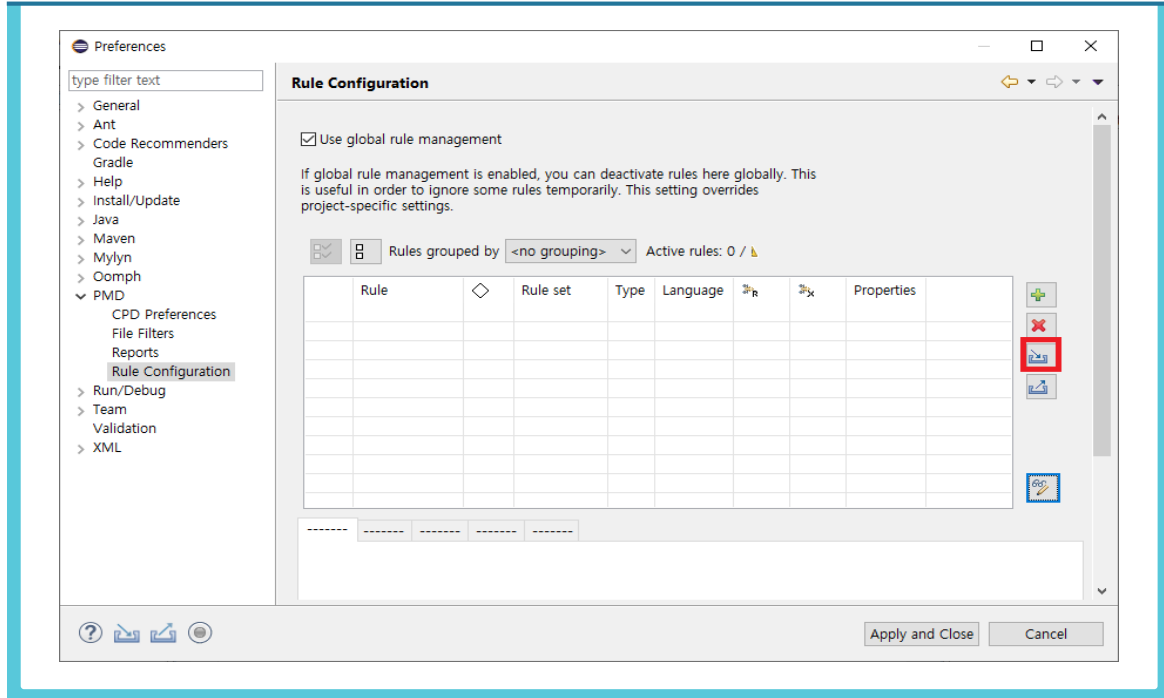
- ② 팝업 창에서 작성한 XPath 룰이 포함된 XML이 나타난 것을 확인한다. 작성된 XPath 룰은 다음과 같은 형식 안에 포함되어 있다: <![CDATA[내용]]>. Name, Message, Priority, Description 등을 작성하면, XML에 자동으로 반영되어 나타난다.



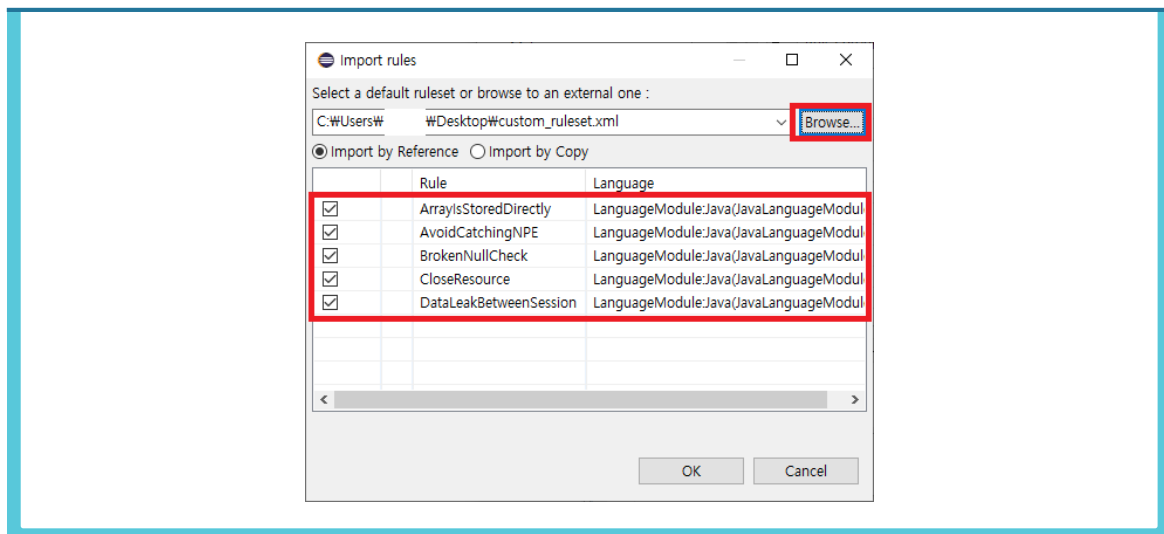
- ③ Eclipse에서 룰셋을 제작한 방식 그대로 XML을 복사하여 룰셋을 제작한다.

8. 룰셋 Eclipse IDE 플러그인에 추가

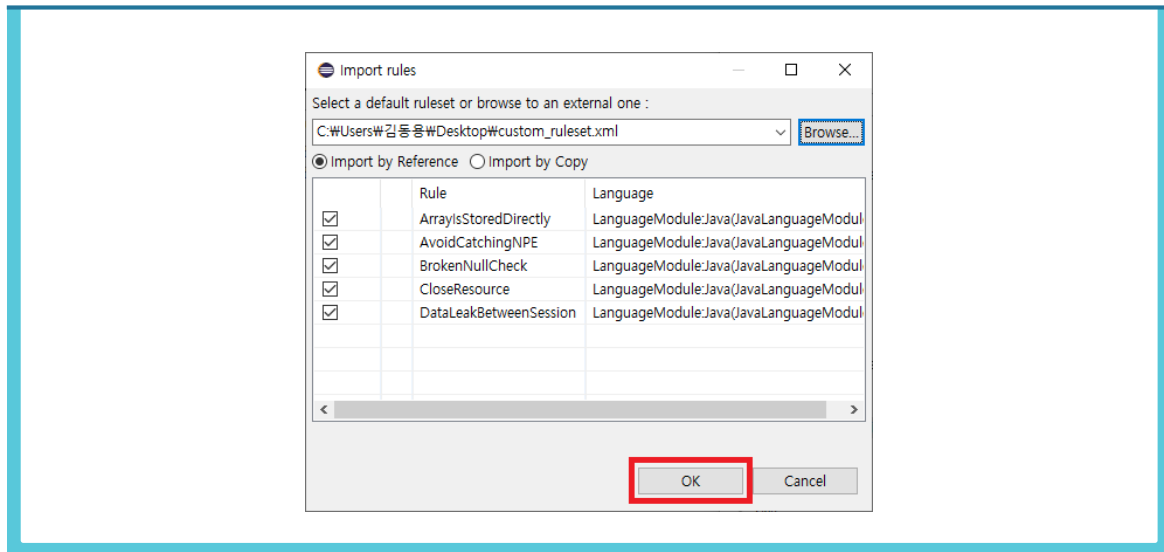
- ❶ 작성된 룰셋을 Eclipse 플러그인에 추가한다. Eclipse > PMD > Rule Configuration 에서 Import rule set... 버튼을 클릭한다.



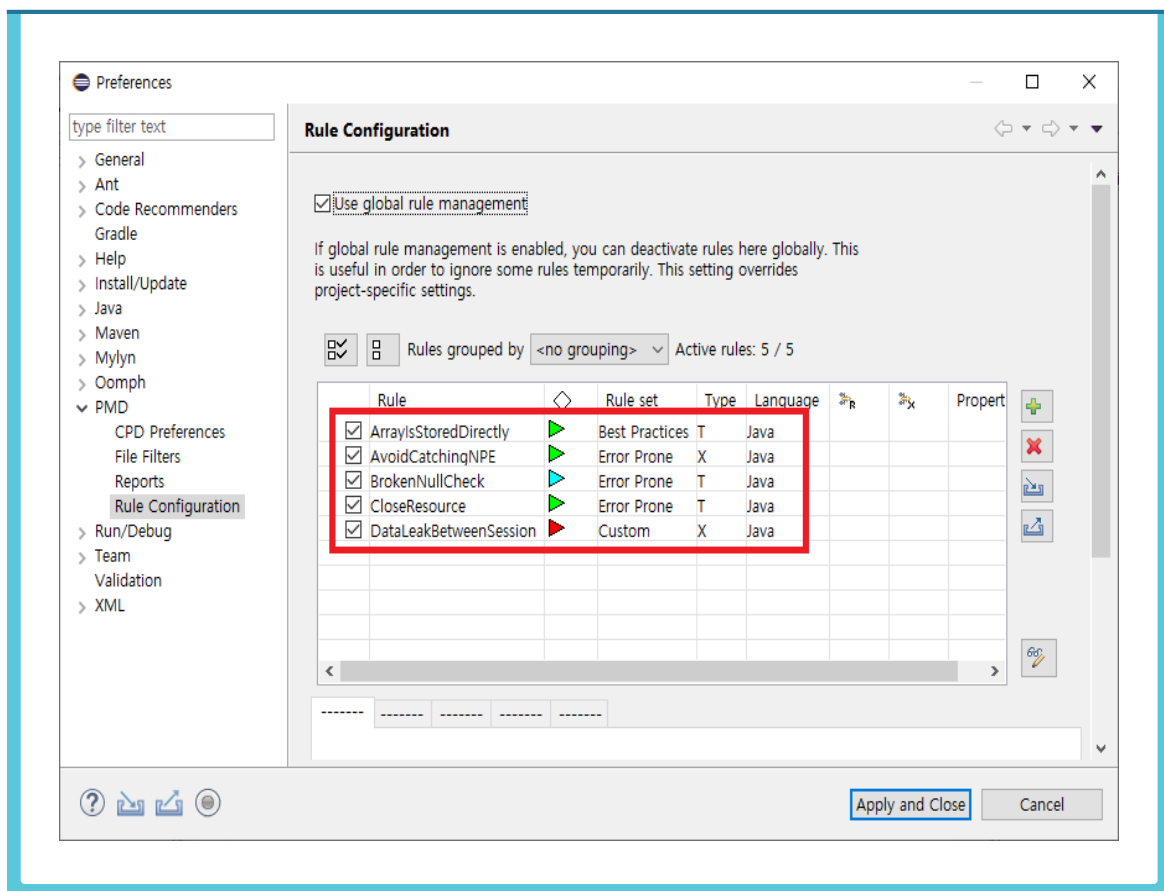
- ❷ 팝업 창에서 Browse...를 클릭하고 제작한 룰셋 XML 파일을 선택한다. 룰셋에 포함된 룰들이 출력된다.



③ 체크박스를 모두 선택하고 OK 버튼을 눌러 룰을 적용한다.

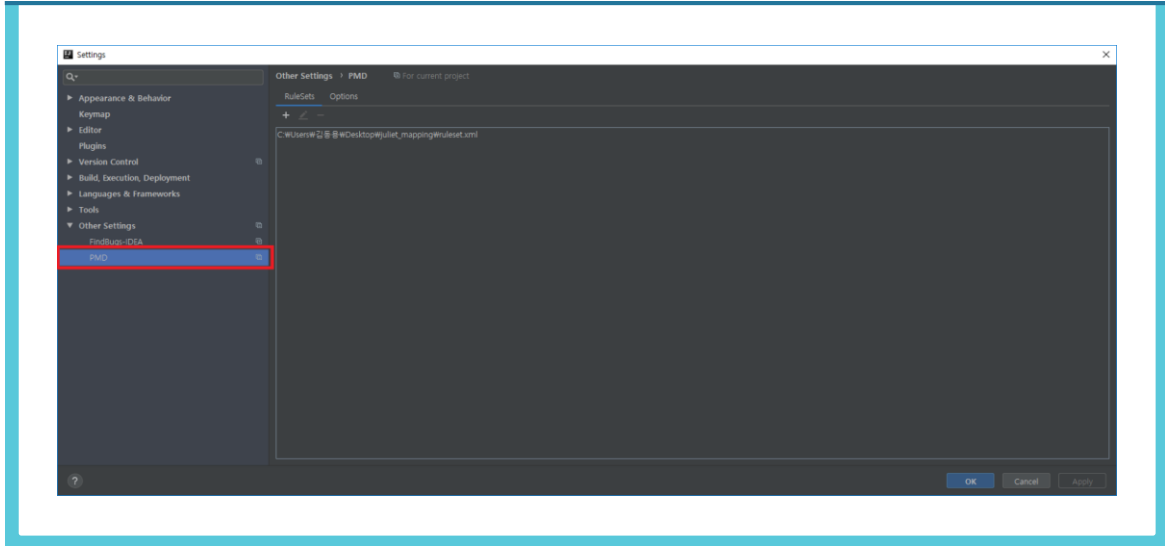


④ 설정 창을 닫고 다시 실행하면 룰들이 적용되어 있는 것을 확인할 수 있다.

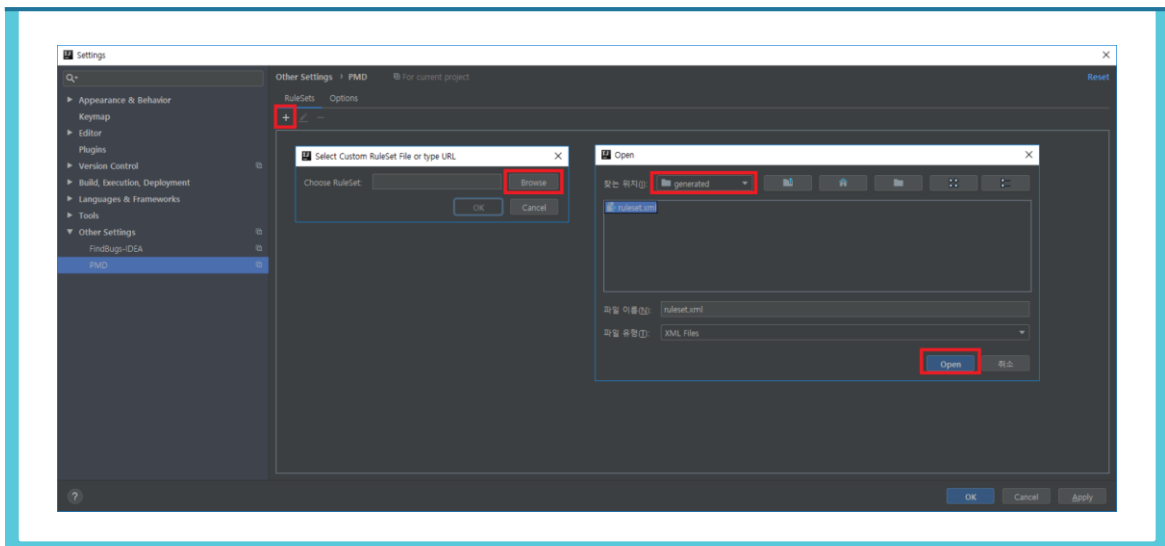


9. 룰셋 IntelliJ IDE 플러그인에 추가

- ① 작성된 룰셋을 IntelliJ 플러그인에 추가한다. IntelliJ에서 File > Setting를 선택하고 Settings 창에서 PMD 탭을 클릭한다.



- ② PMD 탭 중 RuleSets 밑에 “+”버튼을 클릭한 뒤 Browse 버튼을 클릭한다. 제작한 룰셋의 경로를 찾아 Open으로 룰셋을 추가한다.



- ③ Eclipse와 달리 IntelliJ는 적용되는 룰 정보를 따로 제공해주지 않는다.

제4절 오탐(False Positive) 관리

보안약점이 내포된 Bad 코드에서 취약한 함수(또는 메소드) 등의 위치를 정확하게 탐지한 경우 ‘정탐(True Positive)’으로 판정하며 그렇지 못한 경우 ‘미탐(False Negative)’으로 판정한다. 또한, 보안약점이 해결되어 보안약점이 존재하지 않는 Good 코드에서 보안 약점이 존재한다고 진단하는 경우 ‘오탐(False Positive)’으로 판정한다.

구분	코드유형	판정기준
정탐	Bad	보안약점이 내재된 함수 등의 위치를 정확하게 탐지한 경우
오탐	Good	보안약점이 해결된 함수 등을 탐지하는 경우
미탐	Bad	보안약점이 내재된 함수 등을 탐지 못하는 경우

시큐어코딩 점검 시 오탐에 대해서 효과적으로 관리하지 못하면 오탐을 계속 분석을 하게 되어 점검 효율이 떨어지게 된다.

Spotbugs와 PMD는 오탐을 관리할 수 있는 방법을 제공한다. 또한 추가적으로 Jenkins의 리포팅 변화 추이를 활용한 방법을 제시한다. 상황에 맞게 오탐관리 방법을 선택하면 점검의 효율성을 높일 수 있게 된다.

1. Spotbugs에서 제공하는 방법

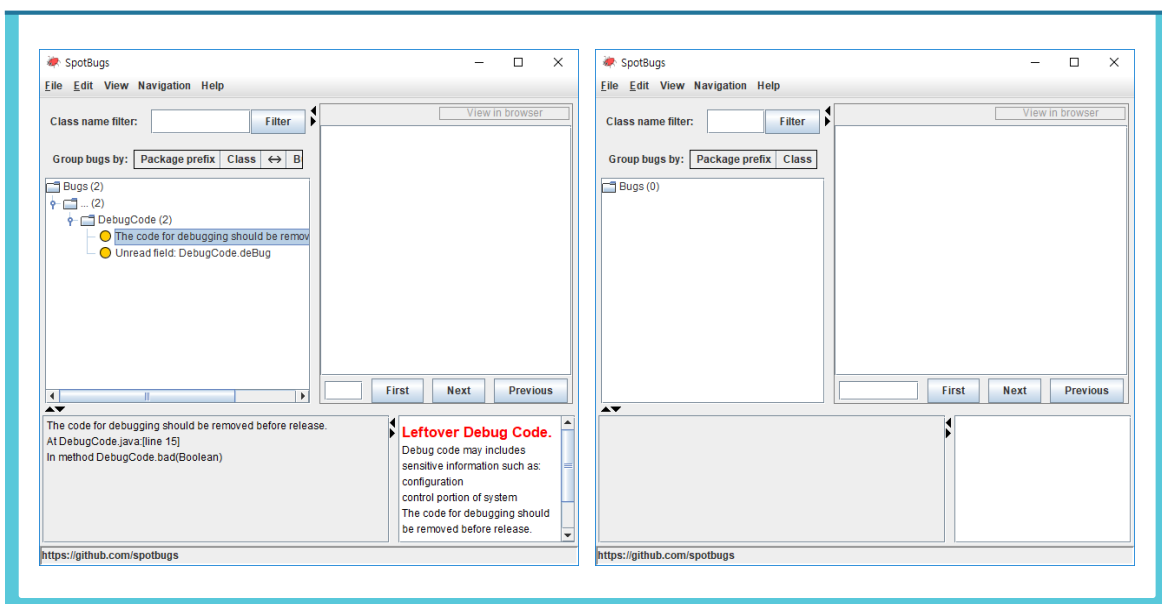
- Spotbugs에서는 예외 처리를 분석 단위인 메소드 혹은 클래스 단위로 진행한다. 예외 처리를 하고 싶은 클래스 또는 메소드 앞에 `@SuppressWarnings` annotation 을 삽입한다.

```

1 import edu.umd.cs.findbugs.annotations.SuppressFBWarnings;
2
3 @SuppressWarnings()
4 public class DebugCode {
5
6     private static boolean DEBUG = false;
7     private boolean deBug = true;
8
9     public static void good(Boolean bool) {
10         // Just do something
11         System.out.println("good");
12     }
13
14     public static void bad(Boolean bool) {
15         if(DEBUG) {
16             // Do something if debug flag is set
17             System.out.println("bad");
18         }
19         // Do something
20     }
21
22     public void main(String[] args) {
23         good(true);
24         bad(true);
25     }
26 }
27

```

- 아래는 `@SuppressWarnings()`을 적용 전, 적용 후의 결과이다. `@SuppressWarnings` 이후에 해당 클래스에서 검출되는 버그가 사라진 것을 확인할 수 있다.



❸ @SuppressWarnings() Annotation을 이용한 오탐 관리는 다음과 같은 단점을 지닌다. 분석 결과에서 즉시 예외 처리할 수 없고 단점 또한 다수 존재하기 때문에 Spotbugs를 이용한 분석 결과는 참고용으로 사용하는 것이 적합하다.

1. 클래스 또는 메소드 단위로 분석 범위를 제외시키기 때문에 해당 클래스 또는 메소드에 탐지되는 모든 취약점 제외
2. 소스 코드에 실제 프로젝트와는 관련 없는 코드인 @SuppressWarnings() annotation이 추가
3. @SuppressWarnings()을 import하기 위해서 의존성이 발생

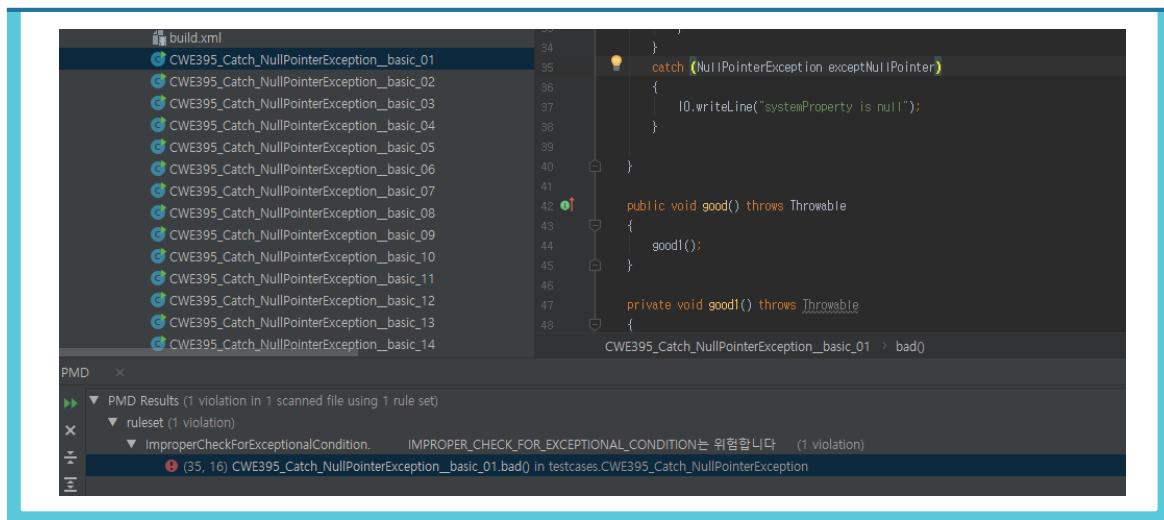
아래는 findbugs.annotations을 위해 pom.xml에 추가되는 의존성 코드이다.

```
<dependencies>
  <dependency>
    <groupId>net.jcip</groupId>
    <artifactId>jcip-annotations</artifactId>
    <version>1.0</version>
    <optional>true</optional>
  </dependency>
  <dependency>
    <groupId>com.github.spotbugs</groupId>
    <artifactId>spotbugs-annotations</artifactId>
    <version>3.1.3</version>
    <optional>true</optional>
  </dependency>
</dependencies>
</project>
```

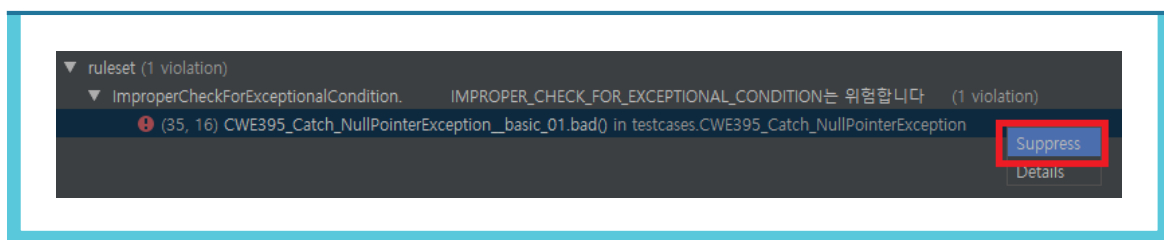
2. PMD에서 제공하는 방법

PMD의 경우 해당 라인에 `//NOPMD`를 추가하면 해당 라인은 검사를 하지 않고 진행한다. 소스 코드의 해당 라인을 예외 처리하는 것이기 때문에 해당 라인에서 발견되는 모든 보안 약점이 예외 처리된다.

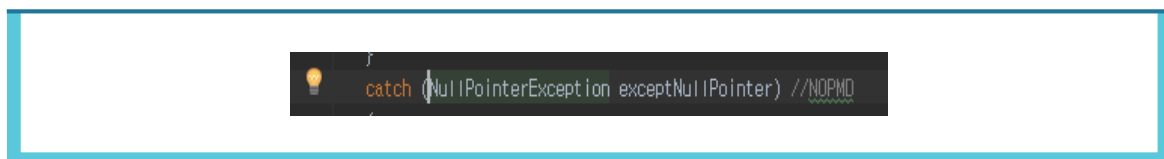
❶ 다음 예시는 커스텀 룰셋으로 `NullPointerException`이 발견된 경우다.



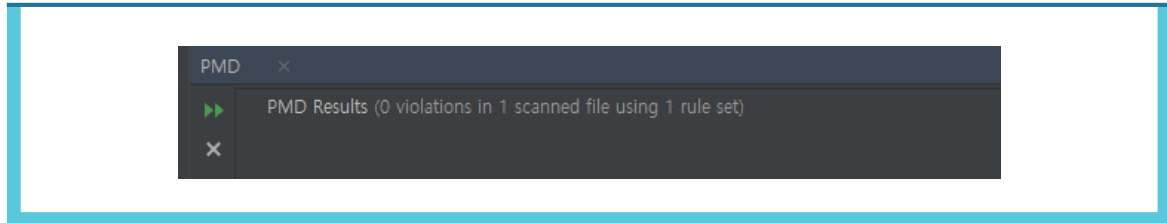
❷ 예외 처리를 하고 싶은 보안 약점에서 마우스 오른쪽 클릭 > Suppress를 클릭한다.



❸ 보안 약점이 발견된 코드에 `//NOPMD`가 자동으로 추가된다.



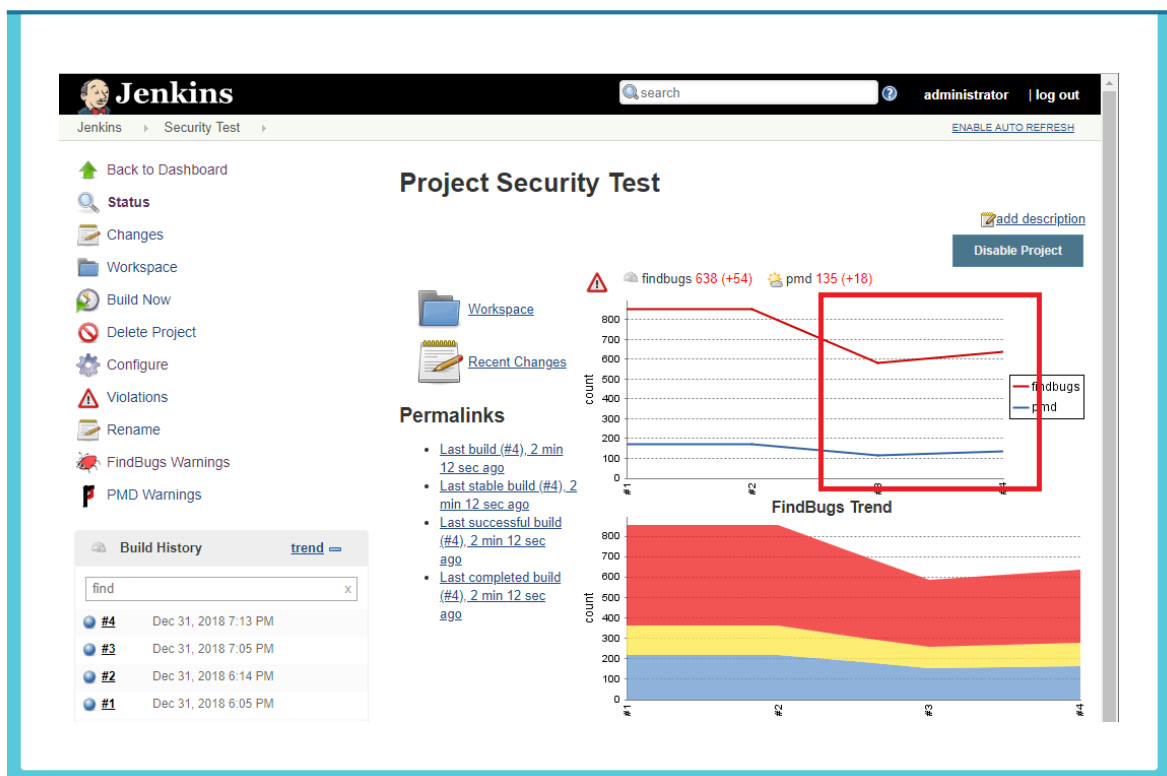
- ④ //NOPMD가 추가된 상태에서 분석을 다시 진행하면, 해당 라인이 예외 처리 되어 취약점이 발견되지 않는 것을 확인할 수 있다.

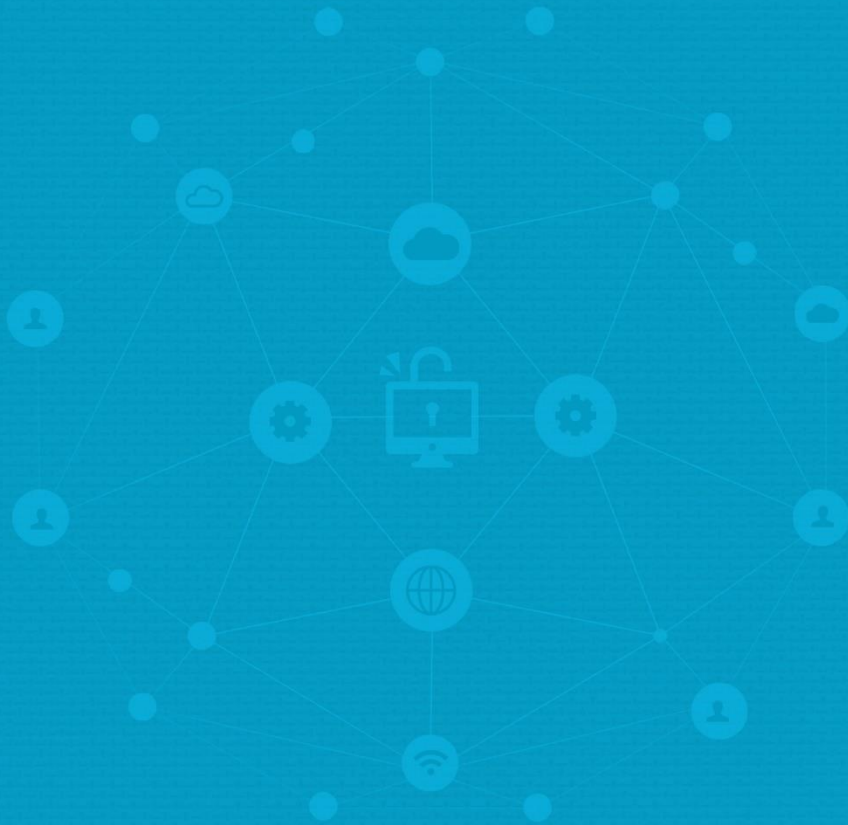


3. Jenkins의 리포팅 변화 추이를 활용하는 방법

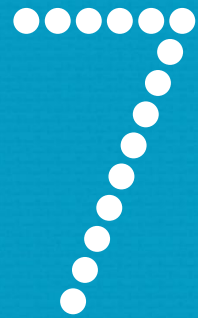
jenkins에서는 이전단계 빌드에서 추가/고정(fixed)된 현황을 모니터링 할 수 있다. 이를 통해서 이전 단계에서 확증된 내용 이외에 추가된 이슈가 무엇인지 확인할 수 있다.

장점으로는 변동 사항에 대해서만 집중할 수 있다는 점이 있다. 단점으로는 변화하는 과정이 아닌 특정 단계에서 오탐과 정탐이 무엇인지 구분하기 어렵다는 점이 있다.





PART



제7장 행안부 47개 보안약점 관련 공개SW 진단도구 룰(Rule) 분석

- 제1절 행정안전부 47개 보안약점 관련 공개SW
도구 룰(Rule) 목록
- 제2절 행정안전부 47개 보안약점 관련 공개SW
도구 룰(Rule) 현황
- 제3절 공개 소프트웨어 진단도구별 성능 비교 분석
- 제4절 공개 소프트웨어 진단도구 다중 효과성
비교 분석



제7장 행안부 47개 보안약점 관련 공개SW 진단도구 룰(Rule) 분석



제1절 행정안전부 47개 보안약점 관련 공개SW 도구 룰(Rule) 목록

다음은 행정안전부 47개 보안약점에 대응되는 현황이다. Spotbugs, FindSecurityBugs, PMD에서 기본적으로 제공하지 않는 룰은 사용자 정의 룰의 개발을 통해서 추가 룰을 개발 할 수 있다. 아래의 룰 매핑 현황은 업데이트 등 지속적인 검토가 필요하며, 공개 소프트웨어 도구를 활용하기 위한 참고 자료로만 활용할 것을 권장한다.

No.	구분	행안부 SW 보안약점	관련 룰 보유 여부			
			SB	FSB	SB+FSB	PMD
1	입력데이터 검증 및 표현	SQL 삽입	O	O	O	
2		경로 조작 및 자원 삽입	O	O	O	
3		크로스사이트 스크립트	O	O	O	
4		운영체제 명령어 삽입		O	O	
5		위험한 형식 파일 업로드		O	O	
6		신뢰되지 않는 URL 주소로 자동 접속 연결		O	O	
7		XQuery 삽입		O	O	
8		XPath 삽입		O	O	
9		LDAP 삽입		O	O	
10		크로스 사이트 요청 위조		O	O	
11		HTTP 응답분할	O	O	O	
12		정수형 오버플로우	O		O	
13		보안기능 결정에 사용되는 부적절한 입력값		O	O	
14		메모리 버퍼 오버플로우	O		O	
15		포맷스트링 삽입		O	O	

No.	구분	행안부 SW 보안약점	관련 룰 보유 여부			
			SB	FSB	SB+FSB	PMD
16	보안기능	적절한 인증없는 중요기능 허용		0	0	
17		부적절한 인가		0	0	
18		중요한 자원에 대한 잘못된 권한 설정		0	0	
19		취약한 암호화 알고리즘 사용		0	0	
20		중요정보 평문저장		0	0	
21		중요정보 평문전송		0	0	
22		하드코드된 비밀번호	0	0	0	
23		충분하지 않은 키 길이 사용		0	0	
24		적절하지 않은 난수 값 사용	0	0	0	
25		하드코드된 암호화 키		0	0	
26		취약한 비밀번호 허용				
27		사용자 하드디스크에 저장되는 쿠키를 통한 정보 노출		0	0	
28		주석문 안에 포함된 시스템 주요정보				
29		솔트 없이 일방향 해쉬 함수 사용		0	0	
30		무결성 검사없는 코드 다운로드		0	0	
31		반복된 인증시도 제한 기능 부재				
32	시간 및 상태	경쟁조건: 검사시점과 사용시점(TOCTOU)	0		0	
33		종료되지 않는 반복문 또는 재귀 함수	0		0	
34	에러처리	오류 메시지를 통한 정보 노출 (시스템 데이터 정보 노출)		0	0	0
35		오류 상황 대응 부재				0
36		부적절한 예외처리	0		0	0
37	코드오류	널(Null) 포인터 역참조	0		0	0
38		부적절한 자원 해제	0	0	0	0
39		해제된 자원 사용				
40		초기화되지 않은 변수 사용	0		0	
41	캡슐화	잘못된 세션에 의한 데이터 정보 노출		0	0	
42		제거되지 않고 남은 디버그 코드				
43		시스템 데이터 정보노출		0	0	
44		Public 메소드부터 반환된 Private 배열	0		0	0
45		Private 배열에 Public 데이터 할당	0		0	0
46	API 오용	DNS lookup에 의존한 보안 결정				
47		취약한 API 사용		0	0	

제2절 행정안전부 47개 보안약점 관련 공개SW 도구 룰(Rule) 현황

다음은 행정안전부 47개 보안약점 관련된 Spotbugs, FindSecurityBugs, PMD 등 공개SW 도구가 제공하는 룰 현황이다. 다만, 관련 룰은 공개SW 도구의 버전 업데이트 등에 따라 지원하는 현황이 달라질 수 있다.

1. 행정안전부 47개 보안약점 관련 Spotbugs 기본 Bug Description

행안부 SW 보안약점	관련 Bug Description
SQL 삽입	SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE SQL_PREPARED_STATEMENT_GENERATED_FROM_NONCONSTANT_STRING
경로 조작 및 자원 삽입	PT_RELATIVE_PATH_TRAVERSAL PT_ABSOLUTE_PATH_TRAVERSAL
크로스사이트 스크립트	XSS_REQUEST_PARAMETER_TO_SERVLET_WRITER XSS_REQUEST_PARAMETER_TO_JSP_WRITER XSS_REQUEST_PARAMETER_TO_SEND_ERROR
HTTP 응답분할	HRS_REQUEST_PARAMETER_TO_COOKIE HRS_REQUEST_PARAMETER_TO_HTTP_HEADER
정수형 오버플로우	ICAST_INTEGER_MULTIPLY_CAST_TO_LONG IM_AVERAGE_COMPUTATION_COULD_OVERFLOW
메모리 버퍼 오버플로우	IL_CONTAINER_ADDED_TO_ITSELF IL_INFINITE_RECURSIVE_LOOP
하드코딩된 비밀번호	DMI_CONSTANT_DB_PASSWORD DMI_EMPTY_DB_PASSWORD
적절하지 않은 난수 값 사용	DMI_RANDOM_USED_ONLY_ONCE
경쟁조건: 검사시점과 사용시점(TOCTOU)	WL_USING_GETCLASS_RATHER_THAN_CLASS_LITERAL MSF_MUTABLE_SERVLET_FIELD
종료되지 않는 반복문 또는 재귀 함수	IL_INFINITE_RECURSIVE_LOOP IL_CONTAINER_ADDED_TO_ITSELF
부적절한 예외처리	REC_CATCH_EXCEPTION

행안부 SW 보안약점	관련 Bug Description
널(Null) 포인터 역참조	NP_BOOLEAN_RETURN_NULL NP_CLONE_COULD_RETURN_NULL NP_EQUALS_SHOULD_HANDLE_NULL_ARGUMENT NP_TOSTRING_COULD_RETURN_NULL NP_ALWAYS_NULL NP_ALWAYS_NULL_EXCEPTION NP_ARGUMENT_MIGHT_BE_NULL NP_CLOSING_NULL NP_GUARANTEED_DEREF NP_GUARANTEED_DEREF_ON_EXCEPTION_PATH NP_NONNULL_FIELD_NOT_INITIALIZED_IN_CONSTRUCTOR NP_NONNULL_PARAM_VIOLATION NP_NONNULL_RETURN_VIOLATION NP_NULL_INSTANCEOF NP_NULL_ON_SOME_PATH NP_NULL_ON_SOME_PATH_EXCEPTION NP_NULL_PARAM_DEREF NP_NULL_PARAM_DEREF_ALL_TARGETS_DANGEROUS NP_NULL_PARAM_DEREF_NONVIRTUAL NP_OPTIONAL_RETURN_NULL NP_STORE_INTO_NONNULL_FIELD NP_UNWRITTEN_FIELD NP_SYNC_AND_NULL_CHECK_FIELD NP_DEREFERENCE_OF_READLINE_VALUE NP_IMMEDIATE_DEREFERENCE_OF_READLINE NP_LOAD_OF_KNOWN_NULL_VALUE NP_METHOD_PARAMETER_TIGHTENS_ANNOTATION NP_METHOD_RETURN_RELAXING_ANNOTATION NP_NULL_ON_SOME_PATH_FROM_RETURN_VALUE NP_NULL_ON_SOME_PATH_MIGHT_BE_INFEASIBLE NP_PARAMETER_MUST_BE_NONNULL_BUT_MARKED_AS_NULLABLE NP_UNWRITTEN_PUBLIC_OR_PROTECTED_FIELD
부적절한 자원 해제	OS_OPEN_STREAM OS_OPEN_STREAM_EXCEPTION_PATH UL_UNRELEASED_LOCK_EXCEPTION_PATH UL_UNRELEASED_LOCK TLW_TWO_LOCK_WAIT SWL_SLEEP_WITH_LOCK_HELD
초기화되지 않은 변수 사용	DC_PARTIALLY_CONSTRUCTED IC_SUPERCLASS_USES_SUBCLASS_DURING_INITIALIZATION SI_INSTANCE_BEFORE_FINALS_ASSIGNED SE_NO_SUITABLE_CONSTRUCTOR NP_NONNULL_FIELD_NOT_INITIALIZED_IN_CONSTRUCTOR UR_UNINIT_READ UR_UNINIT_READ_CALLED_FROM_SUPER_CONSTRUCTOR BAC_BAD_APPLET_CONSTRUCTOR MF_METHOD_MASKS_FIELD LI_LAZY_INIT_STATIC LI_LAZY_INIT_UPDATE_STATIC NP_UNWRITTEN_PUBLIC_OR_PROTECTED_FIELD UWF_FIELD_NOT_INITIALIZED_IN_CONSTRUCTOR
Public 메소드부터 반환된 Private 배열	MS_EXPOSE_REP
Private 배열에 Public 데이터 할당	ME_ENUM_FIELD_SETTER MS_EXPOSE_REP

2. 행정안전부 47개 보안약점 관련 Findsecuritybugs 기본 Bug Description

행안부 SW 보안약점	관련 Bug Description
SQL 삽입	Potential SQL Injection (SQL_INJECTION) Potential SQL Injection with Turbine (SQL_INJECTION_TURBINE) Potential SQL/HQL Injection (Hibernate) (SQL_INJECTION_HIBERNATE) Potential SQL/JDOQL Injection (JDO) (SQL_INJECTION_JDO) Potential SQL/JPQL Injection (JPA) (SQL_INJECTION_JPA) Potential JDBC Injection (Spring JDBC) (SQL_INJECTION_SPRING_JDBC) Potential JDBC Injection (SQL_INJECTION_JDBC) Potential Scala Slick Injection (SCALA_SQL_INJECTION_SLICK) Potential Scala Anorm Injection (SCALA_SQL_INJECTION_ANORM) Potential Android SQL Injection (SQL_INJECTION_ANDROID) Untrusted servlet parameter (SERVLET_PARAMETER)
경로 조작 및 자원 삽입	Potential Path Traversal (file read) (PATH_TRAVERSAL_IN) Potential Path Traversal (file write) (PATH_TRAVERSAL_OUT) Potential Path Traversal (file read) (SCALA_PATH_TRAVERSAL_IN) Tainted filename read (FILE_UPLOAD_FILENAME) Untrusted servlet parameter (SERVLET_PARAMETER)
크로스사이트 스크립트	XSSRequestWrapper is a weak XSS protection (XSS_REQUEST_WRAPPER) Escaping of special XML characters is disabled Potential XSS in JSP (XSS_JSP_PRINT) Potential XSS in Servlet (XSS_SERVLET) Potential XSS in Scala Twirl template engine (SCALA_XSS_TWIRL) Potential XSS in Scala MVC API engine (SCALA_XSS_MVC_API)
운영체제 명령어 삽입	Potential Command Injection Untrusted servlet parameter (SERVLET_PARAMETER)
위험한 형식 파일 업로드	Tainted filename read (FILE_UPLOAD_FILENAME)
신뢰되지 않는 URL 주소로 자동 접속 연결	Unvalidated Redirect (UNVALIDATED_REDIRECT) Unvalidated Redirect (Play Framework) (PLAY_UNVALIDATED_REDIRECT) Spring Unvalidated Redirect (SPRING_UNVALIDATED_REDIRECT)
XQuery 삽입	Untrusted Content-Type header (SERVLET_CONTENT_TYPE)
XPath 삽입	Potential XPath Injection (XPATH_INJECTION)
LDAP 삽입	Potential LDAP Injection (LDAP_INJECTION)
크로스 사이트 요청 위조	Spring CSRF protection disabled (SPRING_CSRF_PROTECTION_DISABLED) Spring CSRF unrestricted RequestMapping (SPRING_CSRF_UNRESTRICTED_REQUEST_MAPPING)
HTTP 응답분할	Potential HTTP Response Splitting (HTTP_RESPONSE_SPLITTING)
보안기능 결정에 사용되는 부적절한 입력값	Untrusted Content-Type header (SERVLET_CONTENT_TYPE) Untrusted Hostname header (SERVLET_SERVER_NAME) HTTP headers untrusted Untrusted Referer header Untrusted User-Agent header
포맷스트링 삽입	Format String Manipulation (FORMAT_STRING_MANIPULATION)
적절한 인증없는 중요기능 허용	Found JAX-WS SOAP endpoint (JAXWS_ENDPOINT) Found JAX-RS REST endpoint (JAXRS_ENDPOINT) Potential external control of configuration (EXTERNAL_CONFIG_CONTROL) Tainted filename read (FILE_UPLOAD_FILENAME)

행안부 SW 보안약점	관련 Bug Description
부적절한 인가	Insecure SMTP SSL connection (INSECURE_SMTP_SSL)
중요한 자원에 대한 잘못된 권한 설정	External file access (Android) (ANDROID_EXTERNAL_FILE_ACCESS) Cookie without the HttpOnly flag (HTTPONLY_COOKIE) Cookie without the secure flag (INSECURE_COOKIE)
취약한 암호화 알고리즘 사용	MessageDigest Is Weak DES is insecure (DES_USAGE) DESede is insecure (TDES_USAGE) RSA with no padding is insecure (RSA_NO_PADDING) ECB mode is insecure (ECB_MODE) Cipher is susceptible to Padding Oracle (PADDING_ORACLE) Hazelcast symmetric encryption (HAZELCAST_SYMMETRIC_ENCRYPTION) MD2, MD4 and MD5 are weak hash functions (WEAK_MESSAGE_DIGEST_MD5) SHA-1 is a weak hash function (WEAK_MESSAGE_DIGEST_SHA1)
중요정보 평문저장	Unencrypted Socket
중요정보 평문전송	Potentially Sensitive Data in Cookie Unencrypted Socket Cookie without the secure flag (INSECURE_COOKIE)
하드코드된 비밀번호	Hard Coded Password (HARD_CODE_PASSWORD)
충분하지 않은 키 길이 사용	Blowfish usage with short key (BLOWFISH_KEY_SIZE) RSA usage with short key (RSA_KEY_SIZE)
적절하지 않은 난수 값 사용	Predictable pseudorandom number generator (PREDICTABLE_RANDOM) Predictable pseudorandom number generator (Scala) (PREDICTABLE_RANDOM_SCALA)
하드코드된 암호화 키	Hard Coded Key (HARD_CODE_KEY)
사용자 하드디스크에 저장되는 쿠키를 통한 정보 노출	Persistent Cookie Usage (COOKIE_PERSISTENT)
솔트 없이 일방향 해쉬 함수 사용	MD2, MD4 and MD5 are weak hash functions (WEAK_MESSAGE_DIGEST_MD5)
무결성 검사없는 코드 다운로드	Struts File Disclosure (STRUTS_FILE_DISCLOSURE) Spring File Disclosure (SPRING_FILE_DISCLOSURE) RequestDispatcher File Disclosure (REQUESTDISPATCHER_FILE_DISCLOSURE) URLConnection Server-Side Request Forgery (SSRF) and File Disclosure (URLCONNECTION_SSRF_FD)
오류 메시지를 통한 정보 노출	Information Exposure Through An Error Message (INFORMATION_EXPOSURE_THROUGH_AN_ERROR_MESSAGE)
잘못된 세션에 의한 데이터 정보 노출	Untrusted session cookie value (SERVLET_SESSION_ID)
시스템 데이터 정보노출	URLConnection Server-Side Request Forgery (SSRF) and File Disclosure (URLCONNECTION_SSRF_FD) Struts File Disclosure (STRUTS_FILE_DISCLOSURE) Spring File Disclosure (SPRING_FILE_DISCLOSURE) RequestDispatcher File Disclosure (REQUESTDISPATCHER_FILE_DISCLOSURE)
DNS lookup에 의존한 보안 결정	Use of ESAPI Encryptor (ESAPI_ENCRYPTOR)

3. 행정안전부 47개 보안약점 관련 Spotbugs 기존 탐지 룰(Detector)

행안부 SW 보안약점	관련 Detector	룰 설명
SQL 삽입	FindSqlInjection	SQL문의 입력값이 고정된 값이 아니고 PreparedStatement를 사용하지 않는 경우 검사
크로스사이트 스크립트	CrossSiteScripting	스크립트가 사용되기 전에 입력 값이 신뢰할 수 없는 소스에서 온 것인지 확인
부적절한 인가	DoInsideDoPrivileged	보안 기능이 고려되지 않은 API를 사용하여 인증 없이 연결하는 경우 탐지
적절하지 않은 난수 값 사용	DumbMethods	적절하지 않다고 알려진 난수 생성 메소드 사용 탐지
경쟁조건: 검사시점과 사용시점(TOCTOU)	SynchronizeOnClassLiteralNotGetClass	getClass()를 이용해 Synchronized 하는 경우 탐지
종료되지 않는 반복문 또는 재귀 함수	InfiniteRecursiveLoop	같은 값으로 재귀 호출을 반복하는 경우 탐지
널(Null) 포인터 역참조	FindNullDeref FindUninitializedGet	할당되지 않은 변수 사용 탐지
부적절한 자원 해제	FindUnreleasedLock	Java.util.concurrent lock을 할당받고 해제하지 않는 경우
초기화되지 않은 변수 사용	FindUninitializedGet	초기화되지 않은 변수 사용 탐지
Public 메소드부터 반환된 Private 배열	FindReturnRef	내부 변수가 변경 가능한 객체로 반환되는 경우 탐지
Private 배열에 Public 데이터 할당	FindReturnRef	내부 변수에 변경 가능한 객체가 할당되는 경우 탐지
취약한 API 사용	DumbMethods	금지된 Method 사용 탐지

4. 행정안전부 47개 보안약점 관련 FindSecurityBugs 기존 탐지 룰(Detector)

행안부 SW 보안약점	관련 Detector	룰 설명
SQL 삽입	SqliInjectionDetector AndroidSqliInjectionDetector	SQL문의 입력값이 고정된 값이 아니고 PreparedStatement를 사용하지 않는 경우 검사
경로 조작 및 자원 삽입	PathTraversalDetector	File, FileWriter 등을 외부에서 들어온 값을 통해 사용하는 경우
크로스사이트 스크립트	XssServletDetector XssMvcApiDetector XssTwirlDetector XssJspDetector	스크립트가 사용되기 전에 입력 값이 신뢰할 수 없는 소스에서 온 것인지 확인
운영체제 명령어 삽입	CommandInjectionDetector	운영체제 명령어를 환경 변수를 통해 받아오는 경우 탐지
위험한 형식 파일 업로드	FileUploadFilenameDetector	신뢰할 수 없는 소스를 이용한 ServletFileUpload 탐지
신뢰되지 않는 URL 주소로 자동 접속 연결	UnvalidatedRedirectDetector SpringUnvalidatedRedirectDetector PlayUnvalidatedRedirectDetector	신뢰할 수 없는 소스를 이용한 sendRedirect 사용 탐지
XQuery 삽입	XQueryInjectionDetector	XQuery 문을 외부 입력값을 이용해 compile하는 경우 탐지
XPath 삽입	XPathInjectionDetector	XPath 문을 외부 입력값을 이용해 compile 또는 evaluate하는 경우 탐지
LDAP 삽입	LdapInjectionDetector	LDAP 필터를 외부 입력값을 이용해 구성하고 search하는 경우 탐지
크로스사이트 요청 위조	SpringCsrfProtectionDisabledDetector	CSR 방어가 비활성화 되어 있는 경우 탐지
HTTP 응답분할	HttpResponseSplittingDetector	외부 입력 값을 통해 addHeader(), setHeader() 등의 메소드 사용 탐지
보안기능 결정에 사용되는 부적절한 입력값	ServletEndpointDetector	신뢰할 수 없는 소스에서 불러온 값을 사용하는 경우 탐지
포맷스트링 삽입	FormatStringManipulationDetector	신뢰할 수 없는 값을 포맷팅하는 경우 탐지
부적절한 인가	InsecureSmtptSslDetector	보안 기능이 고려되지 않은 API를 사용하여 인증없이 연결하는 경우 탐지
중요한 자원에 대한 잘못된 권한 설정	ExternalFileAccessDetector CookieFlagsDetector PersistentCookieDetector	보안 기능이 비활성화된 쿠키를 사용하는 경우 탐지
취약한 암호화 알고리즘 사용	CustomMessageDigestDetector PotentialValueDetector DesUsageDetector TDesUsageDetector RsaNoPaddingDetector CipherWithNoIntegrityDetector	취약하다고 알려진 암호화 알고리즘 사용 탐지

행안부 SW 보안약점	관련 Detector	를 설명
중요정보 평문저장	UnencryptedSocketDetector UnencryptedServerSocketDetector	암호화되지 않는 소켓, 서버 소켓 사용 탐지
중요정보 평문전송	UnencryptedSocketDetector UnencryptedServerSocketDetector	암호화되지 않는 소켓, 서버 소켓 사용 탐지
하드코드된 비밀번호	DumbMethodInvocations ConstantPasswordDetector	데이터베이스의 비밀번호가 고정 값이거나 특정 메소드(setProperty)의 인자값을 고정 값으로 사용하는 경우 탐지
충분하지 않은 키 길이 사용	InsufficientKeySizeBlowfishDetector InsufficientKeySizeRsaDetector	암호화 알고리즘의 권장 키 길이 미만인 경우 탐지
적절하지 않은 난수 값 사용	PredictableRandomDetector	적절하지 않다고 알려진 난수 생성 메소드 사용 탐지
하드코드된 암호화 키	ConstantPasswordDetector	하드코드된 암호화 키 사용 탐지
사용자 하드디스크에 저장되는 쿠키를 통한 정보 노출	PersistentCookieDetector	매우 긴 수명을 가진 쿠키 사용 탐지 세션 만료 시간이 부적절한 경우
솔트 없이 일방향 해쉬 함수 사용	PotentialValueDetector	Update를 통해 솔트 값을 업데이트하지 않는 경우 검사
무결성 검사없는 코드 다운로드	FileDisclosureDetector SSRFDetector	외부에서 받은 값을 검사 없이 그대로 실행하는 경우 탐지
오류메세지 통한 정보 노출	ErrorMessageExposureDetector	스택 정보가 유출되는 경우 탐지
잘못된 세션에 의한 데이터 정보 노출	ServletEndpointDetector	클라이언트에 의해 변조될 수 있는 세션 ID를 사용하는 경우 탐지
시스템 데이터 정보 노출	SSRFDetector FileDisclosureDetector	예외 처리에서 시스템 데이터가 노출되는 경우 탐지
DNS lookup에 의존한 보안 결정	ServletEndpointDetector	getServerName() 또는 getHeader("Host")를 사용해 호스트의 이름을 사용하는 경우 탐지
취약한 API 사용	UnencryptedSocketDetector	암호화되지 않는 Socket 사용 탐지

5. 행정안전부 47개 보안약점 관련 기존 PMD 룰(Rule)

행안부 SW 보안약점	관련 Detector	룰 설명
오류 메시지를 통한 정보노출	AvoidPrintStackTrace	PrintStackTrace 호출 탐지
오류 상황 대응 부재	EmptyCatchBlock	Catch 블록이 비어있는 경우 탐지
부적절한 예외 처리	AvoidCatchingGenericException	Exception 사용 탐지 NullPointerException 탐지
Null Pointer 역참조	BrokenNullCheck MisplacedNullCheck	잘못된 Null 검사 이후 Null Pointer 사용 탐지 Null Pointer 사용 후 Null 검사 탐지
부적절한 자원 해제	CloseResource	자원 할당 후 미해제 탐지
Public 메소드부터 반환된 Private 배열	ArrayIsStoredDirectly	배열이 직접 저장되는 경우 탐지
Private 배열에 Public 데이터 할당	MethodReturnsInternalArray	내부 배열을 반환하는 경우 탐지



전자정부 SW 개발자·진단원을 위한
공개SW를 활용한 소프트웨어 개발보안 점검가이드



PART 부록

제1절 용어정의

제2절 공개소프트웨어 점검도구 목록

제3절 시험 예제코드

제4절 Spotbugs(Findbugs)/PMD 구조

제5절 PMD 상세분석

PART

부 록

제2장 공개소프트웨어 진단도구 소개 및 설치 환경



제1절 용어정의

소프트웨어 개발보안 	소프트웨어 개발과정에서 개발자 실수, 논리적 오류 등으로 인해 소프트웨어에 내재된 보안취약점을 최소화하는 한편, 해킹 등 보안위협에 대응할 수 있는 안전한 소프트웨어를 개발하기 위한 일련의 과정을 의미한다. 넓은 의미에서 소프트웨어 개발보안은 소프트웨어 생명주기의 각 단계별로 요구되는 보안활동을 모두 포함하며, 좁은 의미로는 SW 개발과정에서 소스코드를 작성하는 구현단계에서 보안취약점을 배제하기 위한 '시큐어코딩(Secure Coding)'을 의미한다.
소프트웨어 보안약점	소프트웨어 결함, 오류 등으로 인해 해킹 등 사이버공격을 유발할 가능성이 있는 잠재적인 보안취약점을 말한다.
소프트웨어 보안약점 진단도구	개발과정에서 소스코드상의 소프트웨어 보안약점을 찾기 위하여 사용하는 도구를 말한다.
소프트웨어 보안약점 진단원	소프트웨어(SW) 보안약점이 남아있는지 진단하여 조치방안을 수립하고 조치결과 확인 등의 활동을 수행하는 자를 말한다.
공개 소프트웨어	소스코드가 공개되어 있는 소프트웨어로서 누구나 자유롭게 사용할 수 있고, 배포할 수 있는 소프트웨어를 말한다.

제2절 공개소프트웨어 점검도구 목록

美 국립표준기술연구소(NIST, National Institute of Standards and Technology)에서는 SAMATE 홈페이지(<https://samate.nist.gov>)를 통해 SW의 품질 및 보안성 개선을 위해 공개소프트웨어 점검도구 목록을 제공하고 있다. 아래는 SAMATE에서 제공하는 주요 공개소프트웨어 점검도구 목록이다.

공개소프트웨어 도구명	지원 언어
ABASH	Bash
BOON	C
Clang Static Analyzer	C/C++, Objective-C
Closure Compiler	JavaScript
Cppcheck	C/C++
CQual	C/C++
Csur	C
Spotbugs	JAVA, Groovy, Scala
FindSecurityBugs	JAVA, Groovy, Scala
Flawfinder	C/C++
Jlint	JAVA, C++
LAPSE	JAVA
PHP-Sat	PHP
Pixy	PHP
PMD	JAVA, JavaScript, PLSQL, Apache Velocity, XML, XSL 등
Pylint	Python
RATS	C/C++, Perl, PHP, Python
Smatch	C
Splint	C
UNO	C
Yasca	JAVA, C/C++, HTML, JavaScript, ASP, PHP, COBOL, .NET 등
WAP	PHP

※ 출처 : NIST SAMATE(http://samate.nist.gov/index.php/Source_Code_Security_Analyzers.html)

제3절 시험 예제코드

소스코드 보안약점 진단도구의 규칙 및 분석 성능을 시험하기 위해 SAMATE에서 제공하는 Juliet Test SUITE 시험 예제코드를 활용할 수 있다. Juliet 시험 예제코드는 <http://samate.nist.gov/SARD/testsuite.php> 에서 최신 버전을 다운받을 수 있다. 아래의 표는 행안부 47개 보안약점과 대응하는 Juliet 예제코드의 매핑 현황이다.

구분	#	보안약점 항목	Julice Code
입력 데이터 검증 및 표현	1	보안약점 항목	CWE89
		SQL 삽입	CWE23
	2	경로 조작 및 자원 삽입	CWE36
	3	크로스사이트스크립트	CWE80
	4	운영체제 명령어 삽입	CWE78
	5	위험한 형식 파일 업로드	
	6	신뢰되지 않는 URL 주소로 자동 접속 연결	CWE601
	7	XQuery 삽입	
	8	XPath 삽입	CWE643
	9	LDAP 삽입	CWE90
	10	크로스사이트 요청 위조	
	11	HTTP 응답분할	CWE113
	12	정수형 오버플로우	CWE190
	13	보안기능 결정에 사용되는 부적절한 입력값	
	14	메모리 버퍼 오버플로우	
	15	포맷 스트링 삽입	CWE134
보안 기능	1	적절한 인증 없는 중요기능 허용	CWE566
	2	부적절한 인가	
	3	중요한 자원에 대한 잘못된 권한설정	CWE539
	4	취약한 암호화 알고리즘 사용	CWE327

구분	#	보안약점 항목	Julice Code
보안 기능	5	중요정보 평문전송	CWE256 CWE319
	6	하드코드된 비밀번호	CWE319 CWE614
	7	충분하지 않은 키길이 사용	CWE256 CWE259
	8	적절하지 않은 난수값 사용	
	9	하드코드된 암호화 키	CWE338
	10	취약한 비밀번호 허용	CWE321
	11	사용자 하드디스크에 저장되는 쿠키를 통한	
	12	정보노출	CWE256 CWE259
	13	주석문 안에 포함된 시스템 주요정보	CWE615
	14	솔트 없이 일방향 해쉬함수 사용	CWE759
	15	무결성 검사없는 코드 다운로드	
	16	반복된 인증시도 제한 기능 부재	CWE307
시간및 상태	1	경쟁조건: 검사시점과 사용시점(TOCTOU)	
	2	종료되지 않는 반복문 또는 재귀 함수	CWE674 CWE385
에러 처리	1	오류메시지통한 정보노출	CWE209 CWE535
	2	오류상황 대응 부재	CWE390
	3	부적절한 예외 처리	CWE395 CWE396
코드 오류	1	Null Pointer 역참조	CWE476 CWE690
	2	부적절한 자원 해제	CWE404 CWE459 CWE772 CWE775
	3	해제된 자원 사용	
	4	초기화되지 않은 변수사용	
캡슐화	1	잘못된 세션에 의한 데이터 정보 노출	
	2	제거되지 않고 남은 디버그 코드	CWE489
	3	시스템 데이터 정보노출	CWE209 CWE526
	4	Public 메소드부터 반환된 Private 배열	
	5	Private 배열에 Public 데이터 할당	
API 악용	1	DNS lookup에 의존한 보안결정	
	2	취약한 API 사용	CWE382

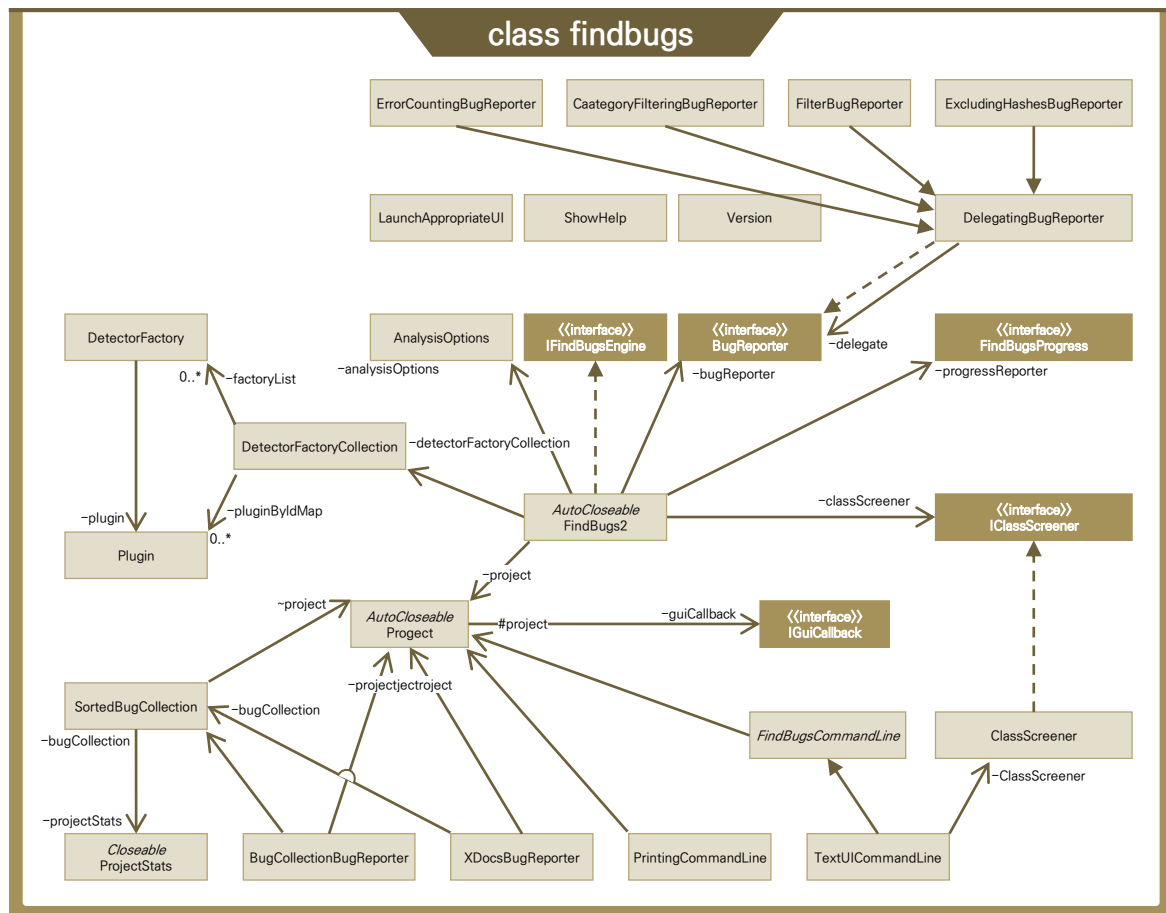
제4절 Spotbugs(Findbugs)/PMD 구조

공개 SW 진단 도구를 기반으로 사용자 스스로 점검도구를 확장할 수 있도록 Spotbugs와 PMD 클래스 개요를 제시한다. 진단 도구의 이해를 통해 전체적인 구조와 동작 개요를 파악할 수 있고 엔진 또는 플러그인(ex. FindSecurityBugs) 개발에 활용하여 진단 도구의 정확도 및 검출력 향상에 기여할 수 있을 것으로 기대한다.

단순히 각 진단 도구의 커스텀 룰(Spotbugs의 detector, PMD의 rule) 제작이 목적이라면 엔진 단의 이해가 반드시 필요하지는 않다. 커스텀 룰 제작을 목적으로 한다면 “제6장2절의 사용자 정의 룰 작성 방법”을 참고하도록 한다.

Spotbugs(FindBugs) 상세분석

1. SpotBugs 클래스 Overview



주요 클래스 기능 설명

클래스	설명
TextUICommandLine	실행 명령어를 파싱하여 IFindBugsEngine에 설정
FindBugs2	SpotBugs 분석 실행, textui를 선택했을 경우 내부적으로 호출되는 메인 클래스
FindBugs	IFindBugsEngine을 사용하기 위해 유용한 정적 메소드 및 필드
gui.Driver	SpotBugs GUI 실행
BugReporter	분석 결과(버그 리포트) 및 보조 정보(분석 오류, 제외된 클래스, 소스 파일 매핑) 기록
DetectorFactory	Detector 객체 생성을 담당하고 해당 detector의 메타 정보를 관리
Project	버그를 분석할 Jar 파일들과 선택적으로 다음 항목을 포함: 1) 프로그램의 소스 코드를 포함하는 디렉토리, 2) 사용자가 분석을 원하지 않는 클래스 경로
Plugin	Spotbugs 플러그인
ShowHelp	도움말 제공
Version	SpotBugs 버전 정보 제공
Filter	분석 결과 필터 기능 제공
SetBugDatabaseInfo	프로젝트 설정/옵션 정보

2. SpotBugs 클래스 상세분석

Spotbugs는 LaunchAppropriateUI.class를 통해 commandLine을 통해 입력받은 UI(textui, gui)에 해당하는 엔진을 호출한다. Spotbugs.jar 파일을 더블 클릭해서 실행하는 경우나 UI를 선택하지 않은 경우에는 기본값인 gui로 실행된다. 분석의 편의를 위해 GUI 등의 기능이 제외된 textui를 선택한 경우로 분석을 진행한다. FindBugs2.class는 'textui'를 선택한 경우 LaunchAppropriateUI.class에서 내부적으로 호출하는 메인 클래스다.

FindBugs2	AutoCloseable
- analysisOptions: AnalysisOptions = new AnalysisOpt... {readOnly} - appClassList: List<ClassDescriptor> - bugReporter: BugReporter - classFactory: IClassFactory - classObserverList: List<IClassObserver> - classPath: IClassPath - classScreener: IClassScreener - currentClassName: String + DEBUG: boolean = VERBOSE Syst... {readOnly} - detectorFactoryCollection: DetectorFactoryCollection - errorCountingBugReporter: ErrorCountingBugReporter - executionPlan: ExecutionPlan ~ explicitlyDisabledBugReporterDecorators: Set<String> = Collections.emp... ~ explicitlyEnabledBugReporterDecorators: Set<String> = Collections.emp... - LIST_ORDER: boolean = SystemPropertie... {readOnly} + PROGRESS: boolean = DEBUG System... {readOnly} - progressReporter: FindBugsProgress - project: Project + PROP_FINDBUGS_HOST_APP: String = "findbugs.hostApp" {readOnly} + PROP_FINDBUGS_HOST_APP_VERSION: String = "findbugs.hostA... {readOnly} - rankThreshold: int - referencedClassSet: Collection<ClassDescriptor> - SCREEN_FIRST_PASS_CLASSES: boolean = SystemPropertie... {readOnly} - VERBOSE: boolean = SystemPropertie... {readOnly}	

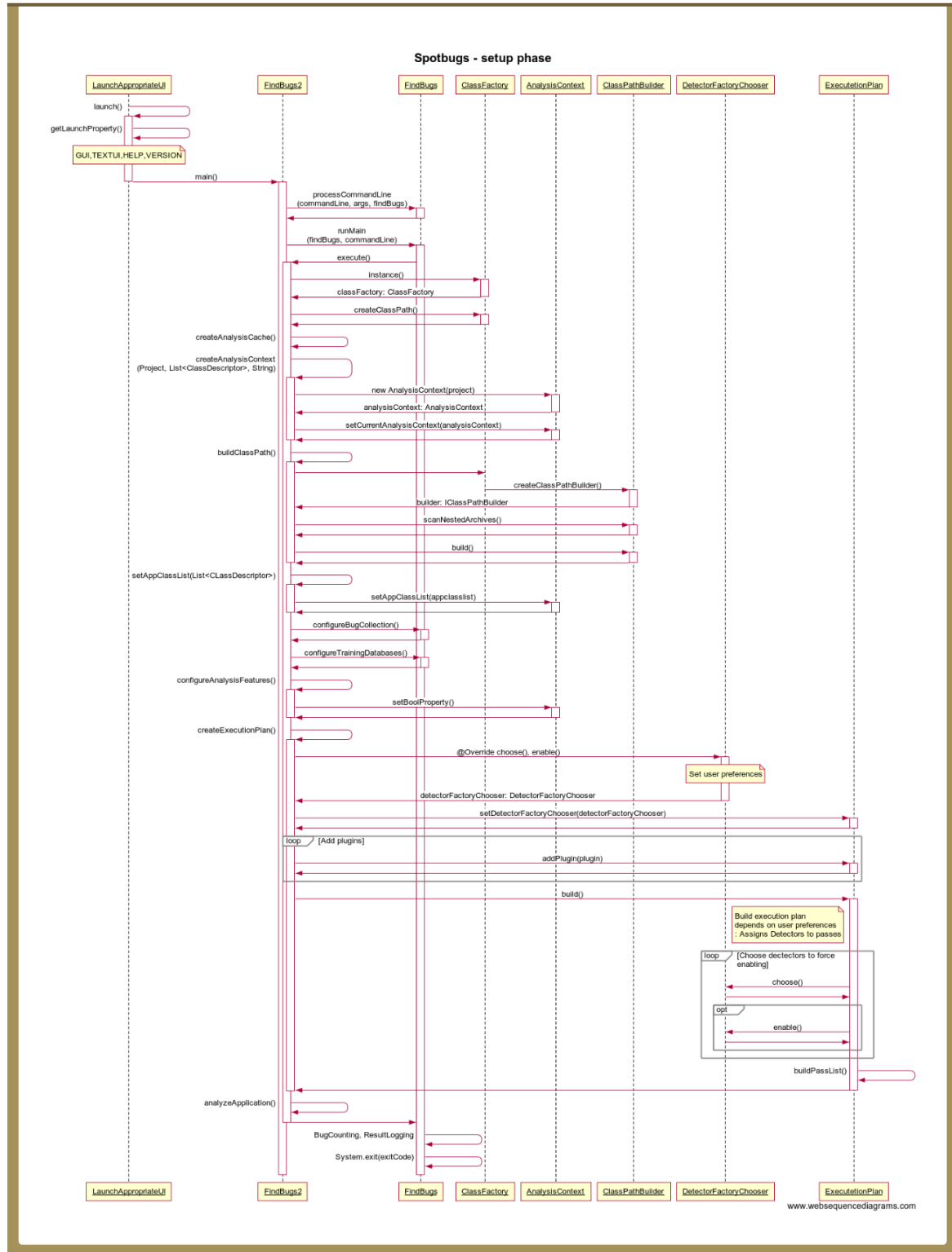
1) FindBugs2 속성

속성	타입	설명
analysisOptions	AnalysisOptions	분석 옵션
appClassList	List <ClassDescriptor>	클래스 식별 정보를 담은 객체의 배열
bugReporter	BugReporter	분석 결과(버그 리포트) 및 보조 정보(분석 오류, 제외된 클래스, 소스 파일 매핑) 기록
classFactory	IClassFactory	Codebase/classpath/classfile 객체를 생성하는 factory

속성	타입	설명
classObserverList	List 〈IClassObserver〉	방문되는 클래스를 관찰, 불러오고 싶은 클래스를 분석 캐시로 부터 명시적으로 가져옴
classPath	IClassPath	리소스들이 불러오는 경로, codebase의 배열
classScreener	ClassScreener	분석되지 않아야 하는 클래스 파일들을 제거
currentClassName	CurrentClassName	현재 분석 중인 클래스 파일 이름
detectorFactoryCollection	DetectorFactoryCollection	분석에 사용될 모든 DectectorFactory를 담고 있는 Singleton 클래스
errorCountingBugReporter	ErrorCountingBugReporter	보고된 버그, 오류 등의 개수를 기록하는 클래스
executionPlan	ExecutionPlan	어플리케이션에 붙일 Detector를 실행하기 위한 계획, plugin 설명에 명시된 순서대로 실행
progeessReporter	ProgeessReporter	분석 진척 상황 보고
project	Project	버그를 분석할 jar 파일들과 선택적으로 다음 항목을 포함: 1) 프로그램의 소스 코드를 포함하는 디렉토리, 2) 사용자가 분석을 원하지 않는 클래스 경로
referencedClassSet	Collection 〈ClassDescriptor〉	클래스 식별 정보를 담는 객체의 배열

3. Spotbugs 시퀀스 다이어그램

① 설정 단계(setup phase)



- SpotBugs가 실행되면 LaunchAppropriateUI 메인클래스로 실행되어 launch()를 호출한다.
- launch()에서 getLaunchProperty()를 호출한다. commandLine을 통해 입력받은 명령어 중 해당하는 UI(textui, gui) 엔진을 호출한다. 특별한 입력 값이 없는 경우 GUI로 실행된다.
- 분석은 TEXTUI로 진행한다. TEXTUI의 경우 FindBugs2.main()을 호출한다. 적절한 UI를 호출한 뒤로는 LaunchAppropriateUI는 사용되지 않으며, FindBugs2.class에서 대부분 호출이 발생한다.
- processCommandLine를 통해서 입력받은 명령어에서 설정값(-quite, -include 등)을 추출한다. 추출된 값들을 기반으로 엔진에 필요한 값들을 초기화한다. 설정을 추출하는 과정에서 오류가 발생하면 도움말을 출력한다.
- FindBugs2에서 runMain()을 호출하면, FindBugs에서 execute()를 통해 FindBugs2 엔진을 실행시킨다.
- instance(), creatClassPath()를 통해 ClassFactory를 불러온다.
- createAnalysisCache(), createAnalysisContext()를 통해 분석에 사용될 built-in 엔진과 사용할 detector들을 저장할 데이터베이스를 등록한다.
- buildClassPath()를 호출하여 분석할 클래스의 경로와 개수를 계산한다. ClassPathBuilder 객체를 생성하고 하위 경로 분석 여부를 설정한다. build()를 호출하여 하위 경로를 재귀적으로 탐색하며 분석할 대상 클래스를 모두 찾고 ClassPath 객체에 발견한 클래스들을 등록한다.
- setAppClassList()를 호출하여 분석 대상 클래스들을 등록한다.
- configureBugCollection()과 configureTrainingDatabases(), setAnalysisFeature()를 호출하여 객체들을 설정한다.
- createExecutionPlan()을 호출하여 실행할 Detector들을 선별한다. 사용자가 지정한 것에 따라 제외되는 Detector가 발생할 수 있다. choose(), enable() 함수들을 오버라이드한 detector FactoryChooser()를 생성하고, setDetectorFactoryChooser()를 통해 등록한다. Detector들의 집합체 DetectorFactoryCollection에서 Plugin들을 추출하여 executionPlan에 등록한다.
- 위의 과정을 통해 설정된 executionPlan을 build()한다. 사용자 설정에 따라 제외되는 plugin들이 발생하며, 의존성에 따라 특정 plugin은 강제로 enable()한다. buildPassList()들을 통해 분석할 클래스에 사용될 Detector들의 list를 생성한다.
- analyzeApplication()을 통해 분석을 시작한다.



- ‘Spotbugs – setup phase’에서 analyzeApplication()을 확장한 시퀀스 다이어그램이다. 설정 단계에서 생성 및 설정된 개체들로 분석을 진행한다.
- getProjectStats().getProfiler()에서 Profile 객체를 받아오며, start(this.getClass())를 통해 분석 시작 시간을 기록한다. 후에 end(this.getClass())과 함께 전체 분석에 걸린 시간을 계산 한다.
- executionPlan에서 AnalysisPass의 객체를 iterator를 이용해 가져온다. AnalysisPass는 의존성에 따른 detector 집합이다. 의존성이 없는 detector 집합부터 시작해 의존성이 강한 detector들이 실행된다.
- instantiateDetector2sInPass()를 호출해 AnalysisPass에 포함된 detector들을 인스턴스화하고 detector들의 리스트, detectorList를 반환한다.
- classCollection에서 분석할 class의 ClassDescriptor를 받아온다. classCollection은 분석 대상이 되는 클래스들의 집합이다.
- 분석할 클래스의 이름을 가져오고 startContext(currentClassName)을 통해 해당 클래스의 분석 시작 시간을 기록한다. 후에 endContext(currentClassName)과 함께 해당 클래스의 분석 시간을 계산한다.
- 분석할 클래스를 setClassBeingAnalyzed()를 이용해 등록하고, detectorList에서 detector를 하나씩 가져와서 클래스를 분석한다. start(detector.getClass()), end(detector.getClass())를 통해 하나의 detector로 클래스를 분석하는 시간을 계산하고 visit(classDescriptor)로 클래스를 방문하여 분석한다.
- 분석이 종료된 후 finishClass(), clearClassBeingAnalyzed(), finishpass(), finishPerClass Analysis(), finish(), reportQueuedErrors(), end(this.getClass())를 통해 분석 결과를 기록하거나 할당된 자원을 해제한다. 시퀀스 다이어그램을 참고하면 대부분이 시작 함수와 짝을 이루고 있는 것을 확인할 수 있다.
- 요약하면, detector들의 집합을 초기화하여 detector의 리스트 detectorList를 획득한다. class Collection에서 분석할 클래스를 하나씩 가져와서, detector들을 하나씩 이용해 클래스를 분석 한다. 모든 detector의 분석이 끝나면 다음 클래스를 가져와 분석을 진행한다.

제5절 PMD 상세분석

1. PMD 주요 패키지 Overview

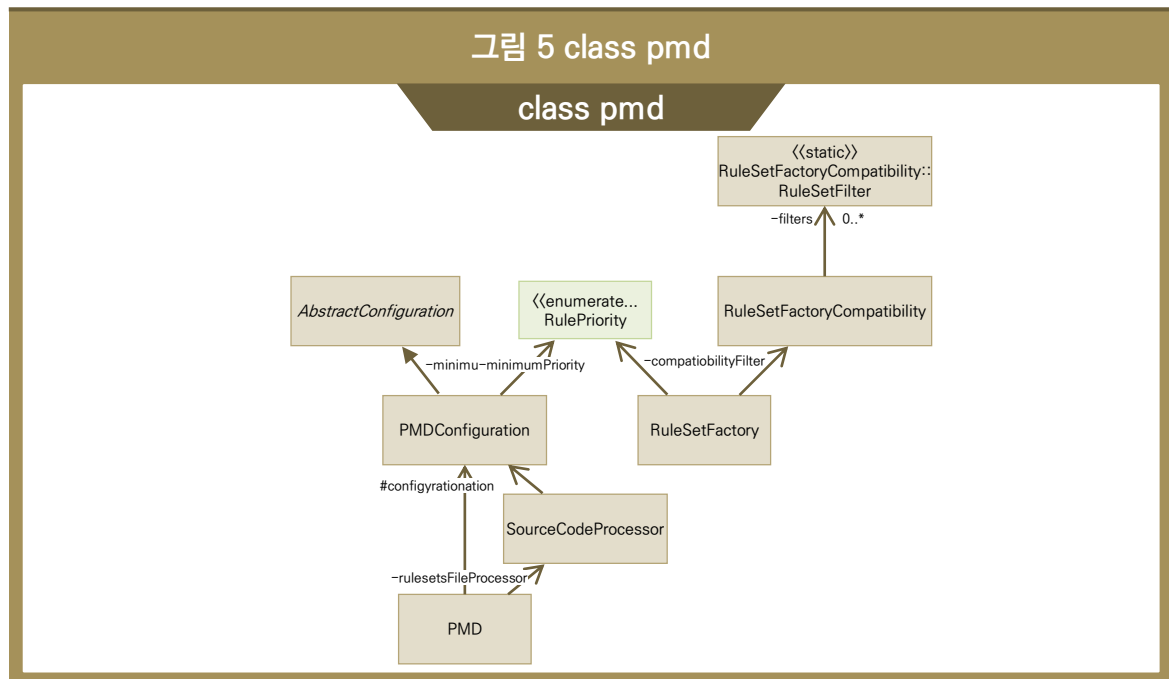
1) 주요 패키지

- pmd: 메인 클래스
- pmd.lang: 언어, 파서, 데이터플로우 분석기를 가지고 있는 모듈
- pmd.lang.ast: 언어의 AST
- pmd.lang.dfa: 언어의 데이터플로우 분석기
- pmd.lang.rule: 규칙을 탐색, 위반했는지 확인하는 모듈
- pmd.lang.symboltable: 데이터플로우 분석기 메모리 모델
- pmd.lang.{cpp, java, fortran, ...}: 개별 언어 실제 분석에 필요한 구체적 모듈
- pmd.properties: 값의 특징들.
- pmd.renderers: 각종 보고서, 화면 출력 렌더러

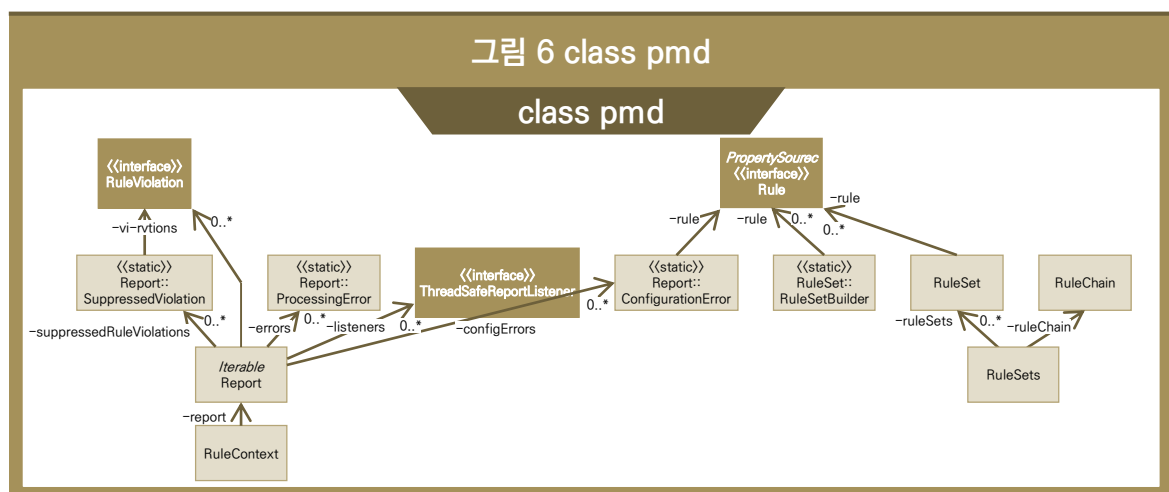
2) PMD

PMD는 크게 실행모듈, 룰셋 모듈, 리포트모듈로 나뉘어 있다. 실행모듈에서 분석을 진행하게 되며, 룰셋 모듈은 분석 룰을 정의하고 있다. 리포트 모듈에서는 분석 된 결과를 리포트한다.

- 그림 5를 보면 PMD에서 SourceCodeProcessor를 이용하여 소스코드를 처리한다. RuleSet Factory에서는 설정된 룰셋을 골라내어 분석 시에 사용할 룰셋을 생성해낸다.



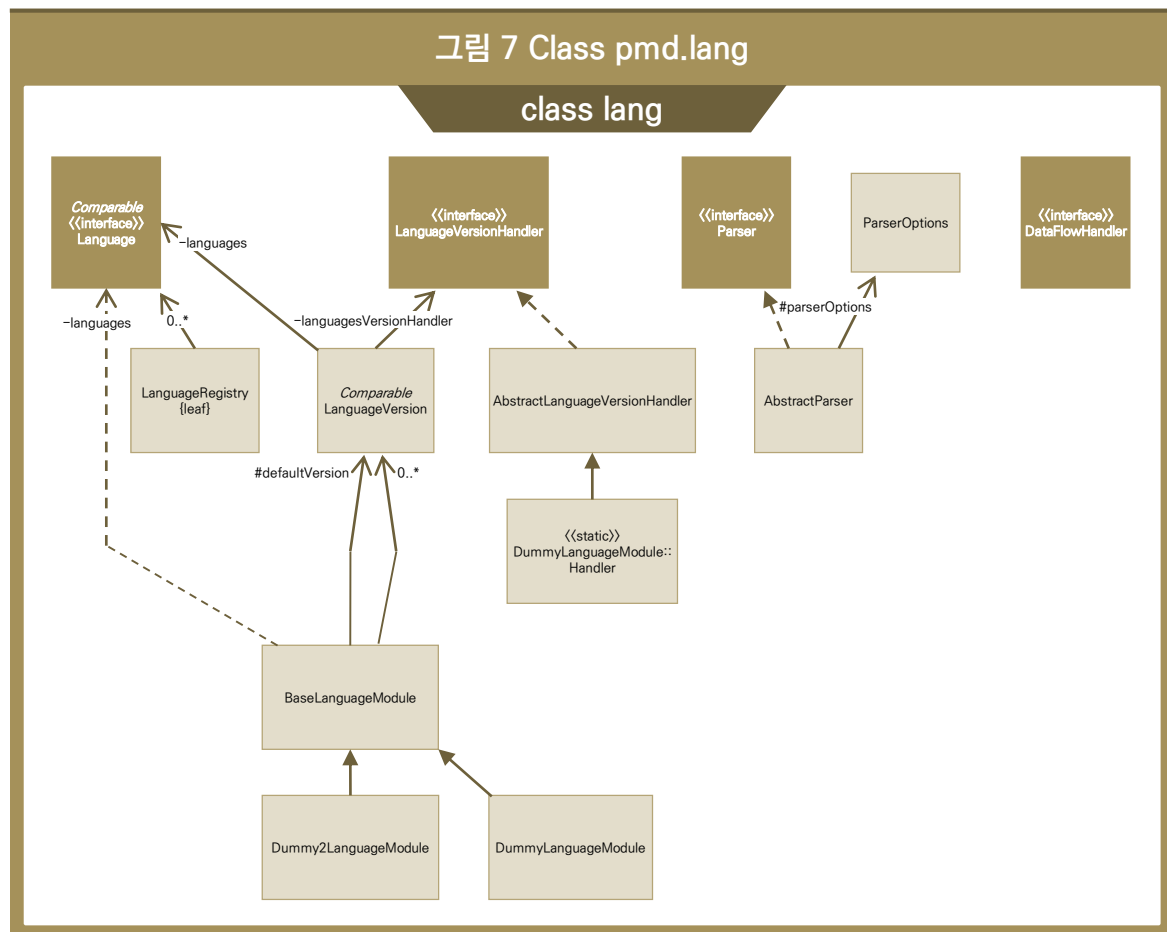
- 그림 6을 보면 Report가 리포트를 수행한다. 이때 Report는 Rule을 함께 참조하여 리포트를 수행하게 된다.



2. PMD 클래스 상세분석

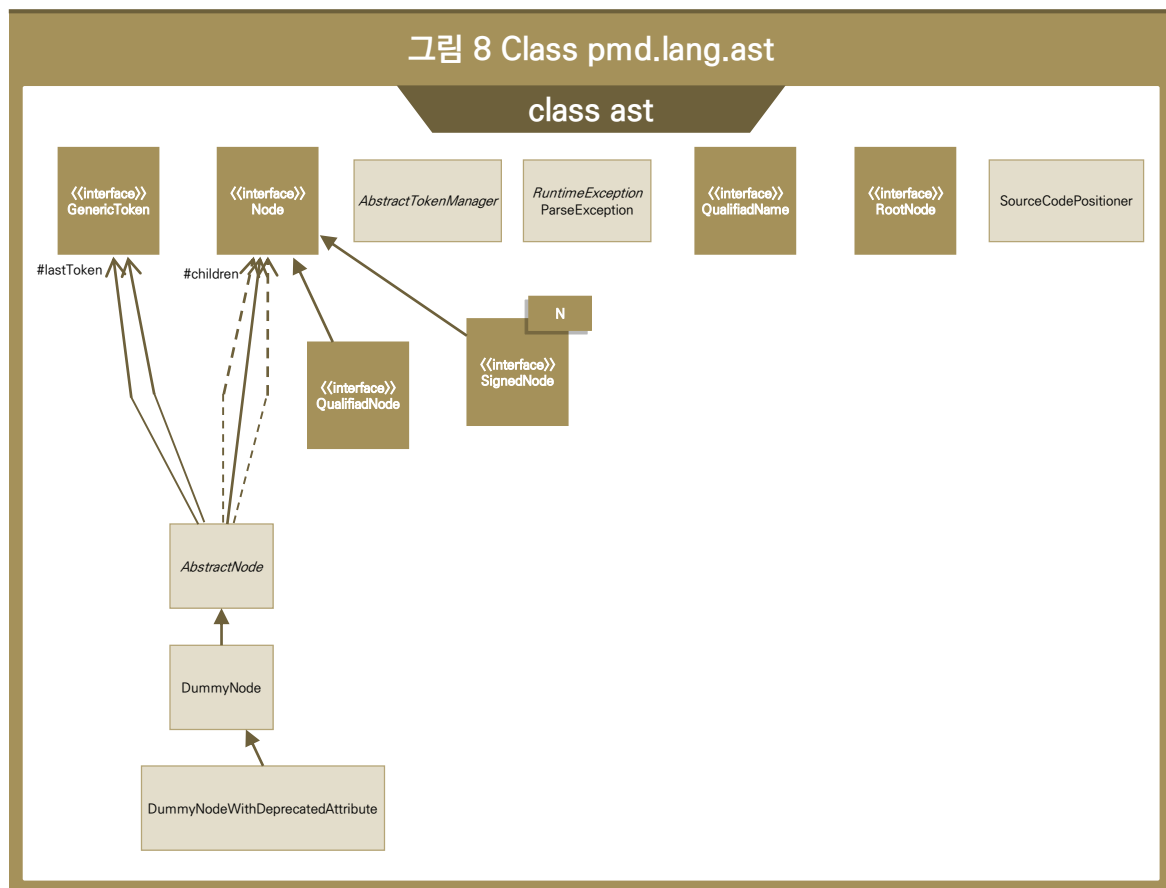
1) pmd.lang

- 그림 7을 보면 Language, LanguageVersionHandler, Parser, DataFlowHandler가 각 언어별로 구현되어있다.
- Language: 언어의 이름, 확장자, 룰 탐색을 위한 visitor, 언어 버전등이 정의되어있다.
- LanguageVersionHandler: 언어를 분석하기 위한 각종 인터페이스가 정의되어 있다. DataFlow Handler, XPathHandler, RuleViolationHandler, Parser, [타입, 심볼, 데이터플로우]를 위한 Visitor등이 정의되어 있다.
- Parser: 코드 텍스트를 구조화된 AST로 읽어들이기 위한 모듈이다.
- DataFlowHandler: AST를 탐색하며 데이터 플로우 관련한 노드들을 생성하게 되는데 이를 위한 유틸이다.



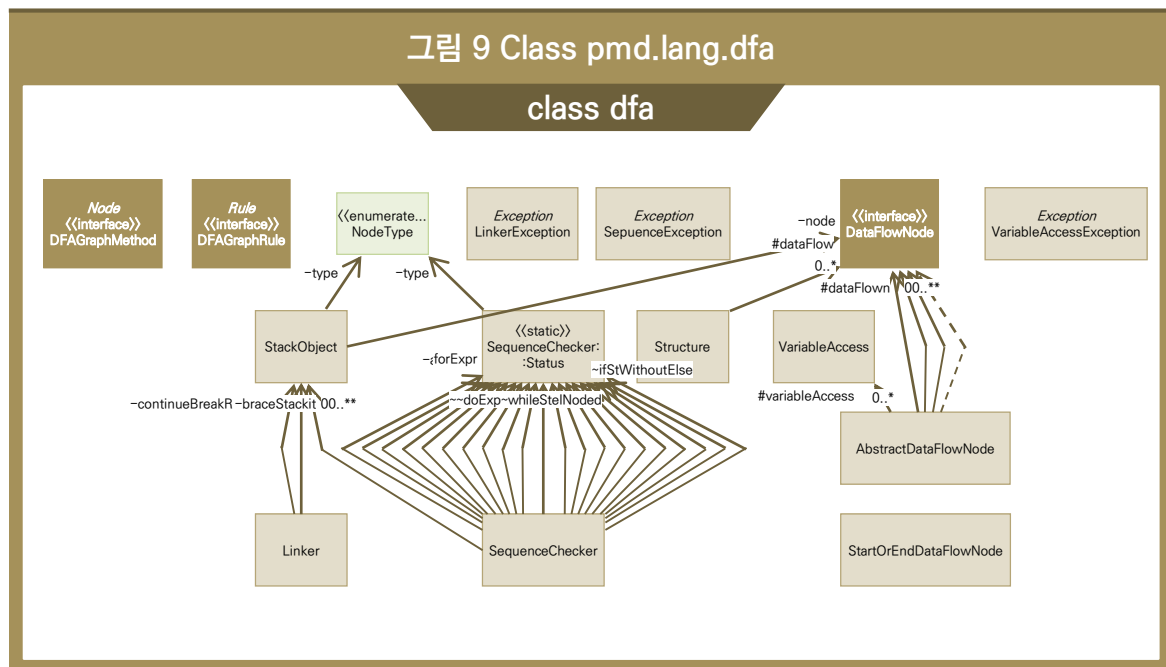
2) pmd.lang.ast

- 그림 8을 보면 AST(간략화 문법 구조)의 각 노드, 소스코드상의 포지션 정보를 관리하는 모듈이 들어있다.
- GenericToken: constant, keyword, comment등 언어와 독립적인 토큰들을 표현한다.
- Node: AST 각 노드의 토큰정보, 부모노드, 자식노드 정보를 포함한다.
- QualifiedName: 클래스의 qualified name이나 리소스 정보를 담는다.
- SourceCodePositioner: 각 노드마다 절대위치가 어디인지 찾는 모듈이다. Rhino, XML, APEX 등이 이 모듈을 사용한다.



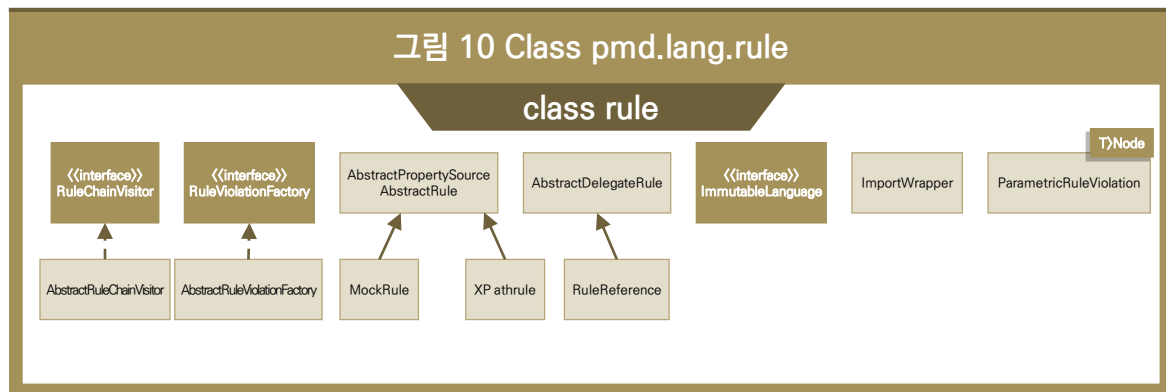
3) pmd.lang.dfa

- 그림 9를 보면 dfa의 각 모듈의 모양을 확인할 수 있다. SequenceChcker가 각 statement, expression들의 상태를 체크하게 된다. DataFlowNode에서 Variable access정보를 저장하여 분석중에 활용할 수 있도록 한다.
- NodeType: 어떤 expression, 어떤 statement인지를 표기하기 위한 Enum이다.
- DataFlowNode: 데이터 플로우 노드를 표현한다. variable access와 같은 정보를 등록하는 메소드, 타입을 확인하는 메소드, 소스코드 정보 등 데이터 플로우 분석을 위해 필요한 정보들이 등록된다.



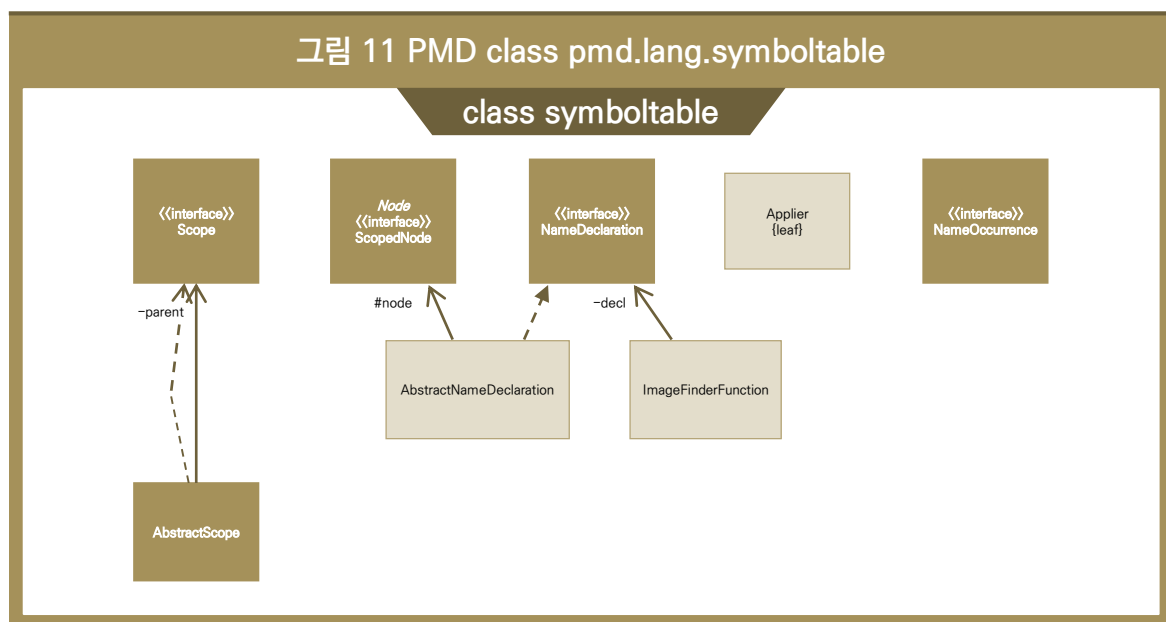
4) pmd.lang.rule

- 그림 10을 보면 RuleChainVisitor, RuleViolationFactory 두 가지 주요한 모듈이 나타나 있다. RuleChainVisitor는 AST에서 룰과 관련된 흥미로운 노드들을 뽑아내는 모듈이다. RuleViolationFactory는 룰이 위반되었는지 체크하는 모듈을 만든다.
- RuleChainVisitor: 각 언어에 대해서 어떻게 AST를 방문해야 하는지에 대한 전략을 담고 있다. 룰을 visitor에 추가 할 수 있다.



5) pmd.lang.symboltable

- 그림 11을 보면 symboltable을 이루는 Scope 모듈이 나타나 있다. 메소드나 variable의 Name Declaration마다 NameOccurence가 어디에 있는지 정보를 Scope에 저장하게 된다.



6) pmd.lang.{각종 언어들} 의 UML

앞서 보인 abstract class들의 실제 구현체이다. 대부분은 상속한 그대로 활용하고 java에 대해 가장 많은 튜닝이 되어있다.

(1) java

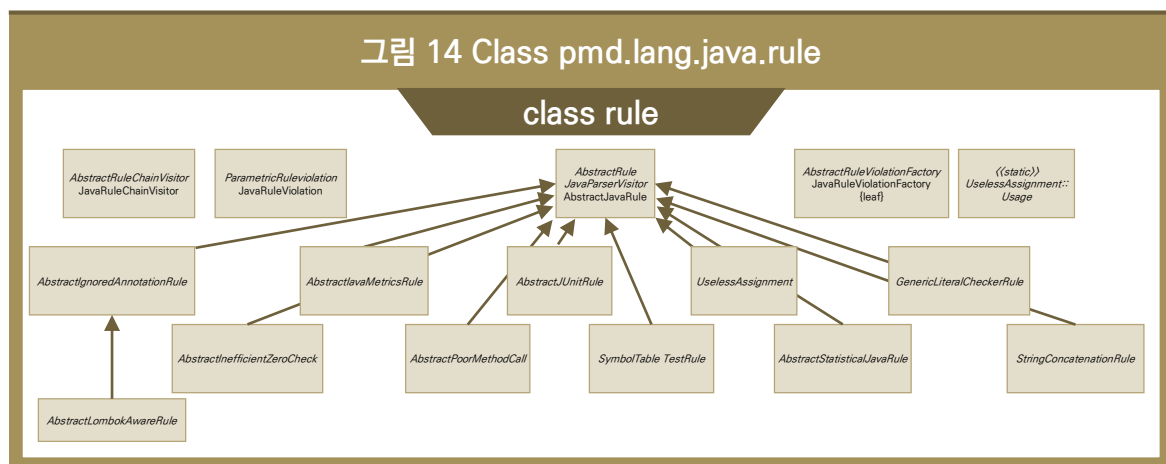
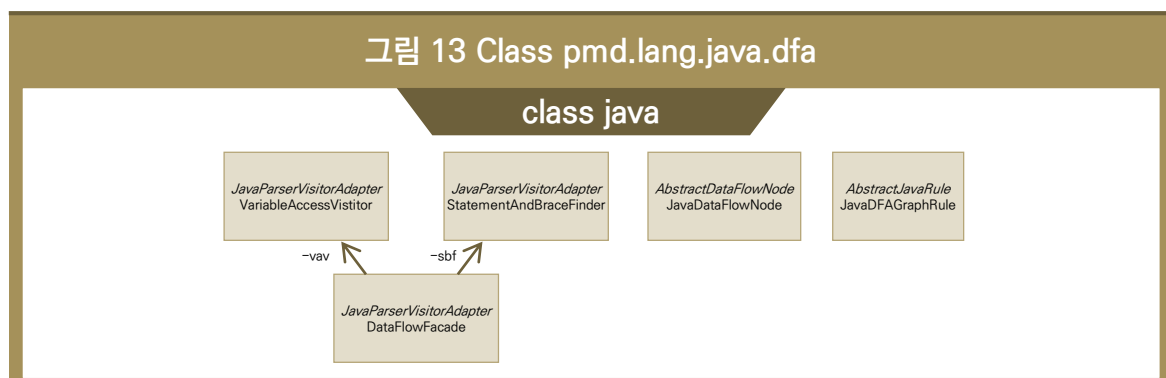
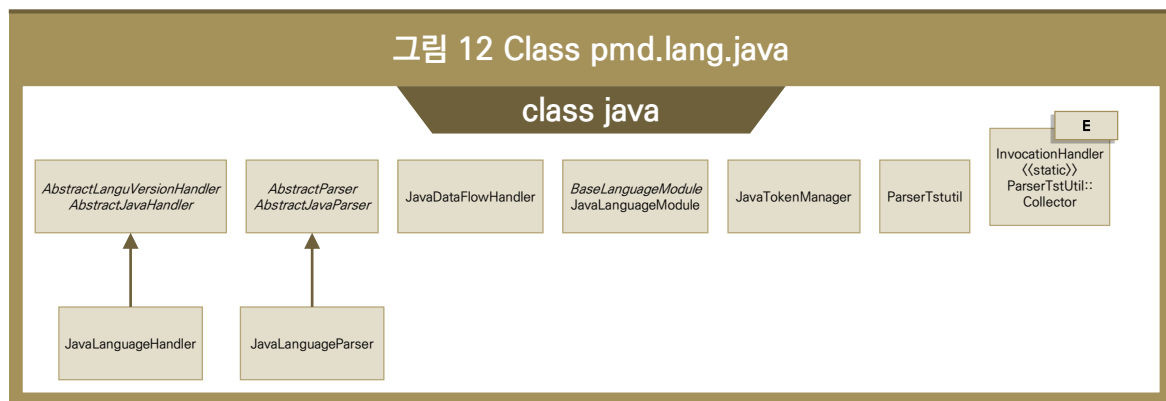
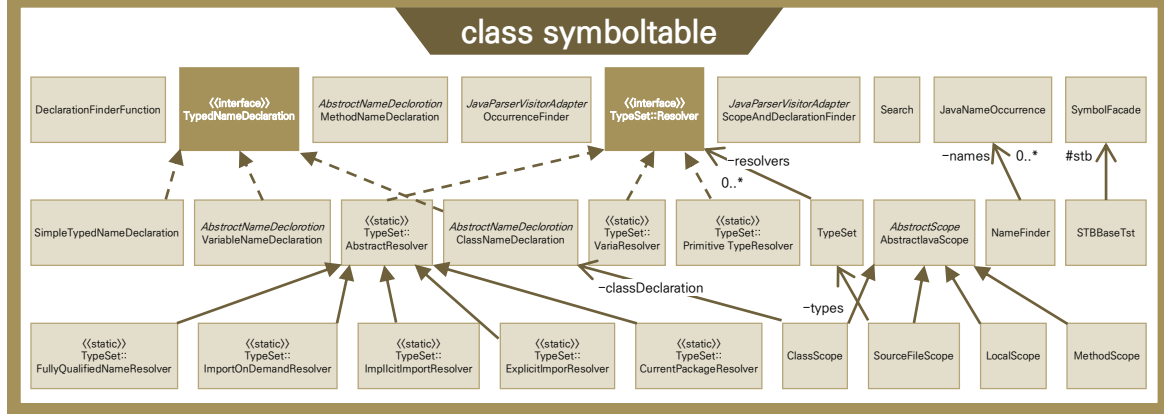


그림 15 Class pmd.lang.java.symboltable



(2) xml

그림 16 Class pmd.lang.xml

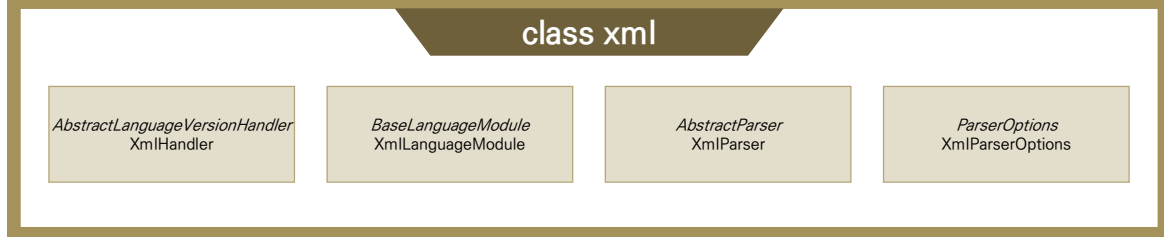
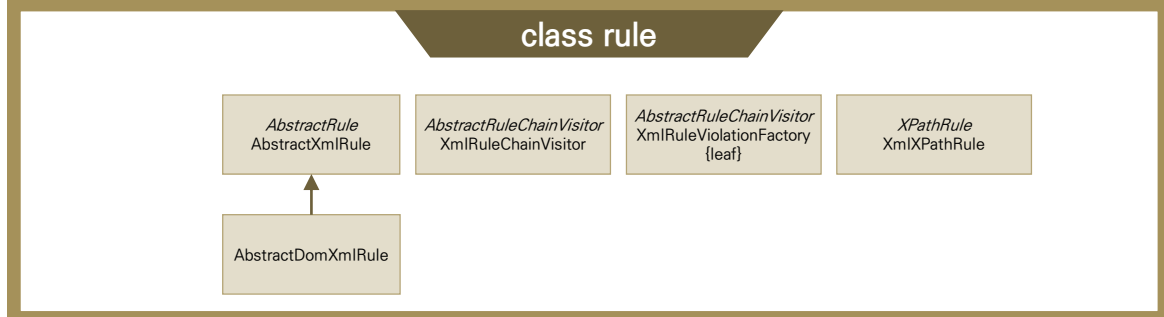


그림 17 Class pmd.lang.xml.rule

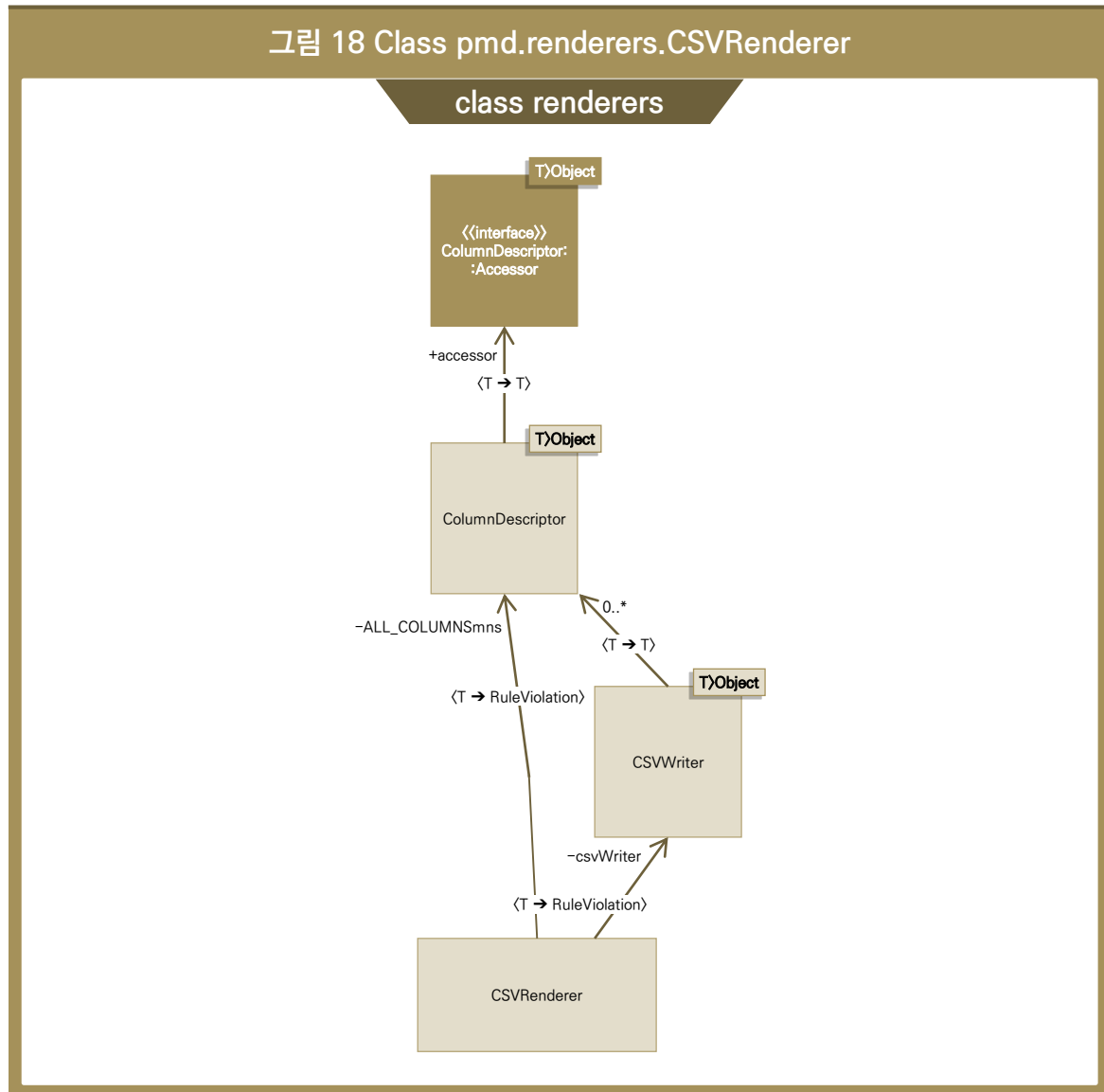


7) pmd.properties

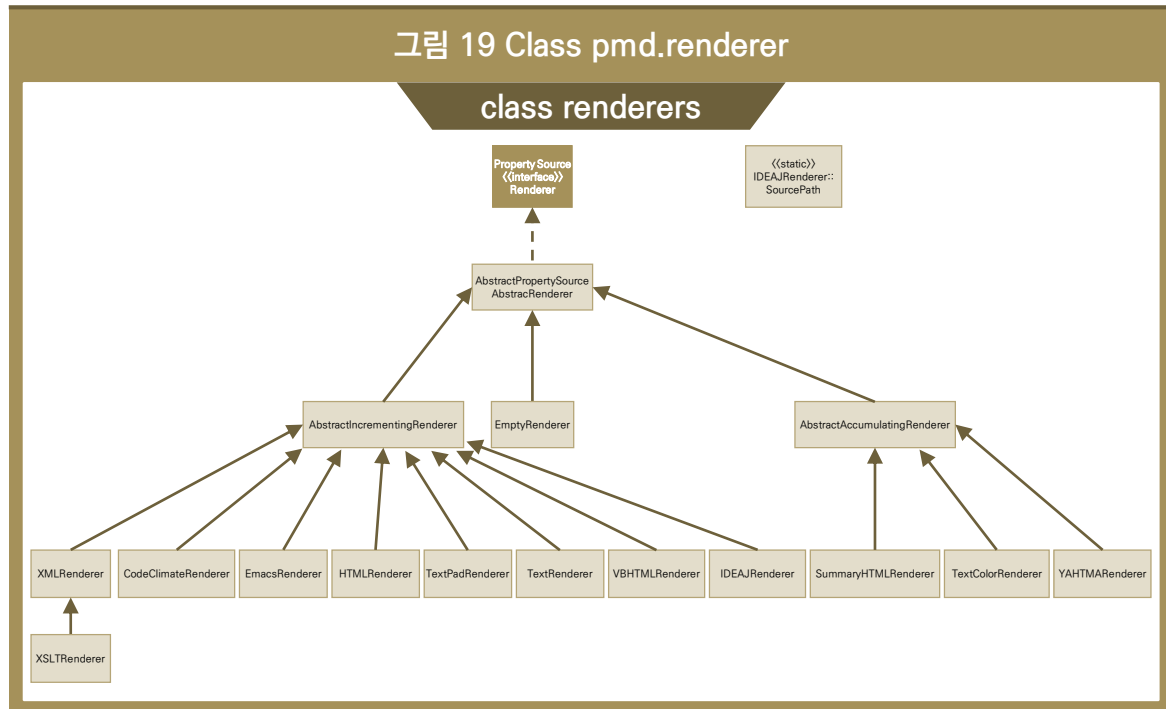
- 각종 값들의 Property들이 구현되어있다. 예를 들면 long 타입의 min값은 어떻게 max값이 어떤지 등등을 기술하고 있다.

8) pmd.renderers

- 그림 18을 보면 CSVRenderer가 존재하여 CSV출력을 담당한다.



- 그림 19를 보면 위의 CSVRenderer와 같은 여러 렌더러들이 구현되어있다. XMLRenderer, Code ClimateRenderer, EmacsRenderer, HTMLRenderer, TextPadRenderer, TextRenderer, VBHTML Renderer, IDEAJRenderer, SummaryHTMLRenderer, TextColorRenderer, YAHTMLRenderer. 각각의 플랫폼에서 분석 결과를 출력하는 것을 담당한다.



제5절 참고사이트

- [1] Spotbugs, <https://spotbugs.github.io/>
- [2] FindSecurityBugs, <http://find-sec-bugs.github.io/>
- [3] PMD, <https://pmd.github.io/>
- [4] Jenkins, <http://jenkins-ci.org/>
- [5] eclipse, <https://www.eclipse.org/>
- [6] IntelliJ, <https://www.jetbrains.com/idea/>
- [7] maven, <https://maven.apache.org>

전자정부 SW 개발자·진단원을 위한 공개SW를 활용한 소프트웨어 개발보안 점검가이드

인 쇄 2019년 6월

발 행 2019년 6월

발행처 **행정안전부**
세종특별자치시 정부2청사로 13

한국인터넷진흥원
전라남도 나주시 진흥길 9

〈비매품〉

※ 본 가이드 내용의 무단 전재 및 복제를 금하며, 가공·인용하는 경우 반드시 “행정안전부·한국인터넷진흥원의 『공개SW를 활용한 소프트웨어 개발보안 점검가이드』”라고 출처를 밝혀야 한다.

※ 본 가이드 관련 최신본은 행정안전부 홈페이지(www.mois.go.kr), 한국인터넷진흥원 홈페이지(www.kisa.or.kr) 에서 얻으실 수 있습니다.